

Chip Hippo

User Guide

Version 1.4.0 · October 13, 2026

Contents

1 Overview

2 Getting Started

3 The Desk & Breadboards

4 Chips & Components

5 Wiring, Nets & Buses

6 Power & Clock Sources

7 The Chip Library

8 Custom Chips

9 Running a Simulation

10 Spice Lite

11 Probing & Net Names

12 Memory Chips & the Inspector

13 Logic Analyzer & Timing

14 Build Guide & BOM

15 Schematic View

16 3D View

17 AI Circuit Builder

18 Arduino Integration

19 Serial Protocol

20 Files, Saving & Undo

21 Projects & Desktops

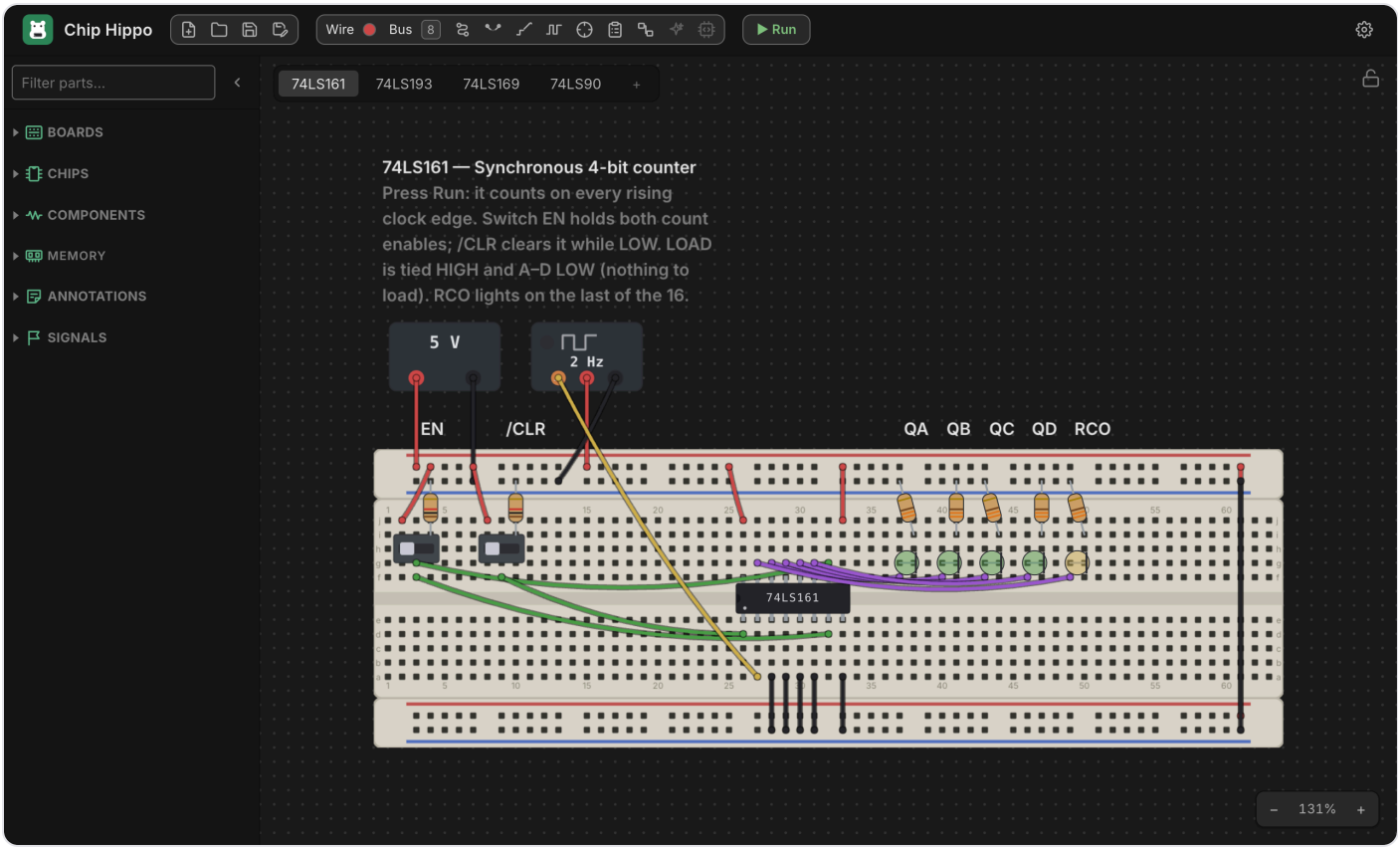
22 Exporting to Other Tools

23 Settings

24 Keyboard Shortcuts

Chip Hippo User Guide

Chip Hippo is a desktop app for designing and simulating **74xx TTL logic circuits** on virtual solderless breadboards. Place a breadboard on the infinite desk, populate it with 74xx DIP chips, wires, switches, LEDs, and power sources, then press **Run** — a simulation engine traces electricity from every power source, resolves each electrical net, and ripples changes through the circuit exactly like a real breadboard would.



What you can build

Anything a logic breadboard bench can hold, from two families — **74LS TTL** and **CD4000 CMOS**: combinational logic (gates, decoders, multiplexers), sequential logic (flip-flops, counters, shift registers), a free-running or manually stepped clock, and small memory circuits backed by ROM/RAM chips — all wired across Full/Half/Tiny breadboards, powered at 3 V to 15 V, and watched settle live. Beyond the 74xx shelf there is a **PROCESSOR group** — a W65C02 and a Zilog Z80A, each a full instruction-set simulation — and an **Interface group** of 65xx peripherals (a PIA and a VIA), for building a small single-board computer out of the same glue logic.

Highlights

- **Breadboard-accurate placement** — chips, wires, and discretes snap to the real 0.1 in hole grid, with the same row-half/trench/rail tie-point rules as a physical board.
- **Live simulation** — Run the circuit and watch LEDs light, chip health badges report power/damage state, and switches drive the circuit live.
- **Deep inspection tools** — a connectivity **probe**, **net names/labels**, and a **logic analyzer** for capturing waveforms.
- **Memory chips with a hex inspector** — file-backed ROM/EEPROM images you program with an in-app programmer, plus a live hex/ASCII viewer.
- **A derived schematic view** — press `Tab` to flip between the physical breadboard and a logical diagram of chip symbols, routed named nets, and bus lines, kept in sync with the desk.
- **Build guide & BOM export** — including a numbered cutting list of every jumper — and **undo/redo** across every edit, per desktop of the open project.
- **Seven languages** — English, German, Spanish, French, Italian, Japanese and Chinese, chosen in Settings ▸ Appearance (this guide is English only).
- **An AI circuit builder** — describe a circuit in words, using your own AI connection, and get a wired design that has already been built and run before you're offered it.
- **A link to real hardware** — Outputs and Inputs carry values between the running circuit and an Arduino over USB, and Chip Hippo writes the sketch's side of the link for you.

Table of contents

Getting started

- Getting Started — install the app, place your first board and chip, and run your first circuit.

Building a circuit

- The Desk & Breadboards — pan/zoom, breadboard kits, strips and rails, snapping and grouping.
- Chips & Components — the parts palette, DIP chips, discretes, placement and rotation.
- Wiring, Nets & Buses — the wire tool, colors, cross-board wires, and multi-bit buses.
- Power & Clock Sources — PSU bricks, voltage and the over-voltage damage rule, and clock sources.
- The Chip Library — the catalog: 74LS and CD4000 logic, memory, the 65xx interface parts and the CPUs, and the pin-assignments/datasheet window.
- Custom Chips — design a chip of your own: its package, its behaviour in a subset of Verilog, and a debugger to step through it while the circuit runs.

Simulating & inspecting

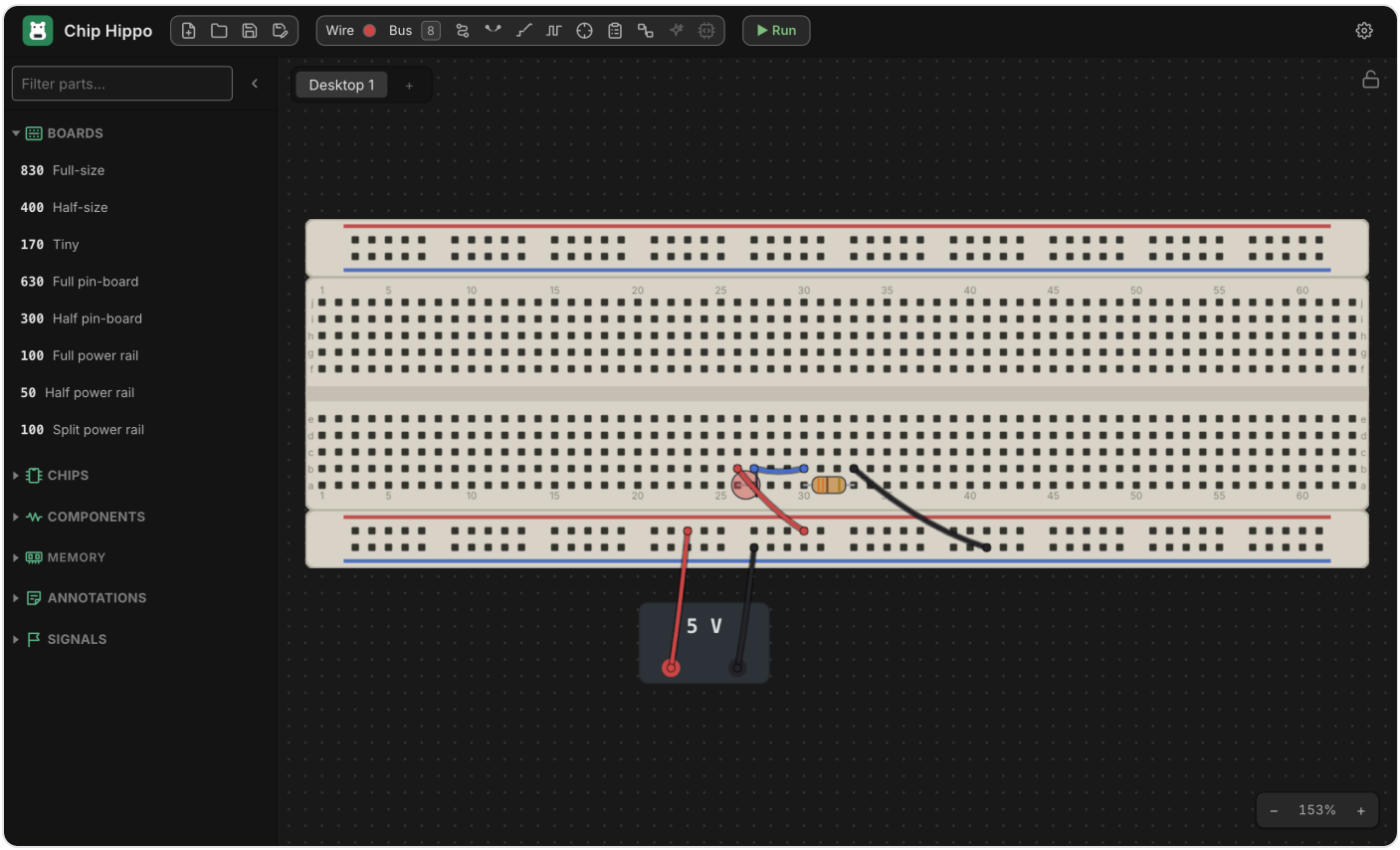
- Running a Simulation — Run/Stop/Pause/Step, the settle model, and live views.
- Spice Lite — the more electrical simulation: charging capacitors, gate delays, output and supply current limits, and wire resistance.
- Probing & Net Names — the connectivity probe and naming nets.
- Memory Chips & the Inspector — ROM/RAM, the programmer, and the hex/ASCII inspector.
- Logic Analyzer & Timing — capturing and reading waveforms.
- Schematic View — the derived logical diagram.
- 3D View — the desk stood up in three dimensions, to orbit round.
- AI Circuit Builder — describe a circuit in words and get a simulation-proven design, using your own AI connection.
- Arduino Integration — pass values between the running circuit and a real Arduino over USB, with the sketch's header generated for you.

Files & reference

- Build Guide & BOM — deriving a bill of materials, a jumper cutting list, and an ordered assembly checklist.
- Files, Saving & Undo — where a desktop and a project are kept, saving them, and undo/redo.
- Projects & Desktops — tabbed desktops, saving a project, and copying a design from one desktop onto another.
- Exporting to Other Tools — writing a desktop out as a KiCad schematic or a Digital circuit.
- Settings — the settings dialog and its options.
- Keyboard Shortcuts — every shortcut in one place.

Getting Started

This page walks through building the smallest possible circuit — a breadboard, a power supply, and an LED — from a completely empty desk to a lit LED. It takes about five minutes and introduces the tools you'll use for every larger circuit afterward.



Launch Chip Hippo

Open the installed app like any other desktop program (or, if you're running from source, `make debug`). Chip Hippo opens on an **infinitely pannable, zoomable desk** — drag to pan, scroll or pinch to zoom, and the header's zoom controls (bottom-right of the desk) reset or step the view. The first time you launch, the desk is empty and shows a hint: *"Open the parts tray and add a breadboard to get started."*

Open the parts palette

The palette is a tray docked to the left of the desk, and it carries its own open/close control rather than a toolbar button. The chevron in the tray's top-right corner shuts it down to a narrow strip along the desk's left edge; the chevron at the top of that strip (or `Ctrl+P` / `⌘P`) slides it back out.

Below that chevron the strip keeps an icon for each of the tray's sections — Boards, Chips, Components, Memory, Annotations and Signals — the same icon that sits beside that section's name in the open tray, and level with it. Clicking one opens the tray straight onto that section, with every other section folded away. The icons are sized from the text, so they grow and shrink with the font size you set in Settings.

At the top of the tray is the **board selector** (Full / Half / Tiny breadboards, plus loose strips); below that, chips and components grouped by function, with a filter box to search by name.

Drag the tray's **right edge** to make it wider or narrower — useful when a long part name is elided. The width you leave it at is remembered between launches, and across closing and reopening the tray.

Add a breadboard

In the palette's **BOARDS** section, click **Full-size** (830 tie points) — the default recommendation for a first circuit; it has the most room and both power rails already attached. A translucent ghost of the whole kit follows your cursor. Move it over open desk space and click to drop it.

The Full kit places three strips as one rigid group: a power rail, the pin-board, and a second power rail — exactly like a real solderless breadboard. You can drag the whole board later by its body, or reopen the palette any time to add more.

Add power

Still in the palette, open the **COMPONENTS ▸ Power** group and click **Power supply**. A small PSU brick ghost follows your cursor — click open desk space near the breadboard to place it. A PSU brick isn't seated on the board; it's a free-standing part with two wireable terminals, **+** and **–**.

Right-click the placed PSU and choose **Properties...** to confirm it's set to **5 V** (new PSUs default to 5 V, but 3 V, 9 V, 12 V and 15 V are also there — anything over 5 V damages a 74LS chip, so save those for CD4000 CMOS parts, which are rated for 3–18 V). **Delete Component**, further down the same context menu, removes the PSU if you want to start over.

Add an LED

Back in the palette, open **COMPONENTS ▸ LEDs** and click **LED**. A small LED ghost follows your cursor, already colored with your "Default LED color" setting (Settings ▸ Appearance — red out of the box). Move it over a free row of breadboard holes and click to seat it — an LED's two legs land in adjacent holes on the same row, anode first. (Press `F` while the ghost is in hand to flip which leg is the anode, or `R` to stand it up on two free ends instead of a footprint row.) To change an LED's color afterward, right-click it and choose **Properties...**.

Give an LED a series resistor, just as you would on a bench. Wired straight between a supply rail and ground it has nothing to limit the current, and the simulation shows it **burnt** rather than lit. One resistor anywhere in the loop is enough — including a single one in a display's common leg, which protects all of its segments at once. (The one thing that limits the current by itself is a CD4000 output at 5 V or below — see Mixing logic families.)

Wire it up

Press `W` (or click **Wire** in the toolbar) to arm the wire tool. Click a free hole or terminal to anchor the first end, then click a second free point to lay the wire — a rubber-band preview tracks your cursor between clicks, and it turns red over an illegal target. The tool stays armed, so you can lay several wires in a row without re-arming it; press `Esc` to cancel a pending wire, or `Esc`/click **Wire** again to disarm the tool.

Lay two wires:

1. **PSU `+` → the LED's anode hole** (or a rail hole in the same row/column as the anode, then the anode's row completes the path through the board).
2. **PSU `–` → the LED's cathode hole**, the same way.

The active wire color is shown as a dot on the **Wire** button; click the dot to open the color palette and pick a different one — the color you pick stays active for every wire you lay afterward, so use different colors for positive and return paths if you want them easy to tell apart at a glance.

Run it

Press **Run** in the header toolbar (or `Space`) to start the simulation. The engine traces power from the PSU, resolves the electrical net at every hole, and settles the circuit — the LED should light immediately, since anode-high through cathode-low is exactly what a driven LED needs.

While running, editing is locked (you can't move parts or lay new wires), but the connectivity **probe**, the **logic analyzer**, and the **build guide** all stay live. Press **Stop** (or `Space` again) to freeze the simulation and return to editing.

If the LED doesn't light, double-check both wires land on the correct terminal (`+`/`–`) and hole, and that the PSU is on and not showing a damage badge.

What next

- **The Desk & Breadboards** — pan/zoom, breadboard kits, strips and rails, snapping and grouping multiple boards together.

- **Chips & Components** — the parts palette in depth: DIP chips, discretes, placement, and rotation.
- **Wiring, Nets & Buses** — the wire tool in depth, cross-board wires, colors, and multi-bit buses.
- **Power & Clock Sources** — PSU voltage and the over-voltage damage rule, plus clock sources for sequential circuits.
- **Running a Simulation** — Run/Pause/Step, the settle model, and how live views (LEDs, chip badges) work.

The Desk & Breadboards

The desk is Chip Hippo's infinite, pannable, zoomable workspace — the place every board, chip, wire, and power source lives. This page covers how to move around the desk and how breadboards actually work under the hood: they are not single parts but individual **strips** — a pin-board plus power rails — that snap and mate together the way real boards do on a bench.



Pan, zoom & fit to screen

The desk has no edges — pan and zoom to work at whatever scale suits the circuit in front of you.

- **Pan**: click-drag an empty patch of desk, or two-finger scroll on a trackpad.
- **Zoom**: scroll/pinch to zoom, centered on the pointer, or use the **+** / **−** buttons in the zoom control (bottom-right of the desk). Keyboard: `Option+Cmd/Ctrl+=` (zoom in), `Option+Cmd/Ctrl+-` (zoom out), `Option+Cmd/Ctrl+0` (reset to 100%). **Option is what makes these the desk's**: the same keys *without* it resize Chip Hippo's own text instead (see Settings).
- **Fit to screen**: `Cmd/Ctrl+F` frames everything currently on the desk in the viewport — the fastest way to find your circuit after panning away. It also slides the whole design back around the middle of the desk, keeping a layout that has crept a long way out from drifting further; that part is a document edit (one `Cmd/Ctrl+Z` puts it back) and is skipped while the circuit is running.
- **Zoom out full**: `Cmd/Ctrl+Shift+F` zooms all the way out, useful for locating a part on a very large or sprawling layout.

A faint **dot grid** marks the 0.1 in hole pitch under everything you place; it coarsens at low zoom and disappears entirely once you're zoomed out too far for individual holes to matter. Zoom and pan position are remembered between sessions.

The desk padlock

A padlock sits in the desk's **top-right corner**, across from the tab strip. Click it (or press `Cmd/Ctrl+L`) to shut it, and the **scroll wheel stops reaching the desk**.

It exists for one input in particular. A Magic Mouse — and some trackpads — reports a scroll from a finger merely resting on the surface, so the wheel is the one thing that moves the desk without being asked to; midway through seating a chip, that's a circuit that slides out from under you. Everything else still works while it's locked: drag-to-pan, the zoom cluster, the keyboard shortcuts above, and **Fit**. It's the wheel that's locked, not the camera.

The padlock's shape changes rather than just its colour — an open shackle when the wheel is live, a shut one when it isn't. It's **saved with the project**: shut it, save, and the project reopens with the wheel still locked. Because it's part of the file, changing it counts as an unsaved change, like renaming a desktop. A new project starts with it open. The schematic view has a wheel of its own and no padlock over it.

Breadboards are strips

A real solderless breadboard is not one molded part — it's a centre **pin-board** with one or two **power-rail** strips dovetailed onto its top and bottom edges. Chip Hippo models this literally: each strip is its own object on the desk, and what looks like "a breadboard" is a small **kit** of strips placed together in one action.

The palette offers three assembled kits:

- **Full 830** — a full-length pin-board with a rail strip above and below.
- **Half 400** — the same layout at half length.
- **Tiny 170** — a bare pin-board only; the real 170-point part ships with no rails at all, so this kit is a single strip.

On a pin-board, rows run top to bottom as `j i h g f`, then the **trench**, then `e d c b a`. The trench is the gap down the centre that isolates the top half of each column from the bottom half electrically — a DIP chip **straddles the trench**, with one row of pins in `f` and the other in `e` (or, for the wide 600-mil chips, in two rows six holes apart — `h` and `d`, say), exactly as it would seat on a real board. A power-rail strip carries both a `+` and a `–` line, each one continuous connection along its whole length, independent of every other strip.

See Chips & Components for how chips and discretes seat into the grid, and Power & Clock Sources for feeding a rail from a PSU.

Kits vs. loose strips

Below the assembled kits, the palette also offers the individual strips on their own — a bare **Full pin-board**, a bare **Half pin-board**, a spare **Full power rail**, a spare **Half power rail**, and a **Split power rail**. Reach for these when a board shipped without enough rails, when you want a rail somewhere a kit wouldn't put one, or when you're building up a custom layout strip by strip. Placing, ghosting, and overlap checking all work identically whether you're dropping a whole kit or a single loose strip.

The split power rail

A **Split power rail** is a full-length rail with both of its lines cut in the middle, like the boards sold for circuits that need two supplies. Holes 1–25 and 26–50 of each line are separate connections, and the printed red and blue stripes break at the cut so you can see where it is. Each half can take its own supply — 5 V for the 74LS chips on the left, 9 V for the CD4000 chips on the right, say — and its own net name. It dovetails, turns, and seats exactly like a Full power rail.

Jumpering the two **–** halves together is fine (two supplies normally share a ground). Jumpering the two **+** halves together is not: two supplies set to different voltages would then fight over one net, and the simulation reports it as a fault (see Power & Clock Sources).

Snapping & mating

Drop a strip within a couple of tenths of an inch of another strip it can dovetail with — matching width when stacked, matching height side by side, meeting flush with no gap — and it snaps into place automatically. This is the same **magnetic pull** whether you're dragging a placed strip or holding a fresh one from the palette; a whole kit snaps as one piece, and the pull only ever engages when the snapped position is still legal (it will never pull a strip into an illegal overlap).

Two strips that end up flush **mate**: they join a shared **group** and from then on drag together as one rigid unit, just like a real breadboard's rails stay attached to its pin-board when you nudge it on the bench. A kit arrives pre-grouped; anything you drop flush against an existing board joins its group (or merges two groups into one, if it bridges a gap). When several strips are mated, clicking any one of them highlights the whole set — the outline traces the union of the group's strips, so flush neighbours read as one board rather than showing a seam between them.

Selecting things

Clicking a board, part, wire, bus or note selects that one thing, and clicking empty desk clears the selection. There are two ways to select **several** at once, and they are complementary:

- **Shift-drag a marquee** anywhere on the desk. It *replaces* the selection with everything the box encloses **completely** — any board wholly inside, any part whose every pin is inside, and any wire with **both** ends inside. Half-covered things are left out on purpose: a wire with one end outside the box would have to be cut to come along.
- **`Cmd`-click** (`Ctrl`-click on Windows and Linux) an individual item to *add* it to the selection, or to take it back out if it's already in. This is how you pick up the four chips you actually want without drawing a box around the eight that are next to each other, and how you drop the one thing a marquee caught that you didn't mean.

`Cmd/Ctrl`-clicking a **board** takes its whole snapped group, the same set a plain click selects. Clicking a **bus** takes all of its member wires — a bus is bookkeeping over wires, and the wires are what a delete or a copy acts on. Clicking **empty desk** with the modifier held leaves the selection alone rather than clearing it: an add that landed on nothing has nothing to add.

Notes and labels are the one thing a multi-selection can't hold, so `Cmd/Ctrl`-clicking one does nothing; a plain click still selects it.

> **On macOS this is `Cmd`, not `Ctrl`** — the modifier the rest of the platform's lists use. `Ctrl`-click there is the system's secondary click, and it keeps that job: it still opens an item's context menu.

Either way the result is one selection: `Delete` removes all of it in a single step, `Cmd/Ctrl+C` copies it (as a whole design, if it includes boards — see Projects & Desktops), and `Esc` clears it. `Cmd/Ctrl+A` selects the entire desk.

A selection also moves as one. Grab any part in it and the whole set travels by the same amount, keeping the arrangement you built; hold **Option** as you press and the wiring comes too — every wire end, and every resistor or LED leg, plugged into any member. The drop is all or nothing, and the move is one `Cmd/Ctrl+Z`. See Moving several parts at once for the whole of it.

Boards are the exception, and it's the one thing to know before you drag a mixed selection. A **board** in the set makes a press on a part do nothing at all — strips have their own drag, which carries everything seated on them under mating and overlap rules a part re-seat knows nothing about — and the selection is left untouched so you can `Cmd/Ctrl`-click the board back out. Grabbing the **board** instead drags that board's own group (and everything seated on it) in the usual way, replacing the selection as any plain click does.

Nothing can be selected while the simulation is **running** — the topology is frozen, so a selection would be one you couldn't act on.

Groups & breaking a snap

Grabbing a mated strip normally drags the **whole group** together. To pull just part of a group apart, hold a modifier while you start the drag:

- **Option-drag** — moves the **reachable run forward** from the strip you grabbed: itself plus every strip mated to it through a below/right edge.
- **Option+Shift-drag** — moves the run **backward** instead, following above/left edges.

Either way, only strips still reachable through the group come along; a strip that merely happens to sit flush elsewhere in the layout is left behind. When you drop a torn-off run, both the piece you moved and what's left of the original group are re-evaluated — each is split into fresh groups based on what's still actually mated within it, so the two halves never end up sharing a group id after the break.

Rotating a rail

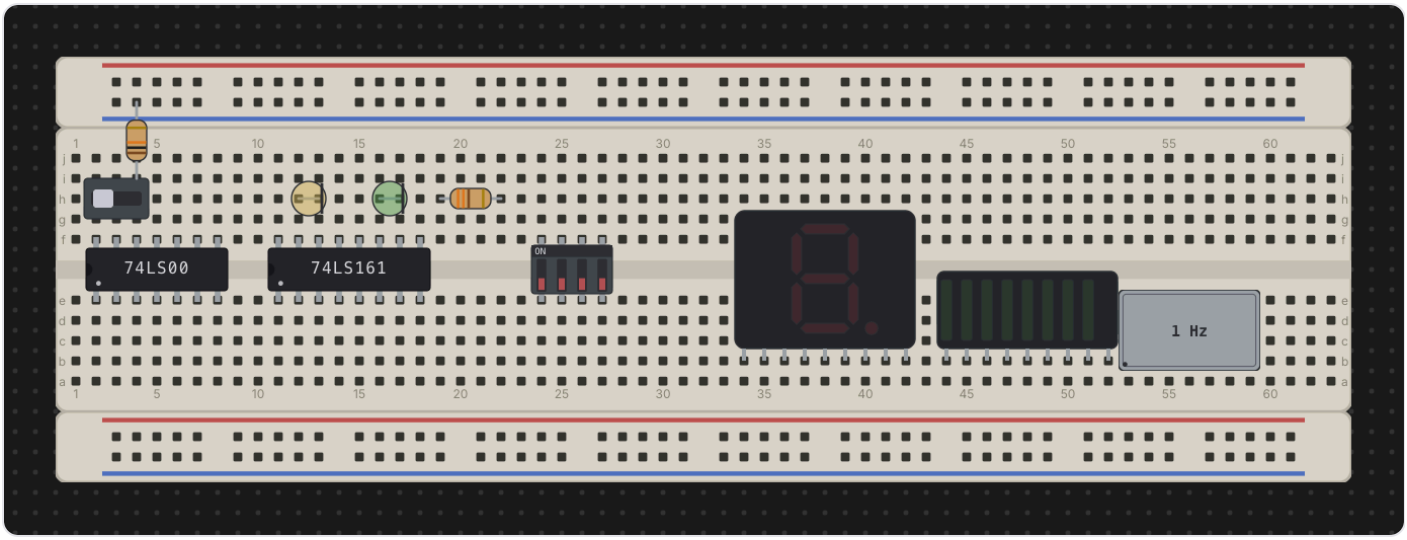
Power rails can rotate; pin-boards cannot. A pin-board's trench (and every DIP seated across it) only makes sense in one orientation, so it's locked at 0°. A rail strip is just two parallel lines of holes, so it reads the same standing on end — turned 90° it becomes a vertical **signal bus** you can run alongside a board and tap into at any point along its length.

Press `R` while a rail is in hand (mid-placement, before you click it down) to cycle its rotation through 0°/90°/180°/270°. Once a strip is placed its angle is fixed; to change it, pick it up again. Rotating doesn't change anything electrically — a rail is one continuous node however it's turned (each half is, on a split rail) — it only changes which way the strip's footprint runs on the desk.

Next: Chips & Components for populating a board, or Wiring, Nets & Buses for connecting everything up.

Chips & Components

Everything that isn't a breadboard strip lives in the **Parts** palette on the left: 74LS and CD4000 logic chips, CPUs, 65xx interface chips, memory chips, switches, LEDs, displays, resistors, capacitors, inductors, diodes, transistors, oscillators, the 555 timer, and power/clock bricks. This page covers finding a part, seating it on a board, and the (surprisingly varied) ways different parts rotate and flip once they're down.



The parts palette

The palette opens with every section collapsed, grouped by function:

- **BOARDS** — the breadboard kits and loose strips, pinned at the top (see The Desk & Breadboards).
- **CHIPS** — the logic chips, folder-grouped by function, ending with the **Timer** group (the **555 timer**), the **Op-amps** group (the **LM358**), the **Interface** group (the 65xx PIA/VIA, the **ULN2003A** Darlington array and the **4N35** and **PC817** optocouplers) and the **PROCESSOR** group (the W65C02 and Z80A CPUs). Which logic family it shows — 74LS, CD4000, or both, each in a folder of its own — is **Settings → Data Sheets → Chip family** (see Choosing a logic family). The 555 belongs to neither family, so it shows in every mode; showing CD4000 alone, it shares the **Timer** group with the CD4000 timers.
- **COMPONENTS** — **Switches**, **Resistors**, **Capacitors**, **Inductors**, **Diodes**, **Transistors**, **Relays**, **LEDs**, **Displays**, **Oscillators**, **Regulators** and **Power**, in that shelf order. Capacitors, Inductors, Diodes and Transistors carry a red **(i)** beside their names: in the digital engine these parts do only part of what the real ones do, and are there for the export and a complete design (see Inductors, diodes and transistors). With **Spice Lite** on they are simulated for real, and the **(i)** goes.
- **Memory** — the ROM/RAM chips, pulled out of CHIPS into a top-level group of their own.

- **ANNOTATIONS** — labels and notes (see Probing & Net Names).
- **SIGNALS** — bench stimulus buttons, pinned at the bottom (see External signals).

With the tray shut, the strip it leaves on the desk's edge carries one icon per section in the same order — click one to open the tray on that section alone. To keep the tray down to one section all the time, turn on **Auto-close tray folders** in Settings: opening any folder then closes the rest.

The tray's right edge is a drag handle: pull it out to give long part names more room, and the width you leave it at is remembered.

Type in the **Filter parts...** box to search by id, title, or description — matching, it forces every group open so results aren't hidden behind a collapsed folder. Click any entry to arm placement — a ghost of the part follows your pointer until you click it down on a board, or press `Esc` to cancel.

Placing a DIP chip

A DIP chip always straddles the trench: half its pins seat in row **e**, the other half in row **f**, running the standard counterclockwise DIP numbering — pin 1 at the anchor hole in row e, pins continuing left to right along e, then wrapping back right to left along f. The **notch** end of the chip (or the dot beside pin 1) marks pin 1 and always faces left. Move the ghost over a pin-board and it snaps to the nearest legal seat; it turns red if the seat is already occupied or falls off the edge of the board.

The **wide chips** — the 24-, 28-, 32- and 40-pin packages: the memories, the 6502 and its peripherals, the Z80 and the 74LS181 — are 0.6 inch across, twice a small chip, and seat at that true width, their two rows of pins six holes apart across the trench. There are four ways to do that, and the chip takes whichever is nearest the pointer as you place or drag it:

Pin 1's row	Other row	Body covers
e	**i**	f, g, h
d	**h**	e, f, g
c	**g**	d, e, f
b	**f**	c, d, e

Every one makes the same connections — each half-column is one connected strip — so pick the one that leaves room for your wiring. The chip's body covers the three rows between its pins, so those holes can't take a wire or another part's lead; wire to each pin from the rows outside it instead. A dragged chip moves from one seat to the next as the pointer travels, wherever you took hold of it. A desk saved before wide chips seated this way keeps them in rows e and f just as they were; move one on its own and it takes its true width, bringing the wires you carry with **Option** out from under its body.

Because a chip's footprint is the same two rows no matter how it's turned, placing one is a matter of picking the column — there's no click-to-rotate step while placing a chip the way there is for a rail or a resistor; instead, rotation happens afterward (see below).

Placing a discrete

Most discretes — slide switches, push buttons, toggle buttons, LEDs (in their default horizontal form), single-digit 7/8-segment displays, and LED bars (bar8) — are **linear**: they seat along a run of adjacent holes in any single grid row (any of `a` – `j`), not just rows e/f. Drop one anywhere its footprint fits and every free hole underneath it is available.

A few parts don't fit that linear model:

- **bar8iso** — the isolated 8-segment LED bar — is packaged as a 16-pin DIP, so it straddles the trench exactly like a chip: anodes A1–A8 in row e, cathodes K1–K8 in row f.
- **DIP switch banks** (**sw-dip1**, **sw-dip2**, **sw-dip4**, **sw-dip8**) are likewise DIP-packaged (2/4/8/16 pins for 1/2/4/8 switch positions), straddling the trench the same way: each position's two facing pins — one in row e, one in row f — are its own independent SPST switch.
- **Oscillator cans** (**osc-full**, **osc-half**) are rigid four-cornered shapes rather than a line of pins — a full can is 7 holes by 4, a half can 4 holes square, with legs only at the four corners. A can can seat anywhere on the grid, including straddling the trench, since its shape (not a row) determines its footprint.
- **Character LCD modules** (**lcd16x2**, **lcd20x4**) *are* linear — a 16-way header along 16 adjacent holes in one row — but the module itself is much bigger than that row. See below.

Component values

A **resistor** has a **Resistance**, a **capacitor** a **Capacitance**, an **inductor** an **Inductance**, a **Zener diode** a **Zener voltage** and a **transistor** a **Part number**. Right-click the part, choose **Properties...**, and either pick from the field's list — the ▾ beside it, or ↓ — or type any value at all, the way it is printed on a parts drawer or a schematic. Typing narrows the list to the entries holding what you have typed (`4` leaves 47Ω, 470Ω, 4.7kΩ, 47kΩ and 470kΩ; `u` finds µ).

Typed	Reads as
`470` , `470R` , `470Ω` , `470 ohms`	470Ω
`4.7k` , `4k7` , `4.7 k` , `4.7 kilo ohms`	4.7kΩ
`1M` , `1meg` , `2M2`	1MΩ, 2.2MΩ
`2R2` , `0.47`	2.2Ω, 0.47Ω
`100n` , `100nF` , `0.1uF` , `0.1µF`	100nF
`4n7` , `10 microfarads`	4.7nF, 10µF
`10uH` , `1 millihenry`	10µH, 1mH
`5.1` , `5.1V` , `5V1`	5.1V

A plain number is in the field's own unit — ohms, farads, henries or volts — so `100` in a Capacitance is a hundred farads, which is out of range and says so. The prefixes are `p` `n` `u` / `µ` `m` `k` `M` `G` , or spelled out (`pico` … `giga` , and `meg` as SPICE writes it). A lowercase `m` is always milli and an uppercase `M` always mega: `1m` on a resistor is a milliohm, refused as out of range rather than guessed at. A prefix letter — or `R` for ohms, `V` for volts — can stand in for the decimal point (`4k7` , `2R2` , `4n7` , `5V1`). A unit belonging to something else is refused by name: `10uF` in a Resistance is "a capacitance, not a resistance".

Press Enter or move out of the field to apply it. A value that reads is shown back in its tidy form — `100 kilo ohms` becomes `100kΩ` — to three figures, or to as many as you typed (`4753` is `4.753kΩ` ; nothing is rounded away). One that doesn't stays as you typed it, in red with the reason under it ("Not a resistance value", "Out of range: 0.1Ω to 100MΩ"), and the part keeps the value it had. Any value in range is taken, on the list or not.

Field	Range	List
Resistance	0.1Ω – 100MΩ	the E12 series, 10Ω – 1MΩ
Capacitance (ceramic)	1pF – 100µF	10pF – 1µF
Capacitance (electrolytic)	100nF – 100mF	1µF – 1mF
Inductance	1nH – 10H	1µH – 680µH and 1mH – 100mH, in E6 steps
Zener voltage	1.8V – 200V	2.4V – 30V, each with its part

When the value a part holds is not a **standard** one — one it is actually made in — a small amber ⓘ appears to the right of the field. Click it for the standard values either side (`3k` offers **2.7kΩ** and **3.3kΩ**; a value past the end of the series, the one end it is past), and click one to take it. It is advice, never a

fault: a value that is not standard is still taken exactly as typed, and a click anywhere else, or Escape, puts the card away. Resistors and capacitors are measured against the **E12** series across their whole range (so `4.7n` is standard though the list does not offer it), a potentiometer against the track values pots are sold in, and an inductor or a Zener against its own list.

A **Zener**'s list pairs each voltage with its part — `5.1V (1N4733A)` — and picking one sets both. Typing a voltage on the list (`5.1` , `5V1`) brings its part number with it, typing a part number on it (`1n4742`) picks its entry, and any other voltage stands alone, with no part number.

A capacitor's **Type** (Ceramic or Electrolytic) and a transistor's **Type** (NPN, PNP, N- or P-channel MOSFET) swap the part where it stands and reopen its card as the new one; like any change to the wiring, the Type is greyed while the circuit runs. The value comes across untouched, so a capacitance outside the new type's range shows red rather than being changed. A transistor's list holds the parts of its own type, and a part number from another type's list is cleared by the swap (one you typed yourself is kept). Typing another type's part number is allowed, with a warning — `2N3906 is a PNP transistor`.

An older project's values load as they were. One that cannot be read is kept exactly as written, and shows red when you open the card.

On the desk a resistor wears its **colour code** — four bands for a value two figures say (`4.7k` : yellow, violet, red, gold), five for one that needs three (`4.99k`) — and a capacitor has its value printed on it. Both values appear in the Schematic View, the BOM, and the KiCad export.

A resistor's value changes nothing in the simulation: a resistor is a weak pull whatever it measures, and a lamp is lit or burnt by whether one is in the loop. Only the timing chips read it (below).

Potentiometers

COMPONENTS ▸ Resistors ▸ Potentiometer is a three-pin trimmer: a resistive track between the two outer pins and a **wiper** on the middle pin that taps it. It seats along one row like a switch, pins 1, W and 3 in three holes side by side. Press `R` while placing it, or with it selected, to turn it end-for-end: pins 1 and 3 swap ends and its screw swings to the other side.

Its **Properties...** card has the track's **Resistance** — picked from the values pots are sold in (100 Ω, 200 Ω, 500 Ω, 1 kΩ … 22 kΩ, 25 kΩ, 47 kΩ, 50 kΩ … 1 MΩ, 2 MΩ) or typed like any resistor's — and the wiper's **Position**, a slider from 0 % to 100 %. The resistance from the wiper to pin 1 is Position × Resistance, and to pin 3 is what is left of the track:

Position	Wiper ⇔ pin 1	Wiper ⇔ pin 3
0 %	0 — a wire	100k
10 %	10k	90k
50 %	50k	50k
90 %	90k	10k
100 %	100k	0 — a wire

(for a 100k part). The slider shows exactly that as you move it — the resistance to pin 1 at its left end and to pin 3 at its right, `15k ●── 85k` — and changing the Resistance updates both. It applies as you drag it, while the circuit runs as well, and the brass screw on the part turns to show where the wiper is.

Each side with track left behaves exactly like a resistor: a weak pull, and a value the timing chips read — a pot as a 555's RB tunes its rate. A side with **none** left is the wiper resting on that pin: a plain wire, so the two nets join, and an LED fed through it with nothing else in the loop **burns**.

Capacitors

COMPONENTS ▸ Capacitors holds two:

- **Capacitor (ceramic)** — a disc, not polarised, leads one hole apart.
- **Capacitor (electrolytic)** — a can with a stripe down its **negative** side, leads one hole apart. Pin 1 is `+`, pin 2 is `−`.

Both place, rotate and bend exactly like a resistor (see Rotating & flipping), and they take holes the way any part does, so a wire or a pin can't share one.

A capacitor carries a value, not charge. It does not filter, smooth, decouple or store anything, and it connects nothing: the simulation treats it as open, even when its legs sit across the two rails. What it is for is the **timing chips** — the 555, the CD4047B, the CD4060B, the CD4098B and the CD4538B read the capacitor (and the resistors) wired to their timing pins and run at the rate the datasheet gives for those values; see Timers. A capacitor still counts as connected when the checks ask whether a net has anything on it.

A reversed electrolytic makes no difference to the simulation. Watch the stripe anyway if you mean to build it.

The first time you place a capacitor a note says all this; **Don't show again** puts it away for good.

Inductors, diodes and transistors

On the standard engine these parts do as much as a logic simulator can honestly do, and no more: no forward drop, no gain, no thresholds, no Zener breakdown, no inductive kick. Spice Lite simulates all of those. What they always do is **export to KiCad as the real part** (see Exporting). Every one of them — and the capacitors — has an optional **Part number** in **Properties...** (`1N4148` , `2N2222` , `2N7000` ...). It is printed on the part, listed in the BOM and written to the export, and it changes nothing about how the part behaves.

Inductors (**COMPONENTS ▸ Inductors**) take an optional **Inductance** (see Component values); leave it blank for a bare inductor. Its **Style** is the part it is, seen from above like everything on the desk: a **Coil** — a toroid standing on edge over its leads, copper wound round a dark ferrite ring — or a **Can**, a drum in a black sleeve, its value printed on top. **Holes between leads** is **1** (leads 0.2 in apart, the smallest part), **2** (0.3 in, the default) or **3** (0.4 in, the biggest). On a part lying along a row it moves the second lead, so a size whose lead would land on a taken hole, or off the end of the board, is refused — with the reason under the choice — and the choice is greyed while the circuit runs, like any other change to the wiring. On the standard engine an inductor **conducts like a wire**: its two leads are one net, whatever it looks like. So one across the rails is a short, exactly as a wire would be. Under Spice Lite one with an Inductance is a real inductor, and its card gains a **Winding** choice.

Diodes (**COMPONENTS ▸ Diodes**) are the **Diode** and the **Zener diode**. Pin 1 is the anode; pin 2, the cathode, is the end the band marks. A diode is **one-way**: a HIGH on its anode passes to its cathode, and nothing ever passes back. A LOW or undriven anode leaves the cathode to whatever else is on it — so two diodes into one net, with a pull-down resistor there, make a **diode-OR**. The HIGH passes at the strength it arrived with: from a rail or a chip output it drives the cathode, through a resistor it only pulls it. Like an LED, a diode wired forward straight across two strongly driven nets (rail to rail, or an output into ground) **burns**. A Zener takes an optional **Zener voltage**, picked with its part number or typed (see Component values), printed and exported, but in the simulation it is exactly a diode: reverse breakdown is analog, so it regulates nothing here.

Transistors (**COMPONENTS ▸ Transistors**) are an **NPN** and a **PNP** bipolar transistor and an **N-channel** and a **P-channel MOSFET**, each standing over three holes in a row with its pin letters printed on it: `E B C` for the BJTs, `S G D` for the MOSFETs. A BJT is a **TO-92** unless you pick **TO-220** (a TIP120 or a TIP31C) under **Package** in its **Properties...**; a MOSFET is a **TO-220** — the power part, its metal tab behind it — unless you pick **TO-92**. The package is how it is drawn, listed in the BOM and exported; on the standard engine it is never how it behaves (under Spice Lite it picks the default **Grade**). Its **Type** turns it into any of the four where it stands. Picking a part number from the list sets its package (and, under Spice Lite, its grade) to match. Real pinouts differ by part number (a 2N2222 is E·B·C, a BC547 C·B·E, an IRLZ44N G·D·S), so select one and press `R` to turn it end-for-end. In the simulation each is a **switch**:

Part	Control	Joins	On while the control is
NPN	Base `B`	Collector `C` and emitter `E`	HIGH
PNP	Base `B`	Emitter `E` and collector `C`	LOW
N-channel MOSFET	Gate `G`	Drain `D` and source `S`	HIGH
P-channel MOSFET	Gate `G`	Source `S` and drain `D`	LOW

When it is on, the two switched pins are joined as a closed switch joins them; when it is off, they are apart. What an **undriven** or undefined control does is where the two kinds differ, as they do on a bench:

- A **BJT** follows its base and remembers nothing: with its base floating or undefined it is **off**.
- A **MOSFET's gate holds charge**: it stays in the last state its gate was driven to — **off** until the gate has been driven at all. So a gate left floating (or driven by a floating CMOS output) keeps the MOSFET on or off rather than passing the confusion down the channel. While it runs on that held charge, the lamp on its face is ringed in amber, its hover says so, and its **Properties…** card warns you: a real gate leaks that charge away, so tie it to a defined level with a resistor.

While the circuit runs, a transistor's lamp lights while it conducts. Put a resistor in an LED's leg as you would on a bench — with none, the LED burns — and a transistor switched on straight across the rails is a short.

A pin wired only to one of these parts **counts as connected**, conducting or not: a reversed diode or an off transistor is a valid board, not a floating input. Supplies don't pass through them, though — a chip powered through a diode or a transistor sees no supply and stays unpowered.

Regulators, relays and the electronic load

These are the parts a bench uses to power and switch things rather than to compute. Each does what a logic simulator honestly can on the standard engine, and what it really does under Spice Lite.

Regulators (**COMPONENTS ▸ Regulators**) are the **LM7805**, **LM7809**, **LM7812** and **LM7815** fixed regulators and the **LM317** adjustable one, each a TO-220 standing over three holes in a row. A 78xx's legs are `IN · GND · OUT` from the left; an LM317's are `ADJ · OUT · IN`, and two resistors set its output — **R1** from OUT to ADJ (240 Ω is usual) and **R2** from ADJ to ground, for **Vout = 1.25 V × (1 + R2 / R1)**: 240 Ω and 390 Ω make 3.3 V. Press `R` while placing one, or with it selected, to turn it end-for-end: its tab swings to the front and its legs read `OUT · GND · IN` (`IN · OUT · ADJ` for an LM317) from the left. On the standard engine a regulator's output **is a supply** — chips on it are powered, like chips on a rail — whenever its input is a supply at least its **dropout** above what it holds (2 V for a 78xx, 1.7 V for the LM317); below that its output is nothing. Regulators chain: a 7805 fed from a 7812's output works.

The relay (**COMPONENTS ▸ Relays**) is a 5 V SPDT relay — the blue cube of a Songle SRD — over five holes in a row: **COIL+**, **COIL–**, **COM**, **NO**, **NC** (`C+ C– COM NO NC` on its face). Its **Coil voltage** in **Properties...** is 5 V or 12 V. The coil has no polarity. On the standard engine COM is joined to **NO** while one coil leg is HIGH and the other LOW, and to **NC** otherwise; its lamp lights while it is pulled in. A real coil wants about 70 mA at 5 V — more than a logic output gives — so drive it from a transistor or a ULN2003A, with a diode across the coil. Press `R` while placing it, or with it selected, to turn it end-for-end: the cube swings to the other side of its row, `NC` now at the left.

The electronic load (**COMPONENTS ▸ Power**) is a bench brick like the power supply, with a red `+` and a black `–`: wire them across a supply, a rail or a regulator's output to draw a current from it on purpose. Its **Mode** is **CC** — constant current, its **Current** from 1 mA to 5 A — or **CR**, constant resistance, its **Resistance** from 0.1 Ω to 100 kΩ; its badge shows the setting. It draws nothing on the standard engine (a logic simulator has no currents); under Spice Lite it shows the volts, amps and watts it is taking. It is left out of a KiCad export, since it is an instrument rather than part of the circuit.

Character LCD modules

The **Displays** group holds two HD44780 character-LCD modules: a 16×2 (the standard 1602A) and a 20×4 (the 2004A). They're drawn to their real sizes — 80 × 36 mm and 98 × 60 mm of PCB — and their screens are **live**: run the circuit, drive the module, and the characters appear on the glass.

Both plug in through a **16-way header along one row**, and the pin assignment is identical between them, so what you learn wiring one applies to the other. The header runs along the module's **top** edge, which means the body hangs **below** the row it plugs into — so seat one on a **bottom row (`a`)** and it clears the board it's plugged into rather than covering it. The placement ghost shows you the whole module, so you can see this before you click.

Driving one is the ordinary HD44780 parallel bus: put a command or character code on `DB0` – `DB7`, set `RS` (0 = instruction, 1 = data) and `R/W` (0 = write), and pulse `E` — the byte latches on `E`'s falling edge. Wire `VDD` / `VSS` to a 5 V rail. `V0` (contrast) and `A` / `K` (backlight) are cosmetic in the standard engine; under Spice Lite the backlight is an LED and `V0` sets the contrast. During a *read* the module drives `DB0` – `DB7` itself, so tri-state anything else sharing that bus.

Both modules show the same controller datasheet in their pin-assignments window — it's one document, because `RS` / `R/W` / `E`, the bus and the address maps are the controller's and are identical across the two sizes.

Rotating & flipping

Rotation behavior is **not one rule for every part** — it depends on what kind of part it is. This is the part worth reading carefully.

Chips (`R` , mid-drag or while selected). A DIP chip's footprint maps onto itself when flipped — same two rows, same columns — so flipping only reverses which physical pin sits where; the chip never has to move. Select a placed chip and press `R` to flip it 180° in place, or press `R` while mid-drag to flip it before you drop it. Either way its pin-assignments window updates to show the new numbering.

bar8iso and DIP switch banks (`R` while selected). The isolated LED bar and every DIP switch bank are DIP-packaged, so they flip exactly like a chip: `R` turns the part 180° in place, the same holes, only the pin numbering per position reverses (a switch bank's own position states don't move — position 1 is still position 1, just wired to the opposite pins now). Neither has the turn-in-hand behavior of a plain LED — treat them as chips for rotation purposes.

Resistor, capacitor, inductor, diode and LED (`R` , both while placing and once placed). These two-lead parts start in a horizontal **footprint** form (pin 1 and pin 2 a fixed span apart along one row). Press `R` while the ghost is armed to turn it into a vertical **two-free-ends** form instead: pin 1 stays at the anchor hole, and pin 2 becomes a free lead that can land on any other free hole — including a hole on a different strip entirely, such as reaching up to a power rail. Each `R` press while placing steps the ghost a further quarter turn, cycling through all four compass directions before repeating. Once the part is placed, select it and press `R` to rotate it 90° at a time — pin 1 stays put and pin 2's lead swings around it, hunting for the next free hole to land in.

Transistors, regulators, potentiometers and relays (`R` while selected, or while placing). These turn end-for-end in place, as the bussed resistor array does: the same holes, with the pin order reversed (`E B C` becomes `C B E`). The body turns with them — a TO-92's flat face, a TO-220's tab, a trimmer's screw and a relay's cube all swing to the other side of the row — while the printing on it stays the right way up.

Oscillator cans (`R` , but the exact step differs by state). A can spins around its own centre, not around one pin, and the step size changes depending on whether you're still placing it:

- **While placing**, `R` steps the ghost a full 90° quarter-turn each press, so you can hunt through every orientation for one that fits.
- **Once seated and selected**, `R` behaves differently for the two sizes: the square **osc-half** can still steps 90° at a time, but the rectangular **osc-full** can jumps straight from 0° to 180° (and 90° to 270°) — because a non-square footprint only has two genuinely distinct orientations once it's down (rotating it a further 90° would just retrace the same two footprints it already swept through while placing).

LED polarity (`F` , while placing only). Independent of rotation, press `F` while an LED's placement ghost is armed to flip which lead is the anode and which is the cathode, before you click it down.

Moving a seated part — with or without its wiring

Dragging a placed part re-seats the part and **leaves every wire where it was**. That is often what you want while a circuit is still bare, but once it's wired it quietly changes the circuit: the pins land on different column-halves, the wires stay in the holes you laid them in, and what was pin 1's input is now pin 3's.

So, exactly as with pulling a mated strip out of a board group, hold a modifier while you start the drag:

- **Option-drag** — moves the part **and everything plugged into it**. Every wire end sitting in a column-half one of the part's pins occupies comes along, keeping its own row and its offset from the part, so the circuit after the move is the circuit before it. The far end of each wire stays put.

A resistor or LED plugged into the part comes too — by the leg that is connected to it. A leg in one of the part's column-halves is attached to it exactly as a jumper in the next hole along is, so it travels; the other leg stays exactly where it is and the part simply **bends** around it, the way a resistor's legs bend on a real bench. Plugged in at *both* legs, it travels whole instead. (A chip's pins can't bend, so a chip next door is never dragged along — it isn't a lead, it's a body.)

Only what is actually connected to the part comes — a wire or a leg in a column the part merely spans without having a pin there (a push button reaches two holes three columns apart, not the one between them) is left alone, because it was never connected to it. And nothing chases further than one step: what travels lands in the same column-half its pin lands in, so a rider never leaves anything of its own stranded behind it.

To see what would come, hold Option. With a part selected, holding Option rings every wire end and every leg that would travel with it, and releasing Option puts the rings away — so you can check before you commit to the drag rather than discover it during one. Something connected at *both* ends gets two rings, so a resistor that will travel whole reads differently from one that will bend; something connected at one end gets one ring, on the end that moves. A part with nothing attached simply shows nothing.

The drop is **all or nothing**. If any of those has nowhere to go — its hole is taken by something that isn't moving, it would run off the end of the strip, or the bend would squeeze a resistor's legs closer than its body is long — the part *and* everything it would have carried turn red, and releasing puts everything back. Half a move would silently cut the connections it left behind, which is the very thing the gesture exists to avoid.

A resistor or LED is dragged by its two ends rather than by a footprint, and that makes one distinction: dragging its **body** moves both legs together, so Option carries its wiring exactly as for any other part;

dragging **one leg** is a re-bend, and carries nothing — that leg can land in any hole, at any angle, on any strip, so there is nothing for a wire to follow. (An LED lying flat is only one hole wide, so there is no body to grab between its legs; stand it up with `R`, or move it as part of a selection.)

Two details worth knowing:

- Whether the wiring comes is decided when you **press**, not when you let go. Once the part is in hand, the set is fixed.
- Moving **across the trench** takes the wiring over with it, keeping the arrangement. Rows `a`–`e` and `f`–`j` are separate nodes, so a wire that stayed in its row would be left in the half its pin had just left — it travels the same number of holes the part did instead, so a wire two holes from the part is still two holes from it, on the same side. Drag far enough that the wiring would run off the edge of the board and the drop is refused; a row nearer the trench fits.
- Moving to **another board** works the same way — the wiring lands in the matching holes over there, and anything the wires reach back to stays where it is. While the part is over the gap between two boards there is nowhere for it to land, so the wiring sits where it really is and turns red until the part is over holes again.

`Shift` is not this modifier: Shift-drag always rubber-bands a selection, including when the press lands on top of a part.

Moving several parts at once

A selection drags as one unit. Marquee a group of parts (Shift-drag), or add them one at a time with `Cmd` / `Ctrl`-click, then drag any one of them: every selected part travels by the same amount, keeping the arrangement exactly as you built it. Power and clock bricks count as parts here, and come too.

Everything above applies unchanged, just to the whole group:

- **Option** takes the wiring with it — every wire, and every resistor or LED leg, riding *any* member. A wire between two selected parts travels at both ends; one that leaves the group keeps its far end where it is; a resistor plugged into a member bends after it exactly as above.
- Holding Option **rings** every end and leg that would travel, across all of them.
- The drop is **all or nothing**. If one part, or one wire end, or one leg has nowhere to land, every member and everything riding it turns red and releasing puts the lot back — a group that dropped some of its members would be a rearrangement you never asked for.
- The whole move is **one undo step**.

A **board** in the selection is the one case that declines: strips have their own drag, which carries everything seated on them under rules a part re-seat knows nothing about. Pressing a part then does nothing and leaves the selection alone — `Cmd` / `Ctrl`-click the board back out and drag the parts, or grab the board itself, which drags its own snapped group with everything seated on it and replaces the

selection the way any plain click does. And a plain click inside a selection narrows it to the part you clicked, exactly as clicking a part outside one does.

One thing a selection does **not** carry: a **wire** you selected that isn't plugged into any of the parts moving. Wires travel because they ride a pin, not because they are highlighted — so a wire selected on its own (to delete or recolor it) stays exactly where it is while the parts move around it.

Occupancy — one hole, one lead

Every hole on a breadboard — and every terminal on a power/clock brick — holds **at most one lead**, whether that's a chip pin or a wire end. Placing a part checks every one of its derived pin positions against every other part and every wire already on the desk; if any pin would land on an occupied hole, or off the edge of a board entirely, the ghost turns red and the drop is refused. This is the same rule a wire's endpoints follow (see Wiring, Nets & Buses) — chips, discretes, and wires all compete for the same holes, with no separate bookkeeping for any of them.

A rotated resistor or LED's free lead is the one case where a pin can legally resolve to **nothing** — if you later move or delete the strip under that lead, the part stays exactly where it is and that leg simply floats, unconnected, just as it would on a real bench.

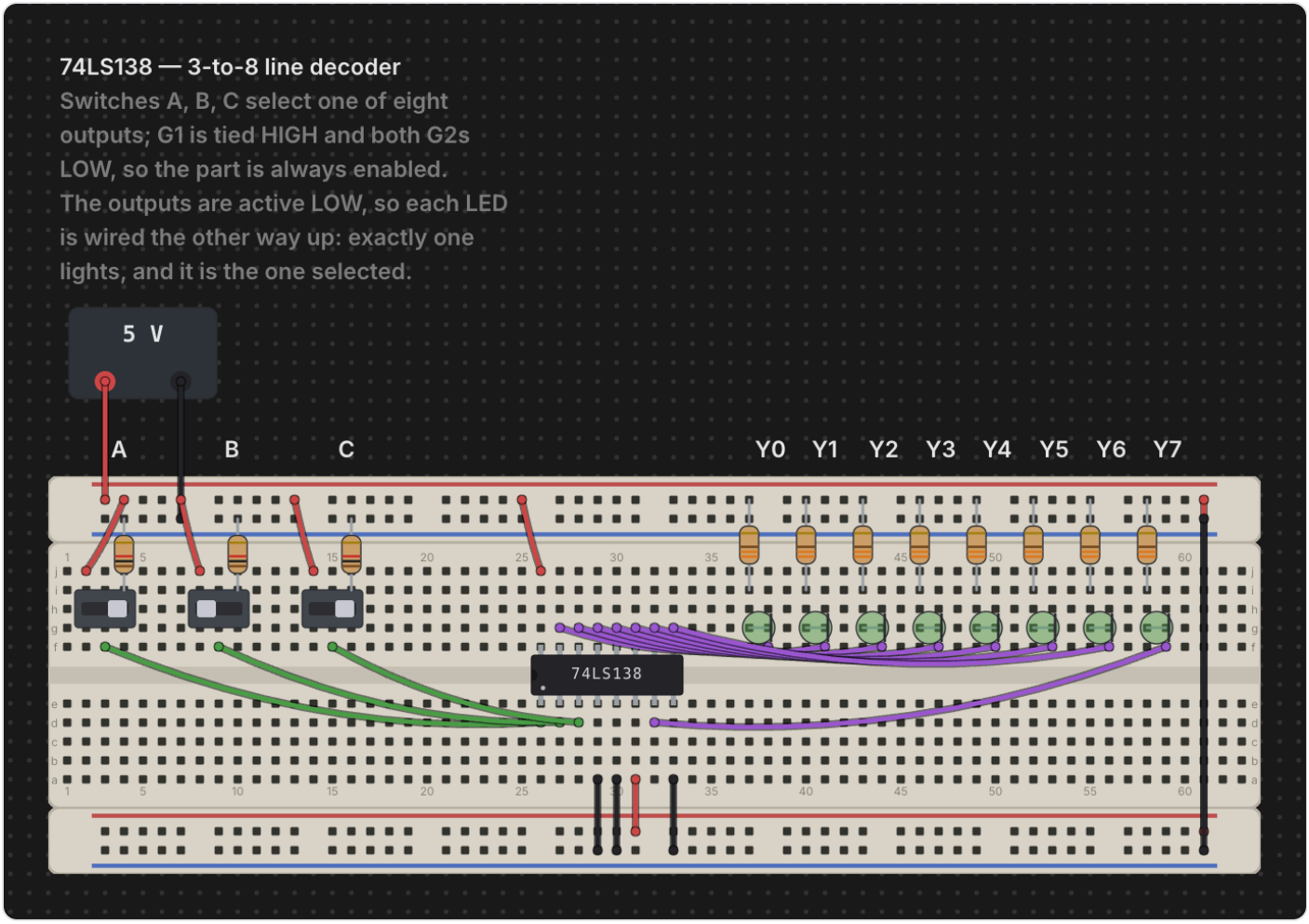
The pin-assignments window

Right-click any placed part — chip, discrete, or brick — and choose **Pin Assignment**, at the top of its context menu, to open its floating pin-assignments window: a diagram of every pin/terminal and, for most chips, a cropped datasheet excerpt below it. A real chip's diagram stays fixed at its canonical layout no matter how you've flipped it on the desk (it matches the physical part, not the placement); a DIP-packaged discrete — `bar8iso` or any DIP switch bank — is the exception: its diagram reflects its current `R` flip, since it has no real notch of its own. See The Chip Library for the full detail on what the window shows and how it sources its datasheet crops.

See also: The Desk & Breadboards for how boards and strips work, and Wiring, Nets & Buses for connecting components together once they're placed.

Wiring, Nets & Buses

Wires are how you connect the holes, rails, and terminals you've already populated with boards and parts into a working circuit. Chip Hippo's wire tool is click-click (no drag-to-draw), every wire remembers the two *addresses* it connects rather than raw screen positions, and a bus lets you lay a whole marching group of wires — a data or address bus — as one gesture. This page covers the wire tool, colors, cross-board wiring, re-routing and deleting a wire, direct and routed wires, auto-routing, addresses, and buses.



The wire tool

Press `W` (or click **Wire** in the toolbar) to arm the wire tool. With the tool armed:

1. Click a free hole or terminal to anchor the first end — a rubber-band preview then tracks your cursor.
2. Click a second free point to drop the wire between them.

The tool stays armed after a wire is committed, so you can lay a whole chain of wires without re-arming — each new wire starts fresh, so it does **not** automatically continue from the last endpoint; just click a new starting hole. A hover ring shows the point under your cursor and turns red the moment it's over an illegal target (already occupied, or the same point you started from) — the rubber band tints red too, so a bad landing is obvious before you click.

Press `Esc` to cancel a pending (anchored-but-not-yet-dropped) wire without leaving the tool, or `Esc`/click **Wire** again to disarm it entirely. `M` also disarms whichever of the wire, bus, or probe tools is currently armed — one key to "put the tools away." The wire tool (like the bus and placement tools) is unavailable while a simulation is running; editing is locked until you press **Stop**.

Wire colors

Chip Hippo ships eight wire colors: **red, black, blue, green, yellow, orange, white,** and **purple**. The active color is shown as a swatch dot on the toolbar's **Wire** button — click the dot itself to open the palette and pick a different one (picking a color sets it *without* arming the tool, so you can set up a color before you start wiring). Clicking the word **Wire** beside it still arms the tool as usual. While the tool is armed you can also press a digit `1`–`8` to jump straight to a color (1 = red, 2 = black, and so on through the list above). The color you pick **stays pinned** between wires — laying a chain of jumpers keeps whatever color you last chose, so pick red for a supply run and black for ground and they'll stay that way until you deliberately switch.

You can also recolor a wire after the fact: right-click it and choose **Properties...**, which offers the same eight swatches alongside the wire's name, description, layout method, and the drawing of the wire itself described below. (**Delete Component** on that menu is how a wire is removed.)

Wire length

The last thing in a wire's **Properties...** dialog isn't a field at all — it's a **picture of the wire**, drawn across the card in its own colour with the sleeve stripped back at both ends to bare tinned lead, and a dimensioned line underneath giving its length in centimetres.

It answers the one question the desk can't. On the desk a wire is a curve between two holes at whatever zoom you're at, so its length is unreadable — and its length is exactly what you need before cutting one off a spool.

Two things about that figure:

- It's the **cut** length, not the run. A real jumper doesn't stop at the surface of the board — it has to reach *into* both holes — so a wire crossing a single 0.1 in pitch is about 13 mm of wire, not 3. The

sleeve in the drawing spans the run; the bare tips are the two stripped ends; the dimension spans the lot.

- The length follows the **drawn shape**, not the straight-line distance: the sagging curve's own arc for a direct wire, the polyline through the waypoints for a routed one, and for a bus member its lead plus the whole ribbon plus its far lead — a conductor inside a ribbon cable is as long as the cable, however short the ends sticking out of it are.

It's the same measurement, and the same wording, as the **Wires** cutting list in the build guide — so a wire can't read one length here and another there. Recolor the wire in this same dialog and the drawing repaints to match.

Cross-board wires

A wire's endpoints don't care which board or brick they land on — you can run a wire from one breadboard to another, from a board to a PSU terminal, or between two separate strips, exactly as freely as within one board. When you drag a board that a wire is attached to, the wire's other end stays put and the whole wire stretches and re-sags live, riding the move; nothing needs to be re-laid. Deleting a board cascades: any wire with an end seated on that board is removed along with it.

Moving a **part** is different, because a part's pins can land somewhere its wires can't follow: an ordinary part drag leaves every wire in the hole you laid it in, which on a wired part quietly rewires the circuit. Hold **Option** as you press and the wire ends plugged into it come along instead — for one part, or for a whole selection of them. See Chips & Components for the rules, and for what the Option-held preview rings show you before you commit.

Re-routing & deleting a wire

Grab a wire two different ways:

- **Its end cap** — drag just one endpoint to a new hole or terminal, re-routing that end while the other stays put. The end follows the cursor and snaps onto a free hole once it's within about a hole's reach of one. A hover ring and red/legal tint on the dragged end work exactly like placing a fresh wire. Letting go drops the end on the hole the ring was showing; with no ring, or a red one, the end snaps back to where it started.
- **Its body** — drag anywhere along the wire itself to translate the whole wire rigidly, keeping its length and orientation and just sliding both ends together onto a new pair of holes. If either landing point isn't free, the wire snaps back on release.

To delete a wire, click it to select it, then press `Delete` or `Backspace` (the same shortcut removes whatever's currently selected — a part, a board, or a wire).

Wire layout: direct or routed

Every wire is drawn one of two ways, its **Layout Method** — right-click a wire and choose **Properties...** to switch it:

- **Direct** (the default) — the wire sags gently from one hole to the other, the way a real jumper lead does. There is nothing to position: the shape follows the two ends.
- **Routed** — the wire runs as straight lines, and *you* decide where it goes. This is the one to reach for when a run would otherwise cross the middle of a chip, or when a tidy right-angled layout reads better than a fan of curves.

A routed wire starts as a single straight line. To bend it:

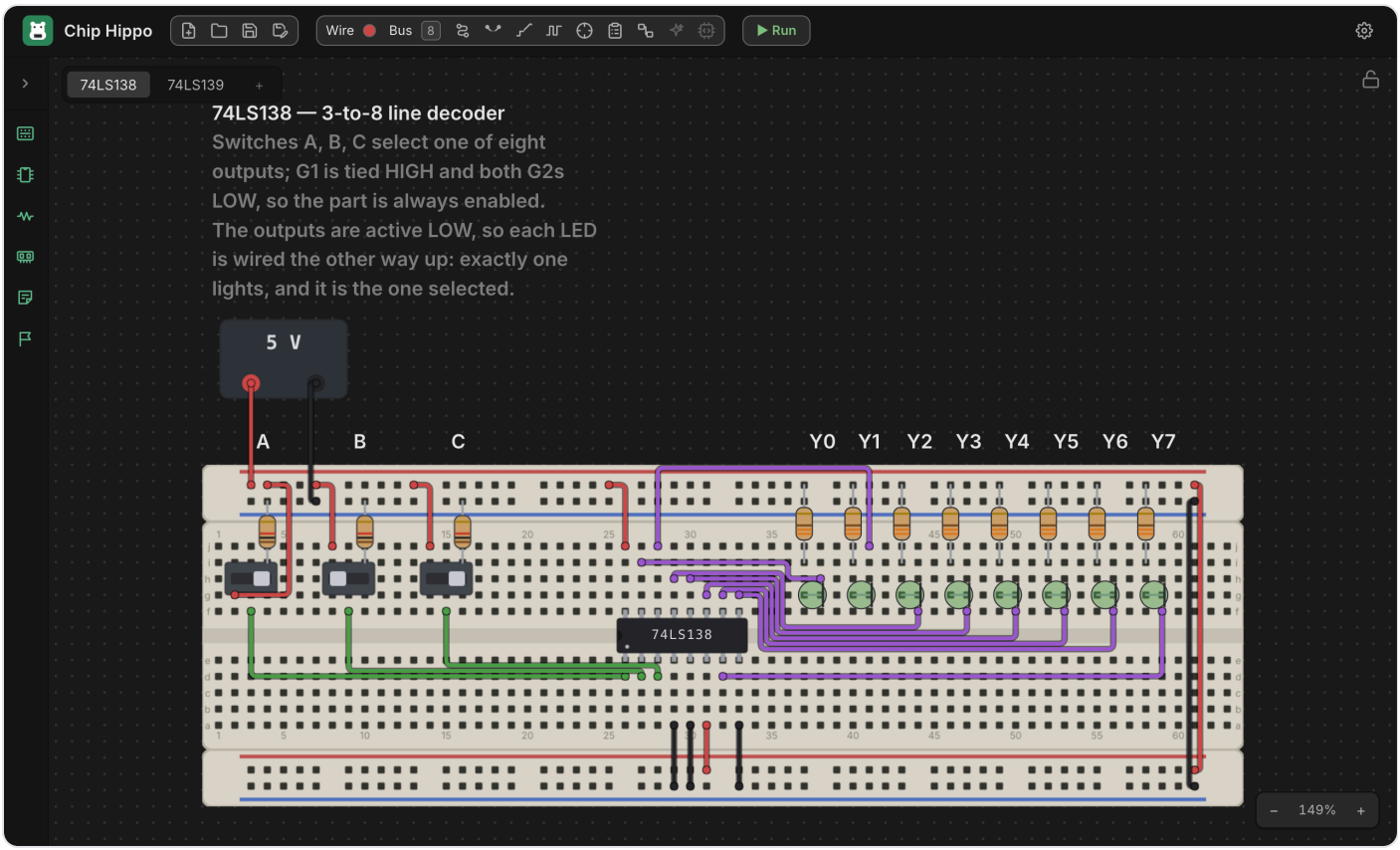
- **Press anywhere along the wire and drag.** A new point is added where you grabbed it, and the wire now runs start → point → end. Repeat to take it around whatever is in the way — up to twenty points per wire.
- **Hover an existing point** and its knob appears; drag it to move that bend somewhere else.
- **Drop a point onto a neighbouring point, or onto either end of the wire,** and it merges away — that's how you take a bend back out. The point you are dragging turns red as it lands, so you can see the merge before you release.
- **Drop an end cap onto one of the wire's own points** and that point is absorbed: the wire now reaches where the bend used to be, so the bend has nothing left to do.

Because a routed wire's body drag is spoken for, it has no rigid whole-wire translate — move it by its end caps instead. Switching a wire back to **Direct** deletes every point it carried and redraws it as the plain sagging curve; switching to **Routed** straightens it out, ready for points. Either way it's one undo step (`Cmd/Ctrl+Z`), as is every individual bend.

Settings ▸ Appearance ▸ **Wire layout** picks which of the two a *newly laid* wire gets. It's read when the wire is laid, so changing it never disturbs wiring already on the desk. Circuits built by the AI builder always arrive direct, whatever that setting says — the builder places holes, not hand-drawn routes.

Auto-routing every wire

Auto-route wires in the toolbar tidies a whole desk in one go: every wire becomes a **routed** wire running straight along the board, around the parts and clear of the other wires, instead of a fan of curves crossing the chips.



It asks first, saying how many wires it will redraw, because it replaces every bend placed by hand. To find a clear lane it may also move a lead to another hole on the **same connection** — along the same power rail, or within the same five-hole strip — so the circuit itself never changes. A wire it can't find a path for is left exactly as it was, and the message at the end says how many. Bus wires are left to their ribbon.

A large desk can take a few seconds; the message says how far it has got, and its **Cancel** button stops it with nothing changed. The whole result is one undo step (`Cmd/Ctrl+Z`). Like any other wiring change, it does nothing while the circuit is running.

Addresses

Wires never store pixel positions — they store **addresses**, the one cross-module currency for anything wireable. An address is `<ownerId>.<point>`:

- `bb1.a12` — a grid hole (board id `bb1`, hole `a12`).
- `bb2.+7` — hole 7 on a rail strip's `+` rail.
- `psu1.+` — the `+` terminal of a PSU brick.

A wire's `from` / `to` are just two of these addresses, plus a `color`. One hole or terminal holds at most one lead — one pin or one wire end — so the occupancy check that keeps the wire tool's hover ring legal/illegal is the same one that governs seating a chip or a discrete. Because everything resolves

through addresses rather than coordinates, a wire keeps working correctly no matter how the boards around it get rearranged.

Buses

For a wide, tidy bundle — a data or address bus — press `B` (or click **Bus** in the toolbar) to arm the bus tool instead of laying eight wires by hand. It's click-click like the wire tool: anchor a start hole, then click a second point to lay the whole run in one go, rendered as a single fat band rather than a fan of individual wires.

Before you click, set the **width**. The circled number on the toolbar's **Bus** button shows the active one, and clicking it opens the presets in a row: 2, 3, 4, 5, 6, 7, 8 and 16 bits, the current one ringed. Picking one sets the width without arming the tool, so you can set up before you start. From the keyboard — while the bus tool is armed — `2`–`8` lay a bus that many bits wide (`4` → `D[3:0]`) and `1` lays the 16-bit bus (`D[15:0]`); no single digit can spell 16, and the widest bus is worth the first key. The width names the bus too (`D[3:0]`); rename a placed one from its context menu.

Landing the second click on a **chip pin group** (a catalog-defined bus of pins, like a chip's data lines) fans the bus onto those pins in bit order, instead of marching it down a second run.

Both ends have to fit. A bus claims one hole per bit at each end, so as you move the cursor every hole it would take is ringed — in the hover color when they're all free and on the strip, in red when they're not, and a red set can't be clicked. Aim an 8-bit bus five columns from the end of a board and you'll see five red rings instead of eight, which is the whole explanation: the run doesn't fit there. That check runs on the **first** click as well, so a start that could never work is refused where you make it rather than looking like the far end is at fault.

A bus is metadata layered over the individual wires it lays — right-click a placed bus to **Rename**, **Recolor** (which recolors every member wire too), **Un-bundle** (keep the wires, drop the bus grouping), or **Delete bus

- wires**. Drag a bus by its band to move every member wire together.

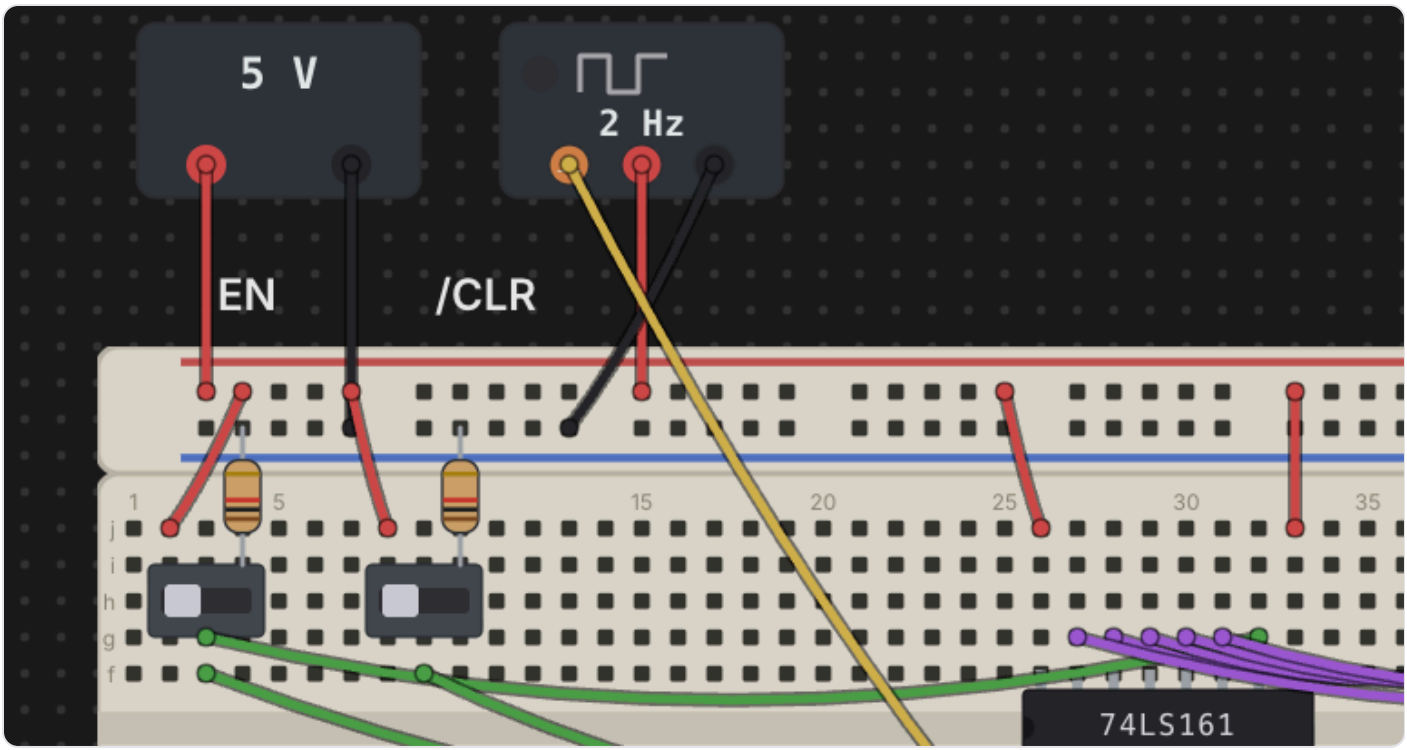
See also

- Chips & Components — placing the parts a wire connects.
- Power & Clock Sources — PSU and clock brick terminals, which wires reach exactly like board holes.
- Probing & Net Names — inspecting the nets your wires and buses form, and naming them.

Power & Clock Sources

Every circuit needs somewhere to draw power from and, for sequential parts, a signal to step them along. Chip Hippo gives you two kinds of desk-level **bricks** for this — a **power supply (PSU)** and a **clock source** — neither of which seats on a breadboard. They sit loose on the desk with their own addressable terminals, and you wire those terminals into a board's rails (or straight to a chip's pins) just like any other wire run.

There is a third kind of source, and it does not live on the desk at all: **external signals**, the bench stimulus buttons pinned down the right-hand edge of the viewport. They are covered at the end of this page.



PSU bricks

Add a **Power supply** from the parts palette (**Power** group) and drop it anywhere on the desk — it isn't tied to a board. It draws as a small body with a voltage badge and two terminal pads: a red `+` and a black `−`. Those pads are addressable wire endpoints, `psu1.+` and `psu1.−` for the first PSU you place, exactly like a breadboard hole — click-click a wire from each terminal into a power rail (or directly to a chip's VCC/GND pins) to energize a circuit.

A PSU has no on/off switch of its own — it's always "live" the moment the simulation is running; what matters is which voltage it's set to and what it's wired into.

Its Properties also set a **Current limit** (1 A unless you change it). The standard engine never reads it. Under Spice Lite the brick shows the current being drawn, and a supply asked for more than its limit droops. To load one on purpose, put an electronic load across it; to get a second voltage from it, a regulator.

Choosing a voltage — and the damage rule

Right-click a PSU brick and choose **Properties...** to pick its voltage: **3 V**, **5 V**, **9 V**, **12 V** or **15 V**. It's a live setting — the dropdown stays available and applies immediately even while the simulation is running, so you can change voltage on the fly and watch the effect.

What a voltage does to a chip depends on the chip's **logic family** — each chip is held to the supply its datasheet rates it for:

Supply	74LS (TTL) and the memory, interface and processor chips	CD4000 (CMOS)
3 V	**underpowered** — inert	runs
5 V	runs	runs
9 V, 12 V, 15 V	**damaged** — magic smoke	runs

- **Runs** — a chip whose VCC net carries a supply inside its range and whose GND net is properly grounded behaves exactly to its datasheet.
- **Underpowered** — the chip is inert (every output floats, as if disconnected) but nothing is harmed. A 74LS part needs 5 V; at 3 V it does nothing.
- **Damaged** — this is Chip Hippo's "magic smoke" rule: any chip (or oscillator can, which is powered the same way) whose VCC net sees more than its range allows is immediately marked **damaged** and goes inert for the rest of that run, independent of anything else on the net. A 74LS part is rated for 5 V, so 9 V, 12 V and 15 V all destroy it; a CD4000 part runs anywhere from 3 V to 18 V. It stays dead while the run lasts — a failed chip doesn't recover while the power is still wrong — and **Stop** makes it whole again.

So one board can run a CD4000 chip happily at 12 V while the 74LS chip beside it smokes — which is exactly why, on a real bench, people ran 4000-series chips at 5 V whenever the two families had to share a supply. The warning says the voltage the chip saw and the supply it is rated for.

A chip can also come up **reversed** — a PSU `−` on its VCC pin's net at the same time as a PSU `+` on its GND pin's net — which is reported separately from plain unpowered/underpowered, since it specifically means the supply leads are swapped.

Live chip health (powered / underpowered / reversed / damaged) shows as a badge on each chip while running — see Running a Simulation for how those badges and the rest of the settle model work. This

page only covers what puts a chip into each state.

Two supplies on one desk

You can place as many PSUs as you like. Give each one its own **+** rail — a separate rail strip, or one half of a Split power rail — and the chips on each rail run at that supply's voltage. The grounds can be wired together, and usually should be.

What you can't do is join two supplies set to **different** voltages on the same **+** net, for example by jumpering the two + halves of a split rail. On a real bench the higher supply pushes current back into the lower one. Chip Hippo reports it as **Supplies meet** — a warning when you run, and a fault in the desk review — naming the voltages involved, and runs the net at the higher one until you fix the wiring. Two supplies at the *same* voltage on one net aren't reported.

Recovering from an over-voltage mistake

Press **Stop**. Every damaged chip on the desk is restored, and you can rewire and run again. You don't lose the part, and you don't have to delete and re-place anything.

Over-voltage is meant to be *noticed*, not to cost you the chip: the red badge and the warning are the point, and they've done their job by the time you stop. Wiring a supply to the wrong rail is a mistake about the circuit — one you can see and fix — so it stays fixable. Find where the high supply is reaching a 74LS chip's VCC net (the probe tool is the quick way), correct it, and press **Run**.

Clock sources

Add a **Clock source** from the palette (**Power** group) for a free-running or manually stepped square wave to drive a sequential chip's clock pin. Like a PSU, a clock brick is desk-level — it doesn't seat on a board — and exposes three addressable terminals: `out`, `vcc` and `gnd` (`clk1.out`, `clk1.vcc`, `clk1.gnd`). Wire `out` to a chip's clock input.

A clock is an instrument, and like every instrument on a bench it needs **power**: wire `vcc` to the + rail and `gnd` to the – rail. Its HIGH is then that supply's voltage, so a clock on a 5 V rail clocks a 5 V circuit. A clock with no power **does not tick** — its output drives nothing and its lamp stays dark — and if its `out` is wired into the circuit, running warns **Clock not powered** until you wire it up.

Right-click a clock brick and choose **Properties...** to set its rate:

- **1 / 2 / 5 / 10 / 20 / 50 / 100 / 250 / 500 Hz, 1 kHz** — free-running. Once the simulation is running, the brick toggles its `out` level on its own at the chosen rate; a small lamp on the body lights while the output is HIGH. The slow end is for watching a single edge land; the fast end is for letting a counter

or a CPU actually get somewhere while you watch. Above about 20 Hz the lamp and any LEDs on the circuit blur together — that's the point at which the logic analyzer becomes the way to see what happened. An LED, or a display segment, that switches on and off faster than the desk is redrawn is drawn at its **average** over each frame, as your eye sees a real one: lit a fifth of the time, it glows a fifth as bright. Once anything on the desk toggles faster than **25 Hz** (the fastest clock or timer, times the speed), every lamp drops its glow halo and shows just its colour: a glow that fast is only a flicker, and drawing it costs more than anything else on the desk.

While the simulation runs, a free-running clock also shows a small ⏸ button in its top-right corner. Click it to hold **that one clock**: it keeps whatever level it was at — its lamp and everything it drives stay as they were — while the rest of the circuit, and every other clock, carries on. The button turns into a ▶; click it again to set the clock going. A clock's own pause belongs to the run, like its phase: **Run** always starts every clock going, **Step** passes a paused clock by, and resuming the transport after **Pause** leaves it held.

- **Manual** — click-to-toggle. No timer runs it; instead, while the simulation is running, clicking the brick's body flips `out` from LOW to HIGH (or back) once per click — handy for single-stepping a counter or flip-flop by hand and watching each edge land.

The rate dropdown, like the PSU's voltage one, is a live setting — it applies immediately and stays available while running, so you can retune a clock's speed mid-simulation.

The card's **Wave type** is **Square** (the default) or **PWM**. A PWM clock is a square whose HIGH lasts its **Pulse width** — a slider from 1 % to 99 % of each period, 50 % (the square) unless you move it — and LOW for the rest: a 10 Hz clock at 25 % comes up 75 ms into each period and goes down 25 ms later. The pulse width, like the rate, applies while running; moving it keeps the clock's place in the half it is in, so dragging the slider never holds an edge off. **Step** moves a PWM clock by the half it is in, and the glyph on the brick is drawn at its pulse width. An LED driven by a fast PWM clock looks as bright as its pulse width: at 250 Hz and 20 %, a fifth as bright as one lit steadily — dimming an LED, as on a bench. The slider is greyed for any other wave, and for a manual clock.

With Spice Lite on, the Wave type also offers **Triangle**, **Trapezoid** — the square with sloped edges — **Sawtooth (ramp up)** — it climbs and drops straight back — **Sawtooth (ramp down)** — it drops and climbs straight back — or **Sine**. Each swings from 0 V to the clock's supply once per period of its rate, starting at its low point (a ramp down starts at the top), and the glyph on the brick shows which. Set its **Amplitude** (peak to peak) and **Offset** (the voltage it is centred on) to swing it anywhere else — 1 V about 2.5 V, say; left blank they are the whole supply about half of it. A swing that would pass 0 V or the supply is clipped flat there, as a generator powered from those rails would be. The standard engine has no voltages, so with Spice Lite off such a clock is the square its levels are, and the brick draws one, whatever the card is set to. A manual clock is always square.

An **oscillator can** (a discrete part that seats directly on a board rather than as a desk brick) behaves the same electrically — it's a free-running square-wave source powered like a chip, with its own rate field in its Properties dialog — but it only ever free-runs; a real crystal has no click-to-toggle pin, so it has no manual mode.

The transport drives the edges

Free-running clocks (and oscillator cans) don't tick on their own outside a simulation — their edges are driven by the **Run/Pause/Step/speed transport** in the header, described fully in Running a Simulation. Briefly: **Run** (`Space`) starts every free-running clock at its configured rate; **Pause** freezes them in place without stopping the simulation; **Step** advances every free-running clock by exactly one half-period and re-settles the circuit, useful for watching a sequential chain edge by edge; and the **speed** control scales every free-running clock's rate together (it has no effect on a manual clock, which only ever moves on a click). A manual clock only responds to clicks while the simulation is actually running — stopped, its body is inert like everything else on the desk.

External signals

A **signal** is bench stimulus: a labelled button pinned to the right-hand edge of the desk, and a pointed **flag** whose tip plugs into any hole on any breadboard. Pressing the button injects a level at that hole. Nothing about a signal takes up board space, and — unlike a push button, which you can only hold one of at a time — **you can assert several signals at once**.

Add one from the parts palette's **SIGNALS** section: pick *Signal*, then click anywhere on the desk. The click position does not matter; the button always goes to the next free slot on the rail, with its flag waiting beside it.

Each button's coloured dot shows the **key** that presses it — `1` for the top button, down to `9`, then `0` for the tenth — and the flag prints the same key inside its body. That key is how you tell a flag's button at a glance. Selecting a flag on the desk also outlines its button on the rail.

Drag the flag onto a board and its point snaps into the hole under the cursor. Press **R** while it is selected to turn it a quarter-turn about that point, so it can lie in whichever direction has room. A flag claims its hole exactly like a wire end or a chip pin does, so nothing else can use it while the flag is there.

To take a flag off the board, drag it onto bare desk, or right-click it and choose **Remove Signal**. Either way the flag goes back beside its button on the rail, and its position and rotation are cleared, so the next drag starts fresh. The signal itself stays. **Delete Signal** on the same menu (or the **Delete** key while the flag is selected) removes the signal altogether, button included.

Momentary, toggle, and the resting level

Right-click a flag (or a button) for **Properties...**:

Field	What it does
Name	The label shown on the button
Description	The button's tooltip
Color	The colour shared by the button's dot and its flag — any of the seven signal colours, even one another signal already uses
Type	**Momentary** asserts while held; **Toggle** flips on each press and stays
Default value	The level the signal rests at — **Low** or **High**

A placed signal is **always** driving its net. A momentary signal rests at its default and asserts the *opposite* while you hold it, which is what makes "Default: High" the natural setting for an active-low `/RESET` line: it sits high, and pressing the button pulses it low. A toggle's current level is reset to the default every time you Run, so a run always starts from a known state.

A signal drives at the same strength as a chip output, so two signals arguing over one net — or a signal fighting a chip's output pin — is reported as a conflict, exactly as two chip outputs on one net would be.

Pressing them

While the circuit is running, **1–9 and then 0 press the signal buttons**, top to bottom — the key each button (and its flag) shows. No modifier, so you can hold several down together — `1` and `3` at once is two signals asserted at once. (Those digits pick the wire colour and the bus width while the Wire or Bus tool is armed, but Run puts both tools away, so the two meanings never collide.) Clicking a button with the mouse does the same thing for one signal at a time.

There are **ten signals** per desktop — one per digit key. The palette's *Signal* row goes grey once all ten are in use.

A new signal takes the next colour in the sequence red, blue, green, yellow, orange, white, purple, starting with the first one no signal is using. Once all seven are in use the sequence starts again from red, so the eighth, ninth and tenth signals share a colour with the first three — their keys tell them apart. Deleting a signal frees its colour for the next one you add. The signal colours are the eight jumper colours **minus black**, which is the bench's ground colour: a black flag would read as a ground tie, and its button's dot would all but vanish against the dark rail.

A project saved before black was withdrawn still opens; any signal that was black is given a colour on load, and keeps its name, type, resting level and wherever its flag was planted. Nothing else on the rail is

recoloured.

See Wiring, Nets & Buses for how to route power-rail and clock wiring generally, and Running a Simulation for how power state and clock edges feed into the settle model and live views.

The Chip Library

The parts palette's **CHIPS** folder holds a broad shelf of DIP logic from two families — **74LS TTL**, everything from a single quad NAND gate up to octal shift registers and 4-bit counters, and **CD4000 CMOS** — every one with a datasheet-accurate pinout and real behavior you can wire up and run. Past them, an **Interface** group carries the 65xx peripherals (a PIA and a VIA) and a **PROCESSOR** group carries the CPUs, while a separate **Memory** group sits below for the address-indexed ROM/RAM parts, which get their own dedicated page. This page is a tour of what's on the shelf and how to read a chip's pin-assignments window once you've placed one.

Choosing a logic family

Settings → Data Sheets → Chip family picks which family the parts palette shows: **74LS** (the default), **CD4000**, or **Both**. With one family, the CHIPS folder is the familiar list of function groups — NAND, NOR, Flip-flop… — holding that family's chips. With **Both**, CHIPS gains a `74LS` folder and a `CD4000` folder, each holding its own function groups. The memory, interface and processor chips belong to no family and are there in every mode.

It only filters the palette. A project's chips always load and run, whatever is chosen, and a family the open project uses is shown anyway — so you can add more of the parts already on its desk. Typing a CD4000 part number into the filter while CD4000 is hidden says where to switch it on.

The families differ in more than part numbers — see The CD4000 CMOS family below.

Combinational gates

The basic gate families — the classic 7400-series building blocks:

Part	Description
`74LS00`	Quad 2-input NAND
`74LS01`	Quad 2-input NAND, open-collector — outputs on 1/4/10/13, *not* the classic quad-NAND layout
`74LS02`	Quad 2-input NOR
`74LS03`	Quad 2-input NAND, open-collector — the variant that *does* keep the classic layout (outputs on 3/6/8/11)
`74LS08`	Quad 2-input AND
`74LS10`	Triple 3-input NAND
`74LS11`	Triple 3-input AND
`74LS20`	Dual 4-input NAND
`74LS27`	Triple 3-input NOR
`74LS30`	8-input NAND
`74LS32`	Quad 2-input OR
`74LS86`	Quad 2-input XOR

Open-collector parts (`74LS01` , `74LS03` , `74LS05` , the `74LS47` 's segment outputs and the `74LS181` 's A=B) pull their outputs low only, as on a real bench: a HIGH output lets go of its net and drives nothing. Give each output a pull-up resistor to +5 V, or it floats when it should be HIGH. Because a released output drives nothing, several can share one pulled-up net — a wired-AND, LOW whenever any of them is.

Alongside them, the inverter and buffer/bus-driver parts:

Part	Description
`74LS04`	Hex inverter
`74LS05`	Hex inverter, open-collector
`74LS14`	Hex Schmitt-trigger inverter
`74LS125`	Quad tri-state buffer, active-low enable per gate
`74LS240`	Octal inverting tri-state buffer/line driver
`74LS244`	Octal (non-inverting) tri-state buffer/line driver
`74LS245`	Octal bidirectional bus transceiver — the one part in the catalog with true bidirectional pins

Sequential & MSI parts

Everything with internal state, plus the mid-scale-integration decoders and multiplexers that build address/data logic around them.

Flip-flops & latches

Part	Description
`74LS73`	Dual JK flip-flop, clear
`74LS74`	Dual D flip-flop, preset & clear
`74LS76`	Dual JK flip-flop, preset & clear
`74LS107`	Dual JK flip-flop, clear
`74LS112`	Dual JK flip-flop, preset & clear
`74LS174`	Hex D flip-flop
`74LS175`	Quad D flip-flop
`74LS173`	4-bit D register, tri-state
`74LS273`	Octal D flip-flop, clear
`74LS75`	4-bit bistable (transparent) latch
`74LS279`	Quad $\overline{SR}$ latch
`74LS259`	8-bit addressable latch
`74LS533` / `74LS573`	Octal transparent latch, tri-state (inverting / non-inverting)

The `74LS73`, `74LS75`, and `74LS76` reproduce their datasheet's **non-standard power-pin placement** — real parts don't always put VCC and GND on the package corners, and neither do these.

Counters & shift registers

Part	Description
`74LS90`	Decade (÷10) ripple counter
`74LS161`	Synchronous 4-bit binary counter
`74LS169`	Synchronous 4-bit up/down counter
`74LS193`	Synchronous up/down 4-bit counter
`74LS164`	8-bit serial-in, parallel-out shift register
`74LS165`	8-bit parallel-in, serial-out shift register
`74LS595`	8-bit shift register with output storage latch

Decoders & multiplexers

Part	Description
`74LS138`	3-to-8 line decoder
`74LS139`	Dual 2-to-4 line decoder
`74LS151`	8-to-1 line multiplexer
`74LS153`	Dual 4-to-1 multiplexer
`74LS157`	Quad 2-to-1 selector
`74LS257`	Quad 2-to-1 selector, tri-state

Arithmetic, comparison & encoding

Part	Description
`74LS47`	BCD-to-7-segment decoder/driver
`74LS85`	4-bit magnitude comparator
`74LS148`	8-to-3 priority encoder
`74LS283`	4-bit binary full adder
`74LS83`	The same adder on its **original** pinout — VCC on pin 5, GND on pin 12, not the later JEDEC corners
`74LS181`	4-bit arithmetic logic unit (DIP-24) — 16 logic or 16 arithmetic operations selected by `S0` – `S3` and `M`, with carry generate/propagate outputs for cascading

The CD4000 CMOS family

The 4000-series CMOS parts, each pinned out from its Texas Instruments datasheet. They are **not** pin-compatible with the 74LS parts of the same function — the CD4001B's NOR outputs are on pins 3, 4, 10 and 11, the 74LS02's on 1, 4, 10 and 13 — so read the pin-assignments window before wiring one.

Part	Description
`CD4001B`	Quad 2-input NOR
`CD4002B`	Dual 4-input NOR
`CD4025B`	Triple 3-input NOR
`CD4078B`	8-input NOR/OR gate — both outputs
`CD4011B`	Quad 2-input NAND
`CD4012B`	Dual 4-input NAND
`CD4023B`	Triple 3-input NAND
`CD4068B`	8-input NAND/AND gate — both outputs
`CD4093B`	Quad 2-input NAND Schmitt trigger
`CD4081B`	Quad 2-input AND
`CD4082B`	Dual 4-input AND
`CD4073B`	Triple 3-input AND
`CD4071B`	Quad 2-input OR
`CD4072B`	Dual 4-input OR
`CD4075B`	Triple 3-input OR
`CD4030B`	Quad exclusive-OR — the original; the CD4070B is its successor
`CD4070B`	Quad exclusive-OR
`CD4077B`	Quad exclusive-NOR
`CD4069UB`	Hex inverter (unbuffered)
`CD40106B`	Hex Schmitt-trigger inverter
`CD4049UB`	Hex inverting buffer/converter — **VCC on pin 1, VSS on pin 8**, not the corners
`CD4007UB`	Dual complementary pair plus inverter — six bare **MOSFETs**, see below
`CD4050B`	Hex non-inverting buffer/converter — same odd power pins
`CD4013B`	Dual D flip-flop with **active-HIGH** set and reset (the 74LS74's are active-LOW)
`CD4027B`	Dual JK flip-flop with active-HIGH set and reset — raising **both** drives Q and $\overline{Q}$ HIGH together
`CD4017B`	Decade counter with ten decoded outputs and a carry-out

Part	Description
`CD4022B`	Octal counter with eight decoded outputs — the 4017's divide-by-8 sibling
`CD4020B`	14-stage binary ripple counter — Q2 and Q3 have no pin
`CD4024B`	7-stage binary ripple counter, in a **14-pin** package
`CD4040B`	12-stage binary ripple counter — counts on the **falling** edge
`CD4060B`	14-stage ripple counter with its own **RC oscillator** — or count an external clock on ϕI (see Timers)
`CD4029B`	Presettable up/down counter, binary or decade (a pin picks which)
`CD4510B`	Presettable BCD up/down counter with reset
`CD4516B`	Presettable binary up/down counter with reset
`CD4094B`	8-stage shift-and-store register — its 3-state **OUTPUT ENABLE is active HIGH**
`CD4028B`	BCD-to-decimal decoder, ten active-HIGH outputs
`CD4511B`	BCD-to-7-segment latch/decoder/driver for a **common-cathode** display
`CD4066B`	Quad bilateral switch — an **analog switch**, see below
`CD4051B`	8-channel analog multiplexer/demultiplexer — **VEE on pin 7**
`CD4052B`	Dual 4-channel analog multiplexer/demultiplexer — VEE on pin 7
`CD4053B`	Triple 2-channel analog multiplexer/demultiplexer — VEE on pin 7
`CD4047B`	Monostable/astable multivibrator — timed by one resistor and one capacitor
`CD4098B`	Dual retriggerable monostable
`CD4528B`	Dual retriggerable monostable — its pulse **lengthens with the supply**
`CD4538B`	Dual retriggerable precision monostable
`CD4541B`	Programmable timer: an RC oscillator and a 16-stage counter

What makes the family worth learning on is how it differs from TTL:

- **A floating input is unknown, not HIGH.** A 74LS input pulls itself up; a CMOS input is a gate with nothing on it. Chip Hippo reads an unconnected CD4000 input as `X`, warns about it, and lets the unknown spread to whatever it feeds — a CD4017 with its RESET left floating loses its count. Tie every CMOS input, spare gates included, as the datasheets ask.
- **A much wider supply range.** A CD4000 part runs from 3 V to 18 V, where a 74LS part needs 5 V — see Power & Clock Sources.

- **Weaker outputs.** One standard CD4000 output can hold only one 74LS input LOW; the CD4049UB/CD4050B buffers drive eight. Driving the other way, a 74LS HIGH is marginal for a CMOS input. Both are reported — see Mixing logic families. The weakness has an upside: at 5 V and below a standard output cannot burn an LED wired straight onto it, so one with no resistor lights.

The Schmitt-trigger parts (CD4093B, CD40106B) behave as plain gates in the standard engine: their hysteresis is an analog property, and their classic RC oscillator will not run there — a capacitor on the desk carries a value for the timing chips, but no charge. Under Spice Lite the capacitor charges and each Schmitt input has its two thresholds, so the one-gate RC oscillator runs. In the standard engine, use one of the timers for an RC oscillator.

A few of the larger parts have habits of their own:

- **The up/down counters' carry is combinational.** CARRY OUT on the CD4029B, CD4510B and CD4516B goes LOW at the end of the count — 15 (or 9) going up, 0 going down — while CARRY IN is LOW, and it follows UP/DOWN at once, without waiting for a clock. Wire one counter's CARRY OUT to the next one's CARRY IN and give them the same clock to chain them. Tie CARRY IN LOW on a counter on its own.
- **A decade counter can be preset to a code that is not a digit.** The jam inputs load any binary number, 10–15 included. The CD4510B's datasheet promises it counts back into 0–9 within two clocks going up and four going down, and Chip Hippo follows that part's logic gates to get there; the CD4029B's decade mode uses the same logic.
- **The CD4511B blanks a code above 9**, where the 74LS47 shows odd symbols. Its latch sits before the decoder, so LE/STROBE HIGH freezes the digit while lamp test (LT) and blanking (BL) still work on it.
- **The analog switches drive nothing — they connect.** A CD4066B switch whose CONTROL is HIGH, or the CD4051B/52B/53B channel that INH and the select pins pick, joins its two pins, so whatever drives one side drives the other, in either direction: the same 4051 multiplexes eight signals onto COM or demultiplexes COM out to eight. An open channel leaves each side to whatever else is on it, so a pin on an open channel with nothing else driving it floats. Chip Hippo passes logic levels only — the switch's on-resistance and analog voltages are out of scope — and a supply that passes through a switch arrives at the strength of an ordinary output: the + rail switched onto an output driving LOW is a fight, and a channel joining + to – is a short.
- **The CD4007UB is six MOSFETs, not gates.** Three N-channel and three P-channel transistors with their terminals brought out, each a switch exactly as a discrete MOSFET is: an N-channel joins its two terminals while its gate is HIGH, a P-channel while it is LOW, and a floating gate is unknown. Pair 1's P already sits on VDD and its N on VSS, so joining 13 to 8 makes an inverter from Q1 GATES (6); pair 3 is an inverter from Q3 GATES (10) to 12 once 11 is on VDD and 9 on VSS; pair 2's four terminals are all free. Thresholds and the part's analog uses — amplifiers, oscillators — are out of scope.

- **VEE is a supply pin.** The 405x parts have a negative supply, VEE (pin 7), for analog signals below ground. Chip Hippo has no negative supply, so tie VEE to VSS, as single-supply digital circuits do; until it is, the chip is unpowered.
- **The CD4094B's outputs follow its shift register only while STROBE is HIGH**, and its OUTPUT ENABLE floats them when it is LOW — the opposite sense of every 74LS output enable. The serial outputs QS and Q'S are never floated; Q'S gives the same bit half a clock later, for chaining a second 4094 on a slow clock.

Timers

Seven parts take their timing from a **resistor and a capacitor** you wire to them, as on a bench: the **555** (CHIPS ▸ Timer), and the CD4000 **CD4047B**, **CD4098B**, **CD4528B**, **CD4538B** and **CD4541B** (CD4000 ▸ Timer) and **CD4060B** (CD4000 ▸ Counter). Give the resistors and the capacitor their values in **Properties...** (see Component values) and the part runs at the rate its datasheet gives for them. There is no analog simulation behind it: each part finds its own resistor and capacitor in the wiring and works the time out from the datasheet's formula.

While the circuit runs, each timer prints its rate or pulse length under its part number, and its **Properties...** card shows the same under **Timing**:

Part	Timing pins	Formula (datasheet)
555 astable	RA from VCC to DISCH, RB from DISCH to TRIG + THRES, C from TRIG + THRES to GND	high 0.693·(RA+RB)·C, low 0.693·RB·C
555 monostable	RA from VCC to THRES + DISCH, C from THRES + DISCH to GND	pulse 1.1·RA·C
555 bistable	THRES to GND — no resistor or capacitor	none: TRIG LOW sets OUT, RESET LOW clears it
CD4047B	R between R (2) and RC COMMON (3), C between C (1) and RC COMMON (3)	Q/Q̄ period 4.40·RC (OSC OUT twice as fast), one-shot pulse 2.48·RC
CD4098B	per section: C from RX CX to CX (or GND), R from RX CX to VDD	pulse ½·R·C
CD4528B	per section: C from T2 to T1 (or GND), R from T2 to VDD — and T1 wired to GND	pulse 0.2·R·C·ln(VDD)
CD4538B	per section: C from RX CX to CX (or GND), R from RX CX to VDD	pulse R·C
CD4060B	Cx from ϕO (9), Rx from ϕ̄O (10), Rs from ϕI (11), all to one junction	oscillator period 2.2·Rx·Cx
CD4541B	Ctc from CTC (2), Rtc from RTC (1), Rs from RS (3), all to one junction	oscillator period 2.3·Rtc·Ctc

Two resistors (or two capacitors) side by side between the same two points count as one, combined the way they would be on a bench. Two in **series** are not followed: put the single value you mean.

The 555 works out what it is

The 555 has no mode setting, because the chip has none. It reads its own wiring the way the silicon does:

- **Astable** — TRIG (2) and THRES (6) on one net with the capacitor from it to GND, and DISCH (7) between the two resistors: RA up to VCC, RB down to the TRIG/THRES net. It free-runs from the moment you press Run, starting HIGH.
- **Monostable** — THRES (6) and DISCH (7) on one net with the capacitor to GND and RA up to VCC, and TRIG (2) connected to something outside. A falling edge on TRIG starts a pulse; a trigger during the pulse changes nothing; a TRIG still held LOW when the time is up keeps the output HIGH until it is released.
- **Bistable** — THRES (6) tied to GND, and TRIG (2) connected to something outside: a set/reset latch with nothing to time. A LOW on TRIG sets the output HIGH and it stays HIGH; a LOW on RESET (4)

clears it and it stays LOW. With both held LOW, RESET wins. The usual build is a pull-up resistor and a push button to GND on each of TRIG and RESET; DISCH (7) plays no part, so grounding it beside THRES, or leaving it open, is fine. It starts LOW when you press Run, and prints no rate on the chip.

Anything else is **not guessed at**. The part draws a warning triangle, a notification says exactly what it is missing ("no timing capacitor from TRIG/THRES (2, 6) to GND", or that the wiring is none of the three), and its output stays LOW. RESET (4) LOW forces the output LOW whatever the configuration and restarts the timing when it is released. CONT (5) is not modelled: a bypass capacitor to GND on it is the usual thing and harmless. The 555 runs from 4.5 V to 16 V.

The CD4000 timers

- **CD4047B** — ASTABLE (5) HIGH or ASTABLE̅ (4) LOW makes it free-run: Q and Q̅ square-wave, with OSC OUT (13) at twice the rate. Otherwise it is a one-shot: +TRIGGER (8) rising while –TRIGGER (6) is LOW, or –TRIGGER falling while +TRIGGER is HIGH, fires one 2.48·RC pulse, which another trigger does not restart. The pulse is the internal oscillator running for whole periods, so RETRIGGER (12) rising during it, or held HIGH, runs it on by one more 2.2·RC period at a time — the datasheet's "will retrigger as long as the RETRIGGER input is high". The datasheet's retriggerable hook-up ties RETRIGGER to +TRIGGER, so every further trigger during the pulse runs it on. EXT RESET (9) HIGH ends it and holds Q LOW.
- **CD4098B**, **CD4528B** and **CD4538B** — two independent one-shots each, on the same pins. A section fires on +TR rising (with –TR held HIGH) or on –TR falling (with +TR held LOW) — the 4528 calls them A and B — is **retriggerable** (a new trigger during the pulse starts the time again), and RESET LOW (the 4528's CD) ends the pulse at once. A section with nothing on its timing pins and nothing on its outputs is unused and says nothing. The 4528 differs in two ways: its T1 pins (1 and 15) are not grounded inside the part, so wire them to GND or the section reports it and times nothing; and its pulse depends on the supply — 0.2·R·C·ln(VDD) is 0.32·R·C at 5 V and 0.54·R·C at 15 V. (Its datasheet is a current second source's, HGSEMI's; TI never documented this part. Makers disagree: another maker's CD4528 sheet gives 0.42·R·C at 5 V, falling to 0.30·R·C at 15 V.)
- **CD4060B** — with the RC network on ϕO, ϕ̅O and ϕI it runs its own oscillator and counts it; RESET HIGH clears the count and stops it. With nothing on ϕO and ϕ̅O, ϕI is an ordinary clock input, counted on its falling edges — the way its example circuit uses it. A network with a piece missing is reported, not guessed at. A crystal (the datasheet's other oscillator) is not offered: the desk has no crystal to put there.
- **CD4541B** — the same idea with a programmable output. With the RC network on CTC, RTC and RS it runs its own oscillator (CTC drives it out, RTC its complement); with nothing on CTC and RTC, RS is a clock input counted on its **falling** edges. A and B pick which counter stage is the OUTPUT: 2^13 with both LOW, 2^10 with B HIGH, 2^8 with A HIGH, 2^16 with both HIGH. MODE HIGH recycles — a square wave at the oscillator ÷ 2^N; MODE LOW makes a **single transition** 2^(N–1) counts after a reset and holds it. Q/Q̅ SELECT HIGH inverts the output. MASTER RESET HIGH clears the count and

stops the oscillator; with AUTO RESET LOW it starts by itself when you press Run, and with AUTO RESET HIGH it waits for a MASTER RESET pulse first.

Faster than the desk can show

A timer is drawn no faster than **1 kHz** at `×1`; the speed control scales it with everything else. A timer set faster than that **still reports its true rate** — the readout says `48.1 kHz`, in amber, and the Timing row adds *Faster than the desk can show: drawn at 1 kHz* — but its output is drawn oscillating at 1 kHz with its duty cycle kept. A CD4060B counting a fast oscillator still divides it down correctly: every stage slow enough to show keeps the true count, and only the stages faster than 1 kHz are drawn at the cap. Likewise a pulse shorter than 0.5 ms is stretched to 0.5 ms, so it can be seen at all; the readout still gives the real length.

Time runs at the transport's speed, so **Pause** freezes it and **Step** moves it to the next edge (see The transport).

Op-amps, drivers and optocouplers

Four parts sit between the logic and the rest of a bench. None belongs to a logic family, so they show whichever family the tray is set to.

Part	Group	What it is
`LM358`	Op-amps	Two op-amps in a DIP-8 from one supply (3 V to 32 V): `1OUT · 1IN− · 1IN+ · GND · 2IN+ · 2IN− · 2OUT · VCC`
`ULN2003A`	Interface	Seven Darlington drivers with clamp diodes, DIP-16: inputs 1–7, outputs 16–10 opposite them, `E` (8) to ground, `COM` (9) to the load's supply
`4N35`	Interface	An optocoupler with its base brought out, DIP-6: LED `A` 1 · `K` 2, phototransistor `E` 4 · `C` 5 · `B` 6
`PC817`	Interface	The everyday 4-pin optocoupler: LED `A` 1 · `K` 2, phototransistor `E` 3 · `C` 4

- **The LM358** compares on the standard engine: each unit's output is HIGH while IN+ reads HIGH and IN− LOW, LOW the other way round, undefined when they agree — which is what an op-amp with no feedback does. Wired as an amplifier its output is a voltage, and only Spice Lite has those.
- **The ULN2003A** needs no supply of its own. Each input HIGH sinks the output opposite it to `E`; LOW, or nothing, leaves it open, so a load runs from its supply into the output. Tie `COM` to the load's supply and each output's diode catches a coil's kick.
- **An optocoupler** shares no connection between its sides, so each may run from its own supply. On the standard engine `C` – `E` is a closed switch while the LED's anode is HIGH and its cathode LOW.

Give its LED a series resistor, like any LED. Its **Current transfer ratio** in **Properties...** (50 % to 600 %; 100 % by default) is what Spice Lite carries across.

Interface chips (65xx)

Past the 74xx groups, the **Interface** group carries two Western Design Center 65xx peripherals — both DIP-40, both clocked off `PHI2`, both wired the same way you'd wire them on a real single-board computer:

Part	Description
`W65C21`	W65C21 PIA (CMOS 6521/6821) — two 8-bit bidirectional ports with per-line data-direction registers, plus four handshake/interrupt lines
`W65C22`	W65C22 VIA (CMOS 6522) — the same two ports plus two 16-bit interval timers, an 8-bit shift register, and four handshake lines

They're **logic-level**, not cycle-accurate: nothing here models wall-clock timing, so the VIA's timers count `PHI2` cycles rather than seconds.

A few practical notes for building with them:

- **Address one of the peripherals** by holding its chip selects (`CS0` · `CS1` high and `CS2B` low on the PIA; `CS1` high and `CS2B` low on the VIA), picking a register with `RS0`–`RS1` (PIA) or `RS0`–`RS3` (VIA), setting `RWB`, and pulsing `PHI2` — writes latch on the falling edge.
- **`IRQB` is open-drain** on both peripherals, so give it a pull-up.
- You can drive the bus **by hand** — set the address, selects and `RWB`, then pulse `PHI2` — or wire a CPU to it from the group below.

Processors

The **PROCESSOR** group carries the two 8-bit CPUs, both DIP-40. Each is a full instruction-set simulation, so a program in a ROM or RAM really does fetch and execute one instruction at a time:

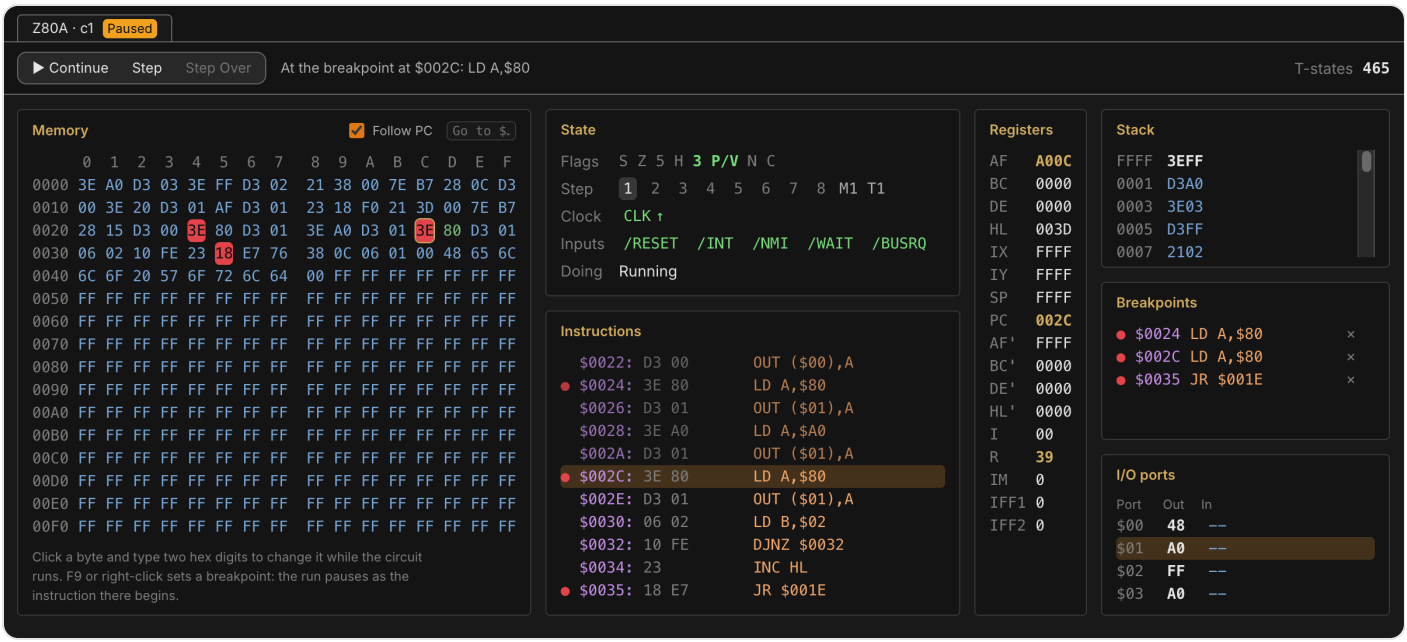
Part	Description
`W65C02`	W65C02S 8-bit CPU — 16-bit address bus (`A0`–`A15`), 8-bit data bus (`D0`–`D7`), `RWB`, `RESB`, `IRQB` / `NMIB`, and `SYNC` pulsing high on each opcode fetch
`Z80A`	Zilog Z80A 8-bit CPU — the same 16-bit address and 8-bit data buses, plus `/MREQ`, `/IORQ`, `/RD`, `/WR`, `/M1`, `/RFSH`, `/HALT`, `/WAIT` and `/BUSRQ` // `/BUSACK`

They disagree about the clock, and it shows in how you wire them. The W65C02 makes exactly one bus access per `PHI2` cycle, so its address bus advances once per clock. Every instruction takes the number of cycles the WDC datasheet gives it, the 6502's "dummy" cycles included: an `INX` reads the byte after it while it works, a read-modify-write such as `INC $0200` reads its address twice before writing, and a branch that is taken spends an extra cycle (two across a page). A dummy cycle is a real read, so a peripheral sees it exactly as it would on a bench. The Z80 is different: one bus access takes several clock cycles. An opcode fetch is four T-states with a memory-refresh cycle glued to its back half, a plain read is three, and an I/O cycle is four; `/M1` marks which cycle is the opcode fetch, and `/RFSH` pulses behind it.

A few practical notes:

- **Both power up in reset.** Wire the reset pin to a push button to hold it. The W65C02 then boots from the reset vector at `$FFFC` / `$FFFD`; the Z80 has **no reset vector at all** and simply starts fetching at `$0000`.
- **Every Z80 control line is active LOW**, which is what the leading slash in its pin names records — the app has no way to draw an overbar.
- **The Z80 gives you real chip-select and strobe lines.** Wire `/MREQ` to a memory's `/CE`, `/RD` to its `/OE` and `/WR` to its `/WE`. That combination matters: during the refresh half of a fetch `/MREQ` pulses low again while `/RD` stays high, so a correctly wired memory does not drive the bus against the CPU's own refresh address.
- `/IORQ` **selects a separate 256-port I/O space**, reached with `IN` / `OUT` and entirely distinct from memory. The 65xx bus has no equivalent — there, peripherals are memory-mapped.
- **Z80 addressing is a little scrambled on the package.** `A0` – `A10` sit on pins 30–40 and `A11` – `A15` wrap round to pins 1–5; the data bus is not in pin order either. The pin-assignments window is worth keeping open.
- Each has an **example circuit** (see below): a small computer, built around it, that types `Hello World` on an LCD as soon as you press **Run**.

The CPU monitor



Right-click a CPU on the desk and choose **Open CPU Monitor** to watch it run. The monitor is a separate window that shows one CPU at a time, laid out like the Chip Designer's debugger. Each CPU on the desktop has a tab across the top; click one to switch to it. The tab shown carries a **Running** or **Paused** badge. While the window is open, selecting a CPU on the desk switches the monitor to it too. Nothing is shown until you press **Run**, apart from the breakpoint list. When you **Stop**, the monitor keeps the run's last moment on screen, dimmed.

Under the tabs is the debugger bar: **Pause/Continue**, **Step** and **Step Over** (below), and a line saying where the CPU is paused, such as `At $8021: BEQ $8039`, or `At the breakpoint at $8021` when it stopped at one.

The window shows:

- **Memory**: 256 bytes. The opcode being run is highlighted, and the byte the current bus cycle is reading or writing is outlined. The monitor reads each address the way the CPU would: it asks your circuit's own decode logic which memory chip answers there. An address that no memory chip answers shows `--`, as do I/O chips such as a W65C22 (reading one of their registers could change it) and addresses where two chips answer at once.
- **State**: the flags register, the step (which bus cycle of the instruction is running; on a Z80, the M-cycle and its T-state), the clock phase, and the CPU's input pins (reset, interrupts, `RDY` / `BE` or `/WAIT` / `/BUSRQ`). An asserted pin is drawn bold.
- **Instructions**: eleven decoded lines. The middle line is the instruction in flight. The five lines above it are the instructions that actually ran, recorded as they happened. The five below are decoded ahead from memory in order, without following branches.

- **Registers**: one register to a line. If the run is paused partway through an instruction, they show how far it has got: PC has moved past the bytes already fetched, and a register being loaded changes once its byte is in.
- **Stack**: the top of the stack, one entry to a line, the newest first and in bold: bytes on the 6502 (from S+1 to the end of page 1), words on the Z80 (from SP up). Click an entry to show its address in the memory block. Reset leaves a 65C02's S at `$FD`, so two entries are listed even before anything has been pushed.
- **Breakpoints**: every breakpoint on this CPU, wherever it is, with the instruction there (below).
- **I/O ports** (Z80 only): the last byte each port was given by an `OUT` and gave an `IN`, with the most recent port highlighted. An I/O port has no memory to read back, so what crossed it is all there is to show. The 6502 has no I/O space: its peripherals sit at ordinary addresses.
- **Cycles** (6502) or **T-states** (Z80): how many clock cycles have run since Run.

The window is as large as it gets, so Stack and Breakpoints share the right column's height evenly and scroll when they are full.

The monitor makes the most sense at a slow clock, or with the clock paused and stepped by hand. At full speed it is a blur.

Moving around memory

The memory block follows the program counter, with PC's row in the middle, until you look somewhere else. Clicking a byte, scrolling the block with the mouse wheel, moving the selection off its top or bottom with the arrow keys (or **Page Up**/**Page Down**), or typing an address into **Go to** (`$8000`, `0x8000` and `8000` all work) and pressing Enter pins the block there. Tick **Follow PC** to have it follow the program again.

Changing memory

While the circuit runs or is paused, click a byte in the memory block and type two hex digits to replace it. The cursor then moves to the next byte, so you can type a run of values one after another. Enter writes a single digit (`4` becomes `$04`), Escape drops it, and the arrow keys move the selection. A byte can only be typed over while it is in the block on screen.

The new byte is written exactly where the CPU would read it, in whichever memory chip answers that address.

What happens to a byte you change in a ROM depends on the **Keep ROM edits after Stop** box. The box appears in the monitor's header as soon as you change a byte in a ROM during a run, and goes away again at Stop:

- **Unticked** (the default): the change lasts until you press **Stop**.

- **Ticked**: every ROM you changed during the run is saved when you press **Stop**, exactly as the memory inspector's **Save** would save it. A message names each chip that was saved. Like any change, it is an unsaved change to your project, and **⌘Z** undoes it.

The box is read when you press Stop, so you can tick it at any point during the run. RAM is never kept, ticked or not: it forgets everything at Stop.

Changing registers

While the run is paused at the start of an instruction (where a breakpoint, a **Step** or a **Step Over** leaves it), click a register and type its hex digits; its last digit writes it, and Enter writes fewer. Escape lets it go, and the up and down arrows move to the next register. Click a flag to flip it. The 6502's `B` flag and its unused bit are not kept by the P register, so they cannot be flipped.

A new PC moves the instruction fetch with it: the CPU carries on from the new address, and the instruction it was about to run at the old one is dropped from the record. If the run is paused partway through an instruction, the registers can only be read; press **Step** first.

What changed

While the run is paused, the registers and flag bits that changed since it last paused are highlighted. After a **Step**, that is what the one instruction changed; after **Continue** to a breakpoint, it is everything that changed on the way there. Changes you make yourself are not highlighted, and **Stop** starts over.

Breakpoints

To set a breakpoint, use any of these:

- Select a byte and press **F9**.
- Right-click a byte and choose **Breakpoint**.
- Click in the left margin of an instruction line.

A byte with a breakpoint turns red, and any instruction line at that address shows a red dot. The **Breakpoints** panel lists them all, so you can see the ones out of view too: click an address there to show it in memory, or **×** to remove it. The list is there before you press **Run**, so you can manage breakpoints before the program starts.

When the CPU reaches the instruction at a breakpoint, the run **pauses** right as the instruction begins: its opcode fetch is under way, and none of it has run yet. The monitor window comes forward on that CPU. From there you can step (below), or press **Continue** (**F8**) to run on until a breakpoint is reached again.

Pausing and stepping

The bar's first button is **Pause** while the circuit runs and **Continue** while it is paused; **F8** presses it either way. They are the same as the main window's **Pause** and **Resume**.

While the run is paused (at a breakpoint, or by **Pause**), the monitor's **Step** button, or **F6**, runs the circuit on until the CPU starts its next instruction, and pauses there again. If an interrupt or a reset is taken instead, the step stops at the start of that sequence. A step also stops at any breakpoint it reaches, on any CPU. The whole circuit runs while it steps: every free-running clock and timed part moves on with the CPU, exactly as the main **Step** button moves them.

Step Over (**F7**) is offered when the instruction in flight calls a subroutine: a 6502 `JSR`, or a Z80 `CALL`, `CALL cc` or `RST`. It runs the whole subroutine and pauses at the instruction after the call, once the stack is back where the call found it (so a recursive call returning to the same place does not stop it early). A subroutine that reads data placed right after the call and returns past it stops where it returns to. A breakpoint inside the subroutine stops it there. A short call is run at once, like a step. A long one, such as a delay loop, carries on as a normal run at the circuit's own speed, and pauses when the call returns; press **Pause** to give up on it where it is. The return needs somewhere to keep its address: on a computer with no RAM behind the stack, the subroutine never comes back to the call.

A step needs a free-running clock or a timed part to move the circuit. On a manual clock, click the clock instead. If the CPU starts nothing new within a few thousand clock edges (it is waiting for an interrupt, say), the step gives up and leaves the circuit paused where it got to.

F6, **F7** and **F8** work only while the monitor window has the focus. If the Chip Designer's debugger is also paused, they act on whichever of the two windows you are in. In the main window the function keys do nothing. While the debugger is holding the circuit in the middle of a clock edge, the monitor's **Step** does nothing: let the debugger finish first (**Continue** or **To Settled**).

Breakpoints fire even while the monitor window is closed. They are saved in the project, with the desktop they were set on: they last through Stop and Run, come back when you switch back to that desktop, and are there the next time you open the project. Setting or clearing one is a change to the project like renaming a desktop, so it marks the project unsaved, but it is not an edit to the circuit: it can be done while the circuit runs, and **Undo** does not take it back. A duplicated desktop keeps its CPUs' breakpoints; an exported one does not carry them.

Memory chips

The **Memory** group carries the address-indexed parts: a couple of generic teaching ROM/SRAM chips plus real-shaped EEPROM/EPROM/SRAM parts on wider DIP packages. Reads and writes are wired up the same way as every other chip in the catalog, but a non-volatile chip's contents live in a real file on disk and are programmed through a dedicated in-app tool rather than by the circuit itself. See Memory Chips & the Inspector for the full story — file-backing, the external programmer, and the hex/ASCII inspector.

The pin-assignments window

Right-click **any** chip — or a package-footprint discrete like the `bar8iso` LED bar, which seats and rotates exactly like a DIP chip even though it isn't one — and choose **Pin Assignment**, at the top of its context menu, to open its **pin-assignments window**: a small, floating window separate from the main desk, showing the physical DIP layout with the notch at the top, pin 1 at the top-left, and pin numbers wrapping down the left side and back up the right to the highest pin at the top-right, exactly as printed on the part.

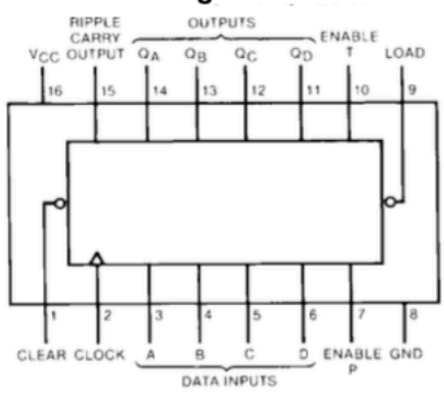
74LS161 · Sync 4-bit binary counter

DIP-16 · pin assignments

1	CLR	IN	VCC	PWR	16
2	CLK	IN	RCO	OUT	15
3	A	IN	QA	OUT	14
4	B	IN	QB	OUT	13
5	C	IN	QC	OUT	12
6	D	IN	QD	OUT	11
7	ENP	IN	ENT	IN	10
8	GND	GND	LOAD	IN	9

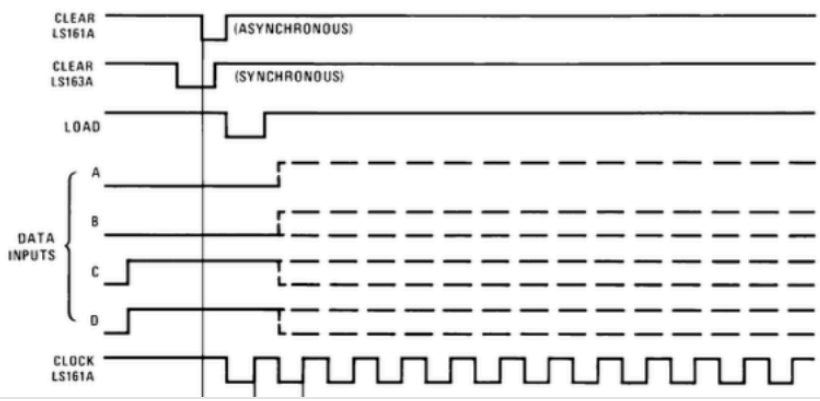
Datasheet — internal diagram & function table

Connection Diagram



Timing Diagram

LS161A, LS163A Synchronous Binary Counters
Typical Clear, Preset, Count and Inhibit Sequences



For a real chip this layout is always the **canonical, fixed arrangement** — it matches the physical part regardless of how you've flipped it on the desk, because a real chip's pin-1 dot is a physical feature of the package, not something rotation changes. `bar8iso` is the one exception: it has no real notch to key off, so its pin-assignments window reflects its **current** `R` **flip** on the desk — rotate it and the corners in the dialog swap to match.

Below the pin map, chips that have one show the manufacturer's **datasheet crop** — a cropped image of the connection diagram or function table pulled straight from the source datasheet PDF. If a chip has no crop on file, that part of the window simply isn't there — the pin map alone is still complete and accurate.

When **Settings → Data Sheets** points at a local folder containing that chip's full datasheet PDF, the window also grows a small document button in its top-right corner; clicking it opens the PDF itself in your system's PDF viewer. This is independent of the built-in datasheet crop — you may have one, both, or neither for any given chip.

Example circuits

A pin map tells you where the pins are; it doesn't show you the part working. So almost every 74LS and CD4000 chip in the catalog ships with a **worked example** — a small bench built around that one part — and the pin-assignments window is where you reach it. Look for the **circuit button** in the window's top-right corner, beside the datasheet one.

Click it and the example arrives as a **new desktop** in the open project, called `74LS138 example` (or whichever part it is), already framed on screen. Every one is the same bench, so once you can read one you can read them all:

- a **5 V brick** feeding both power rails, and a **clock** for the clocked parts;
- **switched inputs** on the left, each throwing between +5 V and a pull-down so an input is never left floating — a part with more inputs than will fit gets a DIP switch bank over a resistor network instead. A CD4000 bench also ties every input it does not switch, spare gates included;
- the **chip under test** in the middle, straddling the trench;
- **LED read-outs** on the right, one per output, through its own resistor. An active-LOW output has its LED wired the other way up, so a lit lamp always means *this output is asserted*;
- a **caption** above the bench saying what the demo shows.

Press **Run** (Space) and flip the switches. Each example opens in a state chosen to show the part doing something.

The bench parts have examples of their own, each drawn as you would build it and captioned with what to try — most of them best run with Spice Lite on:

Part	Desktops
`LM358`	**Comparator** (a pot against half the supply, an LED on the output) · **Amplifier** (gain 2)
`ULN2003A`	**Relay driver** — one channel a relay, another an LED
`4N35`, `PC817`	**Isolated switch** — a 5 V switch driving a 12 V LED on its own supply
`LM7805`	**9 V to 5 V** (a 74LS04 on the regulated rail) · **Under load** (an electronic load drawing 300 mA)
`LM317`	**3.3 V** — set by 240 Ω and 390 Ω
Relay	**Transistor driver** (with its flyback diode) · **No flyback diode**
Electronic load	**Supply limit** — a 500 mA supply under 250 mA, raise it and watch it droop

The two **CPUs** can't be shown by flipping switches either, so each example is a whole computer, wired and ready: the CPU, an **AT28C256** ROM with the program already in it, a **W65C22 VIA** as the interface chip, a **16×2 LCD** on the VIA's ports, and a reset button. Press **Run** and the program sets the LCD up and types `Hello World` a letter about every half second. Hold the reset button and let go to watch it again; open the CPU monitor to follow the program as it goes.

- **`W65C02 example`** — the ROM fills `$8000`–`$FFFF` and the VIA sits in the low half, split by one 74LS04 on `A15`. The clock runs at 100 Hz.
- **`Z80A example`** — the ROM is all of memory, from `$0000` (the upper half repeats it), and needs no glue logic: `/MREQ` is its chip enable and `/RD` its output enable. The VIA sits in the Z80's I/O space, selected by `/IORQ` and written with `OUT (n),A`. The clock runs at 250 Hz, because a Z80 instruction takes more clock cycles than a 6502's. To build it for real, use the CMOS Z84C00: an NMOS Z80A cannot be clocked that slowly.

The ROM's program comes with the example, so you don't load anything. Like every ROM, the copy gets its own backing file, and the program is saved in your project with the rest of it.

The **555** is the exception: there is nothing to switch, because what it does depends on how it is wired. Its example brings **three desktops**, one per mode — `NE555 Monostable example`, `NE555 Bistable example` and `NE555 Astable example` — each built the way you would on a bench, with an LED on the output. Press the button on the monostable for a 1.1-second pulse; press SET and RESET on the bistable to latch the LED on and off; the astable blinks on its own at about 0.6 Hz. You land on the first; the others are the next tabs along.

A few practical notes:

- It's an ordinary desktop and an ordinary unsaved change — it doesn't reach your project's file until you save, and you can rename it, edit it, or delete it like any other.

- Asking for the same example twice doesn't make a second copy; you land back on the desktop you already have. For the 555, only a desktop you have deleted is added again.
- Adding it stops a running simulation, exactly as switching desktops does.
- Parts with no bench have no button: the memory chips and the 65xx peripherals (a RAM can't be demonstrated by flipping switches at it — you can see them working in the CPUs' examples), the CD4000 RC timers (CD4047B, CD4098B, CD4528B, CD4538B and CD4541B — a timer's bench is its resistor and capacitor, not switches), the LM7809, LM7812 and LM7815 (the LM7805's example shows them all), and every other discrete, brick and wire. The CD4060B's example counts a clock brick on ϕI rather than running its oscillator.

Datasheets

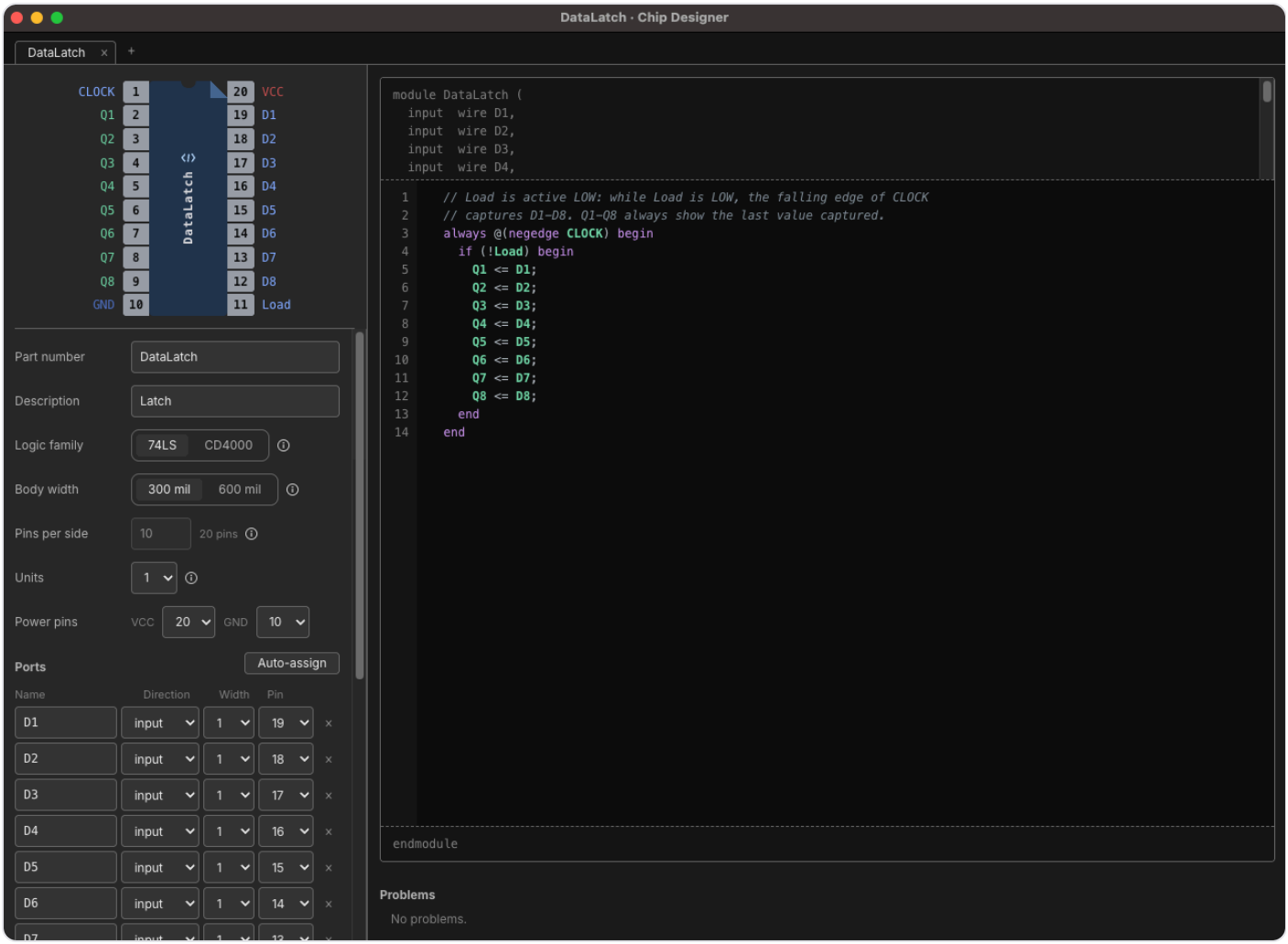
The datasheet crops shown in the pin-assignments window are committed image assets cut by hand from the manufacturer PDFs, not fetched or rendered at runtime — they work offline and load instantly. A part with no crop on file simply shows its pin map with no image below it: four 74LS parts have no matching `74LS*` datasheet (`74LS164` , `74LS193` , `74LS27` , `74LS76`), and the CD4000 parts' crops are still being cut. Pointing **Settings → Data Sheets** at your own folder of manufacturer PDFs is a separate, optional feature — it doesn't add or replace the built-in crops, it just adds the "open datasheet PDF" button for any part whose PDF you have on hand.

Need a part the library doesn't have? Custom Chips covers designing your own.

See also Chips & Components for how chips seat into a breadboard, and Memory Chips & the Inspector for the memory group's file-backing and inspector.

Custom Chips

When the part you need isn't in the library, design it. A **custom chip** is a DIP package you lay out yourself — how many pins, which is which — with its behaviour written in a small, strict subset of **Verilog**. It sits in the parts tray beside the library, places like any chip, and runs in the simulation like any chip. While the circuit runs you can stop inside it and step through its code a statement at a time.



Custom chips are **yours, not one project's**: every chip you design is kept in Chip Hippo's own chip library on this computer, so it is in the tray of every project you open. A project file carries the chips its desktops use, so it opens complete on another computer — and opening a project that uses a chip your library doesn't have adds that chip to your library. Exporting a desktop takes the chips it uses with it, and importing one brings them in the same way.

If a project carries its own, older copy of a chip you have since changed, that project keeps using its copy while it is open, so its circuit is built exactly as it was. Editing the chip there saves the edit to your library too.

Making a chip

In the parts tray, open **CHIPS ▸ CUSTOM** (after the library's chips, or beside the 74LS and CD4000 folders when the tray shows both families) and click **New chip...**. The **Chip Designer** window opens on a fresh design — a two-input NAND on a 14-pin package, so there is something working to change. Right-click a chip in the tray for **Edit Design...**, **Duplicate Design** and **Delete Design**. Deleting a design removes it from your library; a chip placed on a desktop of the open project can't be deleted until it has been removed from the desk.

The designer is one window with a tab per open design. The left half is the package — its pinout drawn as a datasheet draws it, which stays in view at the top, and under a line the fields that define it, which scroll on their own. The right half is the code. Drag the divider between the halves to give the package more room (a chip of several units has a wide pin table); double-click it to put it back. Chip Hippo remembers where you left it.

The package

- **Part number** — what is printed on the chip, up to 16 letters, digits, `-` and `_`. It is also the Verilog module's name.
- **Description** — a line shown beside it in the tray and in the BOM.
- **Logic family** — 74LS or CD4000. It decides what an input left open reads (HIGH for 74LS, unknown for CD4000) and the supply the chip needs, exactly as for the library's parts of that family.
- **Body width** — 300 mil or 600 mil, which is where it seats on the board: a 300-mil chip straddles the trench in rows e and f, a 600-mil one in two rows six holes apart (d and h, or e/i, c/g or b/f — see Placing a DIP chip).
- **Pins per side** — from 2 to 20, so from 4 to 40 pins.
- **Units** — how many copies of the module the package holds, up to six (see Units).
- **Power pins** — which pins are VCC and GND (VDD and VSS for CD4000).
- **Ports** — the module's inputs and outputs: a name, a direction (input, output, or **inout** for a pin the chip both reads and drives, such as a data bus) and a width of 1 to 16 bits, and at the end of the row the pin it is on — one column per unit. A port wider than one bit takes one pin per bit, so its bits are listed under it, each with its own pins. **Auto-assign** puts every bit that has no pin on the next free one. When there are more units than the pane has room for, the pin columns scroll sideways while each port's name stays in view.

While a chip is placed on any desktop of the open project its **size and width are fixed** — its instances are seated in holes, and moving their pins would silently rewire every desk it is on. Everything else, the code included, can still change. To change the package of a chip in use, remove it from the desk first, or duplicate the design. (Other projects that use the chip are not rewired by a change made here: each keeps its own copy in its file.)

The code

The **module header** — `module`, the port list, `endmodule` — is generated from the package and shown above and below the editor, read-only. You write only the **body**: declarations, `assign` statements and `always` blocks. So the code's pins can never drift from the chip's: rename a port in the package and the header follows. The header takes up to two fifths of the code area and scrolls past that; drag the line under it to make it taller or shorter, and double-click the line to put it back.

The code is checked as you type. Problems are listed under the editor with their line numbers, and the line is marked in the gutter. A chip whose package or code has a problem still places, but it **drives nothing** (every output floats), and pressing Run says which chips those are.

Hover a pin on the package to light every use of it in the code; hover a name in the code to light its pins on the package.

The Verilog subset

What is supported behaves as real Verilog does, so code written here reads correctly to anyone who knows the language — and anything outside the subset is **refused with a message saying so**, never approximated.

```
// Ports: CLK, RST_N in; Q[3:0] out — a 4-bit binary counter with an
// asynchronous reset.
reg [3:0] count = 4'd0;

always @(posedge CLK or negedge RST_N)
  if (!RST_N)
    count <= 4'd0;
  else
    count <= count + 1;

assign Q = count;
```

Supported

- `wire` and `reg` declarations, with a constant range (`reg [7:0] r;`) and an optional initial value (`reg q = 1'b0;`); `integer` (a signed 32-bit variable); `parameter` and `localparam` (and `parameter integer`).
- **Arrays** of `reg` or `integer` — memories — with one dimension: `reg [7:0] mem [0:32767];` . Read or write one word at a time, `mem[address]` ; an address that is unknown or out of range reads `x` and

writes nothing. An array may hold up to 32,768 words, and a chip's arrays up to 65,536 words between them. Only an `always @(posedge …)` or an `initial` block may write an array (from `always @(*)` it would be a latch), and an array can't be given a value where it is declared — fill it from an `initial` block. A word starts as `x`, as in a real simulator.

- Continuous `assign`.
- `always @(*)` (or a full list of signals, `@(a or b)`) for combinational logic.
- `always @(posedge CLK)`, with `negedge` as needed, and asynchronous controls in the same list: `@(posedge CLK or negedge RST_N)`, tested in the block's leading `if`, as a synthesis tool expects.
- `initial` blocks, for a starting value.
- `begin … end`, `if` / `else`, and `case` / `casez` / `casex` with `default`.
- **Named blocks** with variables of their own: `begin : fill integer i; … end`. The declarations come first, take no initial value, and are seen only inside the block — the watch panel shows them as `fill.i`.
- **`for` and `repeat` loops** whose bounds are constants (see Loops).
- Blocking `=` and non-blocking `<=`, with Verilog's semantics: a non-blocking assignment takes effect after every block triggered by the same edge has run.
- Operators: bitwise `& | ^ ~ ~^`, reduction (`&a`, `|a`, `^a` and their negations), logical `&& || !`, equality `== != === !==`, comparison `< <= > >=`, shifts `<< >> <<< >>>`, arithmetic `+ - * / % **`, the ternary `?:`, concatenation `{a, b}` and replication `{4{a}}`, and bit, part and indexed part selects (`a[3]`, `a[7:4]`, `a[i +: 4]`, `a[i -: 4]`).
- Numbers in Verilog's forms — `13`, `4'b1010`, `8'hFF`, `'d9` — with `x` and `z` digits.

Values are four-state (`0`, `1`, `x`, `z`) and up to 32 bits wide. A `reg` with no initial value starts as `x`, as in a real simulator. Signedness is Verilog's: a plain number such as `13`, an `integer`, and a parameter given one of them are signed, everything else is unsigned, and an expression is signed only when every value in it is — so `(0 - 1) < 2` is true, while `(count - 1) < 2` with an 8-bit `count` compares unsigned.

Not supported — `while` and `forever` loops (every evaluation has to finish), delays (`#`; the simulation is zero-delay), functions and tasks, system tasks such as `$display` (the debugger's watch panel shows every value), `signed` declarations (use `integer`), arrays of more than one dimension or of wires, gate primitives, module instances (use units instead), and compiler directives.

Also refused, because they are mistakes: assigning an input; driving one signal from two places; a `reg` driven by `assign` or a `wire` set in an `always` block; an `assign` to a bit a variable picks (`assign Y[S] = A;` — choose the bit in an `always @(*)` instead); an `always @(*)` that leaves a signal unset on some path (it would be a latch); a signal set with `<=` and then read in the same `always @(*)`, where it would still hold its old value (use `=` there); an event list that misses a signal the block reads; and a combinational loop (`assign` and `always @(*)` cannot hold state — a clocked block can). A loop's

counter is the one exception to the latch rule: a `for` inside an `if` leaves it unset when the loop does not run, which is fine as long as nothing reads it there. Undriven outputs and unused registers are warnings rather than errors.

Inout pins

An **inout** port is a pin the chip both reads and drives, as a memory's or a processor's data bus is. Drive it with `assign`, giving `z` whenever the chip should let go of it; reading it reads the **pin** — whatever the board has settled it to, the chip's own drive included. An inout can't be set from an `always` block, and, like an output, can't share a pin with another unit.

```
// Ports: A[14:0], CE_N, OE_N, WE_N in; D[7:0] inout — a 32K × 8 static
// RAM on the HM62256's pins. A write takes the byte on the bus as WE_N
// rises.
reg [7:0] mem [0:32767];

always @(posedge WE_N)
  if (!CE_N)
    mem[A] <= D;

assign D = (!CE_N && !OE_N && WE_N) ? mem[A] : 8'bz;
```

Loops

A `for` or `repeat` loop is accepted only when Chip Hippo can tell, before anything runs, how many times it goes round — so that every evaluation is sure to finish:

- A `for` loop sets **one counter** in its first and last parts — `for (i = 0; i < 8; i = i + 1)` — and its start, condition and step use only numbers, parameters and the counter itself. The body may not change the counter.
- A `repeat (n)` count is a constant.
- Declare the counter first: at the top (`integer i;`) or in a named block. Declaring it in the loop header, `for (integer i = 0; …)`, is SystemVerilog, not Verilog. A counter declared at the top may count loops in only one `always` block; give each block a counter of its own.
- In an `always @(*)` a loop may go round at most 1,024 times in all; in a clocked or `initial` block, 65,536 — enough to clear two of the largest arrays. A loop that would never end (a 4-bit counter counting to 16) is refused.

```
// Ports: IN[7:0] in; ONES[3:0] out — how many inputs are high.
always @(*) begin : count
  integer i;
  ONES = 0;
  for (i = 0; i < 8; i = i + 1)
    ONES = ONES + IN[i];
end
```

When the debugger is stopped inside a loop, the counter's value is in the watch panel; a breakpoint in the loop's body stops once for each time round, and one on the `for` line stops each time the counter is set.

One clock per module

A module has at most **one clock**. Every edge-triggered block uses the same clock input, and any other signal in an edge list must be an asynchronous control tested first in the block. Logic on a second clock belongs in a second chip.

Units

A multi-unit package — a quad NAND, a dual flip-flop — is the **same module repeated** on separate pins. Write the module once, set **Units**, then give each unit its own pins in its column of the Ports table. The pinout prefixes each unit's pin names with its number, as the datasheets do (`1A` , `2A` , …). A port can share one pin between units — a common clock or enable — in which case that pin keeps the bare name. An output or inout can never share a pin.

Examples

Every construct the subset accepts appears in at least one of the snippets below, and most show several. Each is a module body as you would type it in the designer; its first comment names the ports it expects — add them in the package, and the generated header declares them. Every snippet compiles as it stands.

- Gates and reductions — comments, a `wire` declared with its value, `assign` (several in one statement), bitwise `~ & | ^ ~^` , reduction `^a ~^a |a ~|a ~&a` , logical `! && ||` .
- Comparisons and a multiplexer — `> >= == != < <=` , the ternary `?:` , and `===` (with `!==`).
- Arithmetic — `+ - * / %` , unary minus, and a concatenation on the left of `assign` .
- Shifts and selects — `<< >>` , part selects `D[3:0]` , bit selects `D[N]` , indexed part selects `D[N +: 2]` , concatenation `{a, b}` and replication `{4{a}}` .
- Numbers — every way of writing a number, `x` , `z` and `?` digits, and `localparam` .

- A decoder — `always @(*)`, `case`, several values on one item, and `default`.
- A priority encoder — `casez`, `casex`, and `begin … end` inside a `case`.
- Sensitivity lists — `always @(a or b)` and `always @(a, b)`.
- A counter — `parameter`, `**`, a `reg` with a starting value, `always @(posedge … or negedge …)` with an asynchronous clear, `if` / `else if`, and `<=`.
- A shift register — `initial`, `negedge`, and the difference between `<=` and `=`.
- Signed values — `integer`, `parameter integer`, a signed comparison and `>>>`.
- A ROM and a register file — arrays, filling one from `initial` with a `for` loop, and a named block with a counter of its own.
- Repeating a statement — `repeat`.
- Inout pins and Loops above have their own: an inout data bus, and a `for` loop in `always @(*)`.

Gates and reductions

Bitwise operators work bit by bit, as the gates in a 74LS00 do; reduction operators fold a whole vector down to one bit; logical operators read a vector as simply true (any bit 1) or false.

```
// Ports: A[3:0], B[3:0] in;
//        NAND4[3:0], XNOR4[3:0], MIX4[3:0], ODD, EVEN, ANY, NONE, NOTALL,
//        IDLE, EITHER out.

/* Bitwise operators work bit by bit, as a gate package does: four
   2-input gates per line here. */
wire [3:0] both = A & B;         // a wire declared with its value
assign NAND4 = ~both,            // ~ inverts; one assign can set several
       XNOR4 = A ~^ B;           // XNOR (^~ is the same operator)
assign MIX4  = (A | B) ^ 4'b0101;

// Reduction operators fold a whole vector down to one bit.
assign ODD    = ^A;              // 1 when A has an odd number of 1s
assign EVEN   = ~^A;
assign ANY    = |A;
assign NONE   = ~|A;
assign NOTALL = ~&A;             // and &A: every bit is 1

// Logical operators read a vector as true (any bit 1) or false.
assign IDLE   = !A && !B;
assign EITHER = A || B;
```

Comparisons and a multiplexer

A comparison answers 1 or 0 — or `x`, when an `x` bit leaves it unable to tell. The ternary picks one of two values, and with an `x` condition keeps the bits both values agree on.

```
// Ports: A[3:0], B[3:0], SEL in; GT, GE, EQ, NE, LT, LE, Y[3:0], SEL_OPEN out.

// A 74LS85-style magnitude comparator…
assign GT = A > B;
assign GE = A >= B;
assign EQ = A == B;
assign NE = A != B;
assign LT = A < B;
assign LE = A <= B;

// …and a 2-to-1 multiplexer: SEL high picks A, low picks B.
assign Y = SEL ? A : B;

// === and !== compare x and z bits exactly, and never answer x
// themselves. A CD4000-family chip reads an open input as x, so on one
// this says whether SEL is connected.
assign SEL_OPEN = (SEL === 1'bx);
```

On a 74LS-family chip an open input reads 1 instead, as a TTL input does.

Arithmetic

An expression is worked out as wide as the widest thing in it — the left-hand side included — so a sum keeps its carry when there is room for it.

```
// Ports: A[3:0], B[3:0], CIN in;
//        SUM[3:0], COUT, DIFF[3:0], NEG[3:0], PROD[7:0], QUOT[3:0], REM[3:0] out.

assign {COUT, SUM} = A + B + CIN;  // a concatenation on the left: the sum is
                                   // worked out 5 bits wide, and its top bit
                                   // lands in COUT
assign DIFF = A - B;               // wraps round, as 4-bit hardware does
assign NEG  = -A;                  // two's complement (unary + also exists)
assign PROD = A * B;               // PROD is 8 bits, so nothing is lost
assign QUOT = A / B;               // dividing by 0 gives x
assign REM  = A % B;
```

Shifts and selects

```
// Ports: D[7:0], N[2:0] in; ROTL[7:0], SWAP[7:0], BIT, PAIR[1:0], FILL[3:0] out.

assign ROTL = (D << N) | (D >> (8 - N)); // rotate left by N: << and >>
                                         // shift 0s in (<<< and >>> are
                                         // the same on unsigned values)
assign SWAP = {D[3:0], D[7:4]};          // part selects, joined the other
                                         // way round by a concatenation
assign BIT  = D[N];                      // one bit, picked by an input
assign PAIR = D[N +: 2];                 // two bits from bit N upwards;
                                         // D[N -: 2] counts down. A bit
                                         // past the end reads x
assign FILL = {4{D[0]}};                 // replication: D[0] four times
```

Numbers

A number may be a plain decimal, or a size, an apostrophe, a base (`b` , `o` , `d` , `h`) and digits. With no size it is 32 bits wide.

```
// Ports: Y[7:0] out.
localparam TEN    = 10;            // a plain number: 32 bits, signed
localparam MASK   = 8'b1010_0101;  // 8 bits in binary; _ is only for reading
localparam LIMIT  = 8'hFF;         // hex — and 8'o377 octal, 8'd255 decimal
localparam SEVEN  = 'd7;           // a base with no size: 32 bits, unsigned
localparam FLOAT  = 8'bz;          // all z: a leading x or z fills the width
localparam ENDS   = 4'b1??1;       // ? is z, read as "don't care" by casez
localparam HALF_X = 8'hx5;         // an x digit is 4 x bits in hex
assign Y = MASK & LIMIT;
```

A decoder

An `always @(*)` block is combinational: it runs whenever anything it reads changes, and it must set its outputs on every path through it, or it would be a latch. A `case` with a `default` covers every path, and so does one that lists every value its subject can take.

```
// Ports: BCD[3:0] in; SEG[6:0] out — segments g…a, 1 = lit, as a
// common-cathode display wants them.
always @(*)
  case (BCD)
    4'd0: SEG = 7'b011_1111;
    4'd1: SEG = 7'b000_0110;
    4'd2: SEG = 7'b101_1011;
    4'd3: SEG = 7'b100_1111;
    4'd4: SEG = 7'b110_0110;
    4'd5: SEG = 7'b110_1101;
    4'd6: SEG = 7'b111_1101;
    4'd7: SEG = 7'b000_0111;
    4'd8: SEG = 7'b111_1111;
    4'd9: SEG = 7'b110_1111;
    4'd10, 4'd11, 4'd12, 4'd13, 4'd14, 4'd15:
      SEG = 7'b000_0000;         // several values, one statement: blank,
                                 // as a CD4511 blanks them
    default:
      SEG = 7'bxxx_xxxx;         // BCD has an x or z bit
  endcase
```

A priority encoder

`casez` treats a `z` or `?` bit, on either side, as matching anything; setting `VALID` before the `case` gives it a value on every path.

```
// Ports: REQ[7:0] in; CODE[2:0], VALID out — the highest request wins,
// as on a 74LS148 (here active high).
always @(*) begin
  VALID = 1'b1;                  // a value first, so every path sets it
  casez (REQ)
    8'b1???_????: CODE = 3'd7;   // casez: a ? (or z) bit matches anything
    8'b01??_????: CODE = 3'd6;
    8'b001?_????: CODE = 3'd5;
    8'b0001_????: CODE = 3'd4;
    8'b0000_1???: CODE = 3'd3;
    8'b0000_01??: CODE = 3'd2;
    8'b0000_001?: CODE = 3'd1;
    8'b0000_0001: CODE = 3'd0;
    default: begin
      CODE  = 3'd0;
      VALID = 1'b0;              // no request at all
    end
  endcase
end
```

`casex` does the same for `x` bits too:

```
// Ports: OP[3:0] in; KIND[1:0] out.
always @(*)
  casex (OP)
    4'b1xxx: KIND = 2'd3;        // casex: x bits match anything too
    4'b01xx: KIND = 2'd2;
    default: KIND = 2'd0;
  endcase
```

Sensitivity lists

`@(*)` is the easy way, but a block may list what it reads instead — and must then list all of it.

```
// Ports: A, B, C in; MAJ, ODD out.
always @(A or B or C)             // a list naming everything the block reads…
  MAJ = (A & B) | (A & C) | (B & C);

always @(A, B, C)                 // …the same with commas; @(*) or @*
  ODD = A ^ B ^ C;                // works it out for you
```

A counter

A clocked block runs on its clock's edge. Any other signal in its event list is an asynchronous control, and must be tested first, in the block's leading `if`; a clocked block may leave a `reg` unset, and it keeps its value.

```
// Ports: CLK, CLR_N, LOAD_N, EN, D[3:0] in; Q[3:0], TC out — a
// 74LS161-style counter.
parameter  WIDTH = 4;                  // a named constant…
localparam TOP   = 2 ** WIDTH - 1;     // …and one worked out from it (15)

reg [WIDTH-1:0] count = 0;             // a reg, with its value at power-up

always @(posedge CLK or negedge CLR_N)
  if (!CLR_N)                          // the asynchronous clear, tested first
    count <= 0;
  else if (!LOAD_N)
    count <= D;
  else if (EN)
    count <= count + 1;
  // with none of them, count keeps its value — a clocked block may

assign Q  = count;
assign TC = (count == TOP) && EN;      // terminal count: about to roll over
```

A shift register

```
// Ports: CLK, D in; Q1, Q2, Q3 out — on each FALLING clock edge every
// stage takes what the stage before it held before the edge.
reg s1, s2, s3;

initial begin                    // power-up values, set once before
  s1 = 1'b0;                     // the circuit runs
  s2 = 1'b0;
  s3 = 1'b0;
end

always @(negedge CLK) begin
  s1 <= D;                       // <= (non-blocking): all three are read
  s2 <= s1;                      // first, and written together when the
  s3 <= s2;                      // block is done
end

assign Q1 = s1;
assign Q2 = s2;
assign Q3 = s3;
```

Written with `=` instead — `s1 = D; s2 = s1; s3 = s2;` — each assignment would take effect before the next line read it, and one edge would copy `D` into all three stages.

Signed values

An `integer` is a signed 32-bit variable. An expression is signed only when everything in it is signed: `temp < FREEZE` compares signed numbers, while `T < FREEZE` would compare `T` with -5 read as an unsigned 32-bit number.

```
// Ports: T[7:0] in; COLD, HOT, HALF[7:0] out — T is a temperature in
// two's complement, −128 to 127.
integer temp;                     // a signed 32-bit variable
parameter integer FREEZE = -5;    // a signed constant (a plain -5 is too)

always @(*) begin
  temp = T;                       // T is unsigned: 8'hFB becomes 251, not -5…
  if (T[7])
    temp = temp - 256;            // …so a negative reading is made so here
  COLD = temp < FREEZE;           // both signed: a signed comparison
  HOT  = temp > 40;
  HALF = temp >>> 1;              // >>> keeps a signed value's sign
end
```

A ROM and a register file

An array holds words; read or write one at a time. An `initial` block can fill one before the circuit runs — here with a `for` loop:

```
// Ports: ADDR[3:0] in; DATA[7:0] out — a 16-word ROM of squares.
reg [7:0] rom [0:15];
integer i;

initial                           // filled once, before the circuit runs
  for (i = 0; i < 16; i = i + 1)
    rom[i] = i * i;

assign DATA = rom[ADDR];          // read a word: any address, any time
```

A clocked block writes its words, a whole array of them in a loop if it likes:

```
// Ports: CLK, CLR, WE, WA[2:0], RA[2:0], D[7:0] in; Q[7:0] out — eight
// 8-bit registers: write one on a rising edge, read any one at any time.
reg [7:0] regs [0:7];

always @(posedge CLK)
  if (CLR) begin : clear          // a named block, with a variable of its own
    integer r;
    for (r = 0; r < 8; r = r + 1)
      regs[r] <= 8'd0;            // an array is written a word at a time
  end else if (WE)
    regs[WA] <= D;

assign Q = regs[RA];
```

Repeating a statement

```
// Ports: X[3:0] in; X4[15:0] out — X to the fourth power.
always @(*) begin
  X4 = X;
  repeat (2)                      // a fixed number of times
    X4 = X4 * X4;                 // squared, then squared again
end
```

Debugging a chip while it runs

Place the chip, wire it up and press **Run**. The Chip Designer window turns into the **debugger** while the circuit runs, and back into the designer when you press Stop. (A design can't be edited while the circuit runs.)

Breakpoints

A custom chip runs like any other until you give it somewhere to stop. Two kinds of breakpoint do that:

- **A line of code.** Click a line number in the editor's margin (or, while the circuit is stopped, press **F9** on the line) and the number turns into a solid red circle. When a change on a chip's pins makes its code run that line, the chip stops there, before the line executes. Click the circle to take the breakpoint off. A breakpoint belongs to the design, so it stops every placed copy of the chip, and you can set it before you press Run or while the circuit is running. It stays with its line as you edit the code above it.

- **Break on Settled** — pause at the moment the board settles after the chip has changed, to inspect where everything ended up. Switch it on for one placed chip from its right-click menu on the desk, or from the debugger bar.

A breakpoint on a line that can never stop the chip — a comment, a declaration, `begin` or `end`, the second line of a statement — is drawn as a **hollow** circle and ignored. Click it to take it off. While the package or the code has an error, the chip runs no code, so every breakpoint is hollow until the error is fixed.

A chip with a breakpoint it can reach carries a small red badge on the board, and a chip paused in its code pulses. When a breakpoint fires the board **stalls** — clock edges wait, nothing else moves — and the debugger window comes forward with the paused statement highlighted.

The debugger bar

The global Run / Stop / Pause / Step controls are unchanged, and remain the only ones that act on the whole simulation. The debugger has its own bar, scoped to the chip in view:

Control	Key	What it does
Continue	F8	Run on until the next breakpoint — later in the same block, or on any chip.
Step	F6	Execute one statement. Stepping past the end of a chip's reaction stops at the next statement it runs.
Step Out	F7	Finish this chip's reaction and hand its outputs to the board.
To Settled	F9	Run until the board settles, ignoring breakpoints, and stop there.
Detach	F10	Stop debugging this chip and let it run — the simulation carries on.
Break on Settled		The settled breakpoint, for the chip in view.
Settled lamp		Green when the board has settled: every chip idle, nothing held.

The keys work anywhere in the debugger window while the circuit runs, and do nothing when their button is greyed out. (On a Mac keyboard, hold **fn** with them unless the function keys are set to act as standard keys.) While the debugger runs, F9 is To Settled; set a line breakpoint by clicking its number.

Detach is not Stop: the chip just ignores its breakpoints for the rest of the run. Switch on **Break on Settled**, or set a breakpoint while its tab is showing, and it stops again. The next Run debugs it as usual.

Tabs

Each chip being debugged has its own tab — several chips reacting to the same change each get one, so you can step them in any order. Tabs are ordered by when their change arrived, alphabetically when

changes arrive together. Selecting a custom chip on the desk switches the window to its tab, and a new tab never takes the focus from the one you are stepping. A tab's badge says what it is doing:

Badge	Meaning
Paused	Stopped at a breakpoint, waiting for you.
Paused · N held	Paused, with N changes from other chips waiting for it.
Idle	Its reaction is finished and its outputs are written; it keeps its last values until a new change arrives.
Settled	Stopped at the settled breakpoint.
Detached	Nothing will stop it (no breakpoint it can reach, and Break on Settled off), or it was detached — it runs normally and never pauses.

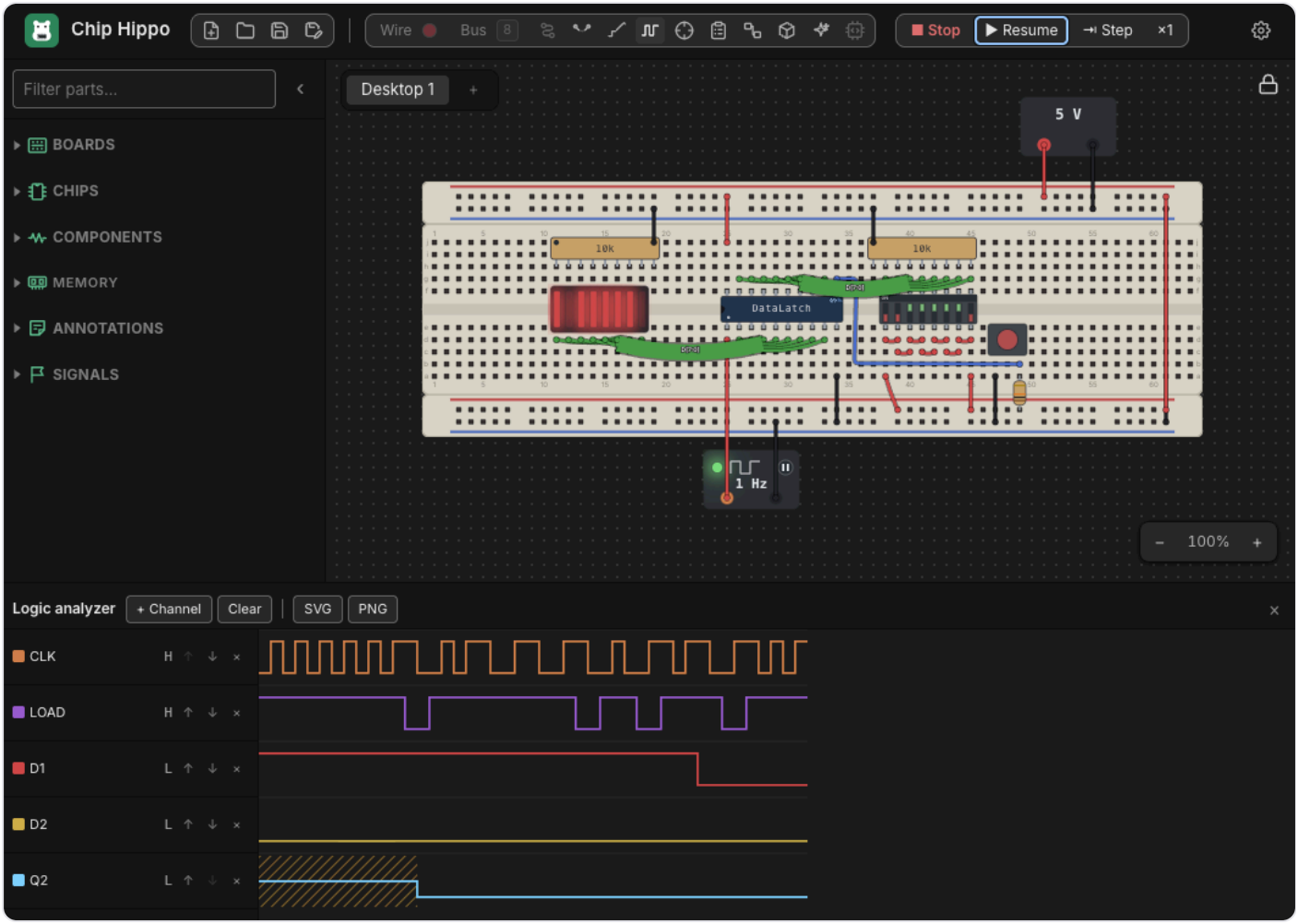
How changes are ordered

Within one pass, every chip that reacts reads its inputs **as they were when the pass began**, and its outputs reach the board when it finishes — the same rule as Verilog's non-blocking assignment. A change that arrives at a chip while it is paused mid-reaction is **held**, not dropped and not applied halfway through; when the chip finishes, it reacts to the held value only if it differs from the one it used. The board is settled only when every tab is idle and nothing is held.

The watch panel

Under the code, the **Watch** panel lists every pin and every internal `wire`, `reg` and `integer` of the chip with its current value — in Verilog's notation, with the decimal beside it when no bit is unknown (an `integer`'s can be negative) — updating as you step. A value a non-blocking assignment is about to write is shown after an arrow beside the old one. An inout pin shows its level, with what the chip **drives** onto it beside it. An array is one row giving its size, with how many of its words are about to be written; **Show words** opens its contents below it, in hex, the words about to change marked. For a multi-unit chip, pick which unit to watch.

On the desk



A placed custom chip is the design you drew. Its **Properties...** card shows the design's **part number** as its *Name* and the design's *Description*. Changing either one there changes the design, so the desk, the tray and the Chip Designer all show the change, and a change made in the designer shows on the card too. The part number follows the same rule in both places. Neither can be changed while the circuit runs.

Pin Assignment opens the same floating pin-assignments window every chip has, drawn from the design's pins and headed by its part number. It stays up to date while you change the design. Where a library chip's window has its datasheet and example-circuit buttons, a custom chip's has one button, which opens the chip in the **Chip Designer**. The right-click menu also has **Open in Chip Designer**.

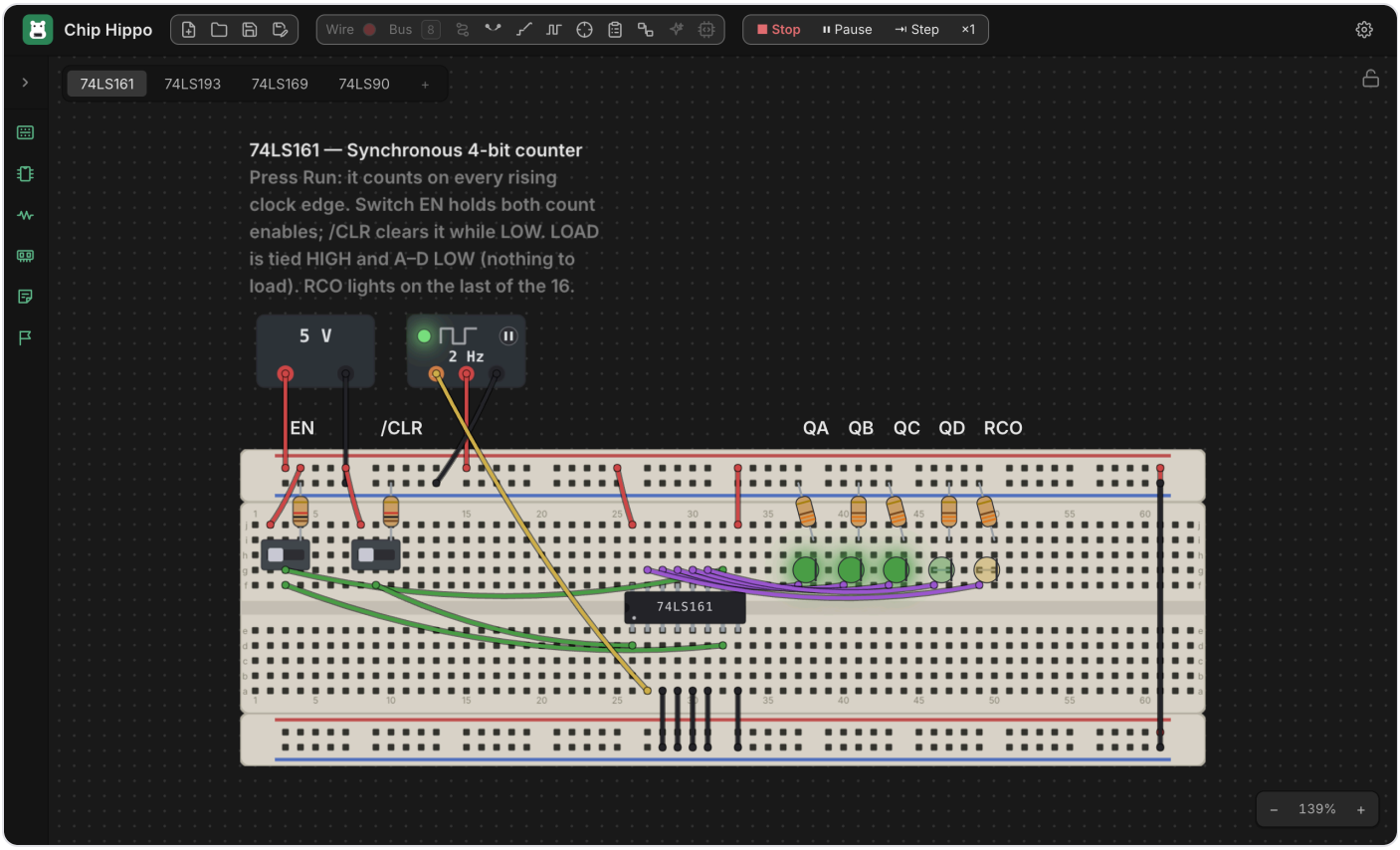
Elsewhere in the app

A custom chip appears as itself in the schematic, the 3D view, the build guide and the BOM (by its part number and description), and in a KiCad export as a DIP of its size, valued with its part number. The

Digital export leaves it out, and says so — Digital has no way to run its Verilog. The AI circuit builder does not use custom chips.

Running a Simulation

Once a circuit is wired up, **Run** hands it to the simulation engine, which traces power from every supply, resolves the electrical state of every net, and settles the circuit — LEDs light, chips report their health, and the probe and logic analyzer come alive. This page covers the transport (Run, Pause, Step, speed), what the signal levels mean, what triggers a re-settle, and what you can still interact with while the circuit runs.



Run & Stop

Press **Run** in the header toolbar — or the shortcut `Space` / `Cmd+R` (`Ctrl+R` on Windows/Linux) — to start the simulation. The button becomes **Stop**, editing locks (you can't move parts, place new ones, or lay wires while running), and the engine cold-starts: it settles the circuit once from scratch and begins driving live views from the result.

Press **Stop** (same shortcut) to freeze the simulation and return to editing. Stopping clears all run-volatile state — net levels, chip health badges, any sequential state (counter values, shift-register contents, clock phase), and **over-voltage chip damage**, so a chip you burnt out is whole again the moment you stop. Nothing that happens during a run outlives it.

`Space` is disabled while a placement, wire, or bus tool is armed (it might want the key for its own gesture) or while a text field has focus — use `Cmd/Ctrl+R` in that case, or just click **Run**.

Signal levels: H, L, Z, X

Every point in the circuit — a hole, a pin, a wire — settles to one of four levels. The probe tool and the logic analyzer both speak this same vocabulary (see Probing & Net Names):

- `H` — logic high, driven by a supply or a chip output.
- `L` — logic low, driven the same way.
- `Z` — floating: nothing is actively driving this point. What a chip input makes of that depends on its **logic family**, exactly as on a real bench:
  - a **74LS (TTL)** input pulls itself up — **a floating input reads as HIGH**, so an unconnected input doesn't just do nothing, it behaves as if tied high;
  - a **CD4000 (CMOS)** input has nothing to pull it anywhere — **a floating input reads as unknown (`X`)**, so whatever it feeds is unknown too. A gate whose other input already decides it still answers (a NOR with one input HIGH is LOW whatever the floating one does), and a counter or flip-flop whose reset or clock floats loses track of its count until a clean reset. Tie every CMOS input — spare gates included.
- `X` — unknown: two drivers disagree on a net (two chip outputs fighting, or opposing supplies wired together), a floating CMOS input, or the net never settled to a stable value at all (an oscillation — see below).

You don't need to know how the engine computes these to use the app, but recognizing `X` on a probe reading is the fastest way to spot a wiring mistake.

What re-settles the circuit

The circuit isn't a one-shot calculation — anything that could change what a net carries triggers a fresh settle while you're running:

- Clicking a **switch** or **button**.
- Turning a **PSU** on/off or changing its voltage.
- Changing a **clock**'s rate or manual/free-running mode.
- A manual clock click, or (for sequential circuits) a **Step** advance.
- A **timer** reaching its next edge — a 555 or an RC-timed CD4000 part keeps its own time and re-settles the circuit at exactly the moment its output changes, with no clock brick involved (see Timers).

Each re-settle starts from the *previous* stable state rather than from scratch — this warm start is what lets a cross-coupled latch or a counter hold its value across small changes instead of resetting every time you touch something.

The transport — Pause, Step & speed

For circuits with a **clock source**, the toolbar grows a small transport cluster next to **Run** the moment you start:

- **Pause** / **Resume** — freezes the free-running clock's edges without stopping the simulation; everything stays lit exactly as it was. Click again (the button relabels to **Resume**) to continue. To hold just ONE clock while the rest of the circuit keeps running, use the ⏸ button on that clock brick instead (see Power & Clock Sources).
- **Step** — advances the circuit by exactly one clock half-period: every free-running clock flips once, and the circuit settles around it. Stepping implies Pause, so you can single-step a counter or shift register edge by edge and watch each bit change. A manually-toggled clock steps only on its own click, not on **Step**.
- **Speed** (`×¼` / `×1` / `×4`) — click to cycle the free-running clock rate up or down. It scales every clock brick on the desk together, so a design with more than one clock keeps their relative timing. Edges are worked out in batches between the frames you see, so the rate isn't capped — but a busy circuit at 1 kHz × 4 (8,000 edges a second) can be more than your computer can work out. Then the circuit runs as fast as it can, and the button shows the speed it is actually managing beside the one you asked for, in amber: `×4 · ×1.7`. Every edge still happens, in order, at its own moment; time simply runs slower. The same readout appears when an Arduino connected through Arduino Integration can't answer as fast as the clock asks: the circuit waits for the board on every output it sends, and that waiting is the run going slower too.

Timers keep simulated time, and the transport owns it. A 555 or a CD4000 timer is told how long the circuit has been running — real seconds since **Run**, scaled by the speed. **Pause** stops that clock along with the clock bricks, so a pulse in progress is frozen where it is; **Speed** scales it, so a 1 Hz 555 blinks four times a second at `×4`; and **Step** moves it on with the clocks — by the fastest clock's half-period, or, with no clock brick on the desk, straight to the next moment a timer's output changes, so a circuit built round a 555 can be single-stepped edge by edge too.

See Power & Clock Sources for placing and configuring a clock brick, and Logic Analyzer & Timing for capturing what these edges actually do to your signals over time.

What lights up while running

Live views render entirely from the settled simulation state — nothing is computed by the views themselves:

- **LEDs** light when their anode net is `H` and their cathode net is `L` — provided something is limiting the current. An LED driven directly between a strong supply and strong ground, with no series resistor anywhere in the loop, shows a distinct **burnt** cue instead of lighting: that connection would fry a real LED, so it fries this one. A net reached through a resistor is only weakly pulled rather than strongly driven, which is what makes the resistor count. This is a physical rule, not a logical one — the levels alone would happily light it. A **CD4000 output at 5 V or below** limits the current by itself, so an LED wired straight onto one lights rather than burns — see Mixing logic families.
- **Chips** show a small health badge the moment they're powered: normal chips show nothing extra, an **underpowered** chip (a 74LS part at 3 V) gets an amber corner dot, and a chip killed by over-voltage shows **damaged** — a red X with a smoke cue. Hover the badge for the exact reason (also unpowered or reversed VCC/GND). A damaged chip stays dead for the rest of that run, and **Stop** restores it — see Power & Clock Sources.
- **Probe highlights** tint whatever net you hover by its live level. Full detail on reading and naming nets lives in Probing & Net Names — this page won't duplicate it.
- **Clock bricks** carry a small pulse lamp that tracks their own current output level in real time.
- **Timers** print their rate or pulse length under their part number — in amber when it is faster than the desk can draw — and show a warning triangle when they cannot make out their own wiring. Hover either for the full sentence; see Timers.

Interacting live

Switches, push buttons, and clock bricks stay fully interactive while running — click a **slide switch** to flip it, hold a **push button** to press it, click one position of a **DIP switch bank** to open or close it, or click a **manual clock** to pulse it by hand. Each of these is exactly the kind of input event described above: it doesn't just update its own view, it triggers a fresh settle of the whole circuit, so downstream LEDs and chips react immediately.

Everything that edits the circuit's *topology* — placing, moving, or deleting a part, board, or wire — is locked out until you Stop. Part state (a switch position, a button press, a clock's phase) is not topology, so it stays live the whole time you're running.

Oscillations & conflicts

"The circuit settles" just means: the engine keeps re-evaluating every net and chip, feeding each result back in, until nothing changes anymore. Most circuits reach that fixed point in a handful of passes, faster than you can perceive.

Two things can go wrong:

- **A conflict or short** — two chip outputs disagreeing on the same net, or opposing supplies tied together — settles immediately, but to `X`, and the app raises a warning naming the net. An analog switch (CD4066B, CD405x) can make one too: a closed channel joins the nets on its two sides, a supply crossing it counts as an ordinary output, so a rail switched onto an output is a conflict, and a channel joining + to – is reported as a short.
- **An oscillation** — a circuit that never stops changing (an unbuffered ring of inverters, for instance) can't reach a fixed point at all. Chip Hippo detects this — after a bounded number of attempts it gives up, marks the still-changing nets `X`, and flags the oscillation rather than hanging.

Either way, the affected nets read `X` on the probe and in the logic analyzer, which is your cue to go find the wiring mistake causing it.

Mixing logic families

A 74LS (TTL) chip and a CD4000 (CMOS) chip can share a board, and at 5 V they can share signals — but the two were not designed to the same voltages, and Chip Hippo says so where it matters. None of these changes a level; each is a warning naming the net, with the fix:

- **A floating CMOS input** — a CD4000 input nothing drives. Spare gates count: their datasheets say to tie every unused input.
- **A 74LS output into a CMOS input** — a 74LS HIGH is about 3.4 V, below the 3.5 V a CD4000 input needs at 5 V. It usually works on a bench, which is why it is marginal rather than wrong; a pull-up resistor from the net to VCC is the textbook fix, and clears the warning.
- **Too many TTL loads** — a standard CD4000 output can hold only one 74LS input LOW. The CD4049UB and CD4050B buffers exist for exactly this: one of their outputs drives eight.
- **Two supplies on one net** — chips on different supply voltages joined by a signal, which needs a level shifter. The CD4049UB and CD4050B are one: their inputs may be driven from a higher supply than their own, so a signal coming DOWN into them is not reported. (Ground is shared by every supply, and is never a signal.)

One difference works the other way. A standard CD4000 output is too weak to burn an LED: at 5 V it can push only about 4 mA through one, a fifth of what an LED is rated for, so an LED wired straight onto a CD4000 output with no resistor **lights** — on the desk as on a bench. The same goes for an LED fed

through a CD4066B or CD4051B/52B/53B switch at 5 V, whose on-resistance (about 470 Ω) does the limiting. Above 5 V it stops being true: at 9 V the output would pass about 15 mA and overheat the chip, and at 12–15 V the LED too, so there the LED burns as it would off a 74LS output. Two parts are built to drive hard and burn an LED at any supply: the CD4049UB/CD4050B buffers when their output is LOW, and the CD4511B's segment outputs. A resistor is still good practice.

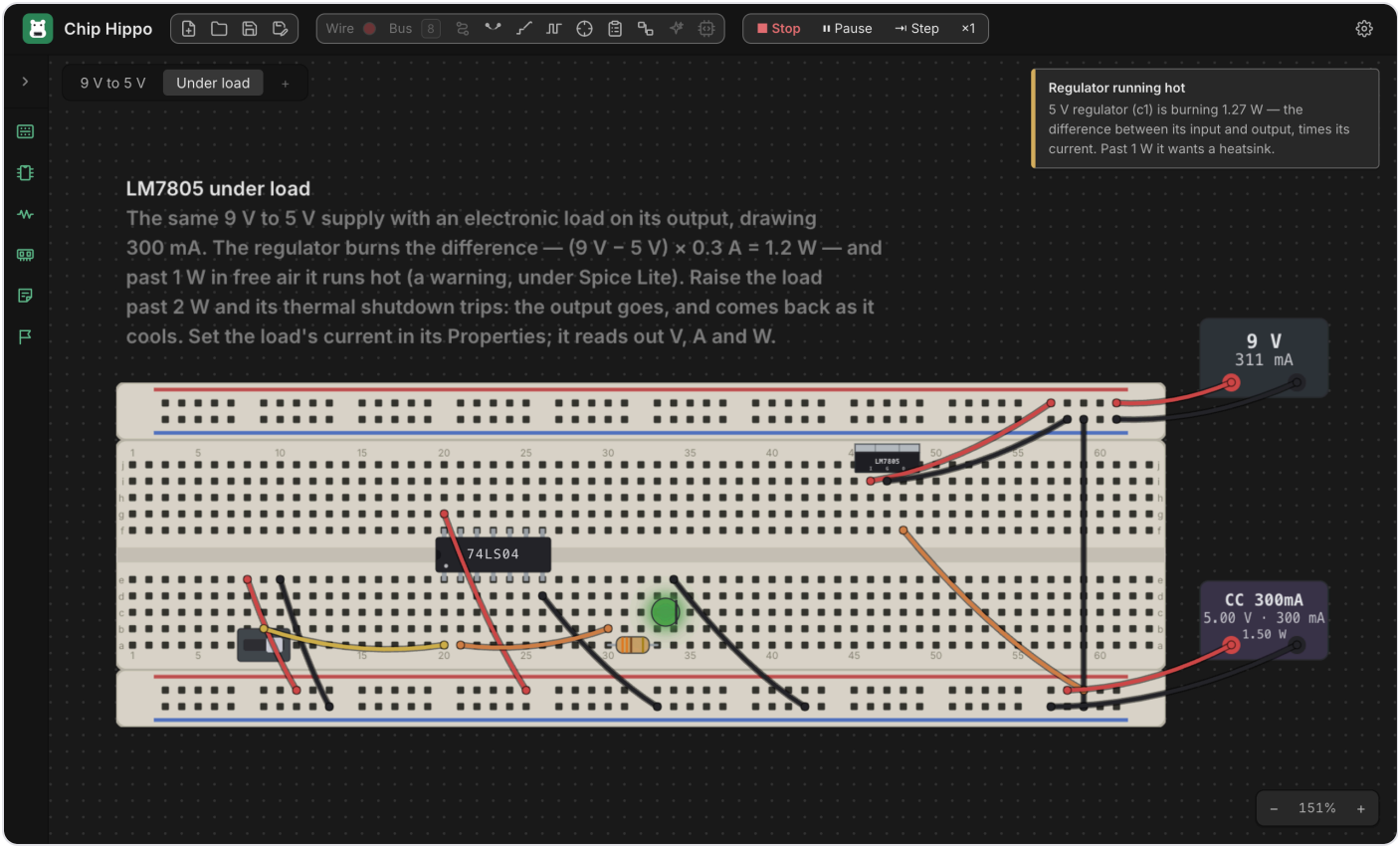
A more electrical simulation

Everything on this page is the **standard engine**: every net is a clean logic level and every change arrives instantly. Settings ▸ Spice Lite switches on a second engine with time, charging capacitors, current limits on outputs and supplies, and the resistance of the wires. See Spice Lite.

Next: Power & Clock Sources for supply voltages, the over-voltage damage rule, and clock bricks in depth, or Probing & Net Names to read exactly what's happening on any net.

Spice Lite

Spice Lite is a second, more electrical way of running a circuit. The standard engine treats every wire as a perfect logic level that arrives instantly. Spice Lite gives **every wire a voltage** and every input reads the voltage on its own pin, and it adds what a real bench has: **time**, **voltages that charge and discharge**, **currents** through every part and the limits of the outputs and power supplies that deliver them, and the **resistance of the jumper wires**. Wire something up wrong and it fails the way the real circuit would, and you can watch a capacitor charge until it crosses a gate's threshold.



It is called *Spice Lite* because it is **not SPICE**: there is no circuit-wide matrix and no manufacturer models. A plain "74LS00" still needs no manufacturer or part choosing. Everything comes from a few shared formulas (Ohm's law, the exponential charge curve) and a short table of numbers per logic family, taken from TI's datasheets, and one common set for the diodes and transistors.

Turning it on

Open **Settings ▸ Spice Lite** and set **Spice Lite** to **On**. The change takes effect the **next time you press Run**: a circuit that is already running keeps the engine it started with.

With it **Off**, nothing on this page applies: the standard engine runs exactly as described in Running a Simulation.

Every wire is a voltage

Each time the circuit settles, Spice Lite works out the **voltage on every wire** from what is on it: each chip output as the circuit it really is, each input's own current, and every resistor, LED, diode, transistor and switch between them. Then **every input reads the voltage on its own pin**:

- At or above its **HIGH threshold** (2 V for 74LS, 3.5 V for CD4000 at 5 V) it reads HIGH; at or below its **LOW threshold** (0.8 V and 1.5 V) it reads LOW. **In between it is undefined**, and the gate's output with it. A Schmitt-trigger input (74LS14, CD40106B, CD4093B) keeps what it last read until the voltage leaves its hysteresis.
- **The level a wire shows** is what the inputs on it read. Most of the time that is exactly what the standard engine shows; where the voltage says otherwise, the voltage wins.

What that changes:

- **A divider reads its voltage.** 10 kΩ to the supply and 1 kΩ to ground makes about 0.6 V at a 74LS input — LOW. The standard engine, with two resistors pulling against each other, can only say undefined. A potentiometer's wiper reads where you set it.
- **A diode has a drop.** A diode-AND (inputs on the cathodes, a pull-up on the joined anodes) gives a diode's drop, about 0.6 V, above a LOW output, which reads LOW. An LED or a diode in a chip's supply drops its voltage too.
- **A 74LS input pushes current out of its pin** while it is held LOW: about 0.23 mA at 0 V, less and less as the pin rises, down to 0.11 mA at 1.08 V, where the gate switches and lets go. Pulled down through **1 kΩ** it sits at about 0.21 V, a good LOW, and through **4.7 kΩ** at 0.7 V, still LOW; through **10 kΩ** it sits at the gate's switch, 1.09 V, in its undefined band — the reason a 74LS input is never pulled down through more than a few kilohms on a real bench. A CD4000 input draws nothing, so 10 kΩ is fine there. The example circuits pull 74LS inputs down through 1 kΩ for this reason.
- **A 74LS HIGH is about 4.3–4.5 V**, unloaded, and falls fast under a load: 3.6 V by 0.2 mA, 3.4 V at 4 mA. That is plenty for another 74LS input, and only just enough for a CD4000 input on 5 V once anything else hangs on the wire; on 12 V it is no HIGH at all. (The higher figure is a gate whose input is grounded; one whose input is held LOW by another 74LS output sits lower, because that input's 0.2 V leaves its output transistor a little on.)
- **A loaded output sags.** A CD4000 output at 5 V lighting an LED through 330 Ω sags to about 3.2 V — under its own family's HIGH threshold, so a CD4000 input on the same wire reads it as undefined. Through 1 kΩ it stays at 4.1 V. The CD4000 example circuits use 1 kΩ for their LEDs for this reason.
- **Two outputs fighting** meet where their strengths put them: a 74LS LOW gives out at about 18.5 mA, all its drive allows, so a 74LS HIGH lifts the wire to about 1.5 V — undefined to the inputs on it. The

conflict is reported, and both outputs carry the current (below).
- **Open-collector outputs** (74LS01, 74LS03, 74LS05, 74LS47, and the 74LS181's A=B) never drive HIGH: wired together with a pull-up they make a wired-AND, in both engines.
- **A wire nothing holds** — inputs alone, or a lead to an LED that is off — is floating, and reads as each family reads a floating input: HIGH for 74LS, undefined for CD4000.

The probe shows the voltage of every wire that has one beside its level (`H · 3.60 V`), and the current through the lead in the hole it is on. A floating wire shows its level alone.

Time and charging capacitors

In Spice Lite every settle pass takes one **gate delay**: about 10 ns for 74LS parts and about 125 ns for CD4000 parts at 5 V (faster at higher supply voltages, following the datasheet's 5 / 10 / 15 V figures). On a desk that mixes the two, the CMOS gates take their proper share longer to answer, and a glitch shorter than a gate's delay never makes it through that gate, as on a real chip. A circuit on its own boards and its own supply keeps its own pace: a CD4000 oscillator beside a 74LS board steps at its own gates' delay, not the 74LS parts', and keeps time as it would on a desk of its own.

A net with a **capacitor** on it becomes an **analog node**. Its voltage follows the real charge curve toward the voltage the circuit round it pulls it to, at the rate the circuit's resistance and the capacitance set (the RC time constant). The whole circuit round it counts: a chain of resistors, a switch or transistor, the resistance inside the chip output driving it, and the current an input on it draws. A CD4000 output charging a capacitor straight can only deliver its limited current (about 4 mA at 5 V), so the capacitor first charges in a **straight line** and then curves the rest of the way.

- **Each input reads the node through its own threshold**: a 74LS input at 1.4 V, a CD4000 input at half its supply. An RC delay feeding both families switches the 74LS gate first and the CMOS gate later.
- **A crossing happens once.** When the voltage passes an input's threshold, that input sees one clean change. The voltage keeps climbing, but nothing happens again until it falls back through the threshold.
- **A Schmitt-trigger input has two thresholds.** The 74LS14, CD40106B and CD4093B switch HIGH at a higher voltage than they switch back LOW (the 74LS14 at 1.6 V and 0.8 V). That gap is what makes the classic one-gate RC oscillator work: a resistor from the output back to the input and a capacitor from the input to ground swing between the two thresholds at a steady rate. Built round an ordinary inverter, the same circuit has nothing to swing between and is reported as oscillating.
- **A 74LS input on a node pushes its own current into it** below 1.1 V, so an RC into a 74LS input starts faster than its R·C alone, and a 74LS14 oscillator needs a resistor of a kilohm or so to swing at all (through 10 kΩ its own input holds the capacitor above its lower threshold).
- **A chip output charges a node only as far as its HIGH.** A 74LS output takes it to about 4.5 V, the last of it slowly; on a desk with a 12 V supply, a CD4000 gate on 12 V, which switches at 6 V, never

sees it change: the reason a 5 V part can't drive 12 V CMOS directly.

- **A node that will never get there is left alone.** If the resistors only pull the node to 0.9 V, a 1.4 V threshold is never crossed, and the circuit doesn't wait for it.
- **Power-up is real.** An empty capacitor from a node to ground starts that node at 0 V. A capacitor to the supply starts it at the supply, which is how a power-on reset works.
- **A capacitor keeps its charge.** If a switch or button ties a node straight to a rail and is then released, the node starts from the voltage the rail left on its capacitor and drains from there, as a debounce or hold-up circuit does.
- **A capacitor wired to nothing is ignored.** One whose other lead sits in an empty column, or off the board, holds no charge and does not slow the node down.
- **A step carries through a capacitor.** A capacitor's charge can't change in an instant, so when the far side of one jumps, the node on this side jumps with it and then settles back through its resistors. A button feeding a 555's TRIG through a capacitor triggers it, and the two-inverter RC oscillators (the CD4060B's and CD4541B's) work the way their datasheets draw them, the junction kicked past the supply each time.
- **Capacitors that see each other move together.** Two or more capacitors joined through resistors (an RC ladder, a filter) or a capacitor between two charging nodes are worked out as one circuit, exactly: a high-pass passes its pulse, a ladder's last node lags its first by the right amount, and the answer is the same however often the desk is redrawn.
- **The analyzer draws voltages.** A channel on a logic analyzer is drawn as its wire's voltage, so a charging capacitor curves up toward its supply instead of stepping from LOW to HIGH, and a 74LS signal swings between about 0.2 V and 4.4 V. The channel's value reads in volts.

Slow curves cost nothing to run: Spice Lite calculates *when* a node will cross a threshold rather than stepping toward it, so a 10-second RC delay runs as cheaply as a microsecond one. If the app falls behind (a busy computer, or its window in the background), it catches up on the crossings it missed, in order.

An oscillation faster than the desk can show (anything above 1 kHz, the fastest clock on offer) is recognised once it has gone round twice the same way. From then on it is drawn at 1 kHz with its duty cycle kept, as the standard engine draws a fast 555, while time itself runs at its true rate: a timer's readout shows the **true** frequency, and a CD4060B or CD4541B counting its own oscillator counts every real cycle. Anything that changes the circuit (a reset, an edited value, a supply moving) ends the fast drawing, and the oscillator is worked out afresh. An RC round an ordinary (non-Schmitt) inverter has no second threshold to swing to: it chatters at its one threshold and is reported as oscillating.

Timers

In Spice Lite a timer is the circuit inside it, not a formula. Each one is built from its datasheet's own block diagram: comparators reading the voltages on its pins against their references, the package's own

resistors, a discharge transistor, and the flip-flops behind them. Nothing in it computes a period. The period is whatever your resistor and capacitor actually do, so the circuits the formulas never covered work too.

- **The 555**: three 5 kΩ resistors from VCC to ground inside it set its two trip points at ⅔ and ⅓ of the supply (CONT is the top tap), the threshold and trigger comparators set and reset its latch, and DISCH is a real open-collector transistor. The familiar 0.693 is really ln 2 and 1.1 is ln 3, so the usual circuits run within a percent or two of the formula; the capacitor starts **empty**, so an astable's **first** HIGH is about 1.6 times as long as the rest. A voltage on CONT moves both trip points, RA built from two resistors in series works, a trigger through a capacitor fires it, RESET reads its own 0.7 V threshold, and a DISCH asked to sink more than its 200 mA rating says so.
- **The CD4047B**: R and C are driven in opposition from its oscillator's input on RC COMMON, which switches at half the supply, exactly the swing its datasheet's own design formulas describe: 4.40·RC astable, 2.48·RC for a one-shot, a retrigger running it on by whole 2.2·RC periods.
- **The CD4098B, CD4528B and CD4538B**: a trigger turns on the transistor that empties Cx, and the pulse ends when Rx has charged it back up to the upper reference. Their datasheets give only the formula (½·RC, 0.2·RC·ln VDD, RC), so the references are worked back from it (the lower at 5 % of the supply), and at 100 kΩ and 100 nF every width comes out within about a percent of the sheet (a CD4528B at 5 V runs 1.2 % long). A small Rx with a large Cx strays further, because the discharge transistor's own resistance starts to count: a CD4538B at 10 kΩ and 1 µF runs 2.3 % long. A retrigger empties the capacitor again, and RESET holds it empty. The discharge transistor is held to its datasheet's **least Rx** — 5 kΩ for the CD4098B, 4 kΩ for the CD4538B (the CD14538B sheet), and 5 kΩ for the CD4528B (the smallest its sheet tests at; it states no minimum). What it carries while it **holds** the capacitor — RESET LOW against Rx — is checked against what that least Rx would let through (the transistor's own resistance included, so an Rx exactly at the minimum is fine): a smaller Rx gives a **Timing resistor too small** warning, and past 100 mW in the transistor the brown smoke. The burst of current as it empties Cx at a trigger is not checked: real silicon does the same, and the datasheets bound it by the largest Cx instead (100 µF for the CD4098B and CD4538B, which a larger capacitor is warned about).
- **The CD4060B and CD4541B**: Rx, Cx and Rs on one junction, as in their datasheets. The junction is kicked past the supply each time an output switches, and Rs keeps the input's protection diodes off it, which is why the datasheets ask for it: leave Rs out and the period is visibly shorter. Both run at about 2.23·Rx·Cx with Rs at twice Rx, the 4060's 2.2 and the 4541's 2.3 on their sheets (so the 4060 runs about 1.5 % slow of its formula and the 4541 about 3 % fast of its own). A CD4541B with AUTO RESET enabled (pin 5 wired LOW) draws the extra supply current its datasheet gives for it — 7 µA at 5 V, 30 µA at 10 V, 80 µA at 15 V. The formula leaves out the chip's own outputs, each about 400 Ω at 5 V (and more as they carry more), which sit in series with Rx and Cx: harmless with Rx at 10 kΩ and up, but at Rx = 1 kΩ a CD4060B's period comes out about 29 % longer than 2.2·Rx·Cx.

A running timer's readout shows what it **measured**: the frequency between its last two rising edges, or the length of its last pulse. The Properties card's Timing row still gives the datasheet's figure, worked out from the values you set. Under Spice Lite a timer never complains that it doesn't recognise its wiring: it does whatever its pins make it do. It still says when a pin it needs is left unwired, or a terminal that must be grounded isn't (a CD4528B's T1): it cannot time at all then. The probe shows the real voltage on every timing pin (THRES and TRIG, CONT, RX CX, RC COMMON, an oscillator's junction).

The CD4000 timers' outputs, and the 555's, are as strong as their datasheets say, and are judged by the same current limits as any other output.

Current and fan-out

Every input wired to an output draws a little current, as its own circuit does: a 74LS input pushes about 0.2 mA out of a pin held LOW (it is a resistor and a diode to its supply inside), and nothing once its pin is past about 1.1 V; a CMOS input, or a memory's or processor's, draws nothing at all. So an output with many inputs on it is pulled away from its level by them, and the voltage it ends up at is the one every input on it reads.

- **A Brownout warning** names an output whose load holds its net where an input on it no longer reads the level the output is driving — though it would at the output's own unloaded voltage. It says the pin, the level and the voltage the net is held at. Too many inputs is one cause; a resistor to ground that is too small for a HIGH (a 74LS HIGH into 100 Ω sits at about 1.8 V) is another. A brownout never damages anything: it is a circuit that does not work, not a part in danger.
- An output that never reaches an input's threshold even unloaded — a 74LS HIGH's 4.5 V into a CMOS input on 12 V — is a level mismatch, not a brownout: that input simply reads it as unknown.

A CD4000 output at 5 V holds one 74LS input LOW at about 0.1 V, two at 0.2 V, five at 0.4 V and six at 0.45 V — all LOW to them, typically, though the datasheet guarantees only one (which is what the standard engine's fan-out warning goes by; under Spice Lite the voltage replaces it). The CD4049UB and CD4050B buffers, whose LOW is about five times stronger, are what belongs between the families in a design that has to work with every part off the shelf.

Every part of a family shares that family's numbers: a 74LS244 bus driver drives as every other 74LS output does, so a circuit behaves the same whichever maker's 74LS parts you picture on the bench. An output that is switched off (a tri-state output not enabled) drives nothing and is no part of a brownout.

Shorts and overloaded pins

What damages an output is the current through it, which Spice Lite knows for every output pin, whatever it is driving, and holds each to what its family is made for:

Family	Warning	Brown smoke
74LS (and memories, CPUs)	over 20 mA through the pin	over 100 mA
CD4000	over 50 mW in its output	over 100 mW

Past the smoke limit the chip is drawn burnt with brown smoke rather than grey, drives nothing for the rest of the run, and a **Brown smoke!** notice says why.

A 74LS HIGH shorted to ground carries about 32 mA (a warning); a LOW shorted to the supply about 150 mA (brown smoke). Two 74LS outputs fighting each other carry about 19.5 mA, just under the warning. An ordinary CD4000 output at 5 V cannot pass enough current to hurt itself; at 15 V a short lets out the smoke. The CMOS parts built to drive more — the CD4049UB/CD4050B buffers' LOW, the CD4511B's segment drivers — are held to the same 100 mW per output transistor their datasheets give every B-series output, so a buffer's LOW shorted to a 15 V supply smokes too. The NE555's output is held to its own sheet's 200 mA.

An **analog switch** channel (CD4066B, CD4051B/52B/53B) is rated for 10 mA: past it a **Switch overloaded** warning, and past 25 mA brown smoke. A CD4066B channel switched on straight across a 5 V supply carries 10.6 mA.

A **transistor** is held to the common limits of its kind:

Transistor	Warning	Smoke
NPN / PNP (TO-92)	over 200 mA, or 312 mW	over 600 mA, or 625 mW
MOSFET, TO-92	over 200 mW	over 400 mW
MOSFET, TO-220	over 1 W (no heatsink)	over 2 W

A transistor past its smoke limit is said with a warning that a real one would have failed, but it carries on conducting: it has no supply pins, so nothing on the desk can be switched off for it.

Inputs have limits too:

- **A 74LS input over 7 V** lets out the smoke — a CD4000 output on 12 V wired into one, for instance.
- **A CD4000 input held beyond its own supply** (more than 0.5 V above its VDD, or below ground) conducts through its protection diode, which is warned about, and lets out the smoke past 10 mA. The CD4049UB and CD4050B have no diode to their supply, which is what lets them take a higher voltage than they run on: they are the level shifters. A **memory's, processor's or peripheral's** input has the same two diodes and the same limits, and so does an analog switch's control pin and a CD4007UB's gate.
- **A CD4000 input left in its undefined band** draws about 0.5 mA from its own supply while it is there, as both of its input transistors are part-way on. (A CD4007UB's gate is a bare transistor's, and draws

nothing of its own: what its pair conducts is the current.)

A **short through a transistor or an analog switch** — one switched on with its two ends on opposite supplies — is reported as a **Short circuit** only when the current really flowing through it is 100 mA or more, or the supply it is on has hit its current limit. The standard engine reports every such join; under Spice Lite a transistor whose base is fed through 10 MΩ, passing microamps, is no short. A short with no part in it (two supplies wired together) is always reported.

Stop restores every chip that let out its smoke.

Power supplies

A **power supply** brick gains a **Current limit** in its Properties (100 mA to 5 A; 1 A unless you change it). While Spice Lite is running, the brick shows the **current being drawn** under its voltage.

- Below its limit, a supply holds its set voltage.
- **Past its limit, the voltage droops** in proportion. Asking 200 mA of a 100 mA supply gives you about half the voltage. The readout turns amber and shows both the voltage and the current.
- **A supply shorted + to –** (its + wired to a ground) is held at its limit and droops to a fraction of a volt, with a **Short circuit** notice; every chip on it is underpowered.
- **A chip whose supply droops below its rating is underpowered** (an amber dot; hover it for the voltage it sees), exactly as if you had set the supply too low. Its load still counts while it is: a chip whose own outputs drag its supply down stays down until you change something, rather than flickering on and off.

The supply's load is every chip's own supply current (1.6 mA for a 74LS part, almost nothing for CD4000), plus everything the voltages make flow: the current through every resistor, LED, diode, transistor and switch from the supply, and out of every chip output into whatever it drives (see LEDs, below): a red LED behind a 330 Ω resistor on 5 V draws about 9.4 mA, and one wired straight across the supply draws everything it can until it burns. Current a chip sinks is routed back through its own ground pin.

Chips powered off the rails

A chip whose supply pins are not wired straight to the rails — fed through a resistor, a diode, a transistor, or from another chip's output — runs at the **voltage that actually reaches its pins**. Its own supply current is a load on whatever feeds it, and so is everything its outputs drive: an LED lit from one of its outputs is fed through its VCC pin, so a 74LS04 fed through 47 Ω sags to about 4.6 V lighting one — underpowered, and still lighting it. A 74LS chip fed through a diode gets about 4.4 V and is underpowered (it needs 4.75 V); a CD4000 chip, which runs from 3 V, works on the same feed, and its

HIGH outputs are the 4.4 V it is running on. Its outputs and inputs are measured from its own pins: a chip whose ground is raised drives its LOW at that ground, and a chip across two supplies' rails (VDD on 12 V, VSS on the 5 V rail) runs on the 7 V between them, its LOW at 5 V and its HIGH at 12 V. The standard engine sees only a supply pin that is not on a supply.

Diodes and transistors

Every diode is one common kind, whatever part number you give it. A transistor is the **Grade** its **Properties...** card picks — a card that offers it only while Spice Lite is on:

Type	Grades (the part each one simulates as)
NPN	**Small signal** (2N3904) · **General purpose** (2N2222A) · **Darlington** (TIP120) · **Power** (TIP31C)
PNP	**Small signal** (2N3906) · **General purpose** (2N2907A) · **Darlington** (TIP125) · **Power** (TIP32C)
N-channel MOSFET	**Logic level** (2N7000) · **Logic-level power** (IRLZ44N) · **Power** (IRF540N)
P-channel MOSFET	**Logic level** (BS250) · **Power** (IRF9540N)

A transistor in a **TO-92** defaults to the first of its type's grades, one in a **TO-220** to **Power**, until you pick one; picking a part from its part-number list picks its grade too. A grade you have picked stays when you change the package. Each grade is that one part's datasheet figures.

The last entry in the list is **Custom...**. It opens the grade's figures under the list, filled in from the grade that was showing — a bipolar transistor's **gain hFE** and **base turn-on VBE** (both at a tenth of its rated current, VCE 5 V), its **breakdown VCEO** and its **max current IC**; a MOSFET's **threshold VGS(th)**, its **on-resistance RDS(on)** (at VGS 10 V), its **breakdown V(BR)DSS** and its **max current ID**. Type any of them the way any value is typed (`1A5`, `300m`, `2.2`); the transistor then simulates as the grade it started from with those figures put back exactly. A P-type's figures are the same magnitudes. Picking a grade again puts its own figures back. A set of figures that are a grade's own simply *is* that grade, so a card reopened on one shows the grade's name, never Custom....

- A **diode** follows a small-signal silicon diode's curve (the 1N4148's): about 0.5 V at 0.1 mA, 0.62 V at 1 mA, 0.75 V at 10 mA and 0.87 V at 50 mA. A **Zener** also conducts backwards at its Zener voltage, so it clamps and regulates. One that carries more than its junction can take burns, like an LED.
- A **bipolar transistor** (NPN, PNP) has **gain**: its base conducts from about 0.6 V, and its collector carries its gain (a few hundred for a small part, a few thousand for a Darlington) times the base current — no more than the circuit lets through, in which case it is **saturated**, a tenth of a volt or less

across it (a Darlington, whose output transistor is driven by another, never below about 0.7 V). Its gain falls at high currents as its datasheet's does. A base fed through 10 MΩ barely turns it on, where the standard engine treats any HIGH base as a closed switch.

- A **MOSFET** turns on from its **gate threshold**, measured from its source — about 2 V for a logic-level part, 3.6 V for the IRF540N, which a 5 V logic output only just turns fully on and a 3.3 V one not at all — its current rising with the square of the gate drive until its channel is fully on, down to its datasheet's on-resistance. Its gate draws nothing and **keeps the voltage it was last driven to** when left floating; the transistor's lamp rings amber while it does. The CD4007UB's six transistors are its family's own output transistors.
- An **analog switch** (CD4066B, CD4051B/52B/53B) closes and opens by the voltage on its control pin, read through its thresholds like any input, and its channel passes whatever voltage is on it through its on-resistance.

Each transistor is held to its own part's ratings: carrying more current than its part is made for, or dissipating more than its package can (a TO-92 about 0.6 W, a TO-220 with no heatsink about 2 W), warns that a real one would fail.

A transistor's lamp on the desk lights while it conducts.

Each transistor also has what a real one has to survive an inductor (below): a MOSFET's **body diode**, which conducts backwards from drain to source like any diode, and a **breakdown** voltage past which the transistor conducts anyway — 40 V across a bipolar transistor's collector and emitter, 60 V across a MOSFET. Neither is reached in an ordinary logic circuit.

Inductors

An inductor with an **Inductance** is a real inductor under Spice Lite (with none it is a wire, as it is on the standard engine). Its **current** cannot change in an instant: switched on, it rises along its time constant, the inductance over the resistance in its path; switched off, it has to go somewhere.

- **Its winding has resistance.** Under Spice Lite an inductor's **Properties...** card gains a **Winding** choice — **Lowest**, **Typical** (the default), **Higher** or **Highest** — each showing the resistance it gives the part as it stands (for example, **Typical — 0.74Ω** for a 1 mH Coil over two holes). The figures come from real ranges of each style and size (a Bourns drum series for the Can, a Bourns toroid series for the Coil), and they move with the Inductance: a bigger value is wound with more, finer wire. The card offers it only while Spice Lite is on; turning Spice Lite off hides it and keeps your choice.
- **Switched off, the current finds a way.** A **flyback diode** across a relay coil carries it round, and it dies away in milliseconds; so does a MOSFET's body diode, or a CMOS output's protection diodes, when one of those is in its path. With **nothing** to carry it, the voltage across the coil rises until the transistor switching it **breaks down**, and an **Inductive kick** warning names the transistor, the voltage

it reached and the energy the coil dumped into it. A real transistor may survive a few of those; it is the reason relay coils have diodes across them.
- **The probe** reads the current through an inductor's lead and the voltage on each side of it, and a coil and a capacitor together **ring** — an LC circuit is solved exactly, its oscillation included.

Clock waves

A clock brick's **Wave type** (its Properties card) turns its square into a **triangle**, a **trapezoid**, a **sawtooth** either way — a *ramp up* climbs and drops straight back, a *ramp down* drops and climbs straight back — or a **sine**, under Spice Lite only. Each swings from 0 V to the clock's supply over one period of its rate, starting at its low point, and drives its `out` as an ideal source, sourcing out of its supply and sinking into its ground. Its **Amplitude** (peak to peak) and **Offset** (its centre) move that swing — a 1 V sine about 2.5 V for an op-amp to amplify; blank, they are the whole supply about half of it. Set past a rail, the wave is **clipped** flat on it, exactly, as a generator running from those rails would be. (A **PWM** clock is not a wave but a level, as the square is: it runs in both engines — see Power & Clock Sources.)

The wave keeps its square's timing: its LOW half is the first half of the cycle and its HIGH half the second, so the triangle peaks and the sine crests at the LOW → HIGH edge, and a sawtooth drops back at the HIGH → LOW one. A trapezoid sits at 0 V for the first 30 % of the cycle, climbs over the next 20 % to reach its supply at the LOW → HIGH edge, holds there for 30 % and falls over the last 20 %, back at 0 V by the HIGH → LOW edge. **Step** moves it half a period, **Pause** stops it where it is, and a clock held by its own ⏸ holds its voltage, picking up from there when it is let go.

It is solved like a capacitor's curve, never stepped: an RC filter fed a sine gives the filter's own gain and phase, a capacitor carries a sawtooth's drop straight through, and an input reads the wave where it crosses its threshold — a Schmitt-trigger input (a 74LS14, a CD40106B) squares a triangle or a sine at its two thresholds, an LM358 follows it to 1.5 V short of its supply. The clock's lamp lights while its output is above half its supply, and the logic analyzer draws the wave, sixteen points a period (at most one every 2 ms) — while it has a channel to record. Each of those points is a whole tick of the desk, so with no channels the wave asks only to be redrawn as often as any other moving voltage.

Bench parts

The regulators, the relay, the electronic load, the LM358, the ULN2003A and the optocouplers are each what they are under Spice Lite, in the same voltage solve as everything else (one common set of figures per kind, from the TI, Vishay and Sharp datasheets):

- **A linear regulator** holds its output at its voltage — a 78xx's above its GND, an LM317's 1.25 V above ADJ — as long as its input stays its **dropout** above that (2 V, 1.7 V for the LM317); below it the output follows the input down, with a **Regulator in dropout** warning. It gives up to **1.5 A** and droops past it

(**current-limiting**), and it never sinks. It burns the difference between its input and its output as heat: past **1 W** in free air it warns that it is running hot; past **2 W** its **thermal shutdown** trips, its output goes, and a second later it tries again — and trips again, until you lighten its load or lower its input. Its hover shows what it is delivering.

- **The LM358** is an amplifier: its output is its open-loop gain (100 000) times the difference between its inputs, about the middle of its swing, and that swing runs from a few millivolts above ground to **1.5 V short of its supply**, behind 20 Ω, sourcing up to 30 mA and sinking 20 mA. With feedback it settles where its datasheet says: two 10 kΩ give a gain of 2. Its inputs draw nothing.
- **The ULN2003A** is seven Darlingtons, each behind its 2.7 kΩ — about 1 V from output to E at a few hundred milliamps, as its sheet's typical figures — with each output's clamp diode to COM. Left with COM open, a coil it switches off kicks its output into breakdown at 50 V. Past 500 mA an output warns.
- **An optocoupler**'s LED is an infrared LED (about 1.2 V); its phototransistor carries the LED's current times its **CTR**, never more — so a load wanting more than that gets less, and the transistor does not saturate. The CTR you set holds from 5 mA through the LED up; below that it falls, as both makers' curves do: to about half at 1 mA and a third at 0.5 mA. So a coupler fed only a milliamp or two needs more CTR, or a larger load resistor, than its setting suggests. Past 60 mA through its LED, or 50 mA through its transistor, it warns.
- **The relay**'s coil is an **inductor** (0.14 H behind its 70 Ω at 5 V, 0.8 H behind 400 Ω at 12 V — about 2 ms to build its current), and its contacts follow that **current**: they pull in once it reaches 75 % of its rating and let go below 10 %. So a relay switched on closes a few milliseconds later, and one with a flyback diode lets go a little late, while the diode keeps its current going. Without one, switching it off kicks the transistor driving it into breakdown (see Inductors). Its contacts are 0.1 Ω, and past 10 A they warn.
- **The electronic load** draws its set current from whatever it is across (**CC**) — until the voltage is too low for it to (under about half a volt at an amp), when its readout turns amber and reads **UNREG** — or behaves as its resistance (**CR**). It never sources. Its brick shows the volts, amps and watts while the circuit runs, and past **25 W** it warns.

LEDs

With Spice Lite on, an LED carries the current its circuit really pushes through it, worked out from its colour's datasheet, and what that current does to it is what it would do on a bench. The standard engine only asks whether something limits the current; Spice Lite asks how many milliamps.

Colour	Datasheet	Forward voltage	Rated	Burns at
Red	Kingbright WP7113ID	1.9 V at 10 mA	30 mA	72 mA
Yellow	Kingbright WP7113YD	1.95 V at 10 mA	30 mA	62 mA
Green	Kingbright WP7113GD	2.0 V at 10 mA	25 mA	54 mA
Blue	Kingbright WP7113QBC/D	3.3 V at 20 mA	30 mA	39 mA
White	Kingbright WP7113QWC/D	3.3 V at 20 mA	30 mA	41 mA

These are ordinary 5 mm LEDs. Every segment of a display and every bar of a bar graph is taken as an LED of its colour.

- **Each colour follows its datasheet's curve.** The current rises steeply from a toe — a red LED carries about 0.4 mA at 1.6 V, 1.5 mA at 1.7 V and 10 mA at 1.9 V; a blue or white one about 1 mA at 2.6 V and 20 mA at 3.3 V — and then more slowly, as its own resistance takes over. A blue LED behind 100 Ω on a 3 V supply glows dimly (2.7 mA); on 5 V it is bright (18 mA).
- **Brightness follows the current.** An LED is drawn at full brightness at the current its datasheet quotes its brightness at (10 mA, or 20 mA for blue and white), dimmer below it, and with a wider glow above it. Below 50 µA it is dark.
- **Past its rating it is overdriven.** It lights brightly and a warning names the current against its rating: a red LED behind 220 Ω on 9 V takes 31 mA. A real one would dim and fail early.
- **Past its maximum junction temperature it burns.** The datasheet gives how hot the junction runs per watt; once the heat the current puts in would take the junction past its maximum, the LED burns out (a red cross and smoke) and stops conducting for the rest of the run. Segments sharing one resistor with it then share its current too. Stopping the simulation restores it. The package taking a few seconds to warm up is not modelled: it burns at once.
- **Backwards, it is rated for 5 V.** More than that across it the wrong way gives a warning.
- **The probe reads its current.** Point the probe at an LED's leg — or at the resistor beside it, or the chip output driving it — to see the milliamps through that lead. No part shows a current on the desk; only a power supply shows its draw.

A chip output is not a perfect source. Each output is modelled as its datasheet says it behaves:

- A **74LS** output follows the curves of the gate's own transistors. Driving HIGH it is about 4.5 V with nothing on it, drops to 3.6 V by a fifth of a milliamp, and then behaves like 120 Ω, so an LED wired straight from it to ground lights at about 16 mA and survives. Driving LOW it sits at 0.2 V and has little in the way, so an LED from the supply straight into it takes nearly 100 mA and burns.
- A **CD4000** output is a small transistor that can only pass so much: about 4 mA at 5 V, 16 mA at 10 V and 28 mA at 15 V. Lightly loaded it is about 400 Ω at 5 V; the more it carries the more it gives way, along the curve a transistor follows, until it can pass no more. At 5 V that is a resistor in all but name.

- The **NE555**'s output is good for 200 mA, and burns an LED wired straight to it. The **CD4511B**'s segment outputs, and the **CD4049UB** and **CD4050B**'s sinks, are stronger than an ordinary CD4000 output, as their datasheets say.

Character LCDs

The LCD modules' **backlight** is an LED of the module's colour between `A` and `K`, behind the 100 Ω resistor the common 1602A and 2004A boards carry for it. Wired to the supply it lights and draws its current (about 18 mA for a blue module on 5 V, 28 mA for a green one); unwired, the panel is dark. Its rating is the module maker's, so it never warns or burns.

`V0` sets the **contrast**. The glass is driven by the voltage between `VDD` and `V0`: full contrast from 3 V (the controller's minimum) up, fading below that, and blank with `V0` at the supply or left unconnected. The usual 10 kΩ potentiometer from the supply to ground, its wiper on `V0`, works as it does on a bench. With Spice Lite off, the backlight and contrast are cosmetic as before.

Wire resistance

Every jumper wire is treated as **24 AWG copper** at its real length, about 0.084 Ω per metre. Current from a supply is routed through the wiring to each chip, and a chip loses the voltage dropped across the wires its current shares with others. A chip at the end of a long chain of jumpers sees a little less than the chip next to the supply.

The effect is honest but small: a 10 cm jumper is under 0.01 Ω, so it takes a heavy load to see it. Drops under a millivolt are ignored.

Decoupling capacitors

Each time a chip's output switches, it draws a brief extra spike of current to charge what it drives. When many outputs switch at once, those spikes add up, and on a supply that is already close to its limit they can tip it over. Spice Lite warns with **Supply spike**.

A capacitor wired **across a chip's supply** (VCC to ground, the usual 100 nF beside every chip) supplies those spikes itself, and the supply never sees them. Spice Lite does not know where along the rails a capacitor sits or what its value is: one capacitor anywhere across the rails a chip's supply pins are on decouples that chip. A capacitor from a signal net to ground is a timing capacitor, not a decoupling one, and doesn't count.

Switch bounce

A real push button or slide switch doesn't close once. Its contact touches, springs off and touches again, three to six times over a couple of milliseconds, before it rests closed. A counter clocked straight from a button counts every touch, which is why real circuits **debounce** their switches: an RC into a Schmitt-trigger input, or a slide switch into an S-R latch.

With **Switch bounce** on (Settings ▸ Spice Lite, Off by default), Spice Lite's switches do the same. Every slide switch, push button (momentary or toggle) and DIP switch bounces as its contacts close, for up to 3 ms, the same way each time you run the same presses. A contact that opens opens cleanly, and a slide switch's old side never touches again while its new one bounces, which is exactly what makes the S-R latch debouncer work. The logic analyzer shows every bounce. With it off, or on the standard engine, every switch closes cleanly.

The Spice Lite settings

Settings ▸ Spice Lite holds:

- **Spice Lite**: On or Off.
- **Settle gap**: how close a capacitor's voltage has to get to its final value before the probe and the analyzer stop redrawing it (an input watching it is told the moment it crosses, whatever this says). 1 % is about five time constants. It never holds the circuit up.
- **Switch bounce**: On or Off (Off by default) — see Switch bounce.
- A **TTL | CMOS** strip showing the families your tray shows (and any family the open project uses). Under each are its datasheet source, a **Reset to defaults** button, and an **Advanced** section with every number Spice Lite uses for that family:
  - **Gate delay**: how long an output takes to follow its inputs.
  - **Output source and sink current**: the strength of the family's output — the current it delivers at its datasheet's test voltage. Twice the figure is an output twice as strong (half the resistance behind it, and twice the current a CMOS output saturates at).
  - **Input current, LOW**: what a 74LS input pushes out of a pin held LOW (the figure is the datasheet's maximum, and the input pushes about half of it at 0.4 V, typically). A CMOS input's is too small to measure, and adds nothing unless you set it to something that is.
  - **Supply current per chip**, **input LOW and HIGH thresholds** and the **switching load** (what each output charges when it switches).

  CMOS values are stated at 5 V: the gate delay and the input thresholds follow each chip's supply, and the currents and the switching load are used as their 5 V figures at any supply. A part in no family

(the memories, processors and peripherals) is no family's: its outputs are a common rail-to-rail stage, and its inputs read at the 74LS thresholds.

Every change, the numbers included, applies at the **next Run**: a circuit that is running keeps the settings it started with. Your changes apply to every project on this computer. Each field has a range — hover over it to see it — wide enough for any logic part (a gate delay of up to 10 µs, a threshold of up to 5 V): a number outside it is set to the nearest end of it, so 20,000 ns becomes 10,000 ns and 0 becomes the smallest value allowed. A field that won't read at all, or a LOW threshold at or above the HIGH one, turns red, keeps the previous value, and says why underneath. A very short gate delay is taken as no more than about 64 times faster than the slowest gate on the desk, so a run never crawls.

What stays the same

- **Exports**: the KiCad and Digital exports ignore Spice Lite entirely and use your parts' real values.
- **The AI builder** and the **example circuits** are proven on the standard engine.
- **The standard engine** is untouched: with Spice Lite off, every circuit behaves exactly as before.

One thing the two engines are allowed to answer differently: **open-collector outputs tied together** (a wired-AND, such as two 74LS05 outputs sharing a pull-up). When one of them sinks and the other's input is undecided, the standard engine — which has no notion of an output that can only pull LOW — shows the net as unknown and warns of two outputs fighting. Spice Lite shows it LOW with no warning, which is what the real circuit does: the undecided output either sinks as well or lets go, and the LOW one holds the net either way. It does so only when open collectors are all that meet there and the LOW really holds the net down: an ordinary output reaching it through a switch channel or a diode, or a pull-up too stiff for the LOW, is still a fight, and Spice Lite says so too.

How close it is to SPICE

Spice Lite is checked against **ngspice**, a full circuit simulator, on the same circuits: each one is built once and run through both, and the answers are compared (the tests do it on every build). Within 2 % is graded **A**, within 10 % **B**.

What it models	Grade	How close
Resistor networks, dividers and potentiometers	A	exact
One capacitor charging, Schmitt-trigger RC oscillators	A	within 0.03 %
Several capacitors together (RC ladders, filters)	A	within 0.01 %
Two-gate RC oscillators, with or without their series resistor	A	within 0.8 %
The 555	A	within 0.4 %, from 1 kΩ to 1 MΩ
The CD4047B, CD4060B, CD4541B and the CD4000 one-shots	A	within 0.3 % of the same circuits in ngspice
LEDs, every colour, and diodes	A	within 0.25 % of their datasheet curves
Transistors, every grade — switched, in between, near threshold	A	within 1.3 % of the maker's model or the grade's own datasheet fit
Inductors: charging, a relay coil with and without its diode, LC	A	within 0.3 %
A CMOS output charging a capacitor	A	within 0.1 % at 5 V, 1.4 % at 10 V
A 74LS input and output, into resistors, LEDs and capacitors	A	within 0.4 % of the gate's own transistors
Circuits mixing 74LS, CD4000 and transistors	A	within 0.6 %
A clock's triangle, sine, sawtooth and trapezoid waves	A	within 0.05 mV, into RCs and a Schmitt input
The ULN2003A	A	within 0.3 % of its Darlingtons' own models
Optocouplers, the LM358, regulators and the electronic load	A	exact to their own laws (no maker publishes a model to check them)

Each grade of transistor is one part's figures (the table under Diodes and transistors), so a transistor whose part number differs from its grade's simulates as that part, not its own.

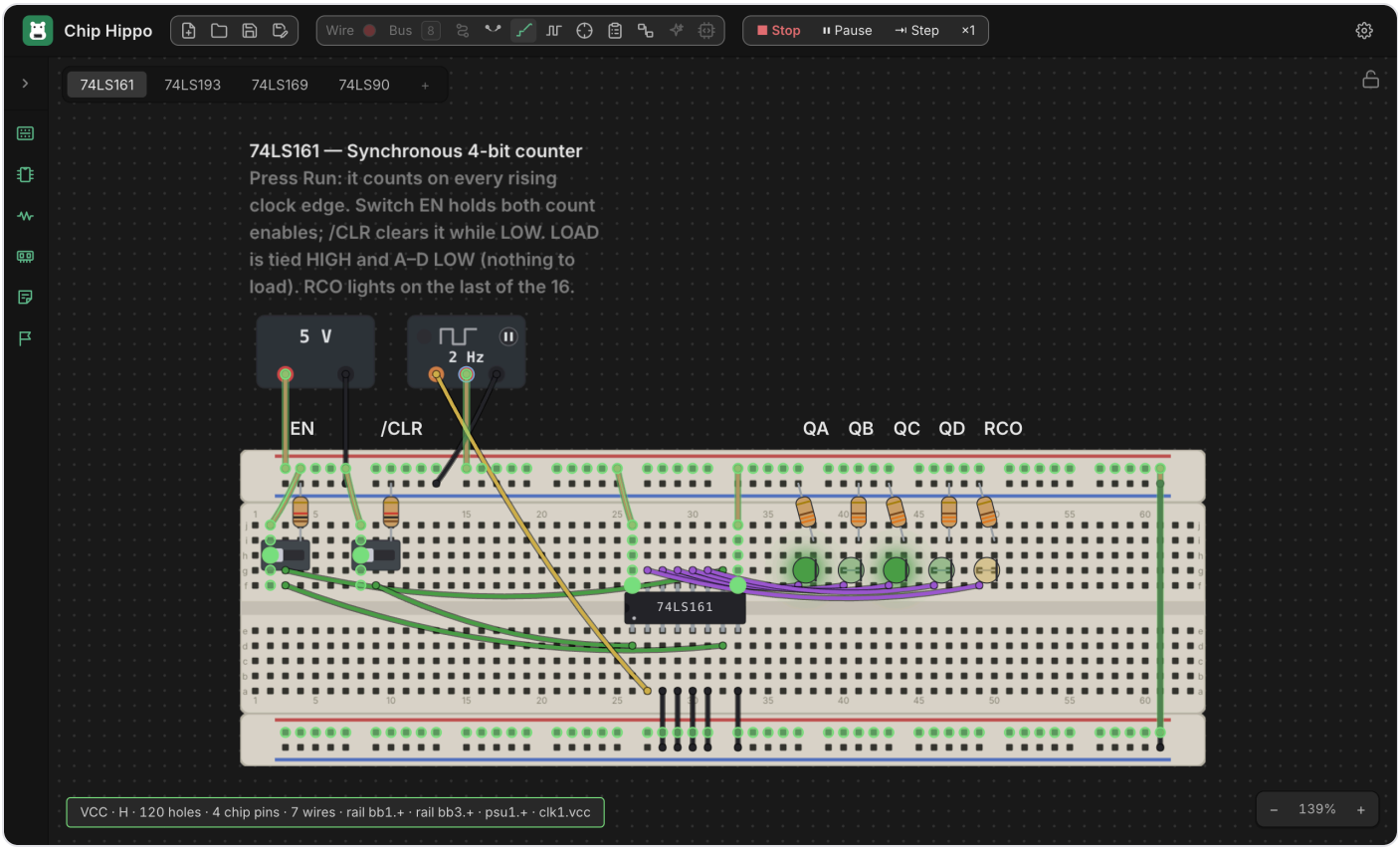
What Spice Lite does not model

To keep it light, some things are left out deliberately:

- a fast oscillation on a desk with a running **clock wave**: the wave never comes round to the same voltage within one of its halves, so an oscillator faster than 1 kHz elsewhere on the desk (a 555 at audio rates) is run crossing by crossing instead of drawn by its schedule, and the desk runs slower — the speed button says so. A wave never swings below 0 V or above its own supply (it is clipped there);
- a gate's **linear region**: an inverter whose own output is fed back to its input through a resistor (a CD4069UB amplifier, a crystal oscillator's bias) sits half way between LOW and HIGH on a bench — an amplifier. Spice Lite says so with a **Gate in its linear region** warning, and leaves the level there undefined;
- anything faster than a gate's delay: edges have no slope, and wires have no capacitance, inductance or ringing;
- the magnetic side of an inductor: its core never saturates and two coils never couple (no transformers). A relay's contacts move with its coil's current, at once — the armature's own travel is not modelled;
- a regulator's own dynamics (its output follows its reference at once) and its temperature (its shutdown trips on the power in it, and it cools in a second); an op-amp's slew rate, offsets and bias currents;
- the timers' comparator references beyond what their datasheets say (the CD4098B, CD4528B and CD4538B are worked back from their formulas) and their internal propagation delays;
- two oscillations both faster than the desk can show, wired into ONE circuit without being locked to each other: only a circuit that repeats as a whole is drawn at 1 kHz, so the pair is reported as oscillating. (Two that share nothing but a supply rail are two circuits, each drawn on its own.);
- a CD4047B's special RC COMMON protection diodes (its datasheet's formulas assume the full swing past the supply, and so does Spice Lite);
- two power supplies wired onto the same rail: the load is booked to the first of them only, and neither shares it with the other;
- current sharing across parallel wires, and the resistance of the breadboard's own contacts;
- heat, beyond an LED's or diode's own junction; signal reflections on long wires; and crosstalk between wires.

Probing & Net Names

Once a circuit is wired up, the probe is how you ask "what's this actually connected to?" — click any hole, pin, or terminal and Chip Hippo highlights every other point that shares the same electrical net, live, without needing to trace wires with your eyes. This page covers arming the probe, reading its net summary, naming a net so it reads clearly everywhere in the app, dropping free-text annotations on the desk, and sending a net to the logic analyzer.



The probe tool

Press `I` or `P` (either key toggles it — no modifier), or click **Probe** in the toolbar. With the probe armed:

- **Hover** a hole, rail point, PSU/clock terminal, chip pin, or a wire and its whole net lights up — a transient highlight that follows your cursor.
- **Click** a point to **pin** the highlight so it stays put while you move the cursor elsewhere (handy for comparing it against a second net). Click the pinned point again to unpin it.
- `Esc` unpins first, then a second `Esc` disarms the tool; `M` disarms whichever of the wire, bus, or probe tool is currently armed.

Unlike the wire, bus, and placement tools, **the probe stays available while a simulation is running** — editing locks on **Run**, but you can keep probing the live circuit the whole time. See Running a Simulation for what Run/Stop locks and what stays live.

Reading the net summary

A highlighted net draws as a glow: dots on member holes and terminals, rings around member chip pins, and a glow stroke along every member wire — brighter when pinned. Below it, a status readout summarizes the same net in one line, for example:

```
VCC · H · 23 holes · 3 chip pins · 2 wires · rail bb1.t+ · psu1.+
```

The pieces, in order, appear only when relevant:

- The net's **name**, if you've given it one (see below).
- Its current **level** (`H` / `L` / `Z` / `X`) while a simulation is running — gone when stopped, since an unpowered net has no level to report.
- With Spice Lite running, the net's **voltage** (every net something holds has one; a floating net shows its level alone), and the **current** through the lead the probe is on: point at an LED's leg, the resistor beside it, the chip output driving it, or a power supply's terminal (for example `H · 9.41 mA`). Nothing on the desk shows a current of its own except a power supply's draw; the probe is where you read it.
- A **connectivity summary** — hole/pin/wire counts, which rails it touches, and any PSU/clock terminals it reaches.

Under the hood, every hole, rail node, wire, active switch/button bridge, and component pin gets partitioned into these nets by a union-find pass over the whole document — the same netlist the simulator resolves each net's level from.

Naming a net

A net's default identity is just its smallest member address (`bb1.a12` , say) — useful internally, meaningless at a glance. Right-click a point while probing to give the net a real name:

- **Name this net…** (or **Rename net…** if it already has one) opens a prompt with quick picks for **VCC**, **GND**, and **CLK**, or type anything you like.
- **Clear name** removes it.

The name binds to the specific point you right-clicked, but resolves to whatever net that point sits on at any given moment — so it survives rewiring around it. If an edit later merges two separately-named nets

into one, one name wins deterministically and the other is dropped; nothing crashes, but it's worth a glance if a net you named seems to have picked up its neighbor's label instead.

Each half of a Split power rail is its own net, so right-click a hole in each half to name them separately — `VCC5` and `VDD9`, say.

Naming isn't just cosmetic: a named net is what makes the Schematic View and the Build Guide & BOM readable — `VCC` / `GND` / `CLK` and your own signal names carry through instead of raw hole addresses.

Annotations

For notes that aren't about a net at all — reminders, section headers, "TODO: add a pull-up here" — drop a free-text **annotation** on the desk from the **Annotations** section at the bottom of the parts palette:

- **Label** — a short one-line caption.
- **Note** — a multi-line block.

Picking one arms a placement ghost like a part; click anywhere on the desk to drop it, which opens an inline editor immediately so you can type right away. Drop it **on top of a part** and it **anchors** to that part — the ghost shows the anchored state before you click — so the annotation rides along whenever that part moves. Drop it on open desk and it just sits at that world position.

Double-click an annotation any time to re-edit its text (`Enter` commits a label; a note keeps `Enter` for newlines, so use `⌘Enter` / `Ctrl+Enter` to commit). Drag its body to reposition it, and right-click for **Edit** / **Remove**. Annotations are pure decoration — they carry no electrical meaning and are invisible to the netlist, occupancy, and the simulator.

Sending a net to the analyzer

Probing a net you want to watch over time? Right-click it and choose **Add to analyzer** to pin it as a channel without leaving the probe tool. See Logic Analyzer & Timing for capturing and reading the resulting waveform.

See also

- Wiring, Nets & Buses — laying the wires and buses whose nets you're probing.
- Running a Simulation — what a net's level means and when the probe keeps working while editing locks.
- Logic Analyzer & Timing — capturing a probed net's waveform over time.
- Schematic View — where net names show up in the derived logical diagram.

Memory Chips & the Inspector

Chip Hippo's memory catalog covers both sides of the RAM/ROM divide: address an 8K/32K/128K SRAM for scratch storage that behaves exactly like real silicon (it forgets everything at power-off), or a ROM/EPROM/EEPROM that holds a real byte image between runs — the same image, opened in a floating **inspector**, that you can browse, hand-edit, and program from a file. This page covers what each kind actually stores, how to get data into a ROM, and how the inspector works.

Volatile vs. non-volatile

Every memory chip in the catalog is one of two kinds, and the difference matters more than the pinout:

- **Volatile (SRAM)** — `ram-8k`, `HM62256` (32K×8), `AS6C1024` (128K×8). Contents live only for the current run: they start as random noise every time you press **Run**, whatever the circuit writes stays readable until you **Stop**, and then it's gone. Nothing is ever saved to disk for these parts — there's no image to lose, because there was never a file.
- **Non-volatile (ROM/EPROM/EEPROM)** — `rom-8k`, `28C16` (2K×8 EEPROM), `AT28C256` (32K×8 EEPROM, pin-for-pin with the `HM62256`, so one socket takes either), `AM27C1024` (64K×16 EPROM). These chips hold a real byte image that persists across runs, and **travels inside the project file**: save the project and every programmed ROM's contents are written into the `.chiphippo` alongside the design, so copying that one file to another machine takes the ROMs with it. An unprogrammed chip reads back random noise too — just like an unprogrammed EPROM off the shelf — until you program it. (Between saves the bytes are kept in a working file in the app's own data folder; which file backs which chip is internal bookkeeping you never need to touch by hand.)

The chip's **blurb** in the parts palette and pin-assignments window always says which kind it is, and its behavior — read timing, chip-enable/output- enable pins, address/data bus width — is otherwise ordinary combinational logic. See The Chip Library for the full catalog and the pin-assignments window.

Programming a ROM

The circuit itself can never write a non-volatile chip. An EEPROM or EPROM's real write cycle isn't something the simulator drives — Chip Hippo treats every non-volatile part as read-only from the circuit's point of view, the same way you couldn't reprogram a 27C-series EPROM just by wiggling its pins on a breadboard. Any write a chip reports during simulation is silently dropped if the chip is non-volatile.

The only way to put data into a ROM is the **external programmer**:

1. Right-click the chip on the desk and choose **Properties...**.

2. Click **Load image... (program)**.
3. Pick a `.bin` (raw bytes) or `.hex` (Intel HEX) file.

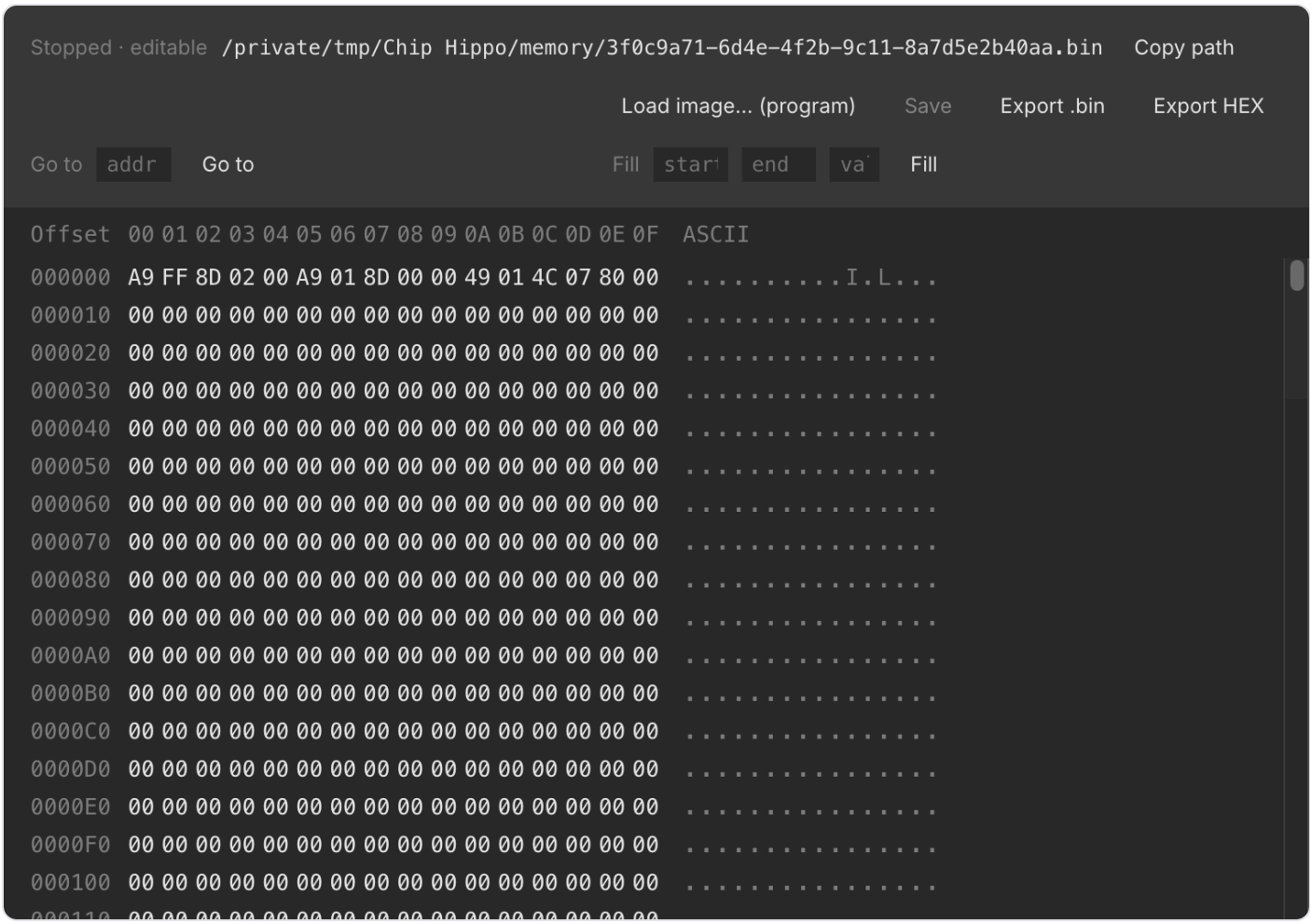
A `.bin` image is copied to the start of the chip's memory. If the file is smaller than the chip, only the start is overwritten and the rest is left alone; if it's larger, it's truncated to fit — either way you get a warning telling you what happened. A `.hex` file says where each byte goes, so its bytes land **at their own addresses**, wrapped to the chip's size, the way an EPROM programmer places them: a program assembled at `$E000` with its reset vector at `$FFFA`, loaded into a 32 KB ROM, puts the code at offset `$6000` and the vector at `$7FFA` — right for a ROM decoded at `$8000`. Bytes the file doesn't mention keep what they held. (Only a HEX file whose data spans more than the chip holds is loaded from its lowest address and truncated.) Once programmed, the chip is flagged **programmed**, which rides undo/redo like any other edit, and the chip keeps its new contents the next time you open the document or press Run.

The chip also **remembers which file you loaded**. The Properties dialog shows it as **Image file**, so you can tell at a glance whether a ROM holds the monitor image or last week's experiment. If you go on to edit the bytes by hand in the inspector, the file is still shown — marked *(edited)*, since it is where the data came from but no longer what the chip holds.

Programming is only available while editing is unlocked (i.e. the simulation is stopped) — see Running a Simulation.

The memory inspector

Every memory chip — SRAM or ROM alike — has a floating **inspector** window: right-click it on the desk and choose **Inspect Memory**. It's a hex/ASCII grid, one row per 16 bytes, with a toolbar for **Go to** (jump to an address), **Fill** (write a byte value across a start/end range), **Save**, and Import/Export.



What you can do depends on whether the simulation is running and what kind of chip it is:

- **Stopped, ROM/EPROM/EEPROM** — the grid is editable. Click a hex or ASCII cell to type a new byte value directly, or select a range and **Fill** it. Typing runs straight through, as in a hex editor: a byte's second hex digit (or an ASCII cell's one character) writes it and moves to the next byte, so a run of values needs no clicks between them. With a byte selected and no editor open, just start typing — the first digit goes into that byte (or, if you last clicked the ASCII column, the character does). **Enter** stops where you are, writing a single digit as that value; **Escape**, or clicking away with only one digit typed, abandons the byte. **Save** writes your edits back to the chip's file and flags it **programmed**, exactly like using the external programmer. Until you save, the chip on the desk wears a warning triangle that says **Edited, not saved**: the circuit reads the chip's file, which doesn't have your edits yet. Closing the inspector without saving discards them.
- **Stopped, SRAM** — shows the chip's last contents from before it was stopped (or noise, if it's never been run) as a read-only snapshot; there's nothing to save, since SRAM has no file.
- **Running, any chip** — the grid is a **live, read-only** mirror of the simulator's own byte image. Bytes the circuit writes are tinted as they change, so you can watch a counter or a shift register fill memory in real time. Editing resumes once you stop.

For a non-volatile part the inspector also names the file its image came from, with **Copy path** beside it; hover that line to see where the app is keeping the raw bytes on disk. It stays open and live-updating as long as the desk document does.

Import & export (Intel HEX)

Beyond loading a raw `.bin`, the inspector's toolbar can **Export** the current contents as either a raw `.bin` or an **Intel HEX** `.hex` file — useful for round-tripping through an assembler toolchain or another tool that expects the hex format. The external programmer (above) accepts either extension coming in: a `.hex` file is decoded before it's written to the chip.

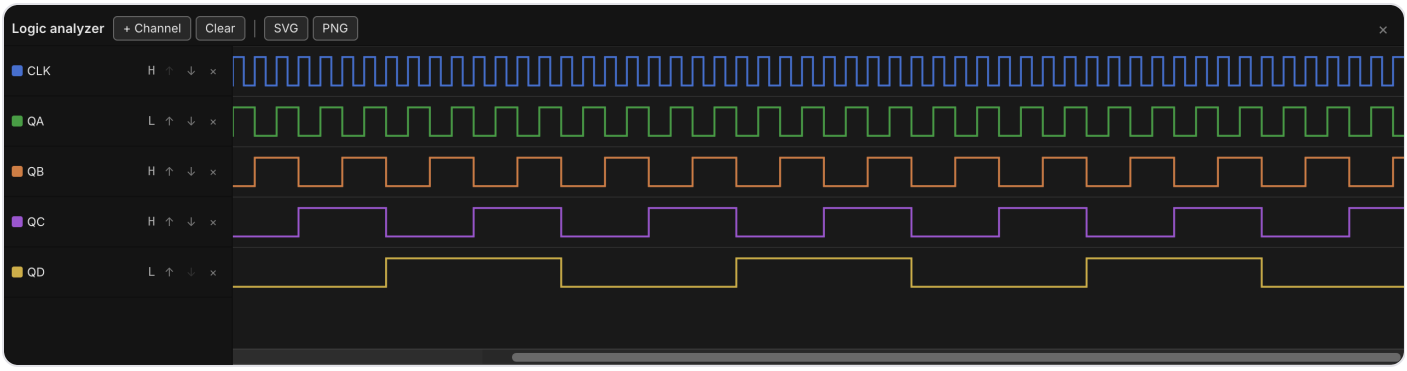
A missing backing file

Because a ROM's contents live in a file separate from the desk document, it's possible for that file to go missing without the document changing — the classic case is deleting the chip and then **undoing** the delete. If Chip Hippo finds a chip flagged **programmed** but its backing file is gone, it recreates the file as fresh noise and warns you that the chip's data was lost, so you know to reload the image rather than trusting stale contents.

See also The Chip Library for the memory chips in context with the rest of the catalog, and Files, Saving & Undo for how the project itself is saved.

Logic Analyzer & Timing

The **Analyzer** is a bottom-docked panel that records the live simulation and draws it as a scrolling timing diagram — one lane per signal you've chosen to watch, a step waveform for a single net or a hex value-lane for a bus, with two draggable cursors for measuring the time between two points. It's a passive recorder: it reads the same broadcast the LEDs and chip badges already render from, so opening it, closing it, or adding channels never changes how the circuit behaves.



Opening the analyzer

Click **Analyzer** in the toolbar, or press `A` — a bare key, like the desk's other tool letters, since `Cmd/Ctrl+A` is Select All — to dock the panel along the bottom of the window. Click the button again, press the shortcut again, or click the panel's **×** to close it — the panel remembers whether it was open and how tall you left it (drag its top edge to resize, from a compact strip up to half the window).

With no channels added yet, the panel shows a prompt instead of an empty grid: **add a net or bus**, or right-click a probed net and choose **Add to analyzer**.

Adding a channel

There are two ways to add a channel:

- **From the panel** — click **+ Channel** in the header. The menu lists every *named* net (see Probing & Net Names) and every bus in the document that isn't already on a channel; picking one adds it.
- **From the probe tool** — arm the probe (`I`), right-click any net on the desk, and choose **Add to analyzer**. This works on ANY net the probe can reach, named or not — it's the way to watch a signal you haven't bothered to name. Using it also opens the panel automatically if it was closed.

Each channel gets a row in the left-hand gutter: a color dot (cycled from a fixed palette unless the channel has its own color — either way the channel keeps it wherever you move it), the net or bus name,

its current value, and three small controls — ↑ / ↓ to move the channel one row, and × to remove it.

To move a channel further, **drag its row** up or down the gutter. The rows and their waveforms make room as you go, so you see the new order before you let go; drop the row where you want it, or press `Esc` to put it back. Hold it against the top or bottom of the list to scroll a long list along. Reordering works while the simulation runs, too.

Channels are saved with the document (`doc.scopeChannels`), so a desktop reopens with its analyzer setup intact, and adding/removing/reordering a channel rides the normal undo/redo stack — a drag is one undo step. A channel that loses its target (the net was deleted, the bus was removed) simply reads as undriven rather than disappearing — it comes back to life if the target returns.

Reading a waveform

A single-bit channel (a **net**) draws as a step waveform inside its lane:

- **High** traces along the top of the lane, **Low** along the bottom.
- **Floating (Z)** draws a dashed line through the lane's midline.
- **Unknown/conflict (X)** — including anywhere the net is undriven — fills the region with a diagonal amber hatch instead of a line. Under Spice Lite there is no hatch: the lane draws the voltage itself, and a net reads X wherever its voltage sits between two inputs' thresholds — a 555's timing capacitor between ⅓ and ⅔ of its supply, all cycle long — which the trace already shows.

The gutter's value column always shows the channel's current level (or its value at cursor A, once one is placed — see below).

With Spice Lite on, a net with a **voltage** — every net something holds — draws that voltage instead: 0 V along the bottom of the lane, the highest supply on the desk along the top. A capacitor is seen to curve up toward its supply rather than to jump from Low to High, a 74LS signal swings between about 0.2 V and 4.4 V, and the value column reads in volts (`3.16 V`). The lane's horizontal axis still counts ticks, not seconds. While a voltage is moving, Spice Lite ticks thirty times a simulated second to draw it, so on its own the curve keeps its true shape; a clock's edges add ticks of their own in between, and a stretch where nothing changes takes no extra width.

Multi-bit (bus) lanes

A channel bound to a **bus** (see Wiring, Nets & Buses) doesn't draw a two-level waveform — decoding eight or sixteen individual traces would be unreadable. Instead it draws a **value-lane**: a band that necks down to a point at every change and displays the bus's current value as hex (`0x1A`, uppercase) centered in the band, wide enough for the label to fit. The bus is decoded from its member wires' bit

positions (as parsed from a name like `D[7:0]`); if even one member is floating or unknown, the whole word is undecodable for that span and the lane hatches just like an unknown net.

Cursors & Δ

Click anywhere in the waveform area to drop **cursor A** at that tick; `Shift`-click to drop **cursor B**. Drag either cursor to slide it to a different tick — the gutter's value column updates to show each channel's value at cursor A while it's placed. With only cursor A down, the header reads `t=<tick>`; with both down, it reads the delta between them — **Δ N ticks**, plus the simulated time between the two columns in milliseconds. Every column remembers the moment its tick ran at, so the figure is right whatever ticked in between — several clocks, a PWM's short half, a switch, a timer — and it is circuit time: a 10 Hz clock's period reads 100 ms at any speed.

While the simulation is running, the view auto-scrolls to keep the newest tick in sight; scrolling the lanes away from the right edge turns this "follow" behavior off until you scroll back. **Clear** in the header wipes the recorded trace and both cursors without touching the channel list; starting a fresh **Run** does the same automatically.

Export

Two buttons in the header export the diagram exactly as currently rendered — every channel, the full retained trace, and a label column identifying each lane:

- **SVG** — downloads a self-contained `timing.svg` (colors resolved to concrete values, not the app's live CSS variables, so it renders correctly outside Chip Hippo).
- **PNG** — rasterizes that same SVG at 2× scale and downloads it as `timing.png`.

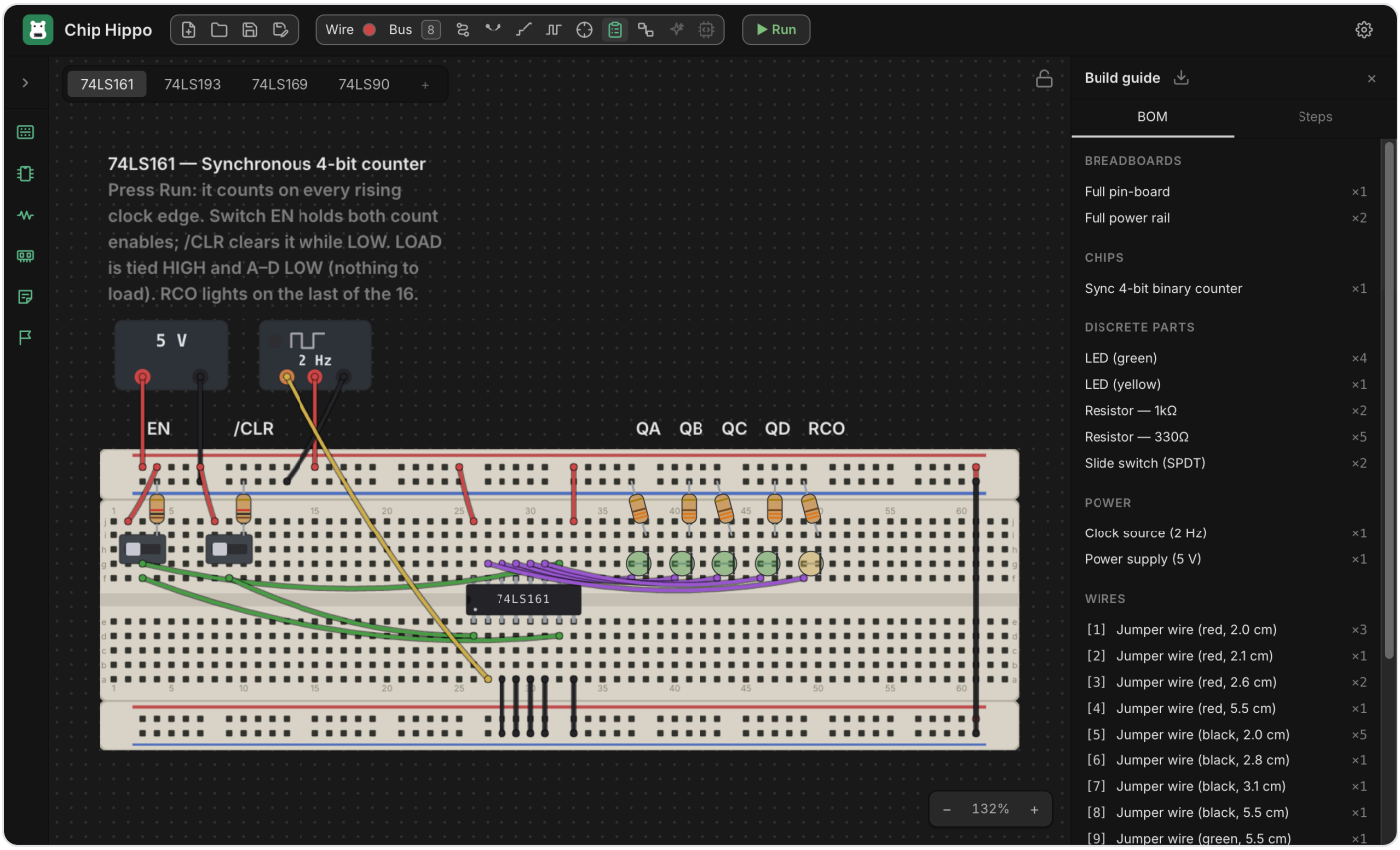
Both are plain browser downloads (no file dialog, no IPC round-trip) and do nothing if there are no channels or nothing has been recorded yet.

See also

- Probing & Net Names — arming the probe, naming a net, and the "Add to analyzer" context-menu entry.
- Wiring, Nets & Buses — laying the buses a bus channel decodes.

Build Guide & BOM

Once a circuit is wired up on the desk, Chip Hippo can turn it into the paperwork you'd actually want on the bench: a bill of materials (what to buy, and how many — down to a numbered cutting list of every jumper), and an ordered assembly checklist that names each of those jumpers as you come to run it. None of this is stored on the document — it's derived fresh from the live desk every time you open or change it, so it never drifts out of sync with what you've actually built.



Opening the build guide

Click the **Bill Of Materials** clipboard button at the right-hand end of the desk-tools pill, or press `Ctrl+B` / `⌘B`, to open the build guide as a right-docked panel. Clicking it again, or the panel's own **×** close button, hides it — and the button stays lit for as long as the panel is open, however you opened or closed it. The panel's open/closed state is remembered between launches, same as the parts palette. It has two tabs — **BOM** and **Steps** — and is read-only, so it stays available even while a simulation is running.

The guide re-derives itself automatically whenever the document changes while it's open (adding/removing a part or wire, renaming a net, flipping a switch) — there's nothing to refresh by hand.

Warnings

Above whichever tab you're on, the guide surfaces anything that looks un-buildable or likely forgotten, with a count badge on the panel's title when there's something to see:

- **Floating leads** — a chip or discrete pin whose lead sits over no hole.
- **Unpowered chips** — a chip whose VCC or GND pin has no connection, so it will never power up.
- **Single-member nets** — a wired net that only reaches one point, which usually means a connection you meant to make and didn't.

The bill of materials

The **BOM** tab lists everything on the desk as counted line items, grouped into five sections (only the non-empty ones show): **Breadboards**, **Chips**, **Discrete parts**, **Power**, and **Wires**. Boards are counted by strip type (a Full 830 kit counts as its constituent rail/pin strips, not as one line), and components are counted by catalog identity — with a few splits that matter for actually buying the right part: LEDs split by color, resistors and capacitors by their value (`Resistor — 4.7kΩ`, `Capacitor (ceramic) — 100nF`), PSU bricks by voltage, and clock sources by rate. Each line reads as `title ×count`.

The cutting list

Wires comes last, because you wire after you seat, and it's a different kind of list: a numbered **cutting list**, one line per colour *and cut length*, tallied.

```
[3] Jumper wire (red, 6.1 cm)  ×3
```

That's three leads to cut the same, which is how you'd actually work through a drawer or a spool — so the length belongs in the line rather than beside it. The length is the **cut** length: the run from hole to hole plus the stripped end that goes down into each hole, so a jumper crossing a single 0.1 in pitch is about 13 mm of wire, not 3. It's the same measurement, and the same wording, as the drawing at the bottom of a wire's own **Properties…** dialog (see Wiring, Nets & Buses) — a wire can't read one length there and another here.

Lines are sorted by the app's colour order, then shortest first, and **numbered in that order**. The number is a cross-reference: every assembly step that runs a wire calls it out as `[3]`, so you cut the pile once, number it, and never have to re-measure anything mid-build. It only moves if the desk itself changes — never because a colour was renamed or the language switched.

Assembly steps

The **Steps** tab is an ordered checklist for building the circuit from scratch, grouped in the order you'd actually work: **Place the boards**, **Power**, **Seat the chips**, **Add discrete parts**, **Run the signal wires**. Each step has a checkbox — ticking it is a session-only visual aid (nothing is persisted), useful for tracking progress while you build.

A few notable details in how steps are phrased:

- A grouped breadboard kit (rails + pin strip snapped together) is one step, not one per strip; a loose strip gets its own step.
- Power steps cover PSU/clock bricks (set to their configured voltage/rate) and then any wire that distributes power — a brick terminal or rail hole at either end.
- A chip's step spells out its straddle, e.g. *"straddling e5–f11, pin 1 at bb1.e5"* (plus a flipped note if it's rotated 180°); a linear discrete lists its resolved lead addresses instead.
- Signal wires are grouped last: whole buses first (one step per bus, one detail line per bit), then the remaining signal nets, each with its wires listed as `[n] from → to`. The leading `[n]` is that wire's item number from the cutting list above, so the callouts form a column you can read down while you cut.

How an endpoint is named

A wire's two ends are resolved to the friendliest label available rather than printed as raw addresses, in this priority order:

1. A component pin at the hole — `"74LS00 pin 3 (1Y)"` (part + pin number + datasheet pin name, when the pin has one).
2. A pin sharing the hole's 5-hole node — a bus tap lands *beside* a pin rather than on it, and still resolves to that pin.
3. A PSU or clock terminal — `"Power supply +"`.
4. A power rail — `"+ rail (bb1)"`.
5. The bare hole address, as a last resort.

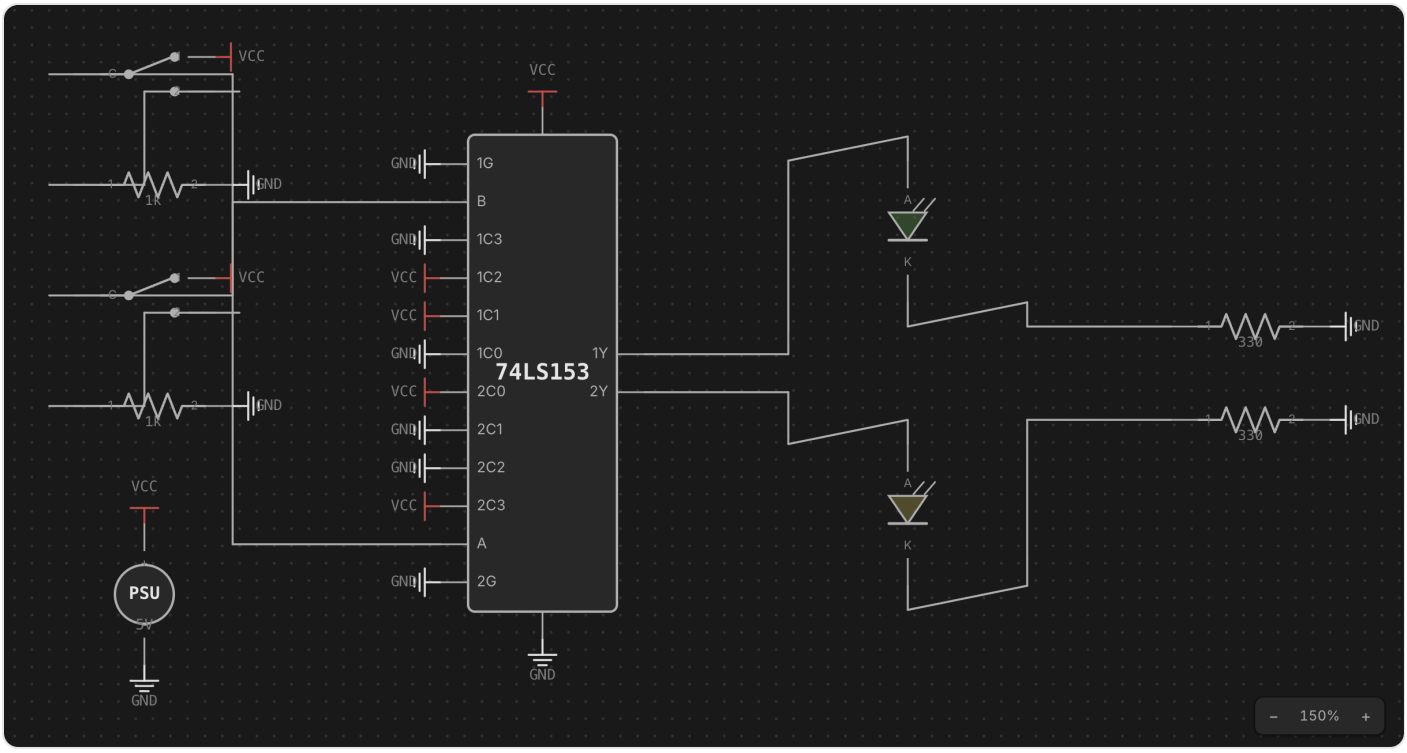
A net you've named (see Probing & Net Names) leads its step with that name instead of an anonymous net id, so naming your nets up front, before opening the guide, makes the whole tab read far more like a real assembly sheet and far less like an address dump.

Downloading the BOM

The download icon in the panel header exports the **whole current plan** — the BOM and the assembly steps together, headings, item numbers and all — as a Rich Text Format (`.rtf`) document, named `<project name>-bom.rtf`. It mirrors the panel's tabs, so what you get on paper is what you were looking at. It's a plain browser download (no save dialog, no main-process IPC involved) generated entirely in the renderer, so it reflects exactly what the panel is showing at the moment you click it. `.rtf` opens in any word processor, which makes it easy to print or hand off as a build sheet alongside the physical parts.

Schematic View

The desk shows your circuit exactly as you built it — a physical breadboard with chips seated in holes and wires running point to point. The **schematic view** shows the same circuit the way an engineer would draw it: chip symbols, named nets, and bus lines, laid out automatically. It's a **projection**, not a second document — the breadboard is still the single source of truth, and nothing you do in the schematic changes a hole, a pin, or a wire on the desk.



Switching views

Press `Tab` to flip between the **Breadboard** and **Schematic** views (not while a text field has focus — typing a `Tab` there behaves normally, e.g. to move between fields), or click the **schematic** button in the toolbar, between **Bill Of Materials** and the **AI builder**. Its icon shows the view it will take you to, and fills in while the diagram is up. The schematic reuses the desk's own camera, so pan and zoom feel identical in both views and your position is preserved when you switch back. The first time you open the schematic in a session it automatically fits the whole diagram in the viewport; after that, panning and zooming are exactly what you left them.

`Cmd/Ctrl+F` re-fits whichever view you are looking at — with the schematic showing it frames and centers the diagram again, the same way it frames the desk. (Fitting the schematic only moves the camera: symbol positions are derived, so there is nothing on the diagram to relocate.)

If the desk has no chips on it yet, the schematic shows a short hint instead of an empty canvas — add a chip on the breadboard and switch back to see it appear.

Chip symbols & routed nets

Every chip is drawn as a labelled box (or, for a plain gate chip, a small distinctive shape — AND/OR/NAND/NOR/XOR/NOT/BUFFER — badged at the top of the box) with named pin stubs instead of physical DIP pins. Discretes get their own recognizable symbols too: an LED's diode triangle, a resistor body, a capacitor's two plates (one curved, with a `+`, for an electrolytic), a switch, a push button, a PSU or clock source block. A resistor and a capacitor carry their value beside the symbol. VCC and GND pins don't route across the page like a signal would — each one drops a small power-rail symbol right at the pin, the way a real schematic keeps power off the signal routing.

Everything else is connected by **routed nets**: orthogonal lines that run chip-to-chip along the net they belong to, labeled with the net's name where there's room. A group of pins wired as a bus (see Wiring, Nets & Buses) collapses into a single **fat bus line** instead of drawing every bit as its own wire — the same visual shorthand a hand-drawn schematic uses for a data or address bus.

Auto-layout & nudging

The schematic lays itself out automatically and deterministically: the same circuit always produces the same diagram, with chips arranged left-to-right roughly along the direction signals flow and edges routed to keep crossings down. You never have to arrange it by hand — but if a particular symbol lands somewhere inconsistent, drag it to a better spot. That drag is stored as a **per-symbol position hint**, not a real coordinate: it only ever affects where that symbol sits on THIS derived diagram, and it has no effect on the component's actual place on the breadboard. Nets simply re-route to follow wherever you drop it.

An **Auto-layout** button in the schematic clears every position hint at once and lets the algorithm re-place everything from scratch — useful after nudging things into a mess, or after a big edit has made your old arrangement stale.

Live simulation tint

Press **Run** and the schematic lights up exactly like the breadboard does: the routed lines tint by their net's live level, an LED symbol lights when its anode reads high and its cathode reads low, and a chip symbol picks up the same health badge (unpowered / underpowered / reversed / damaged) you'd see on its physical package. Both views are driven from the same simulation state, so whichever one you're

looking at when you hit Run shows the identical picture — there's nothing to keep in sync by hand. See Running a Simulation for what each level color and health badge means.

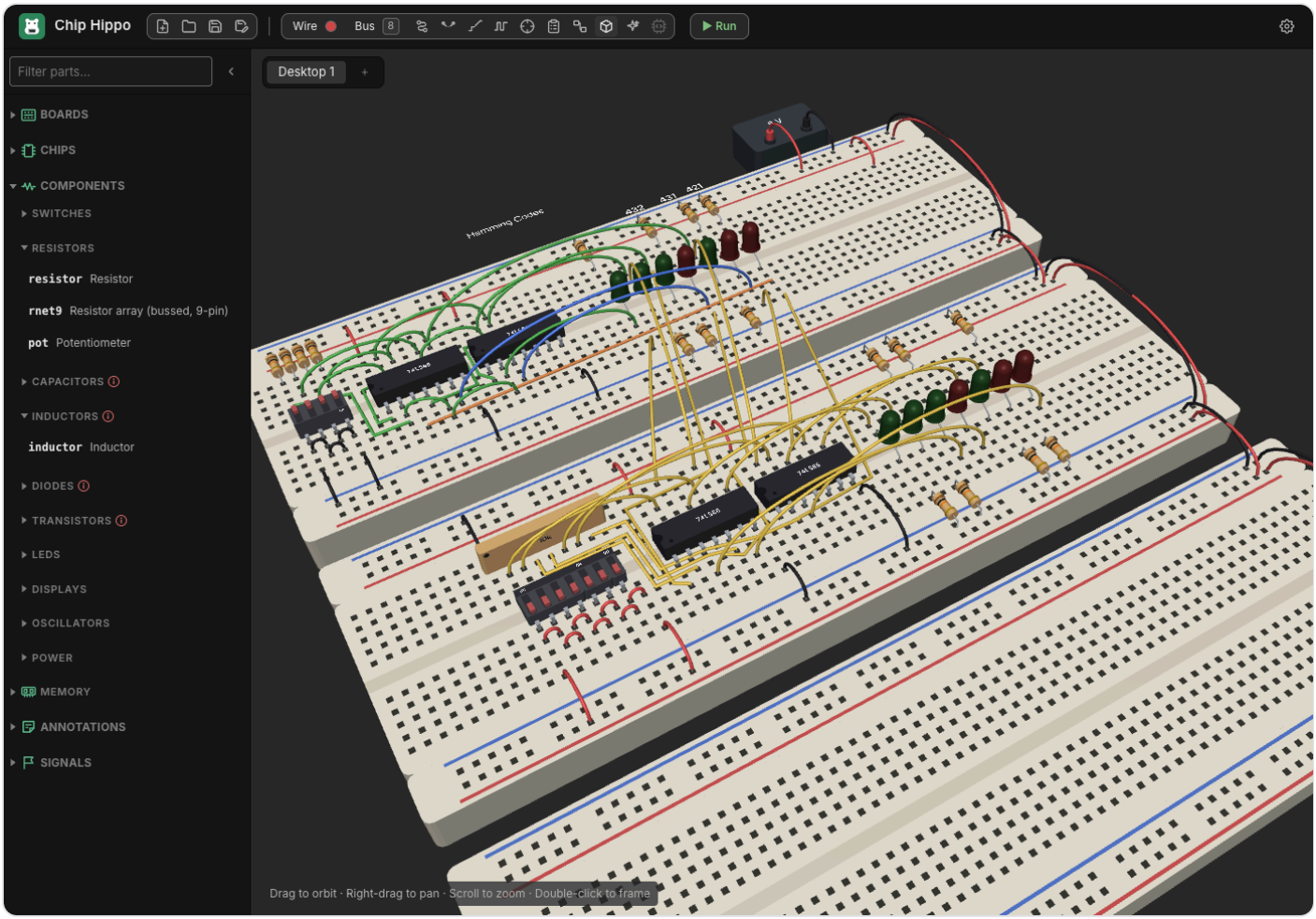
The probe is shared

The connectivity **probe** (see Probing & Net Names) is armed and driven from the breadboard, but its highlight is not confined to it: probing a net on the desk highlights that same net in the schematic too — by its stable net id, not by screen position — so you can hover a wire on the physical board and immediately see which chip symbols and bus lines it corresponds to in the logical diagram. The highlight is applied to both views at once, whichever one happens to be visible, so switching to the schematic after probing shows the net already picked out. It's the same underlying net, just drawn two different ways.

See also: The Desk & Breadboards for the physical view this diagram is derived from, and Wiring, Nets & Buses for how wires and buses become the nets the schematic routes.

3D View

The **3D view** stands the circuit on your desk up in three dimensions: the breadboards with their holes and rail stripes, every chip on its legs, the switches, LEDs, resistors, capacitors and displays, and the jumper wires arching between them — to turn round and look at from any side. It is a way of *looking* at a design, not of building one: like the Schematic View it is a **projection** of the desk, and nothing you do in it changes a part, a hole or a wire.



Switching to it

The 3D view is switched off until you turn it on: in **Settings ▸ Appearance**, set **3D enabled** to **On** and a **3D** button appears in the toolbar. Setting it back to **Off** hides the button again.

Click the **3D** button in the toolbar (a cube, beside the **schematic** button) to show the 3D view in place of the breadboard; it fills in while the 3D view is up, and clicking it again goes back to the breadboard. Chip Hippo always opens on the breadboard. The **schematic** button and `Tab` switch from the 3D view straight to the schematic, as they do from the breadboard.

The first time it is shown — and whenever another desktop or project is opened — it frames the whole desk.

Moving round the desk

Do this	To
Drag	Orbit round the desk
Right-drag, middle-drag or `Shift`+drag	Slide the view across the desk
Scroll, or pinch	Zoom in and out
Double-click, or `Cmd/Ctrl+F`	Frame the whole desk again
`Shift+Cmd/Ctrl+F`	Back right off, the desk small in the middle
`Option+Cmd+=` / `Option+Cmd+−`	Zoom in and out from the keyboard

The desk padlock (`Cmd/Ctrl+L`) locks the scroll wheel here too, so a resting finger on a Magic Mouse can't zoom the view.

The running circuit

The 3D view follows the simulation: while the circuit runs, LEDs and display segments light, a clock source's lamp blinks with its output, a transistor's lamp lights while it conducts, and a character LCD shows what is on its screen. Under Spice Lite an LED glows as brightly as its current drives it — dim on a milliamp, a wider halo when it is overdriven. A part that burns out smokes: a chip wired backwards or killed by too high a supply, or an LED, segment or diode with nothing limiting its current, sends up grey smoke, and a chip Spice Lite overloads sends up brown. They light and smoke from exactly the same decisions the breadboard makes, so the two views always agree.

Edits made on the breadboard appear in the 3D view the next time you switch to it.

What it draws

Every part in the parts tray has a 3D model, built from the same outline and colours the breadboard draws it with — a part sits over exactly the holes it is plugged into, with its legs going into them. Wires arch from hole to hole (the longer the wire, the higher the arch); routed wires lie flat along their bends; a bus is drawn as a ribbon cable. Signal flags, Output/Input tags, and your labels and notes are shown where you put them.

This is a first version, and it does not keep things apart: a long wire can pass straight through a chip, and parts placed over one another overlap, just as they can on the flat desk.

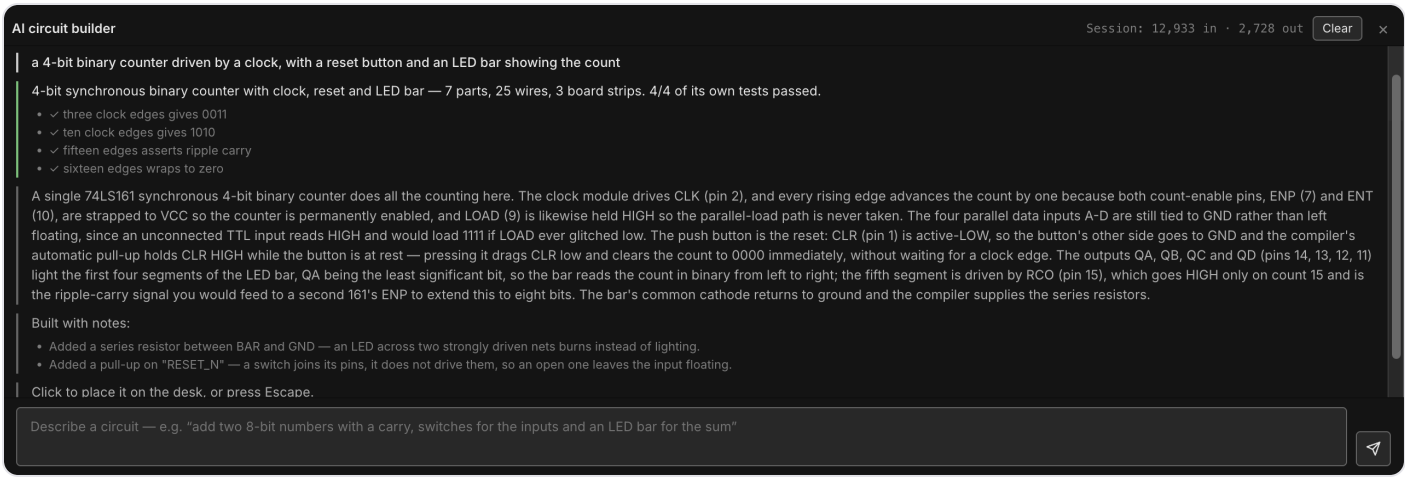
If your computer has no WebGL support, the 3D view says so instead of drawing.

See also

- Schematic View — the circuit as a logical diagram.

AI Circuit Builder

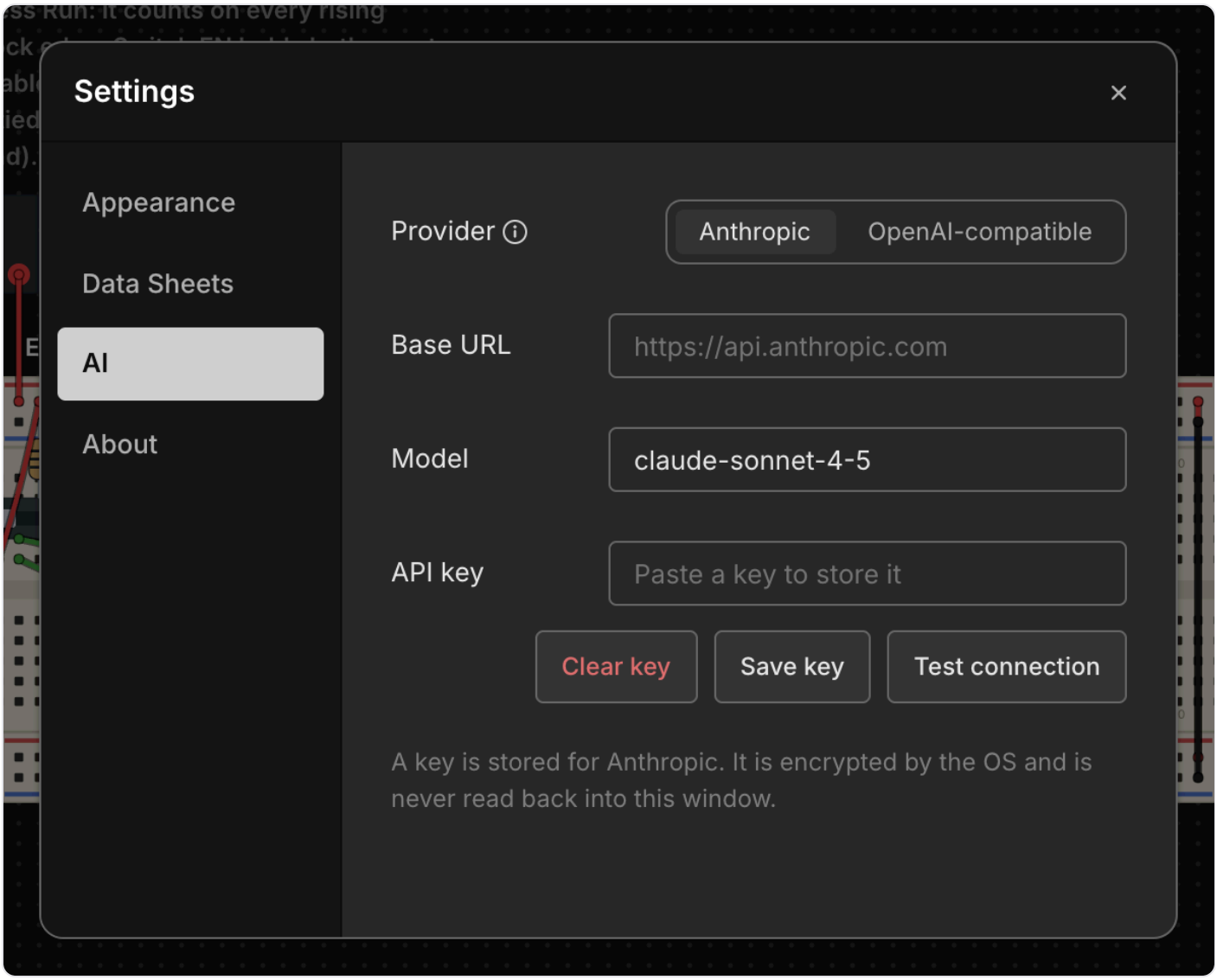
The **AI builder** is a bottom-docked panel where you describe a circuit in plain English and get a real, wired, **simulation-proven** design to place on the desk. It uses *your own* AI connection — your key, your provider, your account — and Chip Hippo contacts no AI provider until you've set one up and asked it for something.



The important thing to understand about it is what the model is and isn't allowed to decide. It never places a chip, never picks a hole, and never draws a wire. It answers one question — *which parts, and which of their pins are joined?* — and Chip Hippo's own compiler does all the geometry, then proves the result by running it through the same simulation engine the **Run** button uses. Nothing that fails is ever offered to you.

Setting up a connection

Open **Settings** (`Cmd/Ctrl+,`) and pick the **AI** tab. There are four things there:



- **Provider** — **Anthropic**, or **OpenAI-compatible**. The second is deliberately not "OpenAI": the same request format is what Ollama, LM Studio, OpenRouter and vLLM all speak, so one setting plus a base URL covers every local and hosted endpoint worth using.
- **Base URL** — leave blank to use the provider's own default (shown as the field's placeholder). Point it at `http://localhost:11434` for a local Ollama, or at any compatible proxy.
- **Model** — also blank for the default. Any model id your endpoint accepts.
- **API key** — paste it and click **Save key**.

Your key is never stored in Chip Hippo's settings file. It goes straight to your operating system's own secure credential store (Keychain on macOS, the Credential Manager on Windows, the desktop keyring on Linux), encrypted, and it is never read back into the app's window afterwards — the panel only ever learns *whether* a key is configured, not what it is. If your system has no secure store available, Chip Hippo **refuses to save the key** and tells you so, rather than quietly writing it to disk in the clear.

Click **Test connection** to check the base URL, key and model in one small request before you rely on them. **Clear key** forgets the stored key.

A local server that needs no key at all works fine — leave the key blank and just set the base URL.

Asking for a circuit

Click the **AI builder** button in the toolbar to dock the panel along the bottom of the window (drag its top edge to resize it; it remembers whether it was open and how tall you left it). Type a description and press `Enter` — `Shift+Enter` gives you a newline for a longer brief.

The toolbar button is **disabled until there's a connection to ask** — no key stored, or a base URL that doesn't parse, and it greys out with the reason as its tooltip. It's the one tool in Chip Hippo that can't work on your own machine alone, so the button says so rather than failing after you press it. (Whether the server will actually *accept* your key is only the server's to say — that's what **Test connection** above is for.)

Descriptions that work well are the ones an engineer would accept:

> add two 8-bit numbers together with a carry, switches for the inputs and an LED bar for the sum

> a 4-bit counter driven by a clock, showing its count on an LED bar

Be specific about the **inputs and the outputs** — what you want to set, and what you want to see. Say which **logic family** you want if it matters: mention CD4000, 4000-series or CMOS and the design uses only CD4000 parts; mention TTL or 74LS and it uses only 74LS parts; say neither and it uses 74LS. It won't mix the two unless you ask, and if you do it adds the pull-up a 74LS output needs to drive a CMOS input. The parts themselves you can leave to it: the model is told, on every request, exactly which chips are in Chip Hippo's catalog and exactly what every pin on each of them is called, so it has no reason to invent a part that doesn't exist.

You don't need to mention power, and you shouldn't. Every chip declares its own VCC and GND pins, so the compiler wires power itself, plants a 5 V supply, and bridges the breadboard's two power rails. You also don't need to ask for a series resistor on an LED — see below — or for the spare gates of a CD4000 chip to be tied off: a floating CMOS input reads neither HIGH nor LOW, so the compiler ties every input of an unused CMOS gate to GND itself.

While it's working the panel shows progress; the same button becomes a **cancel**, so a long or expensive request is always interruptible.

What happens to the answer

Before you are shown anything at all, the answer is put through a ladder of checks. Each one exists because of a specific way a machine-generated circuit goes wrong *while still looking perfectly fine*:

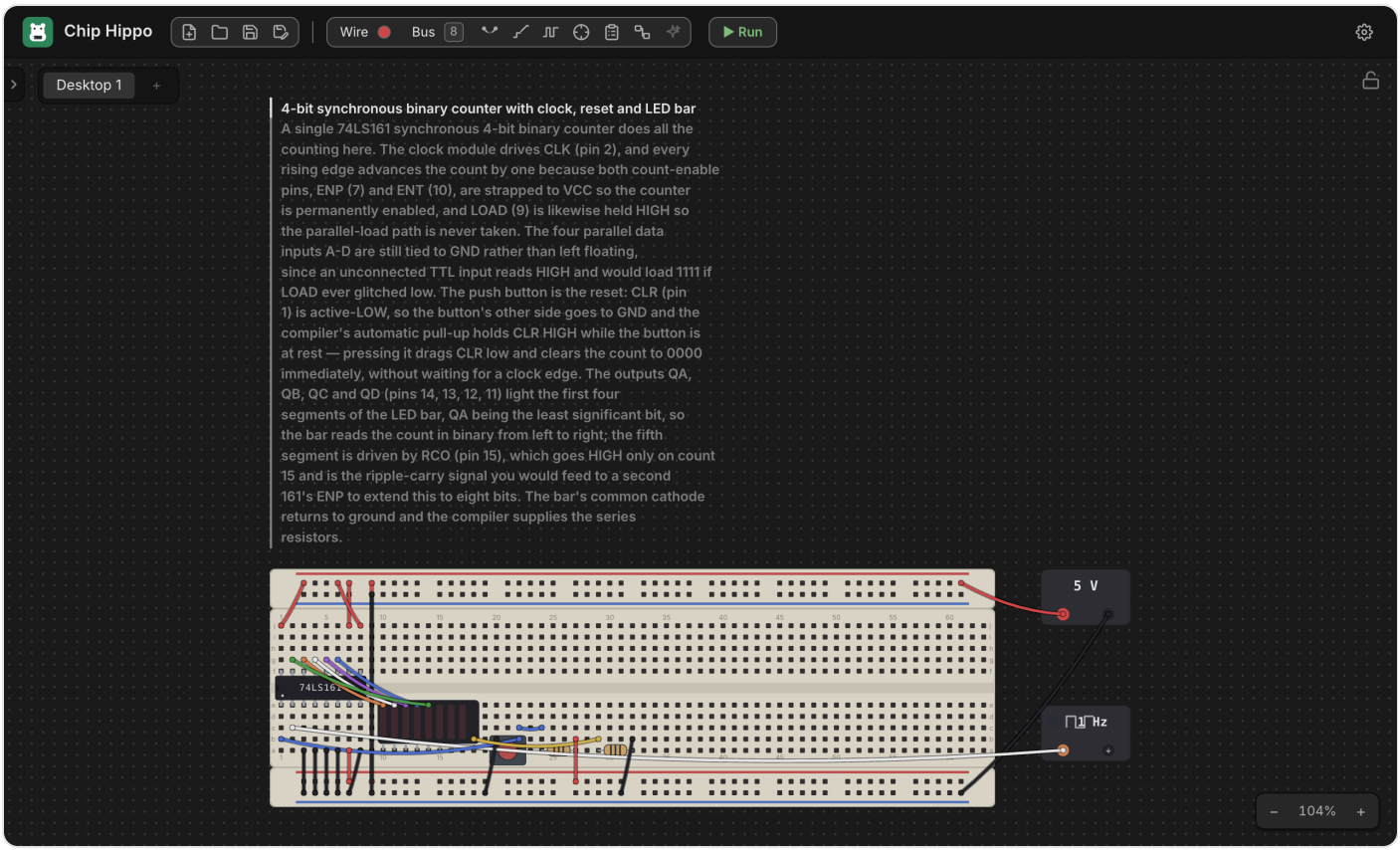
1. **It compiles.** Parts get real seats on real breadboard columns, with an allocator that makes it impossible for two parts to accidentally share a column-half — which on a real breadboard means they're shorted together. Before that, the netlist is held to the electrical rules a simulation would happily let through: no output wired straight to a power rail (the supply silently wins), and no two outputs on one net unless every one of them can be switched off — a tri-state bus.
2. **Everything seated.** No part hangs off the end of a board with pins over nothing. A part like that loads without complaint and is electrically dead.
3. **The nets match.** The connections the design *asked for* are compared against the connections the finished breadboard *actually has*. This is the check that catches an accidental short, which is otherwise invisible — the circuit loads clean and simulates happily while computing something else.
4. **It powers up and settles.** Every chip reports OK, with no conflicts, no shorts, no oscillation, and no LED burning out.
5. **Nothing is left floating.** A signal net nothing drives is reported rather than shipped — and so is an input the design uses but never connects, such as a decoder's enable, which would otherwise quietly read HIGH. (The spare gates of a TTL chip it only half uses may float; a CMOS chip's spare gates are tied off by the compiler.)
6. **Its own tests pass.** This is the interesting one. The design is asked to state its own acceptance tests — "with these switches set this way, after this many clock edges, these LEDs should read this" — and Chip Hippo **runs them**. It's the only check that tests what the circuit *means* rather than whether it's internally consistent, and it's what catches a perfectly-built adder with its bit order reversed. A design must bring at least two, each with different inputs; every test starts from the circuit as built and must spell out every switch position and every segment it reads; and each test's state is checked for shorts and fights too, not just for the lamps it names.

The panel reports which tests ran and whether they passed as part of the summary.

Placing what you get

A design that passes is **armed as a placement ghost** — the whole circuit follows your cursor, snaps against boards already on the desk, and turns red where it won't fit. Click to place it, or press `Escape` to discard it. A circuit you positioned yourself is a great deal easier to trust than one that simply appears.

However large it is, placing it is **one undo step**: `Cmd/Ctrl+Z` removes the entire thing — boards, chips, wires and all — in one go.



Every generated circuit **explains itself**. The description above is stamped on the desk as a caption over the boards, in the design's own words: which chip does the work, why each tie-off is where it is, and what the compiler added on your behalf. You didn't build it and can't ask it why it's wired the way it is — so it says so up front. The caption is an ordinary annotation anchored to the design, so it travels with it, and you can edit or delete it like any other note.

When it doesn't work

Two very different things can go wrong, and the panel distinguishes them.

The design was wrong. A pin named that doesn't exist, an input left floating, an output shared with another output, a test that fails. These go back to the model as a precise list of what failed — not as prose — and it gets up to **two** attempts to fix them. If it still can't, the panel lists what's still wrong and stops rather than looping at your expense.

Chip Hippo was wrong. If the design was described correctly but the compiler couldn't build it, the panel says so plainly and doesn't send it back — re-asking would only spend your tokens on a fault the model has no way to influence.

A few things are outside what a netlist can express, and it will tell you rather than guess:

- **Memory contents.** A ROM, EPROM or EEPROM in a generated design arrives **unprogrammed** — a netlist has nowhere to carry its bytes. The panel says so when it offers the design; load the image through the chip's **Properties** before you run it.
- **Multi-corner cans** — the oscillator packages. The model isn't offered them.
- **Capacitors and the timers** (the 555, CD4047B, CD4060B, CD4098B, CD4528B, CD4538B, CD4541B). A timer's rate lives in the values of its resistors and capacitor, and a netlist has nowhere to state a value, so the model isn't offered them. Build a timer by hand and wire the rest with the builder, or ask for a circuit clocked by a clock brick.
- **Transistors, diodes, inductors and the potentiometer**, and the CD4007UB's bare MOSFETs with them. What these do depends on how they are wired, which way round they go, or where a knob is turned — none of which a netlist can say — so the model isn't offered them either.

Review: asking what's wrong with a circuit

The panel has two modes, picked with the **Build / Review** switch in its header. Everything above is **Build**. **Review** turns the panel around: it reads the desktop you already have, runs Chip Hippo's own checks over it, and explains what it found.

Type a question and press `Enter` — or just press `Enter` with the box empty, which asks *"what is wrong with this circuit?"*.

Chip Hippo finds the faults; the model explains them. This is the same division of labour that keeps Build honest. The app derives every finding itself, from the real netlist and a real settle of the exact circuit on your desk — the model is never asked to read a wiring list and decide whether something is shorted, because a wrong answer to that would be indistinguishable from a right one. What it *is* good at is the sentence afterwards: what a fault means for your design, which one to fix first, and what to do about it.

The checks it runs are the ones that leave a circuit looking perfectly wired and quietly not working:

- **Nothing is powered** — no supply on the desk, a chip whose power pins go nowhere, or one whose VCC net never reaches a rail.
- **An input nothing drives.** The most convincing lie a real circuit tells: a floating TTL input reads HIGH, so the chip *works*, just not as designed. Only inputs that are actually in use are reported — an unused gate on a quad TTL package is idle, not broken. A CMOS input is different: it reads neither HIGH nor LOW, so every floating CD4000 input is reported, spare gates included.
- **A part switched off.** An active-low output enable left unwired reads HIGH, so the chip drives nothing at all while its wiring looks flawless.
- **Two outputs on one net**, whatever they happen to be driving today.
- **Shorts, driver conflicts, and a circuit that won't settle**, straight from the engine.

- **A chip outside its supply range** (a 74LS chip at 3 V or above 5 V), and **an LED with nothing limiting its current**.
- **A family boundary** — a 74LS output driving a CMOS input with no pull-up, too many 74LS inputs on one CMOS output, or two supply voltages on one net.

The findings appear in the transcript immediately, before the request even goes out — so if the connection fails you still have the app's own answer.

Review never changes anything. No part is placed, no wire is moved, and there is nothing to undo afterwards. That's also why it's the one thing here that works **while the simulation is running**: a review can't be refused by a frozen desk, and a running circuit is often exactly when you want to ask what it's doing.

Switching between Build and Review starts a fresh conversation — the two ask the model quite different things — but your transcript stays where it is.

LEDs and resistors

You never need to ask for a current-limiting resistor, and you shouldn't put one in your description. Chip Hippo models the *physical* requirement, not just the logical one: an LED conducting between two strongly-driven nets burns out instead of lighting, exactly as it would on a real bench. So whenever a lamp leg goes straight to a power rail — a display's common leg, a single LED's cathode to GND or its anode to VCC, a segment of an isolated bar — the compiler interposes a series resistor for you (a resistor network, when a display or bar has several legs on a rail). A single LED gets a resistor of its own, plugged in the way you would on a bench: the resistor's top lead goes into the LED's column and its other lead straight into the power rail, so the LED, the resistor and the rail are joined by the breadboard itself, with no jumper wires between them. See Chips & Components for the same rule when you're wiring by hand.

Cost, privacy and the simulation

Requests go to whichever endpoint you configured, billed to your own account — Chip Hippo neither proxies them nor sees them. The parts catalog that makes up most of each request is sent in a form your provider can cache, so a repair attempt costs far less than the first ask.

The panel won't build while the simulation is **running** — press **Stop** first. A generated circuit is a document edit like any other, and the desk is frozen while the circuit runs. **Review** is exempt: it changes nothing, so there is nothing for a frozen desk to refuse.

A review sends a compact description of your circuit — the parts, which pins are joined, and the findings — but never the layout, and never anything about your project beyond the desktop you asked about.

See also

- Settings — the AI tab in full, alongside every other preference.
- Running a Simulation — the engine every generated circuit is proved against before you see it.
- Files, Saving & Undo — how a placed circuit is saved, and how one undo step removes it.

Arduino Integration

Chip Hippo can talk to a real **Arduino** over USB while the circuit runs. An **Output** hands a value from the breadboard to a sketch — a number to show on a real display, a bit to switch a real relay — and an **Input** hands a value from the sketch back onto the breadboard: a sensor reading, a key press, the answer to a question the circuit just asked.

The sketch side is written for you. Chip Hippo **generates a header file** for each board, declaring one function per Output and one small object per Input; you include it, call two functions from `setup()` and `loop()`, and fill in what each Output should do. The link underneath — framing, checksums, acknowledgements, retries — is the header's job and never yours.

No Arduino to hand? Every Chip Hippo has a built-in **Mock** connection that plays the board's part itself (see The Mock connection), so you can build and try an integration before a wire is connected.

What you need

- An Arduino (any board with a USB serial port — an Uno, a Nano, a Mega, a Leonardo…) and the Arduino IDE to upload sketches to it — or just the built-in Mock.
- A **connection** for it in Settings ▸ Serial I/O (below), naming it and saying which port it is plugged into.
- One or more **Outputs** or **Inputs** on the desk, each assigned to that connection.

Nothing here touches the network: a connection is a USB cable, and Chip Hippo opens the port only while the circuit is running.

Connections

A **connection** is a named Arduino: *Display board*, *Sensor Nano*. Open Settings (`Cmd/Ctrl+,`) and choose the **Serial I/O** tab. **+** beside the connection list creates one; give it a name and pick its **Port** from the list of what is plugged in right now (**Re-scan** after plugging a board in). **Baud rate** defaults to 115200, which suits every current board. **Advanced** holds the rest of the serial framing — parity, stop bits and flow control. Leave those alone unless you have a reason. (Data bits are always 8 and there is no XON/XOFF flow control: the link carries binary frames, which need every bit of every byte.)

Apply saves the connection. If the port isn't plugged in at that moment you're told so, and the button becomes **Apply anyway**, which saves it regardless — useful when you're setting up before the board arrives. **Open window…** opens the connection's window, and the bin beside the list (click it twice) deletes the connection. An Output or Input still using it keeps its assignment — its Properties show the connection as *(not on this computer)* — and the circuit won't run until you choose another one there.

A connection whose port is missing, or which still **Needs configuration**, says so in the list, and its **Port** is marked in red.

The list always starts with the **Mock**, marked *Built-in*. It has nothing to configure and can't be removed, and its name is reserved: a connection of your own can't be called *Mock*.

Connections travel with the project

The connection's **name** and serial settings are saved in the project file, so a design opened on another computer still knows which board each Output talks to. The **port** is not — `/dev/cu.usbmodem14101` on your Mac means nothing on someone else's PC — so a connection that arrives in a project is added to that computer's Settings marked **Needs configuration**, and the circuit won't run until a port has been chosen for it.

The Mock connection

The **Mock** is a pretend Arduino inside Chip Hippo. Choose it as an Output's or Input's **Connection** like any other, and press **Run**. There is no port to pick, no sketch to upload and no header to generate. It speaks the same protocol a real board does, byte for byte, so what works with the Mock works with a board.

When a run starts, the Mock's connection window opens beside the desk (without taking focus from it), with a send panel under its stream. You can also open it from Settings ▸ Serial I/O: pick **Mock** and press **Open window…**. It shows:

- **What arrives.** Each value an Output sends appears as a line, named and split into its fields: `→ OUTPUT Display value=0x3F dp=1`. The Mock acknowledges it at once, so the circuit carries straight on.
- **What you send.** There is one row per Input on the Mock. A bit field is a toggle; a byte or a word is a row of bit toggles plus a hex field, and editing either updates the other. **Send** sends that Input's whole value, exactly as a sketch's `send()` would, and it lands on the board like one. **Send all** sends every row in turn. The rows appear once the circuit has run with the Mock, since that is when the window learns your design.
- **Log text.** Type a line and press **Log** (or Enter). It arrives in the log exactly as a sketch's `ChipHippo.println()` would.

Faults, folded away at the bottom, make the Mock misbehave once, on purpose, so you can see how the link copes:

Fault	What happens
Drop next ACK	The Mock swallows its acknowledgement of the next Output. Chip Hippo waits, sends it again, and the circuit carries on — the value still arrives only once.
Corrupt next frame	The next acknowledgement or Input the Mock sends is damaged in transit. Chip Hippo notices and asks for it again.
Ignore handshake	The Mock doesn't answer on the next Run, which stops with *No answer*.
Wrong signature	The Mock answers the next Run as a sketch built for a different design, which refuses to start.

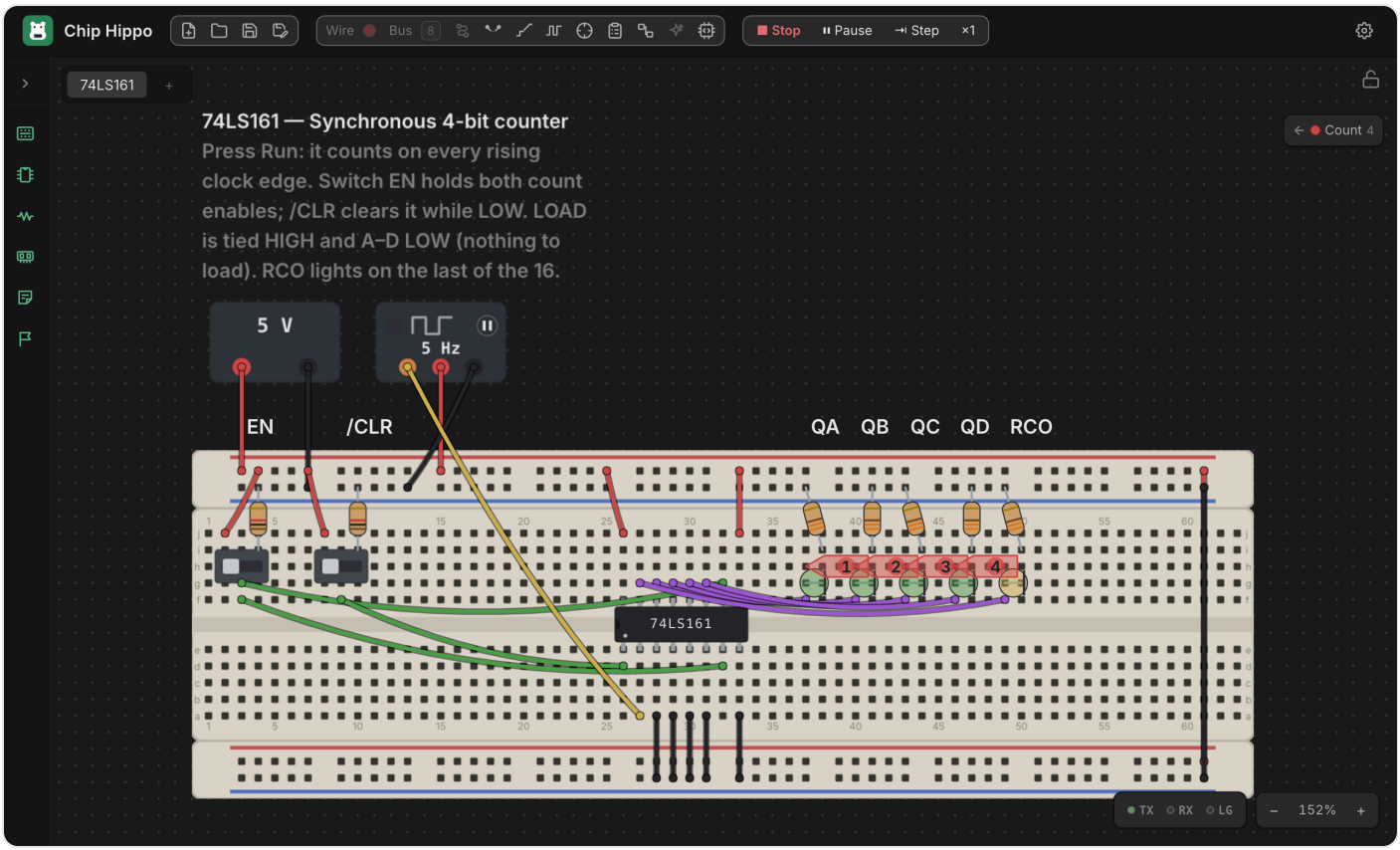
A fault stays armed (its button highlighted) until it has happened. Click it again to disarm it. You can arm one while the circuit is stopped, ready for the next Run. What each fault causes shows up in the stream as a warning line (turn on **Protocol** to see the traffic around it too).

Outputs and Inputs

Add either from the parts palette's **SIGNALS** section: pick **Output** or **Input**, click anywhere on the desk, and choose how many **pins** it has — 1, 2, 4, 8 or 16. You can change that afterwards in its Properties.

Each one appears as a **card** on the right-hand edge of the desk, under the signal buttons: its direction arrow (← Output, pointing out; → Input, pointing in), its colour, its name and how many pins it has. Under the name sits a small numbered chip for every pin that isn't on the board yet, plus one more — the **trigger**, marked with an arrow — unless the element's trigger is **Auto**, which has no trigger tag at all. **Drag a chip onto a breadboard hole** to plant that pin there as a **tag**; drag a planted tag to move it, and press `R` while dragging (or with the tag selected) to turn it. **Delete** unplugs a selected tag back onto its card. The card's right-click menu offers **Properties...**, **Remove All Tags** and **Delete Output** / **Delete Input**; a tag's own menu adds **Remove Tag** and **Add to analyzer**.

A tag is plugged into its hole the way a wire end is — one hole, one lead — and deleting the breadboard under a tag sends it back to the card; the Output or Input itself is never lost with it.



Pins and fields

An element's pins are grouped into **fields**, and each field becomes one parameter of the Arduino function (for an Output) or one setter (for an Input). A field is a **Bit** (1 pin, a `bool`), a **Byte** (8 pins, a `uint8_t`) or a **Word** (16 pins, a `uint16_t`), up to 16 pins in all. The Properties dialog lists them in pin order with a name each; **+ Bit**, **+ Byte** and **+ Word** add one, and the × removes one.

Pin 1 is the **least significant bit** of the element's value, pin 2 the next, and so on. A pin whose tag isn't planted, or whose net is floating or unknown, **reads as 0**.

Properties

Right-click a card (or one of its tags) and choose **Properties…** to edit:

Setting	What it does
Name, **Description**	The name is also the function (Output) or object (Input) name in the header, turned into a valid C++ identifier
Color	The card's dot and its tags — any of the seven signal colours
Connection	Which Arduino it talks to. A new Output or Input joins the connection the desktop already uses, or else your first Arduino — the Mock only when none is set up. The gear beside it opens Settings ▸ Serial I/O
Trigger	**Auto**, **Rising**, **Falling** or **Either**. Auto watches no line: an Output sends whenever its value changes, an Input puts each value on the board as it arrives. The others name which transition of the trigger line counts. A new Output starts on **Rising**, a new Input on **Auto**
Trigger starts	**Low** or **High** — what the trigger line is taken to have been before the run began, so a line that starts high can still fire (or not) on the first settle. Greyed out on Auto, which has no line
Pins	The fields, as above

How values move

Chip Hippo only ever looks at the board **between settles** — when every chip's output has stopped changing. A glitch inside one settle is never sent anywhere, and nothing from the Arduino ever lands on the board halfway through a ripple.

An Output fires on its trigger, or on a change

When an Output's trigger line makes the chosen transition, Chip Hippo samples the Output's pins and **sends the value**. On **Auto** there is no trigger line: after every settle Chip Hippo samples the pins and sends the value if it differs from the last one it sent — and always at the very first settle of a run, so the sketch learns where the circuit starts. **The circuit then waits** — no clock edges, no further settles — until the Arduino's function has *returned* and the board has acknowledged it. So whatever the sketch does in response has already happened before the circuit moves on, and an Input the sketch sends from inside that function lands in the same step: the request/response shape (the circuit asks, the Arduino answers, the circuit carries on with the answer) works with no timing of your own.

Auto sends **every** value the pins settle to, including the in-between ones: flip two switches one after the other and the Arduino hears both steps. When the pins only hold a meaningful value at one moment — a data bus, a latch strobe — use an edge on that strobe instead.

An Output or Input waiting for an edge **won't run without its trigger tag on the board**: pressing Run says which one is missing it, and offers its Properties, where you can switch it to Auto instead.

An Input is live or triggered

An Input drives its pins at the strength of a chip output — two things disagreeing over one net is reported as a conflict, exactly as two chip outputs would be — but **drives nothing at all** until the Arduino has sent it a first value.

- On **Auto**, an Input is **live**: each value the Arduino sends is put on the board at the next opportunity between settles.
- On an edge, the latest value the Arduino sent **waits** and is put on the board only when the trigger line makes its transition — the way a latch loads on its clock. Earlier values the circuit never took are simply replaced.

Switching an element to Auto takes its trigger tag off the board, but Chip Hippo remembers where it was: switch back to an edge and the tag returns to the same hole — unless something else has been plugged in there meanwhile, in which case it waits on the card.

Generating the board's code

The **Generate** button in the desk tools pill opens a card listing every connection this desktop's Outputs and Inputs use. Each has one button: **Generate** while its code has never been generated or is out of date, which generates it and opens the files, and **View files…** once it is up to date, which just opens them. In the file viewer, **Save As…** saves the file on show: save `ChipHippo.h` into that board's sketch folder and include it:

```
#include "ChipHippo.h"

void setup() {
  ChipHippo.onConnect(sendInputs);  // optional — see below
  ChipHippo.begin();
}
void loop() { ChipHippo.poll(); }

// The Output "Display" arrives here: its name + In, its fields the parameters.
void DisplayIn(uint8_t value, bool dp) {
  ChipHippo.print("showing ");
  ChipHippo.println(value);
}

// The Input "Keypad" is sent from here: set its fields, then send it whole.
void keyPressed(uint8_t code) {
  ChipHippo.KeypadOut.setCode(code);
  ChipHippo.KeypadOut.setReady(true);
  ChipHippo.KeypadOut.send();
}

// Runs at the start of every run: give each Input its starting value.
void sendInputs() {
  ChipHippo.KeypadOut.setCode(0);
  ChipHippo.KeypadOut.setReady(false);
  ChipHippo.KeypadOut.send();
}
```

In the sketch, names read from the **Arduino's** side. An Output leaves the circuit and *arrives* at the Arduino, so the Output called *Display* is the function `DisplayIn()`; an Input is *sent* from the Arduino into the circuit, so the Input called *Keypad* is `ChipHippo.KeypadOut`. The two different endings also mean an Output and an Input can share a name. A name that already ends that way isn't doubled (an Output called *DataIn* stays `DataIn`). A name that can't be used as it is (two Outputs with the same name, a field called `delay`) is changed, and the Generate card lists each change under its connection. That includes an Output's field named in capitals, such as `SP` or `HEX`, which becomes `SP_` or `HEX_`: the Arduino cores use names in capitals for their macros, and a parameter spelled like one won't compile.

The file viewer shows the header beside `ChipHippoExample.ino`, an example sketch that uses it, each in its own tab, with line numbers. The example shows the three things a sketch does, each labelled where it

happens: **receiving** (a function per Output that logs what arrived), **sending** (every Input's starting value at the start of a run, then the first Input counting up once a second from `loop()` — a stand-in for reading a switch or a sensor) and **logging** (a line when a run starts, one per Output that arrives, one per value sent). The Python programs do the same, timing the loop with the module's own `ticks_ms()` and `ticks_diff()`, which read the same on every board. Select part of a file and **Copy** (or ⌘C / Ctrl+C) takes just that; with nothing selected it takes the whole file, and ⌘A / Ctrl+A selects all of the file on show. **Save As…** opens where you last saved that file for this board and desktop, so a new version goes over the old one without hunting for the folder. Closing the viewer returns to the Generate card. The header's own opening comment points at the example too.

An Input's `send()` returns `true` once Chip Hippo has the value. It returns `false` when nobody is listening: at once before a run has greeted the sketch, and after one has stopped (the first time, after a second and a half of resends). `ChipHippo.connected()` says which state the link is in.

An Input puts nothing on its pins until its first value arrives, so a sketch should say where its Inputs stand as soon as a run begins. `ChipHippo.onConnect(fn)` is for exactly that: `poll()` runs `fn` at the start of **every** run. That matters on boards that don't restart when Chip Hippo opens the port — a Leonardo, a Micro, most boards with USB built into the chip — where the sketch simply carries on from one run into the next and `connected()` alone can't tell that a new one has begun.

The Mock needs no header. A desktop whose Outputs and Inputs are all on the Mock is told so, and the Mock never marks the Generate button out of date.

The header never contains your code, so saving over it after a change to the design is always safe. Three rules keep the link healthy:

- **Call `ChipHippo.poll()` often**, and keep Output functions short. The circuit is stood still while an Output is being handled; a `delay()` there is time it waits. Chip Hippo resends a value it hasn't had acknowledged after half a second, and stops the run after three tries.
- **Log with `ChipHippo.print()` / `println()`, never `Serial`.** Chip Hippo owns the port while the circuit runs; anything printed straight to `Serial` is noise on the link. What you print through `ChipHippo` appears in the connection window.
- **One header per connection.** Two boards on one desktop get two headers, each declaring only its own Outputs and Inputs.

Python boards (MicroPython and CircuitPython)

A board that runs Python — a Raspberry Pi Pico, an ESP32, an Arduino Nano ESP32 or Nano RP2040 Connect, most Adafruit boards — can take part too. Set its connection's **Language** to **Python** in Settings ▸ Serial I/O, and Generate writes `chiphippo.py` instead of a header: one module that runs on MicroPython and CircuitPython alike. (An Uno, Nano, Mega or Leonardo is too small for Python and stays C++.)

```
from chiphippo import link


@link.display_in  # the Output "Display" arrives here
def display_in(value, dp):
    link.print("showing", value)


@link.on_connect  # runs at the start of every run
def send_inputs():
    link.keypad_out.set_code(0)
    link.keypad_out.set_ready(False)
    link.keypad_out.send()


link.begin()
while True:
    link.poll()
```

It is the header said in Python: the same names in Python's own style (`display_in`, `keypad_out.set_code`), an Output handled by decorating a function with `@link.<its name>_in`, and `link.print()` for the log. The file viewer shows the module beside `main.py` (MicroPython) and `code.py` with `boot.py` (CircuitPython). Save `chiphippo.py` and the program for your board onto it.

Where the link runs is the one real difference between the two:

- **MicroPython** uses the board's own USB serial port — the one its REPL uses — so point the connection at that port. Inside the link's data a `0x03` byte is just data, not Ctrl-C; between messages, Ctrl-C still stops the program, so Thonny and `mpremote` can always get in to replace its files. The example `main.py` switches Ctrl-C off on its first lines, before it imports `chiphippo`; if you write your own, start it the same way. A plain `print()` would land in the middle of the link, so log with `link.print()`. On a board whose USB is a separate serial chip (a classic ESP32 DevKit), MicroPython always talks at **115200 baud, 8N1**: leave the connection there — the Generate card warns if it isn't. A board with USB built in (a Pico, a Nano ESP32) ignores the rate.
- **CircuitPython** gives the link a second USB serial port of its own, which `boot.py` turns on (`usb_cdc.enable(console=True, data=True)`). After the board resets, a new port appears beside the REPL's. For a Python connection the **Port** list names the two — *port 1 of 2: REPL* and *port 2 of 2: CircuitPython data* — so choose the data port. (Any board with several ports shows which of them each one is.)

A function that raises an exception is reported in the connection window and the circuit carries on — on a board there is nowhere else it could be seen. Most Python boards don't restart when Chip Hippo opens the port, so give your Inputs their starting values from an `@link.on_connect` function, as above. `link.begin()` can also be handed a port you have opened yourself — a `machine.UART` or `busio.UART` at the connection's baud rate.

Using the generated files in your own firmware

Chip Hippo itself is free software under the GNU General Public License, but the files it generates for a board are not bound by it. `ChipHippo.h`, `chiphippo.py` and the example programs beside them can go into your own firmware — open or closed, given away or sold — under whatever terms you choose. Each file says so in its opening comment; the full wording is Chip Hippo's LICENSE-EXCEPTION.

Keeping the header current

Every header carries a **design hash** of what it was generated from — the connection's settings and the names, fields and order of its Outputs and Inputs. Change any of those and the header is **out of date**: the Generate button shows a dot, and on the Generate card that connection's button reads **Generate** again. Positions of tags on the board, colours and descriptions don't change the hash. A header generated by an older Chip Hippo whose generated code has since changed shows as out of date too, so that generating it again picks the change up.

Pressing **Generate** is what marks a header up to date — the hash is kept in the project file — so save the new file over the old one (**Save As...** offers the same place) before you rebuild the sketch.

A run checks something narrower: the **layout** the sketch was built for — which Outputs and Inputs there are, in what order, and how each one's pins are split into fields. Names aren't part of it, so a design you've only renamed things in still runs while you get round to regenerating. But a sketch built for a different layout would hand values to the wrong functions or parameters, so that **stops the run** before it starts, asking you to generate the header again and re-upload the sketch. A sketch built by a version of Chip Hippo that speaks a different protocol is refused the same way.

Running with an Arduino

Press **Run** as usual. Before anything simulates, Chip Hippo checks that every Output and Input in use has a connection, that each connection is configured, and that its port is plugged in — and if not, says which and offers to open the right Properties or Settings. "In use" means on the breadboard: a board is checked (and opened) only when at least one of its Outputs or Inputs has a tag planted, so an Arduino you've left unplugged doesn't stop a circuit that isn't using it. It then **opens each port and greets the sketch**, asking again every quarter of a second for up to five seconds; many boards reset when the port

opens, so this is also the time a sketch has to boot. A board that never answers stops the run with the port named, and so does a sketch built for a different layout (above).

While it runs:

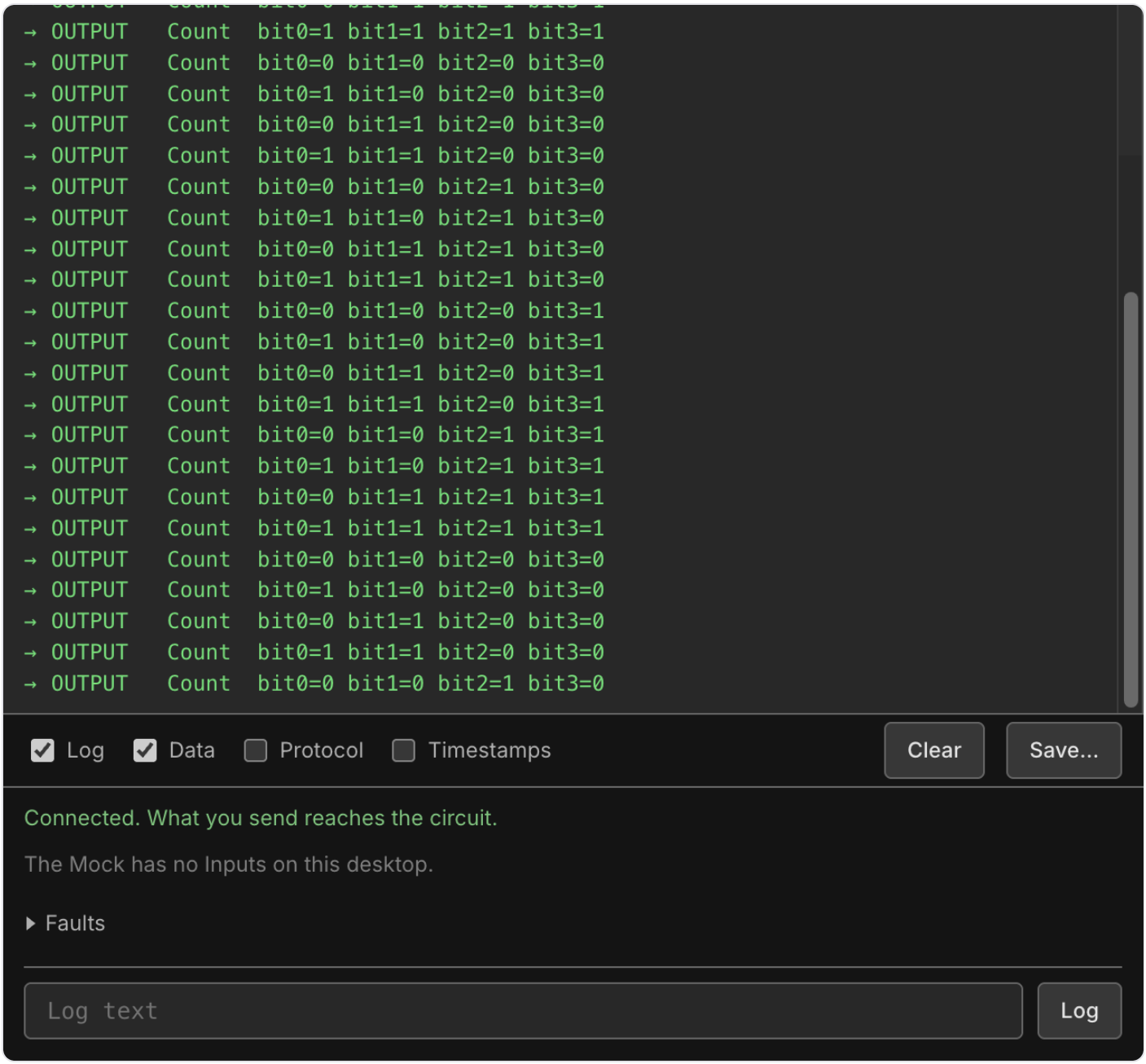
- **TX**, **RX** and **LG** lamps beside the zoom controls flash as values go to an Arduino, come back from one, and as log text arrives. Click any of them to open a connection window (a menu asks which, if there are several).
- A board that **restarts** mid-run (a reset button, a brown-out), one that is **unplugged**, one that fails to acknowledge a value after three attempts, and one whose sketch sends a value that Chip Hippo never acknowledges all **stop the run** with a message naming the connection. A sketch that has left the run has forgotten it, so press **Run** again to reconnect.

Stop closes every port, so the Arduino IDE can have it back to upload a new sketch — there's no need to quit Chip Hippo between uploads.

The connection window

Each connection has a window showing everything that happens on it, in one stream: what its sketch printed through `ChipHippo`, and the values and protocol frames going back and forth. Because it is one timeline, cause and effect read in order — a value arrives, then the line the sketch printed in response.

Open one by clicking the lamps, with **Open Connection Window** on an Output's or Input's right-click menu (it works while the circuit runs), or by picking the connection in Settings ▸ Serial I/O and pressing **Open window...**.



Every line starts with a mark saying what it is:

Mark	Line	Example
(none)	Text the sketch printed	`got 41`
`→`	A value sent to the board (an Output)	`→ OUTPUT Display value=0x3F dp=1`
`←`	A value from the board (an Input)	`← INBOUND Keypad code=0x07 ready=1`
`·`	Protocol traffic: the handshake, acknowledgements, the port opening and closing	`· ← ACK seq 12`
`!`	Something went wrong, and what the link did about it	`! No ACK for seq 12 — resending (attempt 2/3)`

Arrows are always from Chip Hippo's side: `→` leaves Chip Hippo, `←` arrives. Hover over a value's line to see the exact bytes it travelled as. A value that had to be sent again appears once, with the resend as a warning line beneath it.

The footer holds the controls:

- **Log**, **Data** and **Protocol** choose which lines to show — the first two on by default, Protocol off. Hidden lines are still recorded and come back when you turn them on again. **Warning lines are always shown.**
- **Timestamps** adds each line's time since the run started, to the millisecond (`+ 12.345`) — the way to line a frame up with what the sketch printed.
- **Clear** empties the window. **Save…** writes the whole stream to a text file — every line, with its time, whatever the filters are showing.

A new Run starts the window afresh. After **Stop** everything stays, so a failure can be read at leisure — even after closing and reopening the window. It keeps the last 2,000 lines, follows new lines only while you're scrolled to the bottom, and its text can be selected and copied. Each connection's window remembers where it was and how it was set up.

Under the hood

For the curious, or for anyone writing their own firmware: the link is a small framed protocol over the serial port, **version 1**. Every frame is `0x7E · TYPE · SEQ · LEN · PAYLOAD · CRC`, with `0x7E` and `0x7D` escaped everywhere after the start byte and a CRC-16/CCITT-FALSE (polynomial `0x1021`, initial value `0xFFFF`) over the unescaped `TYPE` to `PAYLOAD`. A value's payload is the element's index, its width in bits, and the value as two bytes, little-endian.

At the start of every run Chip Hippo sends a HELLO carrying the protocol version, a fresh 16-bit session number and the layout signature (a CRC-32 of the layout described above), and the sketch answers with

its own version and signature. Both sides take part in the run only if the two match. Outputs and Inputs are then acknowledged and resent on a timeout or a NAK. A resend is recognised by its sequence number, so a function never runs twice for one value, and an Output is acknowledged only once its function has returned. Log text travels in unacknowledged chunks that never split a character.

The whole protocol — every frame, rule and constant, with worked examples — is on the Serial Protocol page. The book icon at the top right of Settings ▸ Serial I/O and of the Generate card opens it too.

Serial Protocol

Chip Hippo serial protocol, version 1. This page defines the bytes that travel between Chip Hippo and a board at the other end of a serial connection. You never need it to use the Arduino integration — the header and module Chip Hippo generates speak it for you — but it is the whole contract. Anything that follows this page (a sketch in another language, firmware in Rust, a test harness on a pseudo-terminal) can take the board's place.

---

1. Conventions

- **Host**: Chip Hippo. **Device**: whatever is at the other end of the connection, usually an Arduino. **Circuit**: the simulated breadboard.
- **Output** and **Input** are named from the circuit's side: an Output's value leaves the circuit for the device; an Input's value comes from the device into the circuit.
- A **data frame** is an `OUTPUT` or an `INBOUND` — the only frames delivered reliably (§6). A data frame is **outstanding** from its first transmission until it is acknowledged or its delivery fails.
- A **session** is one run's conversation on one connection, from the handshake (§5.4) until the port closes.
- **MUST**, **MUST NOT**, **SHOULD** and **MAY** state requirements, as in RFC 2119. A paragraph marked *Note* explains a rule and adds none.
- Byte values are hexadecimal (`0x7E`). Multi-byte integers are **little-endian**. A frame is written as its bytes on the wire, in hex.

---

2. Transport

- A byte stream with no framing of its own: a UART, USB CDC, a pseudo-terminal. It MUST deliver bytes in order. It may lose or corrupt them; §3.5 and §6 recover.
- The serial parameters — baud rate, parity, stop bits — come from the connection (Settings ▸ Serial I/O). Data bits are always **8**: frames are binary, and CRCs, signatures and values use every bit of a byte. The default, and the recommendation, is **115200 baud, 8-N-1, no flow control**.
- **No software flow control.** XON (`0x11`) and XOFF (`0x13`) are ordinary bytes here — `0x11` is the `INBOUND` type — and only `0x7E` and `0x7D` are escaped, so a link that swallows them loses frames. Chip Hippo never opens a port with XON/XOFF.
- One host and one device per connection. The device sees only the Outputs and Inputs assigned to its connection. A run may use several connections at once; each is an independent link with its own session.

- The host writes **nothing but whole frames**: no preamble, no line endings, no break. A device MAY rely on this (the MicroPython library does, §13).
- The host opens the port when the circuit starts running and closes it when it stops. Opening the port resets many boards (a classic Arduino's auto-reset), and a bootloader may swallow bytes for a second or two. Many others — native-USB boards such as a Leonardo, and most Python boards — do **not** reset, and their program carries on from one run into the next. So a device learns that a run has begun from the handshake (§5.4), never from its own start-up.

---

3. Frames

3.1 Frame format

```
START  TYPE  SEQ  LEN  PAYLOAD     CRC_LO  CRC_HI
0x7E   1     1    1    LEN bytes   1       1
```

Field	Size	Meaning
`START`	1	Always `0x7E`, never escaped. Begins a frame.
`TYPE`	1	The frame type (§4).
`SEQ`	1	In `OUTPUT` and `INBOUND`: the frame's sequence number, 1–255 (§6.1); one with SEQ 0 is ignored. In `ACK`: the sequence number acknowledged. In every other frame: 0, which the receiver MUST ignore.
`LEN`	1	The number of payload bytes, counted **before** escaping: 0–255.
`PAYLOAD`	`LEN`	Defined per type (§4).
`CRC`	2	CRC-16/CCITT-FALSE (§3.3) of the unescaped bytes `TYPE SEQ LEN PAYLOAD`, little-endian.

A frame's **body** is everything after `START`, from `TYPE` to `CRC_HI`.

3.2 Size limits

Quantity	Bytes
Payload	0–255
Body, unescaped: `TYPE` + `SEQ` + `LEN` + payload + CRC = 1 + 1 + 1 + 255 + 2	at most 260
Frame on the wire: `START` + every body byte escaped = 1 + 2 × 260	at most 521

- A sender MUST NOT send a payload longer than 255 bytes: `LEN` could not say so.
- Every receiver has a **payload limit**, the longest payload it accepts. The host's is 255. A device's MAY be smaller, but MUST be at least 7 — the longest payload the host sends (`HELLO`).
- A receiver that reads a `LEN` greater than its payload limit MUST treat the frame as damaged at once (§3.5), without waiting for the rest of it.

Note: so a receiver never holds more than 3 + its limit + 2 bytes of a frame — 260 for the host, 12 for a device with the smallest limit. Escaping needs no buffer of its own: each escape pair is undone as it arrives.

3.3 CRC

CRC-16/CCITT-FALSE: polynomial `0x1021`, initial value `0xFFFF`, input and output not reflected, no final XOR. The check value, for the ASCII bytes `123456789`, is `0x29B1`.

```
uint16_t crc16(const uint8_t *p, size_t n) {
  uint16_t crc = 0xFFFF;
  while (n--) {
    crc ^= (uint16_t)(*p++) << 8;
    for (uint8_t i = 0; i < 8; i++)
      crc = (crc & 0x8000) ? (crc << 1) ^ 0x1021 : (crc << 1);
  }
  return crc;
}
```

3.4 Escaping

Every body byte that is `0x7E` or `0x7D` is sent as two bytes: `0x7D`, then the byte XOR `0x20`. No other byte is escaped.

Unescaped	On the wire
`0x7E`	`0x7D 0x5E`
`0x7D`	`0x7D 0x5D`

Escaping applies to every body byte — `TYPE`, `SEQ`, `LEN`, the payload and both CRC bytes — while `LEN` and the CRC are computed over the unescaped bytes. So `0x7E` on the wire is **always** the start of a frame, which is how a receiver finds the next frame after noise, a torn frame or a reset.

3.5 Receiving a frame

A receiver is either **hunting** (between frames) or **in a frame**.

1. While hunting, it discards every byte but `0x7E`, which begins a frame.
2. In a frame, a `0x7E` abandons the partial frame and begins a new one. A frame cut short this way is **torn**, not damaged: it is dropped without a word, and the sender's timer (§6.4) covers it. Any other byte is unescaped (§3.4) and read, in order, as `TYPE`, `SEQ`, `LEN`, `LEN` payload bytes and the two CRC bytes.
3. The frame is **damaged**, and the receiver goes back to hunting, when:
   - a `0x7D` is followed by anything but `0x5E` or `0x5D` — or `0x7E`, which tears the frame instead (step 2);
   - `LEN` is greater than the receiver's payload limit (§3.2) — known as soon as `LEN` is read; or
   - the CRC does not match — known when the second CRC byte is read.
4. A damaged frame is discarded. If the receiver is in a session (ACTIVE, §5.2 and §5.3), it MUST then send a `NAK` (§4.4) at once — **unless** the damaged frame's `TYPE`, as received, is `HELLO`, `HELLO_ACK`, `ACK`, `NAK` or `LOG`: frames never resent in answer to a NAK. A frame damaged before its `TYPE` arrived is NAKed. A receiver not in a session MUST NOT send a `NAK`: before a handshake, damage is most likely a bootloader's noise.
5. An intact frame is dispatched by `TYPE`. A frame of a type the receiver does not know, or never receives (a device never receives `INBOUND`), MUST be ignored.

Note: a `LEN` damaged into a larger value is damage at once if it exceeds the receiver's payload limit (step 3). Otherwise the receiver waits for bytes that belong to the next frame, whose `0x7E` then tears it (step 2), and the sender's timer (§6.4) covers the loss. A `LEN` damaged into a smaller value fails the CRC.

4. Frame types

Value	Name	Sent by	SEQ	Payload	Delivery
`0x01`	`HELLO`	host	0	7 bytes (§4.1)	resent until answered (§5.4)
`0x02`	`HELLO_ACK`	device	0	7 bytes (§4.1)	—
`0x06`	`ACK`	either	the `SEQ` acknowledged	none (§4.3)	—
`0x15`	`NAK`	either	0	none (§4.4)	—
`0x10`	`OUTPUT`	host	1–255	4 bytes (§4.2)	reliable (§6)
`0x11`	`INBOUND`	device	1–255	4 bytes (§4.2)	reliable (§6)
`0x20`	`LOG`	device	0	1–255 bytes (§4.5)	not acknowledged

The values `0x00`, `0x7D`, `0x7E` and `0xFF` are never frame types. Every other unassigned value is reserved for later versions (§11).

4.1 HELLO and HELLO_ACK

The payload of both (7 bytes):

Offset	Size	Field
0	1	Protocol version: **1**.
1	2	Session (§5.1), little-endian.
3	4	Layout signature (§7.3), little-endian.

For protocol version 1, the first three payload bytes are the version and the session. They are all a device needs to answer a `HELLO`, and all the host needs to judge a `HELLO_ACK`'s version. A later version MAY change the `HELLO` and `HELLO_ACK` payloads after establishing that it is speaking to a different version (§11).

- The host sends `HELLO` with its version, the session it has chosen and the signature of the layout it expects.
- A device MUST answer every intact `HELLO` of at least 3 bytes, for any session but 0, with a `HELLO_ACK` carrying the `HELLO`'s session and **its own** version and signature — in any state, and even

when they differ from the `HELLO`'s. Only the host has to explain a mismatch, so only the host needs to see one. (A `HELLO` longer than the device's payload limit is not intact: it is damaged, §3.5.)

- A device MUST ignore a `HELLO` shorter than 3 bytes, or for session 0, without answering it.
- A `HELLO_ACK` for session 0 is the device's **announcement** that it is in no session (§5.3) — sent when its program starts, and when it leaves a session after a failed delivery. It answers nothing.
- A host MUST ignore a `HELLO_ACK` shorter than 3 bytes. What it does with any other is in §5.2 and §5.4.

4.2 OUTPUT and INBOUND

`OUTPUT` carries an Output's value from the circuit to the device; `INBOUND` carries an Input's value from the device to the circuit. The payload of both (4 bytes):

Offset	Size	Field
0	1	Element index (§7.1), within this connection and direction.
1	1	Width, **1–16**: the element's pin count.
2	2	Value, little-endian and right-aligned: bit 0 is the element's pin 1.

- A sender MUST send every value bit at or above the width as 0. A receiver MUST ignore them.
- An element is an ordered list of **fields** — bit (1 pin), byte (8) or word (16), 16 pins in all at most — packed into the value in order from bit 0. A byte followed by a bit puts the byte in bits 0–7 and the bit in bit 8. The fields need no wire support of their own: they only say which bits the generated code hands to which parameter, which is why the layout signature covers them (§7).
- An `INBOUND` always carries the **whole** element — there are no partial updates. The device keeps its own copy of the state and sends all of it.
- A data frame is **unusable** when it is intact but its `LEN` is not 4, its width is not 1–16, its index names no element, its width is not that element's, or (at a device) the program has no handler for its Output. An unusable frame MUST be acknowledged like any other (§6.2) and then discarded: a resend would carry the same bytes.

4.3 ACK

`SEQ` is the sequence number of the data frame acknowledged. The payload is empty, and a receiver MUST NOT examine it. When an `ACK` is sent, and what one means, is §6.2.

4.4 NAK

`SEQ` is 0 and the payload empty; a receiver MUST NOT examine either.

A `NAK` means: *a frame arrived damaged — if you have a data frame outstanding, send it again now*. It names no frame, because the damaged frame's `TYPE` and `SEQ` cannot be trusted; and it needs to name none, because each side has at most one data frame outstanding (§6).

- A side receiving a `NAK` with a data frame outstanding MUST retransmit that frame at once — the same bytes, the same `SEQ` — and MUST NOT advance its sequence counter. The retransmission is one of the frame's sends (§6.4).
- A side receiving a `NAK` with nothing outstanding MUST ignore it.

A `NAK` is never acknowledged, never resent and never answered with a `NAK`, so it can cause no more retransmissions than the send limit allows.

4.5 LOG

The payload is 1–255 bytes of UTF-8 text, with no terminator. A newline (`0x0A`) ends a line; text without a trailing newline continues on the same line when the next `LOG` arrives.

- A sender MUST NOT send an empty `LOG`, and SHOULD split longer text between characters, never inside a multi-byte UTF-8 sequence. A receiver MUST ignore an empty `LOG`, and SHOULD decode `LOG` text as one stream, so that a character split across frames still arrives whole.
- `LOG` is **not acknowledged**: its `SEQ` is 0, and it is never NAKed, resent or de-duplicated. Log text changes nothing on the circuit, so losing a line to a damaged frame is cheap, and blocking the device on every `print()` is not.
- A device MAY send `LOG` in any state, in a session or not. Text printed during start-up reaches a host that is listening, and costs nothing when none is.
- `LOG` never touches the circuit. The host shows the text in the connection's window.

---

5. Sessions

5.1 Session numbers

A session number is 16 bits, **1–65535**. **0** is reserved: it is never a run's session, and a `HELLO_ACK` for session 0 is a device's announcement that it is in no session.

- The host MUST NOT use the session of its previous handshake on the same connection again, and SHOULD make repeating any recent session unlikely. Chip Hippo counts upward, across all its connections, wrapping 65535 → 1, from a random starting point.
- *Note:* the session is what tells a device that did not reset between runs that a new run has begun, and what tells a late copy of this run's `HELLO` from the next run's. A host picking, by chance, the very session a device still remembers from before would be taken for a late copy: the device would keep

the old run's sequence numbers and not see a new run begin. Sixteen bits make that a 1-in-65535 event on each restart of Chip Hippo.

5.2 Host states

State	Accepts	Ignores	Moves on
CLOSED	—	—	Run opens the port → HANDSHAKE.
HANDSHAKE	`HELLO_ACK` for its session; `LOG`	Every other frame — including the announcement and a `HELLO_ACK` for any other session. Damage is not NAKed.	Its session is established → ACTIVE. A mismatch, no answer in 5 s, or the port lost → FAILED.
ACTIVE	`INBOUND`, `ACK`, `NAK`, `LOG`; the announcement	Every other `HELLO_ACK` (a late copy of its own, another session's); types it never receives	The announcement, a failed delivery (§6.4), or the port lost → FAILED. Stop → CLOSED.
FAILED	`LOG`	Everything else. Damage is not NAKed.	The run stops and the port closes → CLOSED.

- The host sends data frames only while ACTIVE: never during the handshake, and nothing at all once FAILED.
- **The announcement while ACTIVE** means the device is in no session: it has restarted, or it has left this session after one of its own deliveries failed (§5.3). Either way it will ignore everything until greeted again, which only a new run does, so the host fails what it has outstanding and stops the run. Without the announcement, a device that had left on an Inputs-only design would simply fall silent.
- **The port lost** — a cable pulled, a board gone — stops the run the same way.
- **On FAILED** the host fails every Output waiting to be sent, stops the run and tells the user which connection failed and why: no answer, another protocol version, another layout, the device leaving the session, a failed delivery, or a dropped connection. It still shows log text until the port closes — often the device's own account of what happened.
- There is no goodbye frame. Stop closes the port, and a device learns of it only when a send of its own fails (§5.3).

5.3 Device states

A device remembers the **last session** a `HELLO` named — 0 when its program starts.

State	Accepts	Ignores	Moves on
WAITING	`HELLO`	Every other frame. Damage is not NAKed, and no data frame is sent.	A `HELLO` for a new session that matches the device → ACTIVE.
ACTIVE	`HELLO`, `OUTPUT`, `ACK`, `NAK`	Types it never receives	A `HELLO` for a new session → ACTIVE if it matches, else WAITING. A failed delivery → WAITING, announced.

- **Start.** A program starts WAITING, its last session 0, and MUST send the announcement once: a `HELLO_ACK` for session 0 carrying its version and signature.
- **A `HELLO` for the last session** — a late copy; the host resends until it hears — is answered and changes nothing else, in either state. In particular it MUST NOT reset the device's sequence numbers: rewound, its next `INBOUND` would look like a duplicate to the host and be dropped without a word.
- **A `HELLO` for a new session** is answered, and starts that session. The device MUST discard everything it held for the previous one: its sequence numbers (§6.1), an `OUTPUT` held for later (dropped), an `OUTPUT` whose handler is running (never acknowledged — the run that sent it has gone) and an `INBOUND` waiting for its `ACK` (given up at once). It records the new session as its last, and becomes ACTIVE only if the `HELLO` **matches**: exactly 7 bytes, version 1, and the device's own layout signature. Otherwise it is WAITING.
- **A handler belongs to its OUTPUT's session.** If that session ends while the handler runs — a new `HELLO`, or a failed delivery — every send the handler makes afterwards MUST fail at once: its Inputs belong to a run that has gone, and must not reach the next one.
- **Becoming ACTIVE** is the moment for the device to send every Input its starting value: an Input drives nothing until its first `INBOUND` (§8). The generated libraries run the program's `onConnect` function (`on_connect` in Python) here — the only sign a board that did not reset gets that a new run has begun.
- **A failed delivery** (§6.4) means nobody is listening: the device leaves the session. It MUST discard a held `OUTPUT`, MUST NOT acknowledge an `OUTPUT` whose handler is running, and MUST then send the announcement, so that a host still listening stops rather than wait on a device that no longer answers. It returns to WAITING — **offline** — and keeps its last session, so only the next run's `HELLO` brings it back. While offline, a send fails at once instead of waiting three times 500 ms, which keeps a program left running after Stop from crawling: there is no goodbye frame, and a board that resets on open does not reset on close. The generated libraries report the state as `connected()`.

Note: the byte stream keeps two sessions apart without a session number in every frame. Whatever a device sent in an old session precedes its `HELLO_ACK` for the new one, and the host accepts nothing but that `HELLO_ACK` until it arrives; after it, the device sends nothing that belongs to the old session. The same holds the other way round.

5.4 The handshake

1. **Open.** The host opens the port. On many boards this resets the device (§2).
2. **Greet.** The host chooses a session (§5.1) and sends `HELLO` at once, then again every **250 ms** — the same session each time — until a `HELLO_ACK` for that session arrives, for up to **5 s**.
3. **Judge.** Of a `HELLO_ACK` for its session, the host:
   - refuses the run — **version mismatch** — if its version is not 1, whatever its length;
   - ignores it and keeps waiting if it is not exactly 7 bytes;
   - refuses the run — **layout mismatch** — if its signature is not the host's;
   - otherwise establishes the session and becomes ACTIVE.
4. **No answer** in 5 s refuses the run.

A run is all-or-nothing: if any of its connections fails to open, or fails its handshake, every port the run opened is closed again.

Note: the device judges the `HELLO` as the host judges the `HELLO_ACK` (§5.3), so after a mismatch neither side is in the session. A device built for another design never runs `onConnect`, or sends an Input, into a run that has refused it.

---

6. Reliable delivery

`OUTPUT` and `INBOUND` are delivered **stop-and-wait**. A side has at most one data frame outstanding, and sends the next only when that one has been acknowledged or its delivery has failed.

6.1 Frame identity and sequence numbers

- A data frame is identified by its **session, direction and `SEQ`**. The two directions number their frames independently: the host's `OUTPUT` 7 and the device's `INBOUND` 7 are different frames, and an `ACK` 7 acknowledges whichever of them its receiver sent.
- Each side has a send counter for the session, starting at 0. A new data frame takes the counter's next value, 1–255, wrapping 255 → 1: a session's first data frame in each direction is `SEQ` 1. `SEQ` 0 is never a data frame's: a receiver MUST ignore an `OUTPUT` or `INBOUND` carrying it — neither acknowledge nor process it.
- A retransmission, after a timeout or a `NAK`, is the same frame: the same bytes and the same `SEQ`. The counter advances only for a new frame.

6.2 Acknowledgement

In a session, a receiver acknowledges every intact data frame — at the time the table below gives — with an `ACK` carrying its `SEQ`. A frame is **delivered** when an intact `ACK` with its `SEQ` arrives while it is

outstanding. Any other `ACK` — another `SEQ`, or nothing outstanding — is **stale**, and MUST be ignored.

Receiver	Frame	The `ACK` is sent…
device	`OUTPUT`	**after** the Output's handler returns.
host	`INBOUND`	as soon as the frame arrives — even while the circuit is stalled. The value is applied at the next settle boundary (§8).

Acknowledging an `OUTPUT` after its handler means everything the handler sent — each `INBOUND`, delivered and acknowledged, and each `LOG` — reaches the host **before** the `ACK`. The host stalls the circuit until every `OUTPUT` is acknowledged, so it always sees the device's reaction before the circuit moves on. That is what makes request and response deterministic (§8).

- A handler that fails still counts as returned: the Python library catches the exception, logs it, and acknowledges.
- A device MUST NOT run a handler from inside another handler, or while it waits for an `INBOUND`'s `ACK`. An `OUTPUT` arriving then is held — the host sends one at a time, so one slot is enough — run afterwards, and acknowledged when it has run.
- Handlers must return promptly: within 500 ms if possible, and certainly within the host's three sends, 1.5 s in all (§6.4). Long work belongs in the program's main loop.

6.3 Duplicates

Each receiver remembers the `SEQ` of the last data frame it acknowledged in the session — none, when the session begins.

- A data frame carrying that `SEQ` is a **duplicate**: its `ACK` was lost and the sender has resent it. The receiver MUST acknowledge it again and MUST NOT process it again. Without this, a lost `ACK` would call an Output's handler twice.
- **Exception:** a duplicate of the `OUTPUT` whose handler is still running, or which the device is holding to run next, MUST be ignored — not acknowledged. Its `ACK` follows when the handler returns. This happens whenever a handler sends an `INBOUND`: the device reads the port while it waits for that `ACK`, and the host's timer can resend the `OUTPUT` in the meantime. Acknowledging it then would let the circuit run on before the handler's `INBOUND` had been applied.

Note: because a sender never has two frames outstanding, a receiver only ever sees the frame it last acknowledged again, or the next one. One remembered `SEQ` is enough to tell them apart.

6.4 Timeout, NAK and retry

- **The timer.** Each transmission of a data frame — the first, or any retransmission — starts an ACK timer of **500 ms** when the frame has been written to the port.

- **Retransmit** the outstanding frame when its timer expires, or at once when a `NAK` arrives (§4.4). The retransmission restarts the timer.
- **The limit.** A frame is sent at most **3 times** in all, the first transmission included, whatever caused the resends. If the timer of the third transmission expires, or a `NAK` arrives after it, the delivery has **failed**:
  - the **host** stops the run (FAILED, §5.2). Whatever was queued behind the frame fails with it.
  - the **device** drops the frame, reports the failure to its caller (the generated `send()` returns false), and leaves the session, saying so with the announcement (§5.3) — which stops a host still listening.
- `ACK`s, `NAK`s and retransmissions are the protocol's own traffic, not the circuit's: they never light Chip Hippo's activity lamps. The connection window's protocol view shows them, a resend as a warning.

6.5 Ordering

Each direction's data frames are processed in the order they were sent, since each waits for the one before. The two directions interleave freely. A side waiting for an `ACK` MUST keep receiving meanwhile: it acknowledges the other side's data frames, answers a `HELLO` and acts on a `NAK` as usual. `LOG` frames sit in the byte stream in the order they were written, among the data frames around them.

---

7. Element indices and the layout signature

Names never travel over the wire. Elements are identified by index, and the layout signature makes sure both ends number them alike.

7.1 Element indices

- Indices are assigned **per connection, per direction**, from 0: Outputs 0 to n–1, Inputs 0 to m–1.
- The generated code, the element index in every data frame, and the layout string (§7.2) all use this one numbering.
- *Note:* Chip Hippo orders each direction by the number in the element's internal id (`out2` before `out10`). That number only ever counts up, so an element's index changes only when one before it, on the same connection and in the same direction, is added or removed.

7.2 The layout string

A connection's layout is written as an ASCII string, with no byte-order mark, no NUL terminator, no newline and no spaces:

```
layout  = [ element *( "," element ) ]
element = kind index ":" width *( "+" width )
kind    = "O" / "I"
index   = "0" / ( %x31-39 *DIGIT )   ; decimal, no leading zeros
width   = "1" / "8" / "16"           ; one field: bit, byte or word
```

- Every Output comes first, in index order, then every Input, in index order.
- An element lists its fields' widths in field order, joined by `+`; they add up to the element's width, at most 16.
- A connection with no elements has the empty string as its layout.

For example, Outputs of fields `[byte, bit]` and `[bit]`, and an Input of one word, are `O0:8+1,O1:1,I0:16`.

7.3 The signature

The **layout signature** is the CRC-32 (IEEE 802.3, also known as CRC-32/ISO-HDLC) of the layout string's bytes: polynomial `0x04C11DB7`, reflected (`0xEDB88320`), initial value `0xFFFFFFFF`, final XOR `0xFFFFFFFF`. It travels little-endian. The check value, for the ASCII bytes `123456789`, is `0xCBF43926`; the empty layout's signature is `0x00000000`.

Layout string	Signature
`O0:8,O1:1,I0:16`	`0xF8ACB506`
`O0:8+1,O1:1,I0:16`	`0x7EF66B41`
`O0:1,I0:1+1+1+1`	`0x0EC32022`
`I0:16`	`0x43F1C7E9`

The signature covers what decides which bits reach which parameter: the elements, their order, and each one's field boundaries, not only its total width. A `[bit, byte]` element reordered to `[byte, bit]` keeps its width of 9 but would hand every value to the wrong parameters, so it must be refused.

Everything else is the circuit's business and changes nothing on the wire: names, colours, descriptions, where the tags are planted, and each element's trigger. Renaming an element or a field changes the generated code's **design hash** — a second fingerprint, written into the generated file as a comment and compared by the Generate button — so the code reads as out of date, but a run still works until it is regenerated. The design hash never travels on the wire.

8. What the values mean

The protocol moves values; this is what Chip Hippo does with them, which a device has to know to answer sensibly.

- **Pins and bits.** Pin *k* of an element is bit *k*–1 of its value. On the host, a pin whose tag isn't planted on the board, or whose net is floating or unknown, reads as **0**.
- **Chip Hippo looks only at settle boundaries** — the moments the circuit has stopped changing. A glitch inside one settle is never sent anywhere, and nothing from the device lands on the board halfway through a ripple.
- **When an `OUTPUT` is sent.** An Output with an **edge** trigger (rising, falling or either) is sent when its trigger line's settled level has made that transition since the previous boundary (its *Trigger starts* setting stands in for "previous" at the first). An Output on **Auto** has no trigger line: it is sent whenever the value its pins settle to differs from the last value it sent, **and always at the first boundary of a run**, so the device learns where the circuit starts.
- **The stall.** Once a boundary has sent its `OUTPUT`s, the circuit stands still — no clock edges, no settles — until every one is acknowledged, that is, until every handler has returned (§6.2).
- **When an `INBOUND` is applied.** A received value is held, and a newer one for the same Input replaces one not yet applied. An Input on **Auto** is *live*: its value goes on the board at the next boundary — at once, if the circuit is idle. An Input with an edge trigger releases its value only when its trigger line makes that transition, the way a latch loads on its clock. All the values eligible at a boundary are applied together, and the circuit settles again. A value that repeats what the Input already drives changes nothing.
- **Request and response.** An `INBOUND` that arrives while the circuit is stalled on an `OUTPUT` still counts for that boundary. So a handler that answers with an Input lands its answer before the circuit moves on.
- **Inputs start undriven.** An Input drives its pins at the strength of a chip output — two drivers disagreeing on one net are reported as a conflict — but drives **nothing at all** until its first `INBOUND` of the run. Send every Input its starting value when a session begins (§5.3).

9. Invariants

The rules above, reduced to what must always hold:

1. `SEQ` is non-zero only in data frames and `ACK`s. `HELLO`, `HELLO_ACK`, `NAK` and `LOG` carry 0; a session travels in the `HELLO` payload; a data frame with `SEQ` 0 is ignored.
2. Each side has at most one data frame outstanding.
3. A retransmission is the same frame — the same bytes, the same `SEQ` — and a data frame is sent at most 3 times.

4. A data frame is identified by session, direction and `SEQ`. The two directions' counters are independent.
5. An `ACK` carries the `SEQ` of the frame it acknowledges, and only an `ACK` with the outstanding frame's `SEQ` delivers it.
6. A receiver processes each data frame at most once: a duplicate is acknowledged again, never processed again.
7. A device acknowledges an `OUTPUT` only after its handler has returned, and never one received in a session it has since left.
8. `HELLO`, `HELLO_ACK`, `ACK`, `NAK` and `LOG` consume no sequence numbers and are never acknowledged.
9. A `NAK` names no frame, and causes no more retransmissions than the send limit allows.
10. `LOG` is never acknowledged, resent or de-duplicated, and may be sent outside a session.
11. Session 0 is never a run's: a `HELLO_ACK` for session 0 is the device's announcement that it is in no session.
12. Only a `HELLO` for a new session resets a device's session state.
13. Neither side enters a session unless the versions and the layout signatures match.
14. A failed delivery ends the session on both sides: the host stops the run; the device leaves it and announces that, which stops a host still listening, and stays offline until the next session.
15. A device sends nothing that belongs to a session it has left — not an `ACK`, and not an `INBOUND` from a handler of that session.

10. Constants

Constant	Value	Meaning
`PROTOCOL_VERSION`	1	This protocol's version.
`START`	`0x7E`	Begins a frame.
`ESC`	`0x7D`	Escapes the next byte.
`ESC_XOR`	`0x20`	XORed into an escaped byte.
`HEADER_BYTES`	3	`TYPE`, `SEQ`, `LEN`.
`CRC_BYTES`	2	The CRC.
`MAX_PAYLOAD`	255	The longest payload.
`MAX_FRAME_BODY`	260	The longest body, unescaped.
`MAX_WIRE_FRAME`	521	The longest frame on the wire.
`DATA_PAYLOAD`	4	An `OUTPUT` or `INBOUND` payload.
`HELLO_PAYLOAD`	7	A `HELLO` or `HELLO_ACK` payload.
`HELLO_PREFIX`	3	The shortest `HELLO` a device answers, and `HELLO_ACK` a host reads: version, session.
`MAX_HOST_PAYLOAD`	7	The longest payload the host sends: a device's smallest payload limit.
`ANNOUNCE_SESSION`	0	The session of the announcement: the device is in no session.
`MAX_SESSION`	65535	Sessions are 1 to this.
`MAX_SEQ`	255	Data `SEQ`s are 1 to this.
`MAX_WIDTH`	16	The widest element, in pins.
`CRC16_POLY`	`0x1021`	The frame CRC's polynomial.
`CRC16_INIT`	`0xFFFF`	The frame CRC's initial value.
`HELLO_INTERVAL_MS`	250	How often the host sends `HELLO` while it waits.
`HELLO_WINDOW_MS`	5000	How long the host waits for `HELLO_ACK`.
`ACK_TIMEOUT_MS`	500	The ACK timer.
`MAX_SENDS`	3	The most times a data frame is sent.

These are protocol values, not settings: no connection overrides them. The names are those of Chip Hippo's source (§13).

---

11. Versions

- This page is version **1**. A host and a device talk only when their versions are equal.
- In version 1, the version is the first byte of the `HELLO` and `HELLO_ACK` payloads, and the session the next two (§4.1). A version-1 device answers a `HELLO` of any version with its own version, and joins only a version-1 `HELLO` (§5.3). A version-1 host refuses a `HELLO_ACK` of any other version as a version mismatch, whatever its length (§5.4).
- Any change to a frame's layout, the frame types, the CRC or the meaning of a field is a new version. A later version MAY change the `HELLO` and `HELLO_ACK` payloads after establishing that it is speaking to a different version.
- *Note:* a version-1 side answers or refuses another version, rather than falling silent, only when it can read the frame: framed as in §3, the version first and the session next — and, sent to a version-1 device, no longer than 7 bytes (§3.2).
- Not part of version 1: a goodbye frame, a device-initiated handshake (the announcement tells the host something happened; it establishes nothing), windowed delivery, flow control and keep-alives.

---

12. Worked examples

These examples use the layout `O0:8,O1:1,I0:16` — signature `0xF8ACB506`, sent as `06 B5 AC F8` — and session `0x1234`, sent as `34 12`. Each frame is written as it is on the wire. They are test vectors: Chip Hippo's test suite checks every frame on this page against its encoder.

12.1 The handshake

The host's `HELLO`: version 1, session `0x1234`, its signature (CRC `0x9C63`).

```
7E  01 00 07  01 34 12 06 B5 AC F8  63 9C
```

The device's answer, `HELLO_ACK`: the same session, with its own version and signature (CRC `0x2DAC`).

```
7E  02 00 07  01 34 12 06 B5 AC F8  AC 2D
```

The same device announcing its start-up: a `HELLO_ACK` for session 0 (CRC `0x4458`).

```
7E  02 00 07  01 00 00 06 B5 AC F8  58 44
```

Escaping reaches the session as well. Session `0x7D7E` is sent as `7E 7D`, both of them markers, so a `HELLO` for it is (CRC `0x0C54`):

```
7E  01 00 07  01 7D 5E 7D 5D 06 B5 AC F8  54 0C
```

12.2 An OUTPUT and its ACK

The host sends `OUTPUT` `SEQ` 1: element 0, width 8, value `0x7E` — deliberately a value that needs escaping.

- Unescaped: `TYPE=10 SEQ=01 LEN=04 PAYLOAD=00 08 7E 00`.
- CRC-16 of `10 01 04 00 08 7E 00` is `0x88E4`, sent as `E4 88`.

```
7E  10 01 04  00 08 7D 5E 00  E4 88
```

The device calls the handler for Output 0 with `0x7E`. The handler returns, and the device sends `ACK` 1 (CRC `0x4D0D`):

```
7E  06 01 00  0D 4D
```

Escaping reaches the CRC too. `ACK` 16 has the CRC `0x7D4F`, and `ACK` 17 `0x4E7E`:

```
7E  06 10 00  4F 7D 5D
```

```
7E  06 11 00  7D 5E 4E
```

12.3 An INBOUND, a LOG and a NAK

The device sends `INBOUND` `SEQ` 1 — its own first data frame, since each direction counts separately (§6.1): element 0, width 16, value `0xBEEF` (CRC `0xB788`).

```
7E  11 01 04  00 10 EF BE  88 B7
```

The host acknowledges it with the same bytes as the `ACK` in §12.2: `7E 06 01 00 0D 4D`. It is the device's frame this `ACK` answers, because the device receives it.

The device logs `ok` and a newline, unacknowledged (CRC `0x6104`):

```
7E  20 00 03  6F 6B 0A  04 61
```

A `NAK`, from either side (CRC `0x640F`):

```
7E  15 00 00  0F 64
```

12.4 A retransmission

The host sends Output 1 (width 1) the value 1, as `SEQ` 2. The frame is damaged on the way, and the device's `ACK` is later lost:

Step	Direction	What happens	On the wire
1	host → device	`OUTPUT` `SEQ` 2, first send. It arrives damaged.	`7E 10 02 04 01 01 01 00 46 B6`
2	device → host	The damage is NAKed.	`7E 15 00 00 0F 64`
3	host → device	Second send, at once: the same bytes.	`7E 10 02 04 01 01 01 00 46 B6`
4	device → host	The handler runs; `ACK` 2. It is lost.	`7E 06 02 00 5E 18`
5	host → device	500 ms after step 3, the third and last send.	`7E 10 02 04 01 01 01 00 46 B6`
6	device → host	A duplicate of the last frame acknowledged: `ACK` 2 again, the handler not run again.	`7E 06 02 00 5E 18`

Had step 6 been lost too, the host would have stopped the run 500 ms after step 5. At no point does the `SEQ` change, and the handler runs once.

---

13. Implementations

Four implementations speak this page, and each is tested against the others.

- **The host** — Chip Hippo's main process: the bytes in `src/app/serial/protocol.js`, the session and reliability rules in `src/app/serial/link.js`. Every number on this page lives in one module, `src/web/scripts/model/serial-wire.js` — the names in §10 are its exports — which the host reads

and the code generators write the devices' constants from, so the two ends cannot drift. The element order and the layout string are one function each (`elementsFor` and `layoutString` in `src/web/scripts/model/integration.js`), called by the generators and by the run-time handshake alike.

- **The C++ header** (`ChipHippo.h` , from Generate, for a connection whose Language is C++) — the device side for Arduino boards. Its payload limit is the smallest allowed, so it buffers 12 bytes of a frame; it computes the CRC a bit at a time (no table, so no flash spent on one); it decides whether a `HELLO` matches by comparing it, byte for byte, with the `HELLO_ACK` it answers with; and it sends log text in frames of up to 60 bytes — at every newline, at every `poll()` , and before every `INBOUND` and every `ACK` of an `OUTPUT` .
- **The Python module** (`chiphippo.py` , for a Language of Python) — the header said again, line for line, for MicroPython and CircuitPython. It picks its port by platform. On **MicroPython** it uses the REPL's own USB port, with Ctrl-C switched off inside a frame (where `0x03` is data); since the host never sends a byte outside a frame (§2), a lone `0x03` between frames can only be a tool such as Thonny or `mpremote` , and it stops the program as Ctrl-C would. (The module reads a damaged frame to the end its `LEN` gives it, so a `0x03` in the rest of one is still that frame's; and for a moment after line damage — a frame whose `START` was lost, or the rest of one whose `LEN` was misread — a stray `0x03` is taken as more of it, not as Ctrl-C. Its example `main.py` turns Ctrl-C off before it imports the module, so a board that starts during a handshake is not stopped by a `HELLO` .) On a board whose USB is a separate serial chip (a classic ESP32 DevKit) that port always runs at 115200 baud, 8N1, so the connection must too. On **CircuitPython** it uses the second USB serial port, which its `boot.py` turns on. Any stream can be handed to `begin()` instead.
- **The Mock** — Chip Hippo's built-in pretend board (`src/app/serial/mock-device.js`): the device side in JavaScript, behind the same port interface as a USB cable, so the host cannot tell the two apart. Its one-shot faults — drop the next `ACK` , corrupt the next frame, ignore a handshake, answer with the wrong signature — exercise the recovery rules (see The Mock connection).

Chip Hippo's test suite checks this page's frames, constants and signatures against the code, drives each device — a generated header compiled with the host's C++ compiler, the Python module under Python and MicroPython, and the Mock — with the real host link, and feeds all of them the same bytes to show they answer alike.

Files, Saving & Undo

Chip Hippo keeps your work in **one ordinary file** you can see, move, and share. This page covers what's in it, the four commands that manage it, the dirty marker, and undo/redo.

One file is the whole design

The desk in front of you is always a **desktop of an open project**, from the very first launch. A brand-new project has no name, no home of its own, and exactly one desktop called `Desktop 1` — which is simply a desk to build on. See Projects & Desktops for keeping several desktops side by side.

The project is the document. A single `.chiphippo` file holds all of it:

- every desktop — its boards, chips, wiring, parts, and net names;
- what those desktops are called, and which one you were on;
- whether you left the desk padlock shut;
- the contents of every ROM you programmed.

Nothing else is needed to open a design. Copy that one file to another machine and it opens whole, ROM contents included — which is what makes a Chip Hippo design something you can actually send to someone.

Before you choose where it goes

Chip Hippo never stops you mid-thought to ask for a file name. A project that hasn't been given a home lives in a **working slot** inside the app's own data directory, and `Cmd/Ctrl+S` writes it there with no dialog at all. Design something, save, quit — it's there when you come back.

Save As… is what gives the project a real file, when you want one. From then on Save just writes that file. When you launch, Chip Hippo opens the unsaved project from its working slot if there is one; otherwise it reopens the **project you used most recently**.

New, Open, Save, Save As

Four commands — on the **File** menu, in the header toolbar's File pill, and under the usual shortcuts — manage the project's file:

- **New Project** (`Cmd/Ctrl+N`) — a fresh unnamed project with one empty desktop. What's open now is asked about first.

- **Open...** (`Cmd/Ctrl+O`) — shows a native file picker. A `.chiphippo` project opens as itself; a loose design or an exported desktop opens as a new project of one desktop.
- **Save** (`Cmd/Ctrl+S`) — writes the project to its file, or to the working slot when it hasn't got one yet. No dialog either way.
- **Save As...** (`Shift+Cmd/Ctrl+S`) — asks where to keep it and writes it there. An unnamed project takes its **name from the file you pick**, so there's no separate naming step.

If the file you pick already exists, your system's own save dialog asks whether to replace it, exactly as it does in any other app — Chip Hippo adds no second question of its own. Decline there and nothing is written.

File ▸ Project Properties... shows the project's Location alongside its Name and Description, without opening any dialog that writes.

Unsaved changes

The window title shows a • when the project has changes that aren't in its file yet: `• 6502 SBC — Clock module — Chip Hippo`. Save clears it. There is one marker because there is one document — it covers every desktop's design *and* the desktops themselves, so adding one, renaming one, or deleting one shows up there exactly as wiring a board does.

Nothing is written to your file until you save. Add a desktop, rename it, delete another — none of it touches your `.chiphippo`, and closing the project without saving takes all of it back. Tabs carry no marker of their own for the same reason: a desktop can't be saved on its own, so a dot on one would have nothing to clear it.

And nothing is lost to a crash either. About every 30 seconds, Chip Hippo tucks a copy of whatever you're working on into its own private folder — not into your file. So both things are true at once: the • still means "not in my file yet", closing without saving still takes it all back, and a power cut or a crash still costs you half a minute at worst.

If the app does go down unexpectedly, the next launch puts that work straight back on the desk and tells you so. It arrives **unsaved** — the • is showing — so you decide the same way you always would: save it, or close without saving to go back to what your file holds. A project you'd never saved anywhere is kept too; it was already living in that folder, and it still opens on its own.

You can ignore all of it. There's no setting, no prompt to answer, and nothing appears while you work. ⌘S is still what commits your work to your file, and still worth pressing when you want it there *now*.

Nothing that would lose your changes happens silently. Starting or opening another project asks first, and so does quitting. Both offer to **save** — and choosing that carries the action through, so you are never made to ask twice.

A project still in the **working slot** is asked about every time, even when nothing is unsaved: the incoming project is about to claim that slot, so there's nowhere for this one to stay. **Save** there means Save As — a file of its own.

When the save is the app's idea rather than yours — quitting with a • in the title — nothing is asked about *where*: the project goes to the file it already has, or to the working slot, where the next launch will find it.

Two things that are deliberately **not** counted as changes: which desktop is on the desk, and where the camera is. Moving between tabs and panning around never mark a design dirty.

Undo & redo

`Cmd/Ctrl+Z` undoes the last edit; `Shift+Cmd/Ctrl+Z` redoes it. Undo/redo covers the full editing history of the circuit — placing and moving boards, chips, and discretes; wiring and unwiring; deleting anything; net names — as far back as your session's history allows, and both menu items disable themselves automatically when there's nothing left to undo or redo.

Undo/redo is **per desktop**: switch away and back, and `Cmd/Ctrl+Z` still undoes that desk's last edit, not the other one's. Adding, deleting, renaming and importing a desktop are not on any undo stack — they're changes to the project's own structure, and what takes them back is closing the project without saving.

Undo/redo does **not** cover simulation state. While the circuit is running, editing — and with it, recording new undo steps — is locked, so nothing that happens mid-run (sequential chip state, clock phase, a chip taking over-voltage damage) ever becomes an undo step of its own. All of it is **run state**: it exists while the circuit is running and not a moment longer.

That includes **over-voltage damage**. Feed a 74LS chip 12 V and it lets its smoke out on the spot — the warning appears, its badge goes red, and it stays dead for the rest of that run, because a chip that's failed doesn't come back while the power is still wrong. Press **Stop** and every damaged chip is whole again. Wiring 12 V to the wrong rail is a mistake about the *circuit*, and a mistake you can see and fix; it was never meant to cost you the chip, so it doesn't. Fix the wiring, run again. What undo/redo restores is always the circuit you built, never a moment in its simulated behavior.

Example circuits

Chip Hippo's project repository ships a handful of ready-to-load example circuits as ordinary `.chiphippo` files — small W65C02-based breadboard computers built from 74xx glue logic, each paired with a `.hex` ROM image. Open one the same way as any saved design: **File ▸ Open...** loads it as a project of its own. Then load its matching `.hex` into the ROM chip via the memory inspector or the external programmer before pressing **Run**.

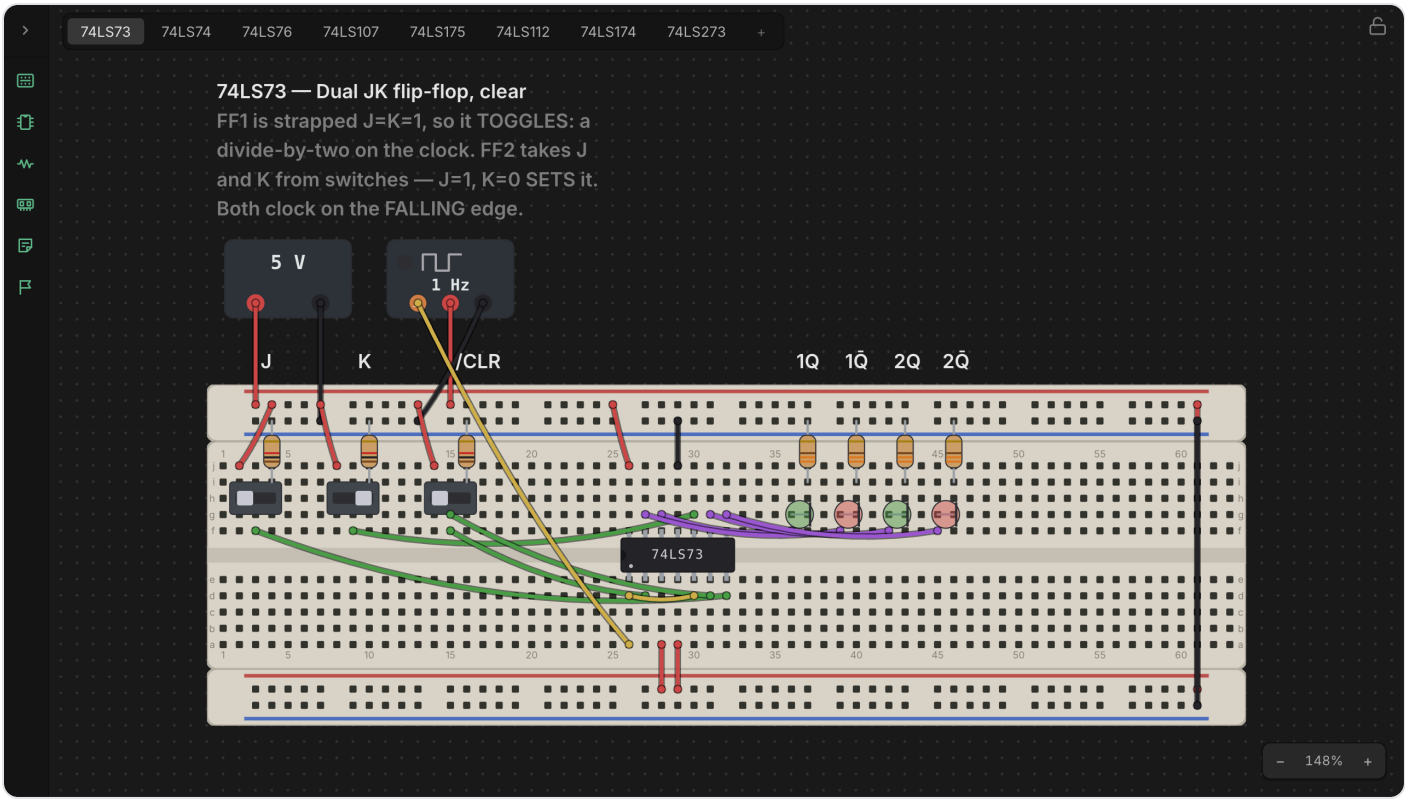
`hello-w65c02.chiphippo` and `hello-z80a.chiphippo` are the exception: they are the two CPUs' example circuits (see The Chip Library), and their program is already in the ROM, so they run as soon as you press **Run**.

See Getting Started for building your first circuit from scratch, and Settings for what else persists between sessions.

Projects & Desktops

A big build eventually needs a bench beside it. You want to work out a clock module, an address decoder, or a memory map *without* doing it in the middle of everything else — and then drop the finished thing into the main design.

That's what a **project** is for. A project is a workspace holding several **desktops**, shown as tabs above the desk. Each desktop is a complete desk of its own — its own boards, chips, wiring, camera position, and undo history — and a design worked out on one can be copied straight onto another.



The project is the document. One `.chiphippo` file holds all of it: every desktop, and the contents of every ROM you programmed — each image stored once however many chips across however many desktops are holding it. That's the whole design in a single file you can email, drop in a repo, or carry to another machine. There are no companion files to keep together and nothing to go missing.

Every desktop is the same. There is no special "main" desk and no lesser one: they are peers, all with the same menu, and which is the build and which is the bench is entirely up to you. Any of them can be renamed, duplicated, or deleted — the only rule is that a project always keeps at least one.

There is always a project open. From the first launch, the desk you are looking at is a desktop of one — a brand-new project with no name, no home of its own, and a single desktop called `Desktop 1`. Nothing has to be created before you can start building, and nothing has to be named before you can add a second desktop.

Adding desktops

Just press + at the end of the tab strip. It asks you nothing: a new empty desktop appears beside the others, named `Desktop 2`, `Desktop 3`, and so on, and you land on it.

Right-click the + for the other way one arrives:

- **New Desktop** — the same thing the plain click does.
- **Import Desktop...** — one read in from a file (see *Moving a desktop between projects* below).

Either way it's an addition, and nothing already there is touched.

There is a third way one arrives, and it doesn't belong on the **+** because it belongs to a *part*: a chip's **example circuit**. Right-click any 74xx chip (or a CPU) on the desk, choose **Pin Assignment**, and click the circuit button in that window's top-right corner — a worked bench for that part lands as a desktop of its own, called `74LS138 example`. See The Chip Library.

The **Desktop** menu has the same two plus the rest, acting on the desktop you're looking at:

- **New Desktop** — the same thing as pressing **+**.
- **Duplicate Desktop** — a copy of this desk, landing right beside it. The copy is brand-new hardware: a ROM on it gets its own backing file, so programming the copy never touches the original.
- **Import Desktop...** — read a `.desktop.chiphippo` (or a loose `.chiphippo` design) in as a new desktop. See *Moving a desktop between projects* below.
- **Export Desktop...** — write this desktop out as a self-contained file.
- **Export To ▸** — write this desktop in another tool's format: a **KiCad** project to take it to a circuit board, or a **Digital** circuit to keep simulating it. See Exporting to Other Tools.
- **Desktop Properties...** — its **Name** and **Description**.
- **Delete Desktop** — remove it from the project.

Adding, renaming, duplicating and deleting a desktop are ordinary unsaved changes, exactly like moving a chip: nothing reaches the project's file until you save it, and closing without saving undoes all of it.

Saving

The **File** menu is the project's, because the project is the only document there is:

- **New Project** (`Cmd/Ctrl+N`) — a blank slate: a fresh unnamed project holding one new, empty desktop. However many desktops you had, a new project always starts as a single one, numbered from `Desktop 1` again.
- **Open...** (`Cmd/Ctrl+O`) — pick a `.chiphippo` project to open.

- **Open Recent** — the last ten projects you saved or opened, most recent first. The same list is one keystroke away as `Shift+Cmd/Ctrl+O`, or a right-click on the toolbar's **Open** button; each entry's × forgets it without opening it.
- **Save** (`Cmd/Ctrl+S`) — write the project to its file.
- **Save As...** (`Shift+Cmd/Ctrl+S`) — write it to a new one.
- **Project Properties...** — the project's **Name**, **Description**, and the read-only **Location** of its file (blank until it has been saved).

The toolbar's four file buttons are the same four actions (with the recent list behind a right-click on **Open**), and the window title's • means the project has changes you haven't saved.

You never have to choose a file. A project that has never been given a home lives in Chip Hippo's own working slot, and `Cmd/Ctrl+S` writes it there with no dialog at all — design something, save it, quit, and it's there when you come back. **Save As...** is what gives the project a real file when you want one; it takes the project's name from the file you pick, so there's no separate "name this project" step. From then on Save just writes that file and moves it to the top of the recent list.

Changing projects

New Project and **Open...** both replace what's open, so nothing is allowed to go with it unasked:

- A project that has **never been given a home** is always asked about, whether or not anything in it is unsaved. It lives in Chip Hippo's one working slot, the project taking its place is about to claim that slot, and there is nowhere else for it to go — so replacing it loses it either way, and even a `Cmd/Ctrl+S` into the slot doesn't change that. You're offered **save** (which gives it a file of its own), **discard**, or **cancel**.
- A **saved** project has a file nothing is claiming, so it's only asked about when it has changes: **save**, **discard**, or **cancel**.

Choose to save and it goes through, then the action you asked for carries on — you're never made to ask twice. Cancel, at that dialog or at a dialog behind it, and nothing happens at all.

Working across tabs

Click a tab to put that desktop on the desk. Everything follows the active tab:

- **Undo/redo** is per desktop. Switch away and back, and `Cmd/Ctrl+Z` still undoes that desk's last edit, not the other one's.
- Each desktop remembers **where you were** — its camera position comes back with it. Panning and zooming are never counted as changes to the design.

- The **simulation stops** when you switch. A running circuit belongs to the desk it's running on, so the transport resets rather than following you to another desktop. Any open pin-assignment or memory-inspector windows close for the same reason: they were pointing at chips on the desk you just left.

Tabs carry no marker of their own. There is one save and one dirty marker, because there is one document: the window title shows the project — `Untitled` until you name it — the active desktop, and a leading • if anything anywhere in the project is unsaved.

Managing tabs

Right-click a tab for its menu:

- **Properties...** opens the same dialog every part, board, and wire has, with the same two fields — a **Name**, which is what the tab reads, and a **Description**, a note on what this bench is for. Hover the tab to see the description.
- **Duplicate Desktop** copies it, landing the copy right beside it.
- **Export Desktop...** writes it out on its own (below).
- **Delete Desktop** removes it from the project. Any desktop can go — there's nothing special about the first one — except the **last** one left, because a project with no desktops would have nothing to open. Its design goes with it, but its file is untouched: close the project without saving and it comes back.

Desktop numbers only ever count up: deleting `Desktop 2` doesn't make the next one `Desktop 2` again, so a name you wrote in a note keeps meaning the same bench. Rename them to whatever the benches actually are — `CPU`, `Clock`, `Decoder` — through Properties...

Moving a desktop between projects

Since a desktop isn't a file, moving one somewhere else is a **snapshot**:

- **Desktop ▸ Export Desktop...** writes the desk to a `.desktop.chiphippo` file — its boards, parts and wiring, plus the contents of any ROM on it. The file is self-contained and has no link back: it is a copy, and nothing you do to the project afterwards changes it.
- **Import Desktop...** — on the **Desktop** menu, and by right-clicking the tab strip's **+** — reads one back as a **new** desktop, keeping the name it was exported under. Import is always an addition, so no file operation can ever replace the desk you're looking at.

An imported desktop is brand-new hardware in the same way a duplicate is: import the same file twice and the two copies' ROMs are independent. Opening a `.desktop.chiphippo` with **File ▸ Open...** works too — it becomes a new project of one desktop.

Copying a design from one desktop to another

For a *part* of a desk rather than the whole of it, copy and paste:

1. On the desktop holding it, **shift-drag a marquee** around the design — the boards, the chips on them, and the wiring. A marquee takes in any board it encloses *completely*, and the selected set is outlined as one block. `Cmd/Ctrl`-click adds or removes an individual item afterwards, so a box that caught one thing too many (or one too few) doesn't have to be redrawn.
2. Press `Cmd/Ctrl+C`. The whole sub-assembly is copied: the boards, everything seated on them (whether or not the box touched it), every wire with **both** ends inside the set, and the buses, net names, and anchored labels riding them. A wire with one end outside is left behind — it would have to be cut.
3. Click the tab you want it on and press `Cmd/Ctrl+V`.

The design appears **under the cursor as a translucent ghost**, following it with no button held — the same way a fresh breadboard does when you pick one from the palette. Move it where you want it:

- Boards outline **green** where the design will land and **red** where it overlaps something already on that desk.
- Bring it near a board that's already there and it **snaps flush**, dovetailing exactly as a board you dragged into place would.
- **Click** to drop it. **Esc** throws it away and leaves the desk untouched.

The drop is all-or-nothing: if any board would land on top of something, the click is refused rather than pasting half a design and silently cutting its wiring. The whole paste is a single undo step, and the pasted design is brand new hardware — fresh boards, fresh parts, and (for a ROM) its own fresh backing file, so editing the copy never touches the original.

The clipboard survives switching tabs, so you can paste the same design onto several desktops, or stamp it more than once on the same one.

Leaving a project

A project stays open between sessions — named or not. Relaunch Chip Hippo and it comes back on the desktop you were last on: the unsaved project from its working slot if there is one, and otherwise the project you used most recently. Starting or opening another one asks first about anything you'd lose (see *Changing projects* above).

Closing the window or quitting asks the same question. Because you didn't ask for a save, it doesn't ask you where anything goes: the project is written to the file it already has, or — having none yet — to

the working slot, where the next launch will find it. Discard instead and the session's changes are gone. Cancel and nothing closes at all.

Opening an older project

Projects saved by an earlier version of Chip Hippo kept each desktop in a `.desktop.chiphippo` file of its own and listed their paths. Opening one brings every one of those designs into the single file, and **nothing of yours is deleted** — the old desktop files are left exactly where they are, and it is the project that changes shape. If one of those files has since been moved or deleted, that desktop opens empty and Chip Hippo tells you which file it was looking for.

Exporting to Other Tools

A design that works on the desk is often only the beginning. You might want to turn it into a real circuit board, or keep simulating it somewhere else. **Desktop ▸ Export To** writes the desktop on screen in another program's format. You can also right-click any desktop's tab and choose **Export To** to export that desktop. There are two formats, one for each of those jobs:

- **KiCad Project...** writes a project for KiCad, the free schematic and circuit-board program. Use it to go from breadboard to PCB.
- **Digital Circuit...** writes a circuit for Digital, a free logic simulator with its own 74xx library. Use it to keep simulating the design with the same chips.

Exporting only reads the desktop, so you can do it while the circuit is running. It never changes your design, and the exported files keep no link to it. Export again whenever the design changes.

The export report

Some things on a breadboard have no exact equivalent in the other program. If anything won't come across as it is, Chip Hippo tells you before writing a single file:

- **Won't come across**: parts the other program has nothing for. Everything wired to them is still exported, and their nets keep their names.
- **Changes**: parts that are exported as something that behaves a little differently. For example, a push button becomes a toggle switch in Digital.
- **Footprints to check**: KiCad parts that got a stand-in footprint (see below). Choose the real one before laying out a board.

Parts are listed by the same names the exported file uses (`U7`, `R1–R3`). Choose **Export...** to go ahead, or **Cancel** to write nothing. If nothing changes in the export, the report is skipped and you go straight to choosing where to save.

Part names and net names

A schematic names every part with a letter and a number, so the export gives each part one:

Prefix	Parts
`U`	Chips
`R`, `RN`	Resistors, resistor networks
`D`	LEDs, diodes and Zener diodes
`L`	Inductors
`Q`	Transistors
`SW`	Switches, push buttons, DIP switch banks
`DS`	7-segment digits, bar graphs, character LCDs
`X`	Oscillator cans
`J`	The power supply and clock source, which become connectors

Parts are numbered in the order you placed them, so the same desktop always exports with the same names. To choose a part's name yourself, set its **Name** to a designator of the right kind (`U5`, `R12`) in **Properties...**, and the export keeps it.

Nets are named the same way in both formats:

- a net you named yourself keeps its name (see Probing & Net Names);
- a wire in a bus is named after the bus and its bit, so bus `D[7:0]` gives `D0` … `D7`;
- the supply rails are `+5V` (or `+3V`, `+12V`) and `GND`;
- every other net is numbered `N1`, `N2`, … in reading order.

Characters the other programs can't use in a net name become `_`. A name marked active-low with a leading `/`, `~` or `!` gets an `n` prefix instead, so `/CLR` exports as `nCLR`.

KiCad

Export To ▸ KiCad Project... asks for a **folder**, not a file name. A KiCad project is several files that belong together, so choose (or create) a folder for it. Chip Hippo writes four files there, named after the desktop:

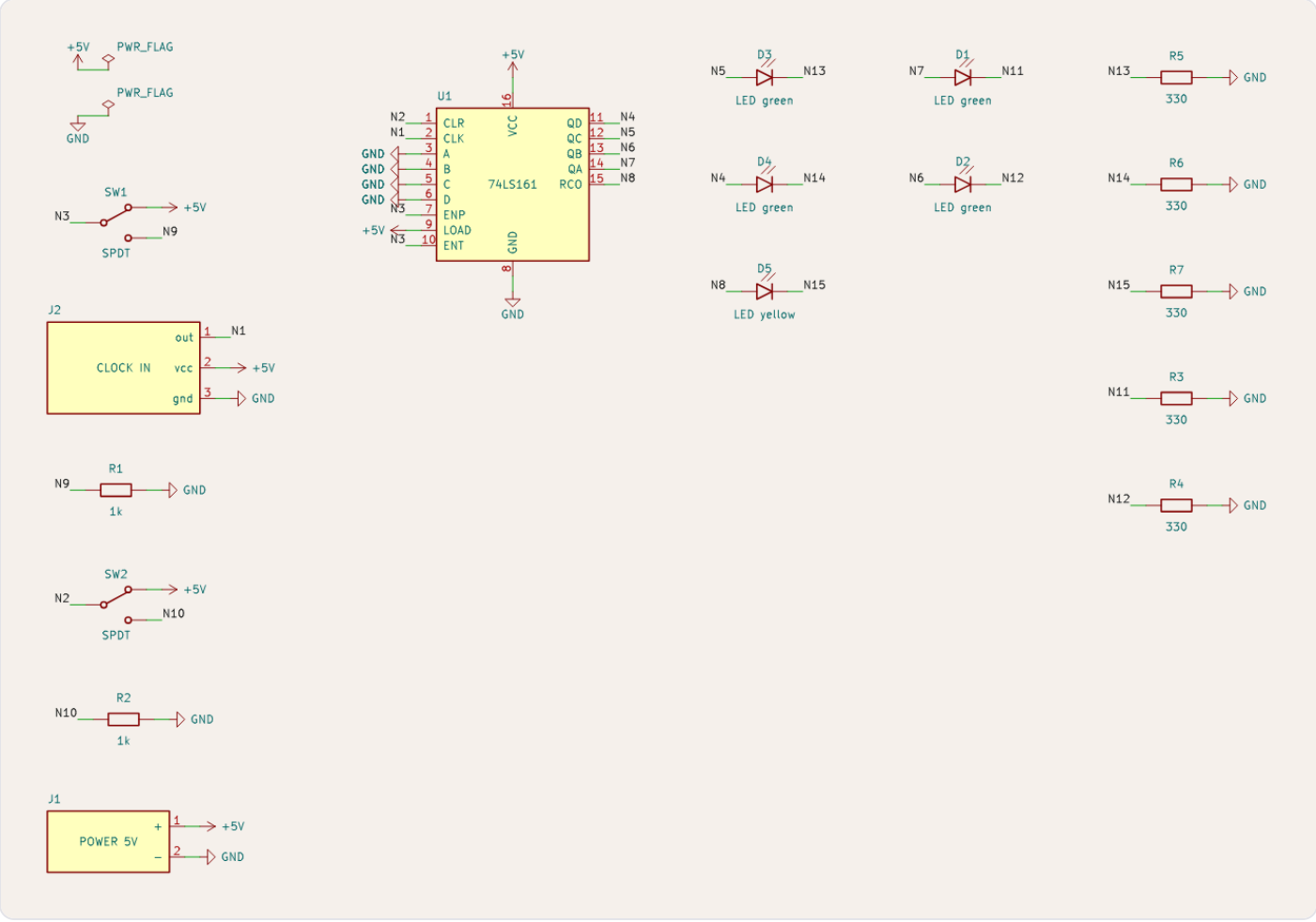
- `<desktop>.kicad_pro`, the project itself. Open this one in KiCad.
- `<desktop>.kicad_sch`, the schematic.
- `chiphippo.kicad_sym`, the symbols for the parts on it.
- `sym-lib-table`, which tells KiCad where those symbols are, so the project opens anywhere without any setup.

The files are written in KiCad 8's format, so KiCad 8 and every later version can open them.

What the schematic looks like

Parts are laid out in the same arrangement as the Schematic View, including any symbols you moved there. Each chip is **one** symbol with every pin in its physical numbering, the way it sits on the breadboard. (KiCad's own library draws a 74xx chip as a set of separate gates.) Inputs are on the left, outputs on the right, and power on the top and bottom.

Pins are connected by **net labels** rather than drawn wires: each connected pin has a short stub ending in its net's name, and KiCad joins every label with the same name. The supply rails use power symbols, with one `PWR_FLAG` per rail so KiCad's electrical rules check knows they are supplied. A pin that isn't connected to anything gets a no-connect flag. The sheet passes KiCad's electrical rules check with nothing to report.



From schematic to circuit board

Every part is given a **footprint** from KiCad's standard libraries:

- Chips get `Package_DIP` footprints at their real width: 300 mil for logic, 600 mil for the wide memory chips and processors.
- Resistors, LEDs, switches, DIP switch banks, resistor networks, oscillator cans and the 16×2 LCD each get their matching standard footprint.
- Capacitors come across **with their value** (`100nF`, `4.7uF`) as their KiCad Value, drawn the way KiCad draws them — the plain capacitor for a ceramic, the polarised one for an electrolytic, whose `+` is pin 1 — on a 5 mm ceramic disc or a 5 mm radial electrolytic footprint. Resistors carry their value the same way (`4.7k`). Every value keeps the figures the part's **Properties...** card shows, written in plain ASCII — `u` for µ, and no `Ω` — as SPICE and BOM tools expect. On the breadboard a capacitor connects nothing; in KiCad it is wired where it sits, which is the point of exporting it.
- Diodes, inductors and transistors come across as KiCad's own shapes — the diode, the Zener (its bar bent), the inductor's coil, and the NPN, PNP and N- and P-channel MOSFET symbols. A diode or a transistor's **Value** is its part number (`1N4148`, `2N2222`), or the plain type (`D`, `NPN`) until it has one; a Zener adds its voltage (`1N4733A 5.1V`); an inductor's is its inductance (`10uH`). Any part number also rides along as an **MPN** field for KiCad's BOM tools. Diodes get a DO-35 footprint on 7.62 mm, its pad 1 the cathode.
- An inductor's footprint is the part its **Style** says, on the pitch its **Holes between leads** says: a **Coil** is a standing toroid, on exactly 5.08, 7.62 or 10.16 mm; a **Can** is a radial drum, and KiCad's round radial inductors are on metric pitches, so it gets the nearest (5.00, 7.00 or 10.00 mm) and the report lists it under **footprints to check**.
- Transistors get a **TO-92** footprint on 2.54 mm — or, for a MOSFET whose **Package** is TO-220, a standing **TO-220** — its pads numbered in the order the desk prints the pins (`E B C`, `S G D`). That is no one maker's pinout, so the report lists every transistor under **footprints to check**: make sure the pads match the real part's datasheet before laying out.
- The power supply and the clock source become pin headers labelled **POWER** (two pins) and **CLOCK IN** (three: the clock, its supply and its ground). That's where power and a clock come in on a real board. The clock needs an oscillator in its place, which the report points out.
- The 7-segment digits, the 8-segment bar graph, the 20×4 LCD and the latching push button get a **stand-in footprint** (a pin header or a tactile button of the right size). Chip Hippo's versions of these are idealized parts, with no single real part to match. Choose the real part's footprint before laying out the board.

With the project open in KiCad, **Tools ▸ Update PCB from Schematic** in the PCB editor brings every footprint onto the board, connected.

Exporting again

To update an exported project after changing the design, export into the **same folder**. Chip Hippo rewrites the schematic and its symbol library, but leaves your `.kicad_pro` and any board you've laid out

alone. Every part keeps the same identity from one export to the next, so **Update PCB from Schematic** only adds, removes and rewires what actually changed, and the parts you've already placed stay put.

Chip Hippo never overwrites a schematic that some other program wrote. If the folder already holds a `.kicad_sch` of that name from somewhere else, the export stops and changes nothing. Choose an empty folder instead. (A schematic Chip Hippo exported is still recognised after KiCad has saved it, by the "Exported from Chip Hippo" line in its title block — leave that line in.)

What isn't exported

Breadboards and jumper wires don't appear as parts: they become the connections between the parts. External signals and Arduino Output and Input elements are stimulus for the bench, not parts, so they are left out (the report lists them). Notes and labels are left out too.

Digital

Export To ▸ Digital Circuit... writes one `.dig` file. Open it in Digital with **File ▸ Open**.

The chips are **Digital's own**: its library models most of the 74xx family as DIL packages, pinned exactly like the real parts. It has no CD4000 folder, but six CD4000 parts have a pin-for-pin twin there, and the export uses it:

Chip Hippo part	Placed in Digital as
CD4002B dual 4-input NOR	744002
CD4017B decade counter	744017
CD4069UB hex inverter	7404
CD4075B triple 3-input OR	744075
CD4077B quad XNOR	747266
CD40106B Schmitt hex inverter	7414

Chip Hippo places each chip and connects its pins through **tunnels** named after their nets, which Digital joins the way KiCad joins labels. Switches, LEDs, bar graphs, 7-segment digits, clock sources, oscillator cans and the power supply each become the matching Digital element. The switches start in the position they're in on the desk. External signals become Digital buttons (momentary) or inputs (toggle), labelled with their names.

These exports have been run in Digital itself and checked against Chip Hippo's own simulation. For the example circuits, every chip output and every lamp reads the same in both, at a range of switch settings.

What changes

Digital is a logic simulator, so some parts have to be exported as what they **mean** rather than what they are:

- **Resistors** are exported as what they do in the logic. A resistor to a supply rail becomes a pull-up or pull-down on the net at its other end, which is what a lamp's current-limiting resistor and a switch's pull-down amount to. A resistor between two signals becomes a plain wire (the report lists it). A resistor from rail to rail does nothing in logic and is left out.
- **Floating inputs**: an unconnected TTL input reads HIGH. Digital treats an undriven input as an error, so the export adds the pull-up that makes it HIGH in Digital too. It doesn't add one where a resistor already sets the level. An unconnected **CMOS** input gets the same pull-up, but there it changes the answer: Chip Hippo reads a floating CMOS input as unknown, and Digital has no unknown to give it. The report names each CD4000 chip this happens to.
- **Open-collector chips** (74LS01, 74LS03, 74LS05) get a pull-up on each output. Chip Hippo simulates their outputs driving both ways, but Digital simulates them as the real parts, which only pull low. The pull-up makes the two behave the same, as it would on a real board.
- **Push buttons** become toggle switches, because Digital has no momentary switch contact.
- **Unpowered chips** are powered, because Digital can't simulate a chip with its supply pins unconnected — and Digital powers every rail it's given, so a chip on rails that no power supply reaches runs there too. Either way the report lists the chip as unpowered, since it does nothing in Chip Hippo.

What doesn't come across

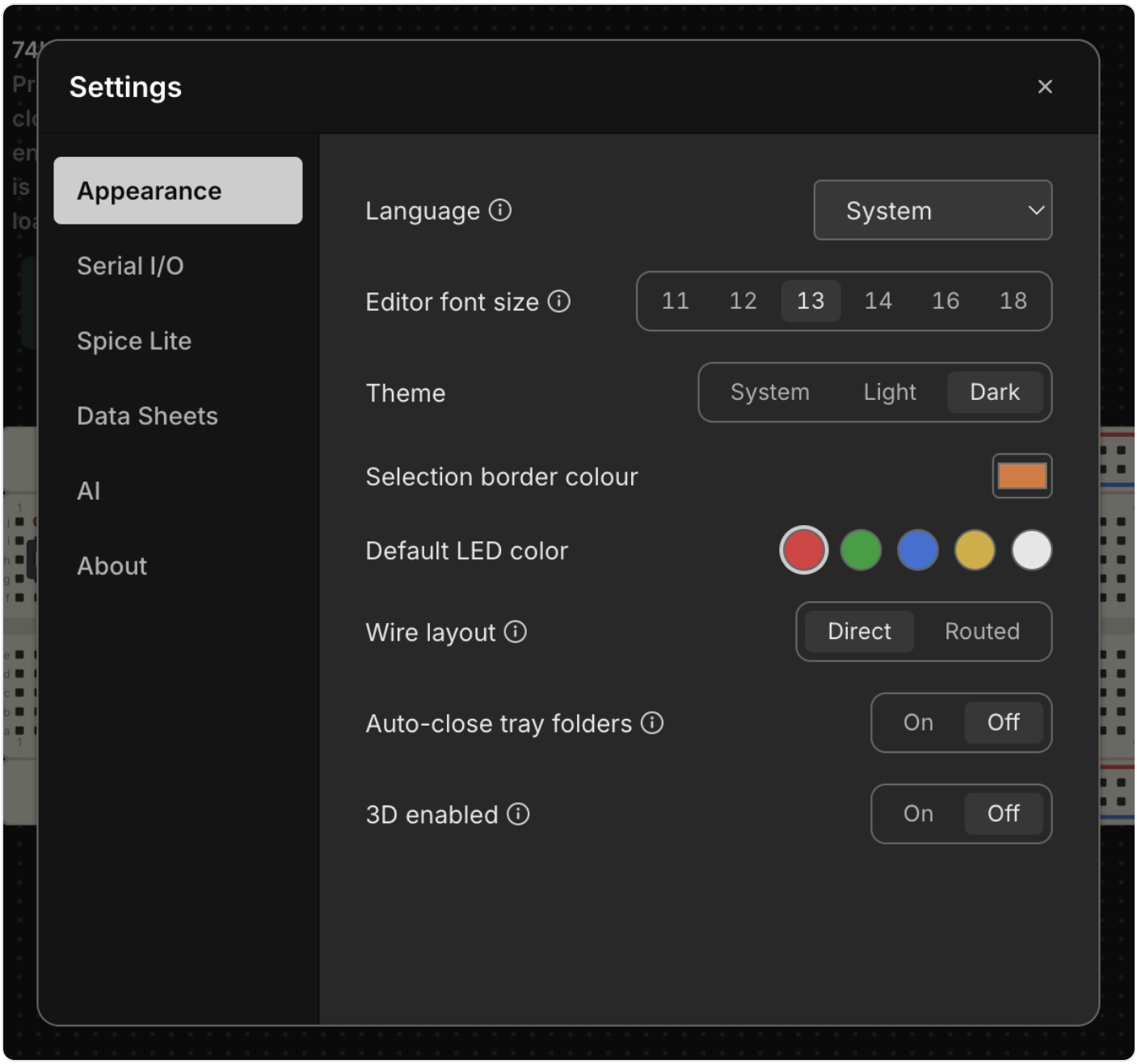
- Seven 74xx parts that Digital's library doesn't include: 74LS73, 74LS75, 74LS169, 74LS240, 74LS259, 74LS279 and 74LS533. Each one's place in the circuit is marked with a note, and everything wired to it is still exported.
- The other 37 CD4000 CMOS parts, which have no twin in Digital's library. The same note marks each one's place.
- The 555 timer, which Digital's library doesn't have either.
- Capacitors. A capacitor connects nothing in Chip Hippo, so leaving it out changes nothing about the circuit.
- Diodes and transistors: Digital has none that pass a level one way, or switch, the way Chip Hippo's do.
- Inductors. An inductor is a wire in Chip Hippo, so the two nets it joins come across **apart** — the report says so; join them in Digital if you need them joined.
- Memory chips, because their contents are stored in files the export doesn't carry.
- The processors and their peripheral chips (W65C02, Z80, W65C21, W65C22).

- The character LCD modules.
- Arduino Output and Input elements.

Digital needs its own library to find the chips. It comes in the `lib` folder of the Digital download, which has to stay beside `Digital.jar`. If a chip shows up as missing, check that folder and Digital's library settings.

Settings

Chip Hippo keeps its handful of app-wide preferences in one small **Settings** dialog — a tabbed master-detail card with a left nav rail and a panel on the right. It's deliberately minimal: five tabs, a few controls each, applied live the moment you change them.



Opening Settings

Open the dialog with `Cmd/Ctrl+,`, or click the gear icon in the top-right corner of the header. Both routes lead to the same dialog seeded with your current settings, so it doesn't matter which you use.

Several settings carry a small ⓘ beside their label. Click it for a note explaining what the setting does and what it doesn't touch; `Esc`, a click anywhere outside the note, or the ⓘ again closes it. The first `Esc` closes the note and the next one closes Settings, so you never lose the dialog by dismissing a note.

Appearance

The **Appearance** tab is the one the dialog opens on:

- **Language** — the language Chip Hippo's own menus, panels, dialogs and tooltips are shown in: **System**, **English**, **Deutsch**, **Español**, **Français**, **Italiano**, or . *System* follows your operating system's own language, falling back to English if Chip Hippo doesn't ship that one. Every language names itself in its own words rather than in the one you're currently reading, so the list stays usable if you land on one you can't. The change applies **immediately and everywhere** — the menu bar and every open window included, with no restart and nothing reloaded, so an unsaved design is never at risk. This user guide is only available in English.
- **Editor font size** — how big Chip Hippo's own text is drawn, in pixels: **11**, **12**, **13**, **14**, **16** or **18**, default **13**. It applies to every window at once, including the floating pinout, memory inspector and user-guide windows, and to the boxes around the text — buttons and panels grow with it rather than clipping it. `Cmd+=` and `Cmd+-` step through the sizes from the keyboard and `Cmd+0` restores the default. What it does *not* change is the desk itself: chip labels, board addressing and your own annotations are printed on the circuit and drawn to the breadboard's scale, so they grow with the desk zoom (`Option+Cmd+=`) instead.
- **Theme** — **System**, **Light** or **Dark**. System follows your operating system's own setting; the other two pin it. The choice applies to every Chip Hippo window at once, including the floating pinout, memory inspector and user-guide windows, and to the native menus and dialogs.
- **Selection border colour** — a `#rrggbb` colour picker for the outline drawn around whatever's selected on the desk. Leave it unset to use the theme's default accent colour instead of a custom one.
- **Default LED color** — which colour a newly placed LED, LED bar or seven-segment digit gets. Unlike the others this one isn't applied live; it's read when you place a part, so changing it doesn't recolour anything already on the desk. Any placed part's colour can still be changed from its **Properties...** dialog.
- **Wire layout** — **Direct** (the default) or **Routed**, the layout method a newly laid wire gets. Direct wires sag from hole to hole; routed wires run straight and can be bent around obstacles by dragging points

into them (see Wiring). Like the LED colour it's read when you lay a wire, so wires already on the desk keep the layout they have — change one from its own **Properties...** dialog.

- **Auto-close tray folders** — **On** or **Off** (the default). When it's on, opening a folder in the parts tray closes every other one, so only the section you're working in stays open, along with the folder it sits inside. Clicking an icon on the shut tray's strip does the same. It acts on the next folder you open, so switching it on doesn't close anything by itself.
- **3D enabled** — **On** or **Off** (the default). When it's on, the toolbar shows the **3D** button that switches to the 3D view; off hides the button, and if the 3D view is showing at the time, Chip Hippo goes back to the breadboard.

Everything but the LED colour and the wire layout takes effect immediately — there's no separate Apply or OK step. (Changing the language is the one setting that closes and reopens this dialog, so the card you're looking at comes back in the language you just picked, on the tab you were reading.)

Serial I/O

The **Serial I/O** tab holds the **connections** — the named Arduino boards that Outputs and Inputs talk to. A dropdown at the top picks one, and the settings under it are that connection's. The list always starts with the built-in **Mock**, a pretend board with nothing to configure (see The Mock connection); its **Open window...** shows what it receives and lets you send values back. **+** beside the list adds a connection, and the bin removes the one picked — click it twice (the first click turns it into a **?**; clicking anywhere else, or `Esc`, puts it back). The ⓘ says what connections are for.

Each connection has a **Name**, the **Port** it is plugged into (**Re-scan** looks again after you plug a board in), a **Baud rate**, the **Language** Generate writes the board's code in — **C++** (the default, an Arduino header) or **Python** (a module for MicroPython and CircuitPython boards; see Python boards) — and under **Advanced** the rest of the serial framing (parity, stop bits, flow control — data bits are always 8, and flow control is None or Hardware, since the link is binary). Unlike the other tabs, nothing here applies until you press **Apply**, which also checks the port is present — if it isn't, you're told so and the button becomes **Apply anyway**. You can edit several connections before applying: each keeps its changes while you look at another, and one with changes not yet applied is marked • in the list. **Open window...** opens the connection's window. The book icon beside the card's close button (shown on this tab only) opens the Serial Protocol reference in the user guide.

A connection the circuit can't run with yet says why in the list — **Port unavailable**, or **Needs configuration** when it arrived in a project from another computer (a project carries its connections' names and settings, but never a port) — and its **Port** is marked in red. See Connections for the details.

Spice Lite

The **Spice Lite** tab switches on the more electrical simulation (see Spice Lite). **Spice Lite** turns it **On** or **Off**, from the next Run. **Settle gap** is how close a voltage that no input is watching gets to its final value before the probe stops redrawing it (1 % by default). **Switch bounce** makes a switch's contacts bounce for a few milliseconds as they close, as a real one's do (Off by default; see Switch bounce). A **TTL | CMOS** strip shows the families the parts palette shows (and any family the open project uses). Under each are its datasheet source, a **Reset to defaults** button, and an **Advanced** section holding every number Spice Lite uses for that family. A value that won't read turns red and keeps the old one. Every change, the numbers included, applies from the next Run, and to every project on this computer.

Data Sheets

Chip family leads the tab: which logic family the parts palette shows — **74LS** (the TTL chips, the default), **CD4000** (the CMOS chips), or **Both**, each in a folder of its own. It applies at once, and it only filters the palette: a project's chips always load and run whatever is chosen here, and a family the open project uses is shown anyway. See Choosing a logic family.

Below it, the tab points Chip Hippo at an external folder of manufacturer datasheet PDFs on your machine. Click **Browse...** to pick a folder with the native file picker; the chosen path is shown next to it, with a **Clear** button that resets it to no folder selected.

Name the PDFs in that folder after each chip's catalog reference (for example `74LS00.pdf`). When a chip's file is found there, its pin-assignments window grows a button that opens the PDF in your system's default viewer. This is separate from the small datasheet crop the pinout window already shows for most chips — the external folder is for the full manufacturer document, not the built-in crop.

If you don't already have a collection, **Download...** fetches one for you. It pulls a datasheet for every part Chip Hippo has a published source on file for, names each file after the part it belongs to, and points the setting above at that folder when it's done — so the buttons appear with nothing else to set up. A progress window counts the files in and names anything that couldn't be fetched; closing it stops the download and keeps whatever already arrived. The sources are:

- Texas Instruments — the current manufacturer's own documents for much of the 74LS family, and for every CD4000 part (several of which share one sheet — it is fetched once and saved under each part's name);
- Microchip — the Atmel EEPROM parts;
- Alliance Memory — the large SRAM;
- the Western Design Center — likewise for the 65xx parts;
- Zilog — the Z80 family user manual;

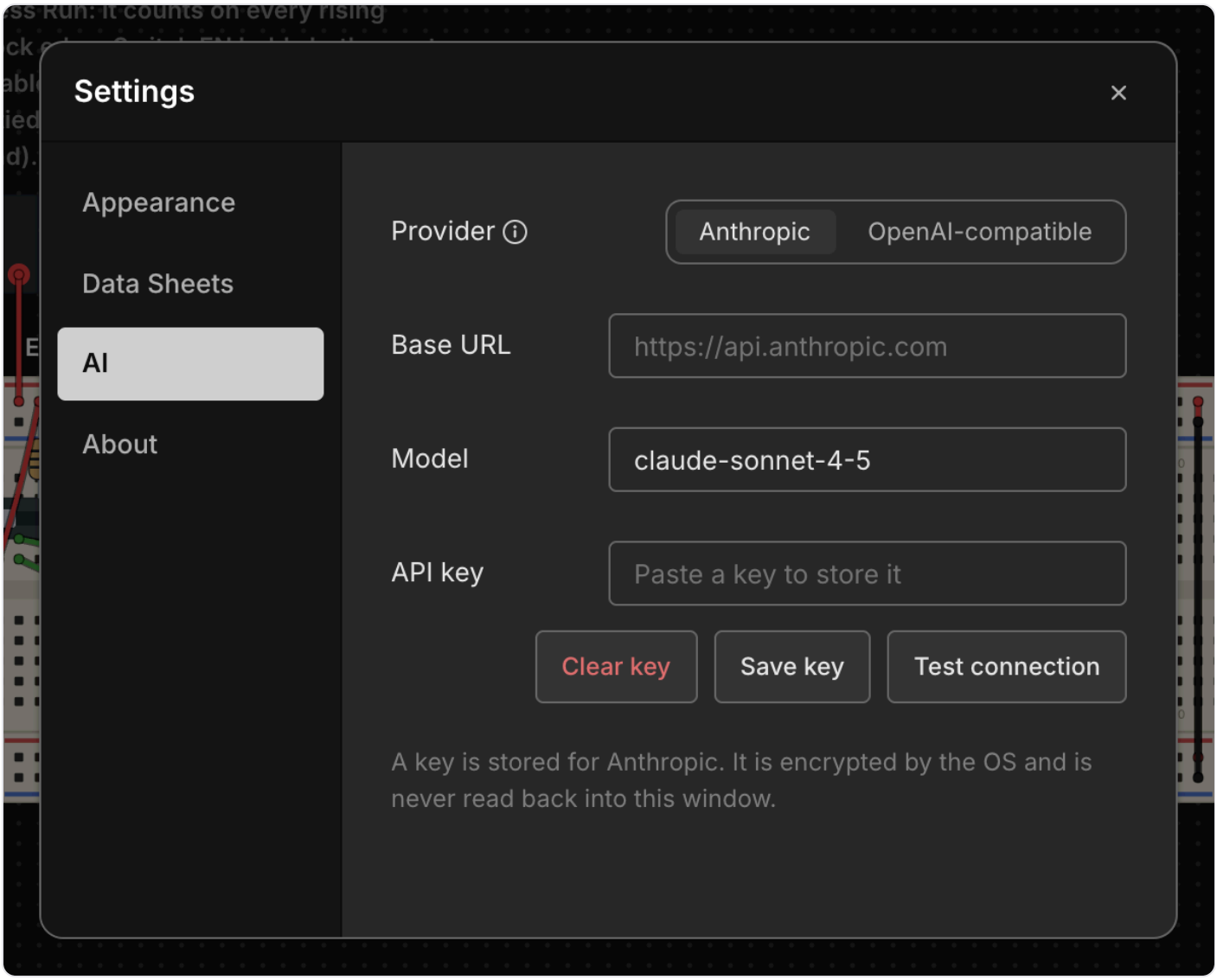
- the USC EE 459Lx reference library — vendor scans of the older 74LS parts, collected for a university course, plus the HD44780, the memory parts and the DIP resistor/oscillator packages;
- a datasheet archive, for the one part whose maker no longer publishes it.

Expect a few tens of megabytes in total; the modern manufacturer PDFs are much larger than the old scans.

Between them they don't carry every part in the catalog, so some chips will still have no PDF button. Press **Download…** again whenever you like — it replaces what it fetched last time. It always writes to Chip Hippo's own folder and never to one you picked yourself, so a collection of your own is never overwritten: **Browse…** and **Download…** are simply two ways of setting the same folder, and whichever you did last is the one in effect.

AI

The **AI** tab is where you point Chip Hippo at your own AI connection, for the AI circuit builder. Nothing here is required — the app works completely without it, and contacts no AI provider until a connection is configured and you ask the builder for something. (The only other times Chip Hippo reaches the network at all are the datasheet **Download…** button above, while you're pressing it, and the update check on the **About** tab below, which is switched off until you turn it on.)



- **Provider** — **Anthropic**, or **OpenAI-compatible**. The second covers Ollama, LM Studio, OpenRouter, vLLM and anything else speaking that request format; you supply the base URL.
- **Base URL** and **Model** — leave either blank to use the chosen provider's default, which is shown as the field's placeholder. Both commit when you leave the field or press `Enter`, not as you type.
- **API key** — paste it and click **Save key**. **Clear key** forgets it. A local server that needs no key works with this left blank.
- **Test connection** — one small request that checks the base URL, key and model together, so a typo is found here rather than on your first build.

The key is never written to Chip Hippo's settings file. It's handed to your operating system's own secure credential store — Keychain on macOS, the Credential Manager on Windows, the desktop keyring on Linux — encrypted, and it's never read back into the app's window afterwards; the dialog only learns *whether* a key is stored. If no secure store is available on your system, Chip Hippo **refuses to save the key** and says so rather than quietly writing it out in the clear.

About

The **About** tab shows which version of Chip Hippo you're running, and is where updates are handled.

- **Check automatically** — **On** or **Off**, and **Off** is the default. On, Chip Hippo asks GitHub once, shortly after launch, whether there's a newer release. Off, it never asks on its own — but the button below still works whenever you want to know.
- **Check for updates** — checks right now and says what it found on the line underneath: that you're up to date, that a version is downloading, or that the check couldn't be made.

When an update is found it downloads quietly in the background; you can carry on working, and there's no progress bar in your way. Once it's ready a **Restart to update** button appears here, and a notice appears in the corner of the desk with a **Restart** button of its own. **Nothing is installed until you say so.** If you'd rather not stop what you're doing, ignore it — the update installs by itself the next time you quit Chip Hippo, and restarting here runs the same "you have unsaved changes" prompt that quitting does, so a design in progress is never lost to an update.

The check itself is the only thing sent, and only to GitHub. Chip Hippo has no analytics and reports nothing about you or your circuits.

You'll see this tab differently depending on where your copy came from. A copy installed from the **Mac App Store** updates through the App Store like every other app there, so this tab shows only the version and says so — there are no controls, because there'd be nothing for them to do. A development build says that updates are only available in installed builds.

What else persists automatically

A few things aren't part of this dialog but are remembered between sessions without any action on your part: the app window's position and size, the desk camera — your current pan position and zoom level — and which panels you left open (the parts tray, the build guide, the analyzer, the AI builder). The ones you can resize remember that too: the parts tray's **width**, and the analyzer and AI builder's **height**. Close Chip Hippo and reopen it, and you're back exactly where you left off.

The desk padlock isn't one of these: it's saved in the project file, so it belongs to the project rather than to the app (see The Desk & Breadboards).

See Files, Saving & Undo for how your circuit itself is saved.

Keyboard Shortcuts

Chip Hippo follows Electron's `CmdOrCtrl` convention throughout, so every shortcut listed here as `Cmd` also works as `Ctrl` on Windows and Linux. Most shortcuts are disabled while a text field has focus, and some are further gated on the current tool or simulation state — those conditions are noted alongside the shortcut.

Desk & simulation

Shortcut	Action
`Space`	Run / Stop the simulation (not while typing, or while a placement/wire/bus tool is armed)
`Cmd+R`	Run / Stop the simulation
`1` – `9`, `0`	Press signal buttons 1–10, top to bottom (`0` is the tenth) — **only while the circuit runs**, and several at once (see External signals)
`Tab`	Switch between the Breadboard and Schematic views (not while typing); from the 3D view, to the Schematic
`Escape`	Abandon a drag in flight, then unpin a probed net, then disarm the probe, then cancel a pending wire/bus, then cancel a placement in hand, then deselect — whichever applies first
`Delete` / `Backspace`	Remove the current selection (a part, wire, bus, annotation, board, or a whole multi-selection); a selected Output or Input tag is unplugged back onto its card instead

Selecting & moving

These are mouse chords rather than keystrokes, but they are the modifiers the desk answers to. The Desk covers building a selection; Chips & Components covers moving one.

Chord	Action
`Shift`-drag	Rubber-band a selection — it replaces whatever was selected
`Cmd`-click	Add one item to the selection, or take it back out (`Ctrl`-click on Windows/Linux; on a Mac `Ctrl`-click stays the context menu)
drag a part	Re-seat that part alone, leaving every wire where it was
`Option`-drag a part	Move it **with everything plugged into it** — the wire ends, and the legs of the resistors and LEDs, in the column-halves its pins occupy
drag any member of a selection	Move the whole selection as one rigid unit
`Option`-drag a member of a selection	The same, carrying everything riding any member of it
hold `Option` over a selection	Ring every hole an `Option`-drag would carry, before committing to the drag (a selected part, or a whole multi-selection)
`Option`-drag a board	Tear off the run of mated strips reachable forward (below/right) from the one you grabbed
`Option` + `Shift`-drag a board	The same, running backward (above/left)

Tools

Shortcut	Action
`W`	Arm/disarm the wire tool
`B`	Arm/disarm the bus tool
`I` or `P`	Arm/disarm the probe tool (works even while the simulation is running)
`M`	Disarm whichever of the wire/bus/probe tool is currently armed
`H`	Fade the wires down to a stub at each end, and back (works while running — it only changes what's drawn)
`1` – `8`	While the wire tool is armed, pick a wire color
`1` – `8`	While the bus tool is armed, pick the bus width — `2` – `8` for that many bits, `1` for 16-bit

Placing & rotating parts

Shortcut	Action
`R`	Rotate or flip the part being placed or selected — see Chips & Components for the exact behavior per part type
`R` (Output/Input tag)	Turn the tag being dragged, or the selected tag, a quarter-turn
`F`	Flip an LED's polarity while its placement ghost is armed
`Cmd+C`	Copy the selection — one part, a whole multi-selection of parts, or (if it includes boards) the whole sub-assembly
`Cmd+V`	Paste a copy as a new placement ghost

View

Shortcut	Action
`Cmd+F`	Fit the desk (recentring it) — or the schematic or 3D view, when it is showing
`Cmd+Shift+F`	Zoom out to fit everything at once
`Cmd+=`	Increase the interface text size
`Cmd+–`	Decrease the interface text size
`Cmd+0`	Reset the interface text size (13 px)
`Option+Cmd+=`	Zoom the desk in
`Option+Cmd+–`	Zoom the desk out
`Option+Cmd+0`	Reset the desk zoom
`Cmd+L`	Lock or unlock the desk against the scroll wheel
`A`	Toggle the Logic Analyzer panel
`Cmd+P`	Toggle the parts palette panel

Both scale shortcuts want the **unshifted** key. On a layout where `+` needs `Shift`, use `Cmd+=` rather than `Cmd+Shift+=` — the shifted chord is deliberately ignored, so it can't collide with anything else.

The two families above do different jobs. `Cmd+=` and friends change **Chip Hippo's own text**: toolbars, panels, menus and dialogs, in every window at once — the same setting as Settings ▸ Appearance ▸

Editor font size. Hold **Option** as well and you move the **desk camera** instead, zooming the circuit, chips, boards, wires and all. Markings on the desk are printed on the circuit, so they follow the desk zoom rather than the text size.

> **On a Mac**, `Option+Cmd+=` and `Option+Cmd+–` are also macOS's own Zoom shortcuts (System Settings ▸ Accessibility ▸ Zoom ▸ *Use keyboard shortcuts to zoom*). If you have that turned on, the system takes the keys first and Chip Hippo never sees them — use the zoom cluster in the bottom corner of the desk, or `Cmd+F` to fit, instead.

Files & editing

Shortcut	Action
`Cmd+N`	New Project (a blank slate, one empty desktop)
`Cmd+O`	Open… (load a saved project)
`Cmd+Shift+O`	Recent projects (the same menu a right-click on the toolbar's Open button drops)
`Cmd+S`	Save the project to its file
`Cmd+Shift+S`	Save As… (give the project a new file)
`Cmd+B`	Toggle the build guide (Bill Of Materials) panel
`Cmd+A`	Select All — every board, part and wire on the desk; in a text field, its text
`Cmd+Z`	Undo
`Cmd+Shift+Z`	Redo

Chip Designer & debugger

In the Chip Designer window. The debugger keys act while the circuit runs, and only when their button on the debugger bar is not greyed out.

Shortcut	Action
`F9`	While designing: set or remove a breakpoint on the caret's line
`F8`	Continue — run on to the next breakpoint
`F6`	Step — execute one statement
`F7`	Step Out — finish this chip's reaction
`F9`	To Settled — run until the board settles, ignoring breakpoints
`F10`	Detach — stop debugging this chip for the rest of the run

CPU monitor

In the CPU monitor window. Each function key acts only when its button on the monitor's bar is not greyed out.

Shortcut	Action
`F8`	Pause the run, or Continue it to the next breakpoint
`F6`	Step — run to the start of the CPU's next instruction (paused)
`F7`	Step Over — run a JSR, CALL or RST's subroutine and pause after the call (paused)
`F9`	Set or remove a breakpoint on the selected byte
`Page Up` / `Page Down`	Move the selected byte 256 bytes, the memory block with it
`Escape`	Drop a typed digit, let go of the selection, or close the window

App

Shortcut	Action
`Cmd+,`	Open Settings
`Cmd+K`	Open the quick Keyboard Shortcuts popup
`Cmd+/`	Open the Chip Hippo User Guide (this guide)
`Alt+Cmd+I`	Toggle Developer Tools